Recommender Systems: From Scratch to Shopee Scale

A hands-on guide with a working marketplace

Built 02 November 2026

Contents

Preface

The big picture in plain words
Who this book is for
The shape of the book
What is in the repository
Setting up
The running example: a simulated marketplace
How each chapter is organised
A note on scale

Chapter 1 — What a Recommender Does

The Monday-morning problem
Intuition: the shop assistant who remembers you
The algorithm: framing the problem
Code walkthrough
Run it
Results: reading the shape of the data
Pitfalls and how it is done in production
Summary
Quiz

Chapter 2 — Measuring success

What you'll learn
Intuition: testing a forecaster on last month's weather
Offline versus online evaluation
Time-based splits, and why random splits leak
What we are measuring: labels and ranked lists
The algorithm: ranking metrics from the ground up
Code walkthrough: `recsys/evaluate.py`
Run it
Results: what does a random recommender score?
Pitfalls, and how it is done in production
Summary
Quiz

Chapter 3 — Popularity baselines

Intuition: the shelf at the front of the shop
The algorithm: five ways to count
Code walkthrough
Run it
Results
Pitfalls and how it is done in production
Summary
Quiz

Chapter 4 — Content-based filtering

Intuition: the librarian who never watches the readers
The algorithm
Code walkthrough
Run it
Results
Pitfalls and how it is done in production
Summary
Quiz

Chapter 5 — Collaborative filtering

Nobody wrote the word "coffee" on that card
The intuition: asking people with similar taste
The algorithm
Code walkthrough
Run it
Results
Pitfalls and how it is done in production
Summary
Quiz

Chapter 6 — Matrix factorisation

The shopper nobody resembles
Intuition: the taste dimensions nobody wrote down
The algorithm
Code walkthrough
Run it
Results
Pitfalls and how it is done in production
Summary
Quiz

Chapter 7 — Embeddings and two-tower retrieval

Building blocks you need first
Intuition: from names to coordinates
The two-tower architecture
The algorithm: training with an in-batch softmax
Code walkthrough
Approximate nearest neighbours
Run it
Results
Pitfalls and how it is done in production
Summary
Quiz

Chapter 8 — Multi-source candidate generation

Intuition: the librarian and the shortlist
Why two stages: the funnel

The four retrievers
Combining the sources: `generate`
Measuring retrievers: recall, pair recall and unique recall
Run it
Results
Hard filters: what must never reach the ranker
Code walkthrough: how the evaluation script is built
Pitfalls and how it is done in production
Summary
Quiz

Chapter 9 — Feature engineering for ranking

Intuition: features are a briefing note
The algorithm: from raw events to a feature row
Code walkthrough
Run it
Results: reading a feature row
Pitfalls and how it is done in production
Summary
Quiz

Chapter 10 — Learning to rank

Intuition: sorting by a score someone learned
The algorithm
Code walkthrough
Run it
Results
Pitfalls and production practice
Summary
Quiz

Chapter 11 — Re-ranking, diversity and business rules

The page that was all baby formula
Intuition: building a shopping window, not picking the best products
The algorithm
Code walkthrough
Run it
Results: what does diversity cost?
Beyond diversity: business rules in the re-ranker
Pitfalls and production practice
Summary
Quiz

Chapter 12 — Cold start, context and bias

Intuition: the new colleague, the restaurant menu and the escalator
The algorithm: ladders, propensities and loops
Code walkthrough
Run it
Results
Pitfalls and how it's done in production
Summary
Quiz

Chapter 13 — Building the system

The intuition: a restaurant with a ledger
The algorithm: the life of one click
Code walkthrough, part 1: capturing behaviour in the browser
Code walkthrough, part 2: the server writes it down
Code walkthrough, part 3: the recommender reads it back
Code walkthrough, part 4: the Rust side of the bridge
The contract between languages: one `.proto`, two code generators
Run it
Results
Pitfalls and production practice
Summary
Quiz

Chapter 14 — From prototype to Shopee scale

Intuition: the night market and the supermarket chain
The algorithm: the same pipeline, scaled out
Code walkthrough: pseudo-code for the missing pieces
Run it: the book's leaderboard
Results: reading the leaderboard
Pitfalls and how it is done in production
Summary
Quiz

Appendix A — Quiz answers

Chapter 1 — What a Recommender Does
Chapter 2 — Measuring success
Chapter 3 — Popularity baselines
Chapter 4 — Content-based filtering
Chapter 5 — Collaborative filtering
Chapter 6 — Matrix factorisation
Chapter 7 — Embeddings and two-tower retrieval
Chapter 8 — Multi-source candidate generation
Chapter 9 — Feature engineering for ranking
Chapter 10 — Learning to rank
Chapter 11 — Re-ranking, diversity and business rules
Chapter 12 — Cold start, context and bias
Chapter 13 — Building the system
Chapter 14 — From prototype to Shopee scale

Appendix B — Glossary

Appendix C — Reproducing every result

1. Tools
2. The offline companion scripts (chapters 1–12)

3. The live system (chapter 13)
4. Charts
5. Screenshots
6. The PDFs

Bibliography

Preface

Open the Shopee app on a Tuesday evening. Before you type a letter, the home screen already shows forty products: a phone case for the phone you bought last month, cat food because you browsed pet products last week, a flash-sale kettle. Nobody wrote those slots by hand. A pipeline looked at millions of products, discarded almost all of them in milliseconds, scored the survivors, and shuffled the winners so the page did not look like a wall of identical phone cases.

That pipeline is a **recommender system**. This book teaches how one works by building one: first a list of the most popular items, then, chapter by chapter, every idea that turns that list into a modern multi-stage system.

The big picture in plain words

A recommender answers one question: "**Out of everything we sell, which few items should this person see right now?**" It does this using clues the shop already has: what the person clicked and bought before, and what similar people did.

The difficulty is size. A marketplace like ours has thousands of items (millions in a real app), and the screen has room for about forty. We cannot carefully judge every item for every visitor within the fraction of a second before the page loads. So real systems work like a shop assistant fetching things from a huge warehouse for a customer:

1. **Retrieval (candidate generation): grab a trolley-load from the warehouse.** The assistant does not inspect every box. They walk to the pet aisle and the phone-accessory aisle, because Sari's history says she likes those, and pull a few hundred boxes quickly. This step must be *fast*, and it may be a little rough. The few hundred items it returns are the **candidates**.
2. **Ranking: look closely and put the trolley in order.** Now the assistant has only a few hundred boxes, so they can afford to examine each one carefully: how likely is Sari to click this, given her price range, her past orders and the time of day? Each candidate gets a score and the list is sorted from best to worst. This step is *precise* but slow per item, which is why it only sees candidates.
3. **Re-ranking: arrange the shop window.** Sorted by score alone, the top ten might be ten nearly identical phone cases. The assistant reshuffles for variety, pushes out-of-stock products down, and follows shop rules, such as not showing the same seller five times in a row. This final polish is **re-ranking**.

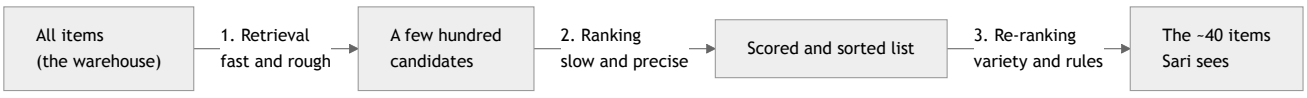


Figure P.1: *The three-stage funnel. Each stage looks at fewer items than the one before, so it can afford to do more work per item.*

The funnel exists because of a trade-off: careful judgement costs time, and time is limited, so we spend cheap effort on many items and expensive effort on few. Without it, you would either show slow pages (judging everything carefully) or poor recommendations (judging everything roughly).

We build exactly this shape step by step. First, in Chapters 3 to 7, one method tries to do the whole job. Then Chapters 8 to 12 split the work into the three stages above.

Who this book is for

The book is written for **mid-level software engineers**: you read Python comfortably, know REST and SQL, and have shipped production code. No machine-learning background is needed; each piece of mathematics is explained the first time it appears and used on a tiny worked example. Everything runs on a laptop in minutes, with no GPU.

Two promises shape everything that follows:

1. **Every algorithm is explained step by step.** You will see the intuition, the formula, a worked example, and then the real code that implements it.
2. **Every number in the text comes from a program you can run.** No result in this book was typed by hand. Each chapter has a companion script; its output is pasted into the book directly from a file captured when the script ran. If you re-run the script you will get the same numbers, because all random seeds are fixed. A **random seed** is like rolling dice in a game where everyone rolls the same numbers in the same order: the "random" parts of the simulation and training come out identical on every run, so your numbers match the book's.

The shape of the book

The book follows the historical and practical path of the field. Each chapter adds machinery only after the previous approach has shown its limits on the same data.

- **Foundations (Chapters 1–2).** The problem, the data, and the ruler: precision, recall, NDCG and time-based evaluation. In short, these are three scores for "how good is this list?" and the habit of testing a recommender on days it has not seen.
- **Single-model recommenders (Chapters 3–7).** One method does the whole job. Popularity (show what everyone likes), content-based filtering (suggest items whose description resembles what you liked), collaborative filtering (suggest what people similar to you liked), matrix factorisation (compress everyone's tastes into a few hidden numbers), and embeddings with two-tower retrieval (turn users and items into lists of numbers so that "close" means "a good match"). Each term is explained from scratch in its own chapter.
- **The multi-stage system (Chapters 8–12).** Real marketplaces run a funnel, described in the next section: a cheap stage fetches a few hundred **candidates**, a precise stage **ranks** them, and a final stage **re-ranks** for diversity and business rules. These chapters cover candidate generation, feature engineering (turning raw logs into useful numbers for the model), learning to rank, re-ranking, and cold start, context and bias.
- **Production (Chapters 13–14).** A working Vue (web page), Rust (server), Postgres (database) and Python gRPC application in which you follow one click end to end, then what changes at tens of millions of users.

The appendices hold quiz answers, a glossary, reproduction commands and the bibliography.

What is in the repository

The book lives inside the repository of a complete working project. The repository contains the book, the code the book teaches, and the application that uses it.

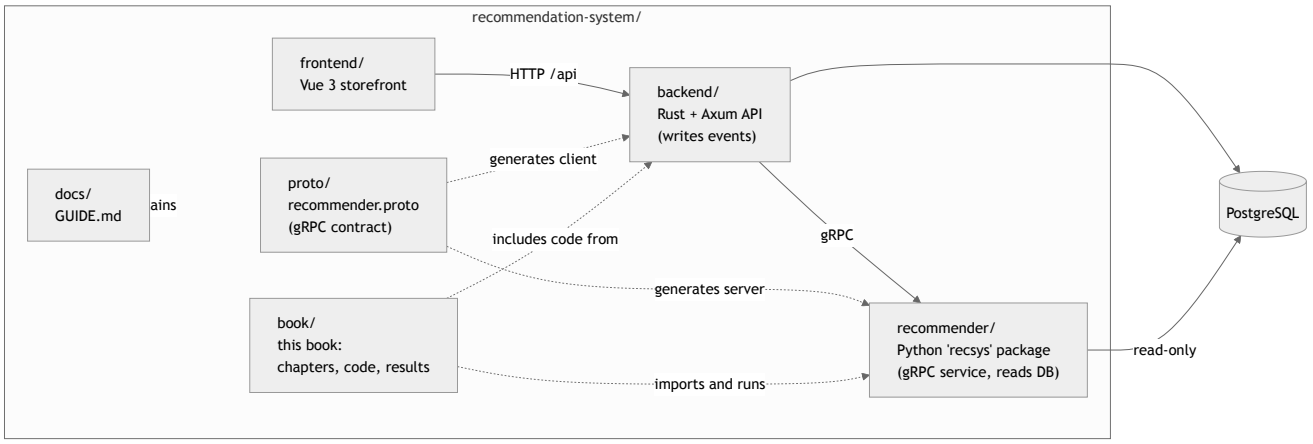


Figure P.2: *The repository at a glance. Solid arrows are runtime calls; dotted arrows are build-time or documentation relationships.*

Each directory has one job:

Directory	What it holds	Where it appears in the book
`recommender/`	The Python package `recsys`: the marketplace simulator, candidate retrievers, feature builders, the LightGBM ranker, the re-ranker and the gRPC service	Chapters 1–3 and 8–13
`book/code/`	One companion script per algorithm chapter, plus `common.py` shared by all of them	Every chapter
`book/results/`	The captured output (`.txt`) and metrics (`.json`) of those scripts	Every `output` block
`backend/`	The Rust and Axum API: authentication, catalogue, cart, checkout, event logging, the gRPC client	Chapter 13
`frontend/`	The Vue 3 storefront with the "Why this?" explanations and the live event log	Chapter 13
`proto/`	`recommender.proto`, the single file that defines the contract between Rust and Python	Chapter 13
`docs/`	`GUIDE.md`, a plain-language walkthrough of the running application	Chapter 13
`book/`	The chapters, charts, screenshots and build tooling for the PDF you are reading	–

Two design decisions recur throughout. **Rust writes all user behaviour to Postgres; Python only reads it**, through a read-only connection (one that can look at the data but cannot change it, so the recommender can never corrupt the shop's records). And **recommendations are computed on request**, meaning the list is built at the moment you open the page, so a click made five seconds ago can

already change the home page, while the model (the learned part) is retrained periodically in the background.

A few words in the table above, in plain terms. **gRPC** is a way for two programs, here the Rust one and the Python one, to call each other's functions over the network. A **`.proto` file** is the written agreement describing exactly which calls exist and what they carry. **LightGBM** is a library that learns to score items; Chapter 10 explains it.

Setting up

You can read the book without running anything, but you will learn more if you change a number and watch what happens.

What you need

Tool	Used for
`uv`	Python environments for `recommender/` and `book/`
Node.js and `npm`	The Vue frontend, and the `mermaid` and `katex` libraries used to render diagrams and math in the PDF
PostgreSQL	Only for the live application of Chapter 13. Listening on `localhost:5432`, user `postgres`, password `postgres`
Rust toolchain (`cargo`)	Only for the backend
`protoc`	Only to compile the gRPC contract for Rust

The algorithm chapters (1–12) need **only `uv`**. Postgres, Rust and Node are needed only when you want to run the full web application.

Companion scripts and the book tooling

Clone the repository, then from the repository root run:

```
cd book && uv sync && uv run playwright install chromium && npm i
```

`uv sync` installs Python dependencies, including the `recsys` package from `../recommender` in editable mode, so the scripts import the same code the chapters show. The `playwright install` step downloads a headless Chromium, which is used both to render the PDF and to take the screenshots. `npm i` fetches `mermaid` and `katex`.

Every companion script runs from the `book/` directory:

```
uv run python code/ch01_explore.py
```

To regenerate every result file in order, use `uv run python tools/run_all.py`. Appendix C lists all commands, including those that rebuild the figures and the PDF.

The web application

For Chapter 13 you need the three services running. The authoritative, step-by-step instructions are in `docs/GUIDE.md`, section "Run it". In outline:

1. Create the database and apply the schema: `createdb marketplace`, copy `backend/.env.example` to `backend/.env`, then `cargo run -- migrate` inside `backend/`.
2. Fill the database with the simulated marketplace: `uv run recsys seed-db --yes` inside `recommender/`.
3. Start three terminals: `uv run recsys serve` (Python, port 50051), `cargo run` (Rust, port 8080) and `npm run dev` (Vue, port 5173).

Then open `http://localhost:5173`. The seed data includes demo accounts such as `user43@demo.test` with the password `password`; the login page lists the others.

The running example: a simulated marketplace

Real marketplace logs are private and public datasets lack positions (where on the page an item was shown), dwell time (how long the user stayed on it) and stock, so the book uses a **simulated marketplace** shipped with the repository: a small pretend store, run by a program, with pretend shoppers who browse and buy. Because we wrote the simulator we also know the hidden truth behind the behaviour, which Chapter 1 uses to show how logged data distorts what users want.

The default simulation, configured in `recommender/recsys/simulate.py`, has 1,500 users, 2,500 items, 150 shops and 60 days of activity, with fixed seeds. The Chapter 1 script confirms the table sizes:

TERMINAL OUTPUT

```
=== Table sizes ===
categories          25
shops              150
items            2,500
users            1,500
sessions        19,455
searches         8,680
events         276,378
```

Beyond sheer size, the simulator reproduces the patterns that make recommendation interesting. Some terms below are new, so each comes with a plain meaning:

- **Popularity is lopsided.** A small share of items attracts a large share of interactions, exactly as in a real store. Plot items from most to least popular and you get a few tall bars followed by a very long, low tail; hence the name **long tail**. The danger is that a recommender which only repeats the tall bars never shows shoppers the rest of the shelf.
- **Tastes are personal.** Each user has hidden preferences for categories, price range and brands. Sari might love cat food and cheap phone cases; another user might prefer premium electronics.
- **Time matters.** Traffic is higher on weekends and around paydays (the 25th through the 1st of each month), and the simulated window contains the **9.9 mega-sale**, a day on which traffic and orders spike. Chapter 1 quantifies these effects and charts them.

- **Behaviour is a funnel.** Every action is logged as an *event*. A user sees an item on screen (an **impression**), some impressions become **clicks**, fewer become **add-to-carts** (the item goes into the shopping cart), and fewer still become **orders** (the purchase). Each step has fewer people than the one before, like a funnel narrowing. Some items get *hidden* (the user taps "not interested") and some purchases are *returned*.
- **Stock runs out and new items appear.** Sellers list products during the window and some sell out. A brand-new item that nobody has clicked yet is hard to recommend, because we have no behaviour to learn from; this is the **cold-start problem**, together with the availability problem, in Chapter 12.

One time split for the whole book

The most important convention in the book is a fixed point in time called the **cutoff**. The simulated marketplace covers 60 days. We pick **day 53 as the cutoff** and apply one rule everywhere:

> Every algorithm may learn from, and compute features from, events that happened **before** the cutoff. It is then judged on what users actually did in the **7 days after** it.

Think of a weather forecaster. You give them records up to 1 March and ask, "What will 2 to 8 March look like?" You already know the answer because it is in your records, but you hide it, then compare the forecast with what really happened. That comparison is the score. *Cheating* would be letting the forecaster read the 5 March newspaper first. In our marketplace, the **history** is the records, the cutoff is "1 March", and the **label window** (the days after the cutoff) is the hidden answer key. The same rule applies to our running shopper Sari: the recommender sees only what Sari did before day 53 and is then checked against what she did in days 53 to 60. If it were allowed to see her day-55 purchase while learning, its score would look wonderful and mean nothing.

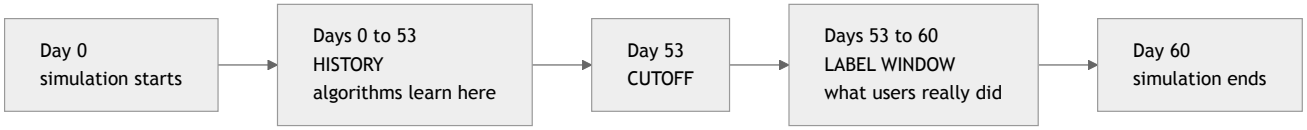


Figure P.3: *The time split used by every companion script. Nothing from the label window is ever visible during training or feature building.*

The shared file `book/code/common.py` enforces the split and provides one `evaluate` function, so numbers are comparable across chapters.

Think of `evaluate` as the single answer key that every algorithm is marked against. If each chapter used its own marking scheme, you could not tell whether a new idea was better or just graded more kindly.

How each chapter is organised

Chapters 1 to 14 follow the same template, so you always know where you are:

1. **An opening story** set in a Shopee-like scenario, and a short list of what you will learn.
2. **Intuition**, using an everyday analogy before any formula.

3. **The algorithm**: the formulas, numbered steps, and a worked example small enough to do with a pencil.
4. **Code walkthrough**: real code from the repository, explained piece by piece.
5. **Run it**: the command to execute and its captured output.
6. **Results**: charts and an interpretation tied to the actual numbers.
7. **Pitfalls and production practice**: what goes wrong, and how large systems handle it.
8. **Summary** and **Quiz**.

Visual and typographic conventions

Code is pulled directly from the repository files, and output blocks are copied from captured results files. Any number in a paragraph also appears in an output block or a JSON file under `book/results/`. Inline math looks like $a \cdot b$; Chapter 2 builds NDCG from nothing.

Three quality scores appear from Chapter 2 onward. You do not need them yet, but here is the one-line meaning of each so they feel familiar. **Precision** asks: of the items we showed, what share did the user really want? **Recall** asks: of all the items the user really wanted, what share did we manage to show? **NDCG** (Normalised Discounted Cumulative Gain) is one score that rewards putting the right item near the *top* of the list rather than at the bottom. Chapter 2 builds each of them with a tiny example.

Four kinds of callout boxes break up the text. They look like this in the PDF:

"**Did you know?**" **boxes** (blue) carry a verifiable fact from the research literature or the history of the field, with its source named at the bottom of the box and listed in the bibliography. Every chapter has at least two of them.

> **Tip**
>
> **A tip** is a practical shortcut or a habit that will save you time.

> **Warning**
>
> **A warning** marks a mistake that is easy to make and expensive to discover, such as evaluating on data the model has already seen.

> **Try it yourself**
>
> A "**Try it yourself**" **box** suggests a small experiment: change one parameter in a script, re-run it and compare. These are the best way to turn reading into understanding.

Finally, every chapter ends with a **quiz** of eight to ten questions: multiple choice, short answer, a code-reading question, a calculation and one hands-on exercise that modifies a companion script. Answers with short explanations are in Appendix A. Try to answer before you look; the effort of retrieving an answer is what makes it stick.

A note on scale

Where we describe how large platforms work we rely on published papers and invent no internal numbers. When a simple method beats a fancier one on this dataset, the book says so and explains why.

With the map in hand, turn the page. Chapter 1 begins where every recommender project begins: with the data, and with an honest look at what it can and cannot tell us.

Chapter 1 — What a Recommender Does

The Monday-morning problem

It is 08:55 on a Monday and Rina, a product manager at a marketplace not unlike Shopee, is staring at a dashboard. Her platform lists millions of products. Her buyers open the app, swipe for ninety seconds, and leave. Some of them leave happy, with a phone case in their cart. Most of them leave empty-handed, because the thing they would have loved was number 48,213 in a list nobody scrolled to.

Her engineering lead says the sentence that starts every recommender project: "We need to show each person the right things." Rina asks the question that matters: *what are the things, who is the person, and how will we know we got it right?* This chapter answers those questions before any modelling code, and introduces the simulated marketplace used in every experiment in this book.

What you'll learn

- How to frame "recommendation" as two separate jobs, *retrieval* and *ranking*, and why personalisation is the point of both.
- The difference between explicit and implicit feedback, and why e-commerce almost always lives on the implicit kind.
- What the *simulator's hidden truth* is, why the data you log is only a shadow of it, and why a real platform never sees the truth.
- The seven raw tables of the book's marketplace and the six event types that fill the `events` table.
- How to measure sparsity, the long tail, per-user activity and time effects (paydays, the 8.8 and 9.9 mega-sales) with real numbers from the companion script `ch01_explore.py`.

Intuition: the shop assistant who remembers you

Imagine two shop assistants in a physical market. The first stands at the entrance and shouts, "Today's best sellers are over here!" every time anyone walks in. The second watches who comes in. When you arrive she remembers that last week you bought cat food, that you looked at a ceramic mug for a long time, and that you ignored the sneakers table twice. She says: "The new cat-food brand arrived, and we have mugs on sale." Neither assistant is lying. But the second one is *personalising*: her output depends on who you are and what you have done.

A recommender system is the second assistant, automated and multiplied by a few million customers. It has three ingredients, and it helps to name them now because the whole book is a variation on them:

1. **Users**: people (or accounts) who act.
2. **Items**: the things that can be shown, clicked, bought.
3. **Interactions**: the record of who did what to which item, and when.

Here are all three for one shopper, **Sari**, in one tiny moment:

Ingredient	In Sari's moment
User	Sari (account #1)
Item	a 2 kg bag of cat food (item #17)
Interaction	"Sari clicked item #17 at 10:03"

A *recommender system* (or *recommender*) is a program that looks at the interactions it has recorded and, for each user, picks the few items to show.

The assistant's brain has to solve a problem that sounds easy and is brutal in practice: given a user and a very large set of items, produce a short ordered list that the user is likely to value. "Short" matters because screens are small. "Ordered" matters because people do not scroll. "Likely to value" matters because we have to turn a fuzzy human notion into something a computer can optimise.

She also does not compare you against *every product in the market*. She first walks along the shelves and grabs thirty plausible things in a quick, rough pass. Only then does she think carefully about which one to show first, second and third. This two-step habit is exactly what every large recommender does, and the next section gives the two steps their proper names.

The algorithm: framing the problem

Retrieval and ranking

A recommender needs a way to say "how good is item i for user u?" in a form a computer can sort. The simplest form is a **score**: one number per user-item pair, where a bigger number means "show this earlier". If Sari's scores are 0.9 for the cat-food bag, 0.5 for a mug and 0.2 for sneakers, her list is cat food, mug, sneakers. Every method in this book is a different way of producing that number.

Formally, let U be the set of users and I the set of items (so $|I|$ reads as "how many items there are"). We want a scoring function

$$s(u, i) \in \mathbb{R}$$

which reads: "give me a user and an item, and I return one real number" ($\mathbb{R}$ is just "any ordinary number"). We want it such that, for each user u, sorting items by $s(u, \cdot)$ puts the ones u would value first.

Why not simply score every item every time? If $|I|$ were 2,500, as in this book's marketplace, we could: score every item for every request. If $|I|$ is 100 million, as on a major marketplace, it is impossible. A good scoring model needs real time per item, and the app has a **latency budget**: the time the shopper is willing to wait, a few hundred milliseconds. A few hundred milliseconds divided by 100 million items leaves no time to think about any of them.

So production systems split the work into stages, like the assistant who grabs thirty things and then chooses:

- **Retrieval** (also called *candidate generation*) is the quick, cheap first pass. It reduces millions of items to a few hundred **candidates**: the items that will be considered further. It may be wrong about the order and may even include junk, but it must not *miss* items the user would have

loved, because nothing downstream can bring them back. We call this property **high recall**: of all the items the user would love, the candidate pile contains most of them. If Sari would love the cat-food bag and retrieval leaves it on the shelf, no later stage can rescue it.

- **Ranking** is the slow, careful second pass. It takes those few hundred candidates and scores each one with a richer model that uses many **features** (facts about the user, the item and their relationship: the item's price, Sari's city, how many cat products she clicked last week). It decides the final order. We want **high precision** here: the items at the top of the final list should really be good ones, not merely plausible ones.
- **Re-ranking** (chapter 11) applies business rules and diversity on top: no five identical phone cases in a row.



Figure 1.1: *the funnel shared by every large recommender. Chapters 3 to 7 build retrievers, chapters 9 and 10 build the ranker, chapter 11 the re-ranker.*

This funnel is the skeleton of the book. Chapters 3 to 7 are about producing *a* score, with progressively smarter methods, and we will evaluate each of them as if it alone had to do the whole job. From chapter 8 onwards we put them together the way real systems do. If a stage name is unfamiliar when you meet it later, come back to this figure.

Personalisation

A **non-personalised** recommender returns the same list to everyone ("most popular this week"), like the first assistant shouting at the door. A **personalised** one changes the list depending on the user, like the second assistant. A non-personalised method is also called a **baseline**: a simple reference that any cleverer method must beat to be worth building. Popularity baselines are more competitive than newcomers expect (chapter 3), so personalisation must win where it matters: where the popular choice is wrong for this person.

"Right" can mean different things. It can be the user's answer (items I would enjoy), the business's answer (orders, revenue) or the ecosystem's answer (a healthy spread of sellers, not only giants). In this book behaviour is the signal, which merges the first two.

Explicit versus implicit feedback

How does a system learn what a user likes? There are two families of signal. Think of a restaurant. Asking a guest to fill in a review card is one way. Watching what they actually order and leave on the plate is the other.

Explicit feedback is a deliberate statement: a 1-to-5 star rating, a thumbs-up, a "not interested" tap. It is the review card. It is clean, because the user meant it. It is also rare. Most people never rate anything, and those who do are not representative of those who do not.

Implicit feedback is the by-product of normal behaviour, with no extra effort from the user: an item was shown, clicked, kept in view for 40 seconds, added to a cart, bought. It is watching what the guest orders. It is abundant, because every visit generates it. It is also **noisy** (mixed with accidents and random habits) and ambiguous. A click may be curiosity or an accident. A missing click does not mean

dislike; the user may never have seen the item. A purchase is a strong yes, but it is also biased towards cheap, convenient and heavily promoted items.

Each logged action is called an **event**: one line in the app's log, such as "at 10:03, Sari clicked item #17". For Sari buying cat food, the events could be:

Time	Event type	What it means
10:03	`impression`	the app showed the bag to Sari
10:03	`click`	she opened its page
10:04	`add_to_cart`	she put it in her cart
10:09	`order`	she paid

Each kind of event is an **event type**. The first four are the positive ones; two more are negative and appear below.

On a marketplace, implicit feedback dominates, and the simulator logs exactly the kinds of events a real app would. A stronger action deserves more credit than a weaker one, so we assign each a **weight** (a strength score), defined in `schema.py`:

$$w(\text{click}) = 1, \qquad w(\text{add_to_cart}) = 3, \qquad w(\text{order}) = 5.$$

Here $w(\cdot)$ is read "the weight of this event type": a click is worth 1 point, an add to cart 3, an order 5. A user who clicked an item twice, added it to the cart once and later bought it has an **interaction strength** (the total points) of $2 \cdot 1 + 1 \cdot 3 + 1 \cdot 5 = 10$ for that item. Hand-sized, but this is precisely how the interaction matrix of chapters 5 and 6 is built from raw events. Without weights, a casual click and an actual purchase would count the same.

Two event types are *negative*: `hide` (the user tapped "not interested") and `return` (the user bought it and sent it back). They are rare, but one `hide` says more about dislike than a thousand silent impressions.

Hidden truth versus logged data

Here is a subtle point that separates a data scientist from someone who merely runs scikit-learn (a popular Python machine-learning library). Behind every real log there is a **truth** you never observe: what the user *really* prefers, how good each item *really* is, how price-sensitive each buyer *really* is. Statisticians call such unobserved quantities **latent** (hidden). The log is a lossy shadow of that truth, filtered through what you chose to show. Think of the way a shadow on a wall tells you the outline of an object but not its colour.

Because our marketplace is simulated, we get to do something real platforms cannot: we write the truth down ourselves. Each simulated user has three hidden properties:

- a **taste vector**: a short list of numbers saying how strongly the user leans towards each kind of product, for example (cat and pet 0.9, electronics 0.1, fashion 0.4, ...; these numbers are only illustrative);
- a **sparse category preference**: "sparse" means mostly zeros. A user really cares about only a few categories and is indifferent to the rest;

- a **price sensitivity**: how strongly a high price puts this buyer off.

Each item has a hidden **quality** (how good it is, regardless of who looks) and its own taste vector, so that user and item can be compared. Then the simulator generates **sessions** (a *session* is one visit to the app, from opening it to leaving), shows twelve items per session, and lets a **utility** decide whether each shown item is clicked, carted or bought. The utility is one number that says "how attractive is this item to this user right now", made by adding pluses and subtracting minuses.

To turn that number into a chance, the simulator uses the **logistic sigmoid**, written σ: a function that squeezes any number into the range 0 to 1. A very negative input gives a probability near 0, an input of 0 gives exactly 0.5, and a very positive input gives a probability near 1. With pos the slot the item was shown in (0 is the top), the click probability is

$$P(\text{click}) = \sigma\big(\text{utility}(u, i) - 4.2 - 0.12 \cdot \text{pos}\big).$$

In words: start from the utility, subtract 4.2 (most shown items are never clicked, so a click needs a good reason), subtract a little more for each slot further down the screen, and squash the result into a probability.

The utility is a sum of readable terms: taste match, category preference, an "intent" bonus when the item is in the category the user came for, item quality, a price-fit penalty scaled by the user's price sensitivity, the effective discount, free shipping, same-city shipping, a bonus for re-buying a consumable (something used up, like cat food), and a heavy penalty for a durable item the user already owns (you do not buy a second fridge).

Worked example. Suppose Sari's utility for the cat-food bag is 3.0. Then:

Slot (`pos`)	Argument of σ	$P(\text{click})$
0 (top)	$3.0 - 4.2 - 0 = -1.2$	$\sigma(-1.2) \approx 0.231$
5	$3.0 - 4.2 - 0.6 = -1.8$	$\sigma(-1.8) \approx 0.142$

Same item, same user, but moving it from the top slot to the sixth costs it about 39% of its click probability ($0.231 \to 0.142$). This is **position bias**: people click what is near the top more, regardless of quality. Because the simulator logs the `position` of every event, we will be able to study and correct it in chapter 12. A utility of 5.0 in the top slot gives $\sigma(0.8) \approx 0.690$; a utility of 1.0 gives $\sigma(-3.2) \approx 0.039$. The utility mostly decides *whether* you click, and the position nudges the probability up or down.

The crucial design rule is written in the docstring (the explanatory comment at the top) of `schema.py`: *the latent variables are never written to disk*. Only what an app would log leaves the simulator. Every algorithm in this book has to rediscover the truth from the shadow, exactly as it would in production. And because we know the truth, we also know the data is *learnable*: when a method fails, we can reasonably suspect the method rather than a pure-noise dataset.

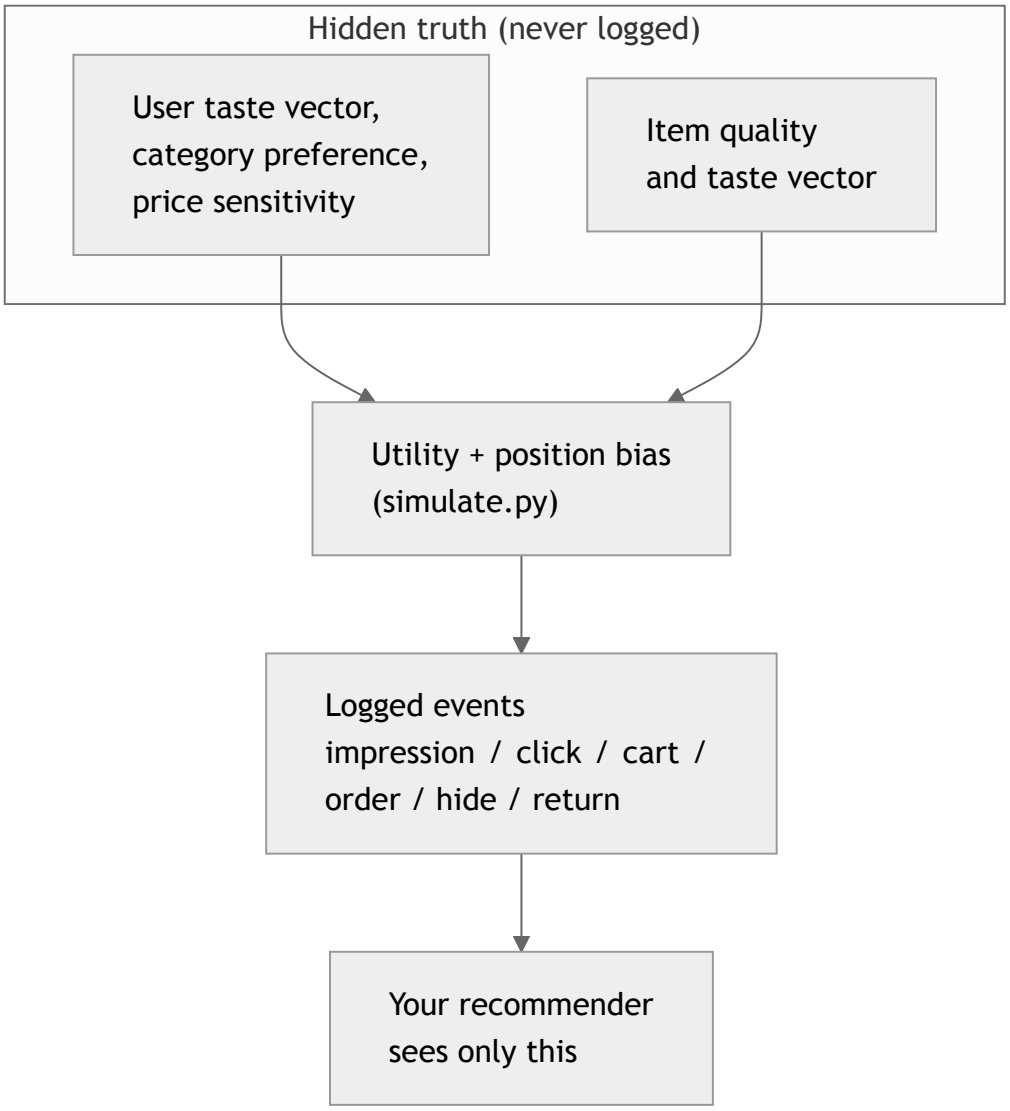


Figure 1.2: *the simulator keeps the truth private. The recommender sees only the logged events and the catalog tables.*

> **Did you know?**
>
> Collaborative filtering began with *explicit* ratings and a hint of the implicit. In the GroupLens system for Usenet news (1994), a rating was a number from 1 to 5, optionally accompanied by the number of seconds the reader spent on the article, a time signal the authors recorded but their rating servers did not yet use. Thirty years later, time on page (our `dwell_sec`) is one of the standard implicit signals.
>
> Source: Resnick et al., ACM CSCW, 1994

Time: the calendar is a feature of the world

Marketplace traffic is not **stationary**, meaning it is not the same every day. People shop more on weekends, more right after payday, and enormously on mega-sale days. Online marketplaces in Southeast Asia run "double date" campaigns (9.9, 10.10, 11.11 and so on), where the month number equals the day number. A recommender has to cope with the fact that on those days user behaviour changes: more discounts, more clicks, higher **conversion** (the share of visits or clicks that end in a purchase).

The simulator reproduces this with a small, readable function, `_day_weights`. A **weight** here is a "how busy is this day" score. Every day starts with weight 1. Then the weight is multiplied by 1.2 on

Saturdays and Sundays, by 1.4 on payday days (the 25th through the 1st of the next month) and by 3.0 on a "double date" sale day, which it detects as *month equals day*. Finally the weights are **normalised**, meaning divided by their total so they sum to one. They then become a **probability distribution over days**: the chance that a randomly chosen session falls on each day. For example, three days with raw weights 1, 1 and 3 become 0.2, 0.2 and 0.6, so the third day receives 60% of the sessions. In this 60-day window starting 1 August 2026, that means the 8.8 sale (8 August) and the 9.9 sale (9 September) both occur. The module docstring only mentions 9.9, but the rule `d.month == d.day` fires for any double date, which is why 8.8 is also boosted.

Worked example. A Saturday sale day (such as 8 August) has raw weight $1.2 \times 3.0 = 3.6$ against 1.0 for an ordinary Wednesday, before normalisation. It is expected to be 3.6 times as busy.

The raw tables

The simulator generates the seven raw tables defined in `recommender/recsys/schema.py`. A *table* is a grid of rows and columns, like a spreadsheet sheet, stored in a database. This file is deliberately "the single source of truth": the simulator writes it, the Postgres loader (the code that copies the data into a Postgres database) reads it, and the feature builder (the code that turns raw tables into numbers for models) checks against it. The tables fall into three groups:

Group	Tables	One row is...
Catalog	`categories`, `shops`, `items`	a leaf category (a lowest-level category such as "phone cases", with none below it), a seller, a product listing
People	`users`	a buyer account
Behaviour	`sessions`, `searches`, `events`	an app visit, a submitted search query, one user action on one item

The phrase "one row is..." has a name: the table's **grain**. It states what one row stands for. We come back to why it matters in the code walkthrough.

`events` is the main interaction log, and it is the table the rest of this book leans on. Its columns are `event_id`, `ts` (timestamp), `user_id`, `session_id`, `item_id`, `event_type`, `position`, `dwell_sec` and `price_paid`. Four of those deserve a comment.

- `event_type` takes one of six values: `impression`, `click`, `add_to_cart`, `order`, `hide`, `return`.
- `position` records the slot, so we can study position bias.
- `dwell_sec` is filled only on clicks: seconds spent on the product page ("dwell" means "stayed"). A long stay hints at real interest.
- `price_paid` is filled only on orders, after discounts.

The **impression** is the least glamorous and most important event. It says "this item was shown to this user in this slot". Without impressions you cannot compute a **click-through rate** (CTR: clicks divided by impressions), because you do not know the denominator (the bottom number of the fraction). Imagine a shopkeeper who counts how many people bought a cake but never counts how many saw it in the window: she cannot tell a popular cake from one that sat in the window all day. You also cannot distinguish "ignored" from "never seen".

Sparsity and the long tail

Two statistical facts shape every method in the next chapters.

Sparsity. *Sparse* means "mostly empty". Arrange users as rows and items as columns, like a spreadsheet. Each cell holds the interaction strength between a user and an item. Here is a tiny version with 3 users and 4 items:

	cat food	mug	sneakers	phone case
Sari	10	1	.	.
Budi	.	.	3	.
Rina	.	.	.	.

Only 3 of the 12 cells are filled (a dot means "never touched"). In real catalogs almost every cell is empty. We measure this as **density**, the fraction of non-empty cells; here it is $3/12 = 25\%$.

$$\text{density} = \frac{\text{non-zero cells}}{|U_{\text{active}}| \cdot |I_{\text{active}}|}, \qquad \text{sparsity} = 1 - \text{density}.$$

Now a realistic-sized illustration: a store with 1,000 users and 10,000 items where the users have between them produced 50,000 distinct user-item interactions has density $50{,}000/10{,}000{,}000 = 0.5\%$. Reading the formula: the top is the count of filled cells, the bottom is the number of all cells ($1{,}000 \times 10{,}000$). Sparsity is the leftover, 99.5%. The practical problem: most algorithms in the later chapters must infer a user's taste from a handful of filled cells in a mostly empty row, like guessing someone's favourite food after seeing three meals. That is why **collaborative filtering** (chapter 5: recommending what similar users liked) needs neighbours and **matrix factorisation** (chapter 6) compresses the matrix into low-dimensional vectors (short lists of numbers per user and per item).

The long tail. Interactions are not spread evenly across items. Imagine five items with 90, 5, 3, 1 and 1 interactions out of 100: one item takes 90% of the attention. A few items are extremely popular; most are rarely touched. Sort the items by interaction count and the resulting curve drops steeply and then runs along the floor for a long time: a "head" (the popular few) and a "tail" (the many rarely touched ones). A useful summary is the share of all interactions that the top $p\%$ of items receive. The more concentrated the share, the more a popularity baseline wins by default, and the harder it is to make a recommender that visibly beats it.

The tail is where personalisation pays: it holds the products matching unusual tastes, and sellers who never get a chance. Without personalisation, a tail item has almost no way to reach the one person who would love it.

> **Did you know?**
>
> The idea that the "tail" of a popularity distribution is economically meaningful was popularised by Chris Anderson's article "The Long Tail" in *Wired* in October 2004. He argued that online retail, free of shelf-space limits, can profit by serving niche products that in aggregate rival the hits. Whether the tail is really where the money is remains debated, but the vocabulary of head and tail is now standard in recommender work.
>
> Source: Anderson, Wired, 2004

Code walkthrough

We will read three pieces of real code: the schema helper that prints the data contract, the simulator's configuration, and the day-weight function that implements the calendar.

The schema as a data contract

`schema.py` models tables with two tiny frozen dataclasses. A **dataclass** is a Python class that mainly holds named fields, and *frozen* means its values cannot be changed after creation, so the contract cannot be altered by accident. A `Column` is a name, a type label and a human description. A `Table` is a name, a grain, and an ordered tuple of columns.

recommender/recsys/schema.py · Table

```python
@dataclass(frozen=True)
class Table:
    name: str
    grain: str  # what one row represents
    columns: tuple[Column, ...]

    @property
    def column_names(self) -> list[str]:
        return [c.name for c in self.columns]
```

The word **grain** matters: for `events`, one row is one user action on one item, and every later aggregate is a `GROUP BY` (SQL's "bucket the rows by this key, then count or add inside each bucket") over it. If you wrongly treat a table as one row per user-item pair when it is really one row per action, a user who clicked twice is silently counted twice. A wrong grain silently double-counts.

The `describe` function turns the contract into a printable report, which is what `uv run recsys schema` shows:

recommender/recsys/schema.py · describe

```python
def describe() -> str:
    """Human-readable dump of the schema (used by `recsys schema`)."""
    out: list[str] = ["RAW TABLES", "==========="]
    for t in RAW_TABLES:
        out.append(f"\n{t.name}  (one row = {t.grain})")
        for c in t.columns:
            out.append(f"  {c.name:<18} {c.dtype:<14} {c.description}")
    out += ["", "FEATURES (one row = one (user, item) candidate at cutoff time)", "=" * 62]
    group = None
    for f in FEATURES:
        if f.group != group:
            group = f.group
            out.append(f"\n[{group}]")
        out.append(f"  {f.name:<26} {f.description}")
    return "\n".join(out)
```

`describe` prints each raw table with its grain, then the features (the numbers prepared for models, from chapter 9 on) grouped by feature group, so documentation changes whenever the schema does.

Configuring the world

Everything about the simulated world is controlled by one frozen dataclass:

recommender/recsys/simulate.py · SimConfig

```python
@dataclass(frozen=True)
class SimConfig:
    n_users: int = 1500
    n_items: int = 2500
    n_shops: int = 150
    n_days: int = 60
    start_date: str = "2026-08-01"  # 60 days -> includes 9.9 sale and two paydays
    seed: int = 7
```

Six numbers define the experiment: 1,500 users, 2,500 items, 150 shops, 60 days from 1 August 2026, and a seed of 7. The **seed** is the starting number of the random number generator. A computer's "random" numbers are really a fixed sequence that starts wherever the seed says; with the same seed the simulator generates the same world, byte for byte, like replaying a recorded game. That is how every result in this book is *reproducible*: if you re-run a script you get the same numbers I quote.

The calendar

The function below is the whole time model:

recommender/recsys/simulate.py · _day_weights

```python
def _day_weights(start: pd.Timestamp, n_days: int) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
    """Traffic multiplier per day + flags for payday and mega-sale."""
    days = pd.date_range(start, periods=n_days, freq="D")
    payday = np.array([d.day >= 25 or d.day == 1 for d in days])
    sale = np.array([d.month == d.day for d in days])  # 9.9, 10.10, ... style sales
    w = np.ones(n_days)
    w[days.day_of_week >= 5] *= 1.2
    w[payday] *= 1.4
    w[sale] *= 3.0
    return w / w.sum(), payday, sale
```

`payday` is true for day-of-month at least 25 or exactly 1; `sale` is true when `d.month == d.day`. Weights are multiplied in by boolean masks (arrays of true/false flags, one per day, that switch a multiplier on or off), then normalised. The flags are returned because the session loop reuses them: on sale days the effective discount rises by 0.2 and order probability gets a bonus, so time changes behaviour, not only traffic.

We do not paste the long body of `simulate()`: it builds the catalog and users, then for each session picks a day, a **surface** (the screen where items are shown: `home_feed` or `search`) and an intent category (what the user came to look for), shows 12 items, computes the utility, and samples the events (draws random outcomes with the probabilities computed). Search for `utility =` to read it.

Run it

The companion script `book/code/ch01_explore.py` loads the simulated tables through the shared helper `common.load_tables()` (seed 7, cached: computed once and reused) and prints six summaries. Run it from the `book/` directory:

```
cd book
uv run python code/ch01_explore.py
```

Its captured output is below. Every number I quote in this chapter comes from here or from the matching `ch01_explore.json`.

TERMINAL OUTPUT

```
=== Table sizes ===
categories          25
shops              150
items            2,500
users            1,500
sessions        19,455
searches         8,680
events         276,378

=== Event funnel (all 60 days) ===
event_type         count    % of impressions    % of previous step
impression       233,460             100.00%
click             24,726              10.59%               10.59%
add_to_cart       10,619               4.55%               42.95%
order              5,858               2.51%               55.17%
hide               1,594               0.68%
return               121               0.05%

=== Events per day ===
median clicks/day on ordinary days : 347
mean clicks/day on payday days     : 488
sale day 2026-08-08 (month == day): 1,386 clicks, 405 orders
sale day 2026-09-09 (month == day): 1,262 clicks, 410 orders
weekend vs weekday mean clicks/day  : 496 vs 376

=== Popularity concentration (clicks + carts + orders per item) ===
top  1% of items (  25) get  12.5% of interactions
top 10% of items ( 250) get  50.3% of interactions
top 20% of items ( 500) get  68.6% of interactions
items with zero interactions: 317 of 2500

=== Sparsity of the user x item matrix (before cutoff) ===
users with interactions : 1,408
items with interactions : 2,120
non-zero cells          : 18,256
density                 : 0.612%  (sparsity 99.388%)

=== Clicks per user (before cutoff) ===
p10 :      1
p25 :      5
p50 :     11
p75 :     20
p90 :     31
p99 :     56
mean: 14.2   max: 106   users with 0 clicks: 92
```

Results: reading the shape of the data

Size and funnel

The world has 25 categories, 150 shops, 2,500 items, 1,500 users, 19,455 sessions, 8,680 searches and 276,378 events: small by industry standards, big enough for the effects we care about.

Next comes the **event funnel**: how many events survive each step from showing to buying. Of 276,378 events, 233,460 are impressions; 24,726 are clicks, which is 10.59% of impressions: the **click-through rate** (CTR), meaning that about one shown item in ten is clicked. Of those clicks, 42.95% are followed by an add-to-cart (10,619 events). Of those cart additions, 55.17% are followed by an order (5,858

events). Compounding the three rates (multiplying them: 0.1059 × 0.4295 × 0.5517) gives 2.51% of impressions that end in an order. Meanwhile, 1,594 `hide` events (0.68% of impressions) and 121 `return` events (0.05%) are the rare negatives.

The funnel is the first reason that "predict a click" is not the same job as "predict a purchase". If we only optimise for clicks we would favour cheap, attention-grabbing items; the ranker in chapter 10 therefore trains on **graded** labels (labels with levels: click 1, cart 2, order 3, a different scale from the interaction weights above), so that an order counts more than a click.

Notice, too, how few positive events there are for a system that shows over two hundred thousand impressions. Training a ranker is a **rare-event problem**: the thing we want to predict happens seldom, so there is little signal to learn from. We pay attention to it throughout: when we say "positives" in later chapters (examples of things the user did want), we mean these few tens of thousands of rows.

Time

The "Events per day" block shows the calendar at work. Two kinds of average appear below. The **median** is the middle value when you sort the days (half the days are below it, half above), and it ignores spikes. The **mean** is the ordinary average, which spikes pull upwards. On ordinary days, the median is 347 clicks. On payday days, the mean rises to 488 clicks. On weekends, the mean is 496 clicks per day against 376 on weekdays. The two double-date sales stand out dramatically: 1,386 clicks and 405 orders on 2026-08-08, and 1,262 clicks and 410 orders on 2026-09-09.

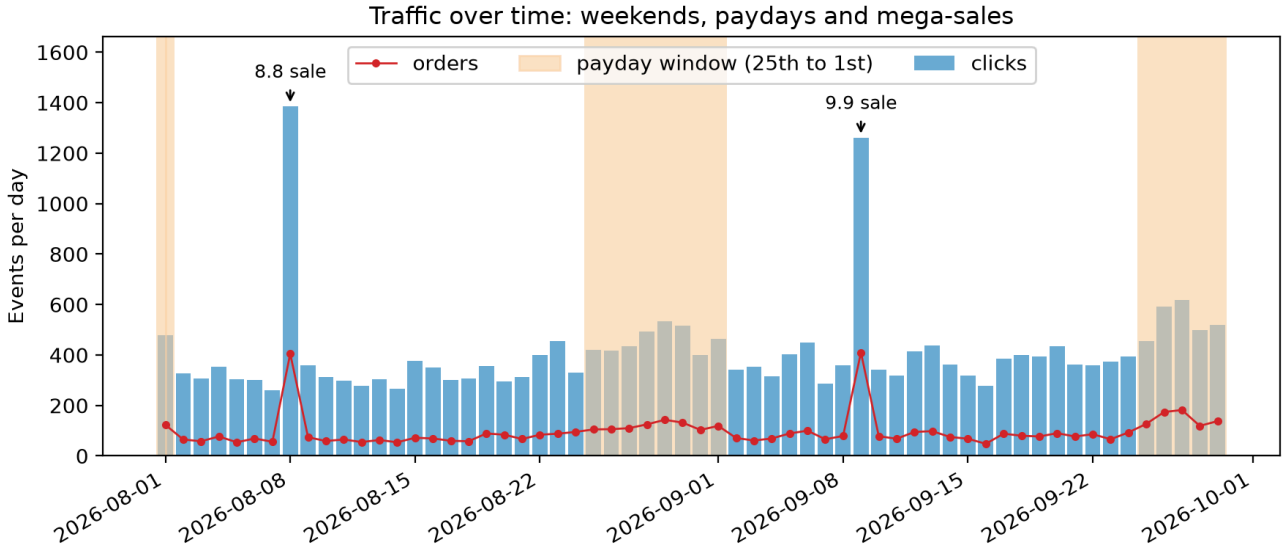


Figure 1.3: *Daily clicks (bars) and orders (line) over the 60 days; payday windows are shaded and the 8.8 and 9.9 sales are annotated.*

The payday windows lift the bars moderately; the sale days are spikes of almost four times the ordinary level for clicks (1,386 against 347).

Concentration: the long tail in numbers

The popularity block counts every click, add-to-cart and order per item. The top 1% of items (25 of them) receive 12.5% of all interactions. The top 10% (250 items) receive 50.3%, and the top 20% (500 items) receive 68.6%. At the other extreme, 317 of the 2,500 items have *zero* interactions, which

is over one in eight of the catalog. In other words, if you ranked the shelves of this shop by how often customers touched them, the first 250 shelves would account for half of all the touching.

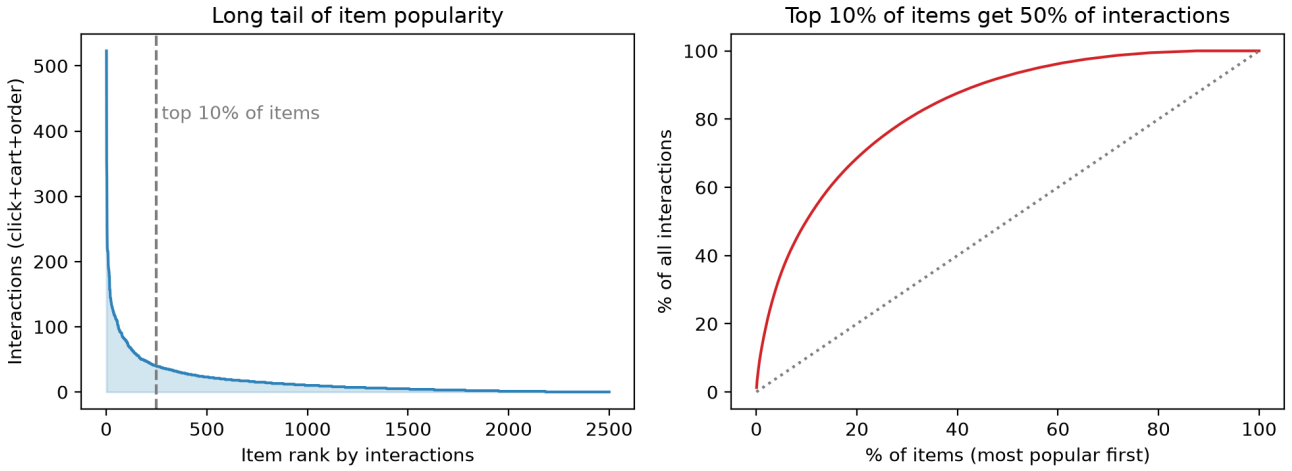


Figure 1.4: *Left: item interactions sorted from most to least popular, showing a steep head and a long flat tail. Right: the cumulative share of interactions captured by the top fraction of items; the dotted diagonal is a perfectly even catalog.*

The left panel is the classic head-and-tail curve. The right panel, a cumulative share, tells the same story: the further the red curve bows away from the dotted diagonal, the more concentrated the interactions. Half the interaction volume comes from only a tenth of the catalog. This is realistic: marketplaces typically have a small number of items that everyone looks at, and a very large number of items that almost nobody does. It is also *kind to popularity baselines*, which we will exploit and then try to beat.

The 317 zero-interaction items are the **cold-start** items of chapter 12. *Cold start* means "the system has no history for this item (or user) yet", so it has nothing to learn from. Only content features (facts like category and price), boosts (extra points added by hand) and exploration (showing something deliberately to gather data) can surface them.

Sparsity

Before the **time cutoff** (the moment we freeze the world: recommendations may use only data from before it; chapter 2 explains why), 1,408 users and 2,120 items have at least one interaction. They are connected by 18,256 non-zero cells in the user-by-item matrix, so the density is 0.612% and the sparsity is 99.388%. In other words, a typical active user has touched only a very small fraction of the items they could have, and the rest of their row is unknown. The matrix is small enough to hold in memory, but the same shape at real scale (hundreds of millions of users, hundreds of millions of items) is the reason that every method in the next chapters has to be designed for a mostly empty matrix.

Users are not equal either

The final block shows how many clicks each user made before the cutoff. A **percentile** answers "what value do $p\%$ of users stay at or below?": the 90th percentile is the click count that 90% of users do not exceed. The **quartiles** are the 25th, 50th and 75th percentiles; the median is the 50th. The median user clicked 11 items; the quartiles are 5 and 20 (25% of users clicked 5 or fewer, 75% clicked 20 or fewer); the 90th percentile is 31 and the 99th is 56. The mean is 14.2 clicks, the maximum is 106, and

92 users have no clicks at all. Those 92 users are the **cold-start users**: newcomers the system knows nothing about. For them, the best we can do is show popular or trending items, or use context such as city or device tier. We will see how the live app does this for anonymous visitors on its home page in chapter 13.

> **Did you know?**
>
> Recommendations are not a side feature at large streaming services. Netflix's engineers reported that their recommender system influences choice for about 80% of the hours streamed on the service, with the remaining 20% coming from search. The number was published in an article on the algorithms and business value of the Netflix recommender system.
>
> Source: Gomez-Uribe et al., ACM Transactions on Management Information Systems, 2015

Pitfalls and how it is done in production

Mistaking the log for the truth. The log is a sample of what the system *chose to show*. An item never shown has no clicks, which says nothing about whether people would click it. This is **exposure bias**: the data over-represents whatever the system already chose to show. Production teams log impressions with position and surface so models can account for it.

Treating all events as equal. One impression is not one order. The weights 1 / 3 / 5 used in this book are arbitrary but sensible; in production the weights, or the choice of a *primary* objective such as orders or revenue, are a product decision that should be revisited when the business changes.

How production differs. Large marketplaces log the same event types through a stream (a continuous pipe of events; Kafka is a popular tool for that), with validation, de-duplication (removing repeated copies of the same event), a warehouse (a big database for analysis and training) and a feature store (a fast lookup of prepared features, used when serving live requests). Impressions are costly at scale, so many platforms log them client-side (from the user's app rather than the server) in batches, as our Vue frontend does (chapter 13).

> **Tip**
>
> Before building any model, spend an hour with a notebook and the raw tables. Print table sizes, the event funnel, the distribution of events per user and per item, and a time series of events per day. These five checks catch the majority of data bugs, and they set honest expectations for what a model can achieve.

> **Warning**
>
> Never compute click-through rate as clicks divided by *users* or *sessions*. CTR needs the number of **impressions** as the denominator. If your system does not log impressions, you cannot tell a popular item from an item that was merely shown very often.

> **Try it yourself**
>
> Open `book/code/ch01_explore.py` and find the loop `for pct in (1, 10, 20)`. Change the tuple to `(1, 5, 10, 50)`, re-run the script and compare the shares. Then look at `common.history()`, which builds the point-in-time view of the data that the rest of the book will use.

Summary

- A recommender turns a very large set of items into a short, ordered, personalised list; to be tractable it splits the work into cheap, high-recall *retrieval* and expensive, high-precision *ranking*, with *re-ranking* on top.
- Explicit feedback (ratings) is clean but rare; implicit feedback (impressions, clicks, carts, orders) is abundant but noisy, and absence of a click is not dislike.
- The simulator holds a hidden truth (tastes, quality, price sensitivity) and logs only what an app would see. Your methods must rediscover the truth from the logs.
- The marketplace has seven raw tables; `events` is the main log, and its grain is one action on one item. Impressions are essential as CTR denominators.
- The funnel runs 233,460 impressions to 24,726 clicks (10.59%), 10,619 carts and 5,858 orders; the matrix is 99.388% sparse.
- Popularity is heavily concentrated: the top 10% of items collect 50.3% of interactions, and 317 items have none.
- Time matters: weekends, paydays and the 8.8 and 9.9 sales change traffic volume and conversion, which is why we split by time and window features.
- 92 users have no clicks before the cutoff; every real system needs a cold-start fallback.

Quiz

Q1.1 Which statement best describes the *retrieval* stage of a two-stage recommender?

A. It orders 20 items with a large model using hundreds of features.
B. It cheaply narrows millions of items to a few hundred candidates, favouring recall.
C. It enforces diversity and business rules on the final list.
D. It computes the A/B test result.

Q1.2 Which of the following is an example of *implicit* feedback?

A. A user rates a product 4 out of 5 stars.
B. A user writes a text review.
C. A user spends 40 seconds on a product page and adds it to the cart.
D. A user taps a thumbs-up on a recommendation.

Q1.3 In this book's simulator, why is it important that the latent tastes and item qualities are *not* written to the output tables?

A. It saves disk space.
B. It forces recommenders to learn from the same kind of data a real app logs, while the truth stays available for reasoning about what is learnable.
C. Latent variables cannot be stored in pandas DataFrames.
D. Postgres rejects floating-point columns.

Q1.4 In one or two sentences, explain why an `impression` event is needed to compute a click-through rate, and what goes wrong if the system logs only clicks.

Q1.5 Explain what *sparsity* means for a user-by-item interaction matrix and name one consequence for the design of recommendation algorithms.

Q1.6 Read the `_day_weights` function above. What is the raw (pre-normalisation) weight of a Saturday on which a double-date sale falls, such as 8 August 2026, and of a Sunday that is a payday but not a sale? Why does the function return the `payday` and `sale` flags in addition to the probabilities?

Q1.7 Using the funnel rates from the chapter (click-through rate 10.59%, click-to-cart 42.95%, cart-to-order 55.17%), how many orders would you expect from 10,000 impressions? Round to the nearest whole number.

Q1.8 A marketplace has 1,000 active users and 10,000 active items, and 50,000 distinct user-item pairs have at least one interaction. What are the density and sparsity of the matrix, as percentages?

Q1.9 *Hands-on.* In `book/code/ch01_explore.py`, change the loop `for pct in (1, 10, 20)` to `for pct in (1, 5, 10, 50)` and re-run the script. What share of all interactions does the top 50% of items receive, and what does that tell you about the bottom half of the catalog? Also state what you would expect the share of the top 5% to be relative to the top 1% and the top 10%.

Q1.10 A colleague proposes treating every impression that was not clicked as a "dislike" label for training. Give two reasons this is a risky assumption on a marketplace app.

Chapter 2 — Measuring success

It is a Monday morning at a marketplace company, and two engineers are arguing in front of a dashboard. Dina has just finished a new ranking model for the home feed. On her laptop it "looks great": the top items for her own test account are all things she would click. Raka, who owns the checkout funnel, is not convinced. Last quarter a different "obviously better" model went live, conversions dipped for a week, and nobody could say whether the model was at fault, the weekend sale was, or the dip was just noise.

Both of them are right to worry. A recommender has no single correct output, and no unit test says "these ten items are right for user 43". What we have is **behaviour**: what people did after we showed them something. Measuring success means turning that behaviour into numbers that are honest enough to decide with, cheap enough to compute a hundred times a day, and well-behaved enough that a higher number really means a better product. Every later chapter ends with a results table produced by the same code, so it is worth understanding exactly what that code does, and where it can mislead you.

What you'll learn

- The difference between **offline** and **online** evaluation, and why you need both.
- Why a **time-based split** is the only honest way to evaluate a recommender, and how a random split quietly leaks the future.
- How to compute **precision@K**, **recall@K** and **average precision** by hand, and what each one rewards.
- How **DCG** and **NDCG** are derived step by step, and why the repo uses gains of 1, 3 and 7 for click, add-to-cart and order.
- What **coverage** and **candidate recall** measure, and why a recommender can look "accurate" while being useless.
- How `recommender/recsys/evaluate.py` implements all of this in about forty lines of pandas, line by line.

Intuition: testing a forecaster on last month's weather

To test a recommender, we **pretend it is the past, ask it to predict, then check against what actually happened next.**

Think of a weather forecaster. You give them weather records **up to 1 March** and ask, "What will 2–8 March look like?" You already know what happened, because it is in your records, but you hide it from them. You compare their forecast with reality; that comparison is the score. *Cheating* would be letting them read the 5 March newspaper first.

Now the same test for a shop recommender. Our records cover 60 days, and shopper Sari did this:

Day	What shopper Sari did
1–52	Browses cat toys, clicks several cat-food brands, adds one to cart, doesn't buy
53 = cutoff	*we freeze time here*
55	**Orders a 2 kg bag of cat food**

The recommender sees only days 1–52 and produces Sari's top 20 items. We then open the answer key (what happened after day 53): did the cat-food bag appear in her list, and how high? If it is in the list, we score points, more if it is near the top. If the list is twenty phone cases she never touched, we score nothing.

The **cutoff** (day 53 here) is the moment where we freeze the world. It has two jobs:

1. **It stops cheating.** Everything the recommender is allowed to use (clicks, carts, orders, item popularity, any feature computed from them) must come from *before* the cutoff.
2. **It defines the answer key.** The **labels** are what users did in the 7 days *after* the cutoff, from day 53 up to the end of the data at day 60. These are the "right answers" we grade against.

Leakage is cheating by accident: information from after the cutoff slips into what the recommender sees. Here is a concrete case. Suppose we build a feature called "times this user ordered this item", and we compute it over all 60 days instead of days 1–52. For Sari and the cat-food bag it would contain the very order on day 55 that we are trying to predict. The recommender then looks brilliant, because it is simply reading the answer. But in production, on day 53, the day-55 order has not happened yet, so the feature is empty and the model collapses. Leakage is the number-one cause of results that look wonderful offline and fail in production. In chapter 9 we build such a leaky column on purpose: it lifts the ranking score NDCG@20 (explained below) from an honest 0.2116 to 0.5035, a gain that is entirely fake.

The second lesson: **one metric is never enough.** A recommender can score well on accuracy and still show every user the same twenty bestsellers. So we need metrics for accuracy, for ranking quality, for breadth, and for the ceiling imposed by earlier pipeline stages. Different answers also deserve different weights: an order is worth more than a click, and slot 1 more than slot 18.

Offline versus online evaluation

There are two places to measure a recommender, and they answer different questions.

In weather terms: **offline** evaluation is the test above, run on recorded history. **Online** evaluation is letting the forecaster publish real forecasts today and seeing how the public reacts.

Online evaluation means putting the model in front of real users and watching what happens. The standard tool is the **A/B test**: randomly split users into a control group (the old model) and a treatment group (the new one), then compare business metrics, such as click-through rate $\mathrm{CTR} = \frac{\text{clicks}}{\text{impressions}}$, conversion rate, orders per session, or gross merchandise value (GMV) per user. Online results are the ground truth. They capture everything: novelty effects, page layout, latency, the fact that users react to what they are shown. But they are slow (days or weeks to reach statistical significance),

expensive (a bad model hurts real revenue while you learn it is bad), and you can only run a handful of experiments at a time.

Three terms in that paragraph in plain words. An **A/B test** is a fair race between two versions: a coin flip decides, for each user, whether they see version A (the current model) or version B (the new model), at the same time. Because both groups live through the same week, a weekend sale or a holiday affects both equally, so a difference between them is caused by the model. **CTR** (click-through rate) is the share of shown items that got clicked: 3 clicks on 100 impressions (times an item was displayed) is a CTR of 3%. **GMV** (gross merchandise value) is the total money value of everything sold.

Offline evaluation replays history. You take logged behaviour, pretend it is the day before something happened, produce recommendations, and score them against what happened next. It is fast (minutes), free, repeatable, and you can test a hundred variants in an afternoon. Its weakness is that history only contains what users did in response to the *old* system's choices. Items that the old system never showed have no clicks; a model that would surface them gets no credit. Offline numbers are a **proxy**.

In practice offline evaluation is a cheap filter in front of a few expensive online experiments: ideas that lose offline are discarded, and only the survivors get an A/B test. Offline metrics are for **rejecting bad ideas quickly** and for choosing between close variants. A good offline result is necessary, rarely sufficient.

> **Did you know?**
>
> When Netflix ran its famous prize competition for movie-rating prediction, the target was a proxy: lowering the root mean squared error (RMSE) of predicted star ratings by 10% was meant to stand in for "better recommendations". The winning team's blend of hundreds of models did cross the line, yet Netflix reported that the additional accuracy gains it measured offline did not seem to justify the engineering effort needed to put that ensemble into production. A great offline score is not the same thing as a great product.
>
> Source: Amatriain & Basilico, Netflix Technology Blog, 2012

Time-based splits, and why random splits leak

A **time-based split** means we pick a date, use everything before it to build the recommender, and grade it only on what happens after it, as in the Sari table above. The usual alternative, a **random split**, shuffles all events together and sets some aside for testing. That is fine for photos of cats and dogs, but not here, as the next paragraph shows.

Machine-learning tutorials usually say "shuffle your data and hold out 20% for testing". For recommenders this advice is dangerous. Suppose we have 60 days of events and we shuffle them into 80% train and 20% test. A click on day 52 might land in the training set while a click by the same user on day 14 lands in the test set. The model then "predicts" day 14 using knowledge of day 52. Concretely it can learn:

- **Popularity from the future.** An item that exploded on the 9.9 sale is a bestseller *because of* clicks that happen after the test event.

- **User taste from the future.** If a user switched from phones to cat food in week 7, a model trained on week 7 "predicts" her week-3 clicks by memorising where she ended up.
- **Session leakage.** Clicks inside one session are strongly correlated (people browse similar items in a row). Splitting a session across train and test makes the test set a near-copy of the training set.

In production you can never see the future, so a model that depends on it fails the first day it runs. The fix is to respect the arrow of time.

The split used in this book

The repo uses a **rolling-origin time split**. Pick a cutoff t_c. Everything strictly before t_c is *history*; the next $H = 7$ days are the *label window*; and the model is scored on how well it predicted that window.



Figure 2.1: *Features and candidates come only from history; labels come only from the label window after the cutoff.*

For the evaluation numbers in this book, the companion scripts fix the dataset end to the last simulated day, set the cutoff 7 days before it, and use the final week as the label window. For *training* a ranker (chapter 10) we need a second, earlier split, so that the training labels sit entirely before the evaluation cutoff:

Purpose	History ends at	Label window
Train the ranker	end − 14 days	end − 14 days to end − 7 days
Evaluate	end − 7 days	end − 7 days to end
Serve (production)	now	the future, unobserved

This is exactly the diagram in the docstring of `recommender/recsys/pipeline.py`. The model never trains on the week it is later judged on.

A tiny worked example of a leak

Suppose Rina clicks toy C on day 13 and orders it on day 14. A **random** split may put day 14 in training and day 13 in test, so "will Rina click toy C?" is asked of a model that has already seen her order of C. A **time** split at day 12 forces the model to guess from her earlier clicks only, as a live system must.

The same rule applies to features

Splitting events correctly is only half of it: a feature such as "this item's orders in the last 7 days" must be computed from events before the cutoff, not "up to today". Chapter 9 calls this **point-in-time correctness**. The rule: *at prediction time t, you may only use data with timestamp $< t$.*

What we are measuring: labels and ranked lists

Two plain-words definitions first. A **ranked list** is just the items we show a user, best first (the top of a product feed). **Graded relevance** means that not every success counts the same: a user who *orders* an item tells us more than one who merely *clicks* it, so we give each outcome a number (a *grade*) instead of a plain yes or no. The table below is that grading.

Before the formulas, let us fix vocabulary. For a given user u and cutoff t_c, the recommender produces a **ranked list** of items $r_1, r_2, \ldots, r_K$, best first. We compare it with what the user actually did during the label window. The repo derives a graded **label** for every (user, item) pair the user touched:

Behaviour in label window	Label
Clicked the item	1
Added the item to the cart	2
Ordered the item	3
Nothing	(not in the label table)

If the user did several things with the same item we keep the strongest (the maximum); this is done by `data.labels` in `recommender/recsys/data.py`. It returns *only* pairs with a positive label, so users who did nothing in the label window never appear: they cannot be scored and are excluded from every average below. The numbers are averages over *users who came back and did something*.

Let us call the items with a positive label the user's **future positives**, and let R be their number. Everything that follows is about how well the ranked list captures them.

The algorithm: ranking metrics from the ground up

Precision@K and recall@K

The suffix **@K** (read "at K") just means "look only at the first K items of the list", for example the top 20 of the page. A **hit** is an item in that list that the user did later engage with.

The two most basic questions about a top-K list are:

- **Precision@K**: of the K items we showed, what fraction did the user engage with?
- **Recall@K**: of the things the user engaged with, what fraction did we show?

$$\text{precision@}K = \frac{|\text{hits in top } K|}{K}, \qquad \text{recall@}K = \frac{|\text{hits in top } K|}{R}$$

A "hit" is a list item that is a future positive. For one user these are just a count divided by a different number. Precision cares about *waste* (every slot is expensive screen space), recall cares about *coverage of what the user wanted*. In a marketplace with thousands of items per category, and users who engage with a handful in a week, precision values are naturally tiny; recall is easier to read and is what this book reports most often. When K is fixed and large relative to R, precision and recall carry the same information up to scaling, which is why we mostly show recall.

Tiny example with Sari. Suppose we show 5 items. Later, Sari engages with 4 items (the bag of cat food, a toy, a litter box, a brush), and 2 of those were in our 5. Then precision@5 is $2/5 = 0.4$ ("two in five slots were useful") and recall@5 is $2/4 = 0.5$ ("we showed half of what she wanted"). If we showed 50 items and caught all 4, recall would be 1 but precision only $4/50 = 0.08$: that is why we look at both.

Both ignore **order** inside the top K: finding a hit at slot 1 or slot 20 gives the same score. On a real page that is wrong, since slot 1 is seen by nearly everyone and slot 20 requires scrolling. We need position-aware metrics.

Hit rate and MRR: two quick position checks

Papers and dashboards often use two simpler cousins of the metrics above. This book does not report them, but you will meet them, so here they are.

Hit rate@K asks a yes/no question per user: was *at least one* of the top K items a hit? The hit rate is the share of users for whom the answer is yes. If 4 of 10 users found something relevant in their top 20, hit rate@20 is 0.4. It is easy to read, but it gives the same credit for one hit as for ten.

MRR (mean reciprocal rank) asks how far down the list the *first* hit is. For each user take $1/\text{rank}$ of the first hit: first hit at rank 1 scores 1, at rank 2 scores 0.5, at rank 4 scores 0.25. The MRR is the average over users. For three users with first hits at ranks 1, 2 and 4: $(1 + 0.5 + 0.25)/3 = 0.583$. It rewards putting *something* good at the very top, and ignores everything after the first hit.

Average precision (AP)

Average precision rewards putting hits early. Walk down the list; every time you land on a hit at position k, record the precision at that cut-off $P@k$. Then average those values over *all* the user's future positives:

$$\text{AP@}K = \frac{1}{R} \sum_{k=1}^{K} P@k \cdot 1[\text{item at } k \text{ is a hit}]$$

Hand-sized example. A list of three items has hits at positions 2 and 3, and the user has $R = 2$ future positives. Then $P@2 = 1/2$ and $P@3 = 2/3$, so

$$\text{AP} = \frac{1}{2}\left(\frac{1}{2} + \frac{2}{3}\right) = 0.583.$$

With hits at positions 1 and 2 instead we would get $\frac{1}{2}(1 + 1) = 1$. Dividing by R (not by the number of hits) penalises positives that never appear. AP is binary, though: an order and a click are the same "hit", so for graded relevance we use DCG.

Cumulative gain, and the discount

Plain version of what comes next: we give each hit some points (more for an order than for a click), and shrink the points for items lower in the list, because fewer people scroll that far. Adding it all up gives **DCG**. Dividing by the best score that was possible gives **NDCG**, a number between 0 and 1. The steps below build it piece by piece.

Let rel_i be the label (0, 1, 2 or 3) of the item at rank i, counting ranks from 1. The most naive graded score just adds them up:

$$\mathrm{CG@}K = \sum_{i=1}^{K} \mathrm{rel}_i$$

Two problems. First, a label of 3 is not merely "three times" a click: an order is a far stronger and more valuable signal. We fix this by converting the label to a **gain** that grows exponentially:

$$g(\mathrm{rel}) = 2^{\mathrm{rel}} - 1 \quad \Rightarrow \quad g(0) = 0,\ g(1) = 1,\ g(2) = 3,\ g(3) = 7.$$

These are exactly the numbers in the `GAIN` dictionary of `evaluate.py`, and the same ones the LightGBM ranker uses as its `label_gain` in chapter 10, so training and evaluation agree on what "good" means.

Second, CG does not care about order. We fix that with a **position discount**: an item at rank i is worth less the further down it sits. The classic choice is

$$d(i) = \frac{1}{\log_2(i+1)}$$

so $d(1) = 1$, $d(2) = 0.63$, $d(3) = 0.5$, and the discount keeps shrinking, slowly. Putting both fixes together gives **discounted cumulative gain**:

$$\mathrm{DCG@}K = \sum_{i=1}^{K} \frac{2^{\mathrm{rel}_i} - 1}{\log_2(i+1)}$$

Why a logarithm? The drop between slots 1 and 2 is big, between slots 19 and 20 negligible, because most users never reach them; a $1/i$ discount would be far harsher. The logarithm is a convention, not a law of nature, but it is the one the field converged on. The figure below shows both curves, and the gains this repo uses.

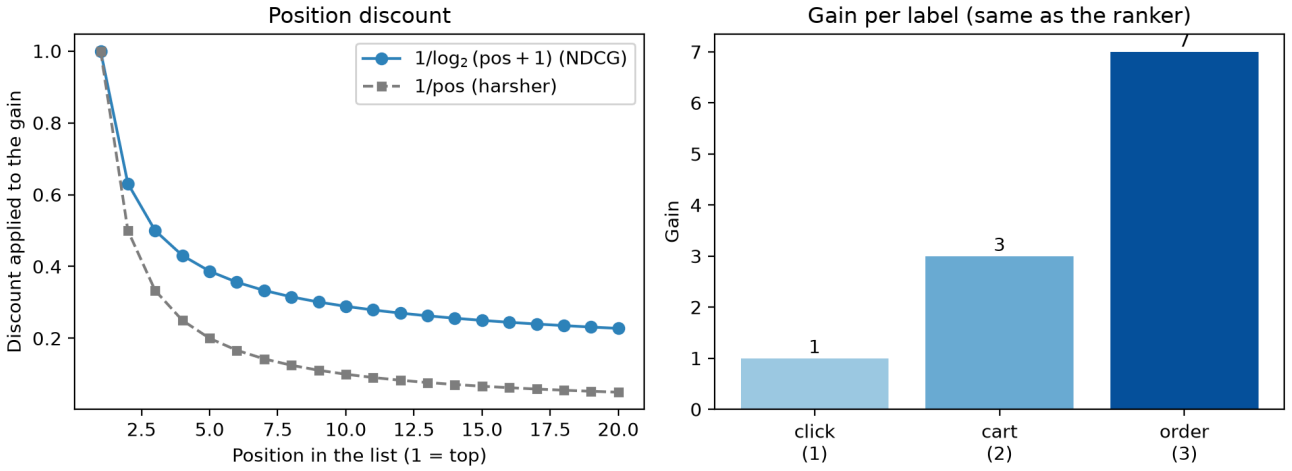


Figure 2.2: *Left: the position discount $1/\log_2(\mathrm{pos}+1)$ against the harsher $1/\mathrm{pos}$. Right: the gain assigned to a click, an add-to-cart and an order (1, 3 and 7).*

Normalising: NDCG

In one sentence: **NDCG is your DCG as a fraction of the best DCG you could have had**, like scoring 17 out of 20 on a test. The scale makes users with many future positives comparable to users with one.

DCG is not comparable across users. A user with one future click has a maximum possible DCG of 1; a user with five future orders has a maximum far higher. To average over users we need a common scale. The fix is to divide by the **ideal DCG** (IDCG): the DCG of the best possible ordering of that user's future positives, namely sorted by gain, highest first.

$$\mathrm{NDCG@}K = \frac{\mathrm{DCG@}K}{\mathrm{IDCG@}K} \in [0, 1]$$

An NDCG of 1 means "you listed every future positive, best first". A score of 0 means you found none. The final reported number is the mean NDCG over all evaluated users.

A 3-item walk-through. Our list has three slots. Slot 1 is a phone case Sari never touched (label 0). Slot 2 is the cat-food bag she ordered (label 3, gain 7). Slot 3 is a toy she added to the cart (label 2, gain 3). She also clicked a brush that we never listed (label 1, gain 1). In words, each slot contributes (gain) divided by (discount of that slot), with discounts 1, 0.631 and 0.5 for slots 1, 2 and 3.

$$\mathrm{DCG@3} = \frac{0}{1} + \frac{7}{\log_2 3} + \frac{3}{\log_2 4} = 0 + 4.417 + 1.5 = 5.917.$$

The ideal ordering of her positives puts the order first, then the cart, then the click: $\mathrm{IDCG@3} = 7 + 3/\log_2 3 + 1/\log_2 4 = 7 + 1.893 + 0.5 = 9.393$. So $\mathrm{NDCG@3} = 5.917/9.393 = 0.630$. Had we put the order in slot 1 and the cart in slot 2, we would have reached $7 + 1.893 + 0 = 8.893$, an NDCG of 0.947: the same hits, a better order, a higher score.

Note the subtlety in the ideal list: it contains *every* future positive the user has, including items that no recommender could have found (for instance an item that was out of stock at the cutoff, or one the user discovered via a friend's link). That makes NDCG@K honest: it is measured against the user's real behaviour, not against what was recommendable.

> **Did you know?**
>
> The DCG family of measures was introduced by Järvelin and Kekäläinen in a 2002 paper on evaluating information-retrieval systems with graded relevance. They proposed cumulating the gain a user receives down a ranked list and discounting items by their rank, precisely so that retrieving a highly relevant document early counts for more than retrieving it late. Two decades later, nearly every search and recommender leaderboard still reports NDCG.
>
> Source: Järvelin & Kekäläinen, ACM Transactions on Information Systems, 2002

Coverage

Accuracy metrics are averages over users, so a recommender can score well by showing nearly the same popular items to everyone. **Catalog coverage** measures breadth:

$$\mathrm{coverage@}K = \frac{\left|\bigcup_u \text{top-}K(u)\right|}{|\text{recommendable items}|}$$

Of all items we are allowed to recommend, what fraction ever appears in some user's top K? For a marketplace this is not just aesthetics: sellers whose items are never shown never sell. Coverage is *not* a quality score on its own (a random recommender has perfect coverage and terrible accuracy), so use it as a **guardrail** next to the accuracy metrics.

Plain version: coverage asks "how much of the shop window do we ever use?" If a shop has 2,000 items and across all users we only ever recommend 200 distinct ones, coverage is $200/2000 = 10\%$.

Diversity is a related but different idea: variety *inside one user's list* (for example, how many different product categories appear in her top 20), whereas coverage looks at variety *across all users*. A list of twenty phone cases is not diverse even if the shop's coverage is high. Chapter 11 measures diversity as the number of distinct categories in the top 20.

Candidate recall: the ceiling

Production recommenders, including ours, are **multi-stage**: a cheap first stage picks a few hundred *candidates* out of thousands (or millions) of items, and a smarter second stage orders them. The second stage can only reorder what the first stage provides, so

$$\text{candidate recall} = \frac{|\text{future positives that appear in the candidate set}|}{R}$$

(averaged over users) is an **upper bound** on the recall of the final ranking: if candidate generation never retrieves the item the user buys, no ranker can put it at slot 1. When a new ranker disappoints, check this number first. Chapter 8 is about raising it.

A tiny example: suppose Sari's future positives are the cat-food bag and a toy, and the first stage (the shortlist of 300 candidates) contains the toy but not the cat food. Candidate recall is $1/2 = 0.5$, and even a perfect ranker can never reach a recall above 0.5 for her. This maximum, set by the first stage, is called the **recall ceiling**.

Code walkthrough: `recsys/evaluate.py`

The whole evaluation module is short enough to read in one sitting. Start with `metrics`:

recommender/recsys/evaluate.py · metrics

```python
def metrics(recs: pd.DataFrame, labels: pd.DataFrame, k: int) -> dict[str, float]:
    """`recs`: user_id, item_id in display order (top K per user). `labels`: user_id, item_id,
label."""
    recs = recs.groupby("user_id").head(k)[["user_id", "item_id"]].copy()
    recs["pos"] = recs.groupby("user_id").cumcount()
    hits = labels.merge(recs, on=["user_id", "item_id"], how="left")

    per_user = hits.groupby("user_id").agg(n_pos=("label", "size"), n_hit=("pos", "count"))
    recall = (per_user["n_hit"] / per_user["n_pos"]).mean()

    orders = hits[hits["label"] == 3]
    ou = orders.groupby("user_id").agg(n=("label", "size"), h=("pos", "count"))
    order_recall = (ou["h"] / ou["n"]).mean()

    found = hits.dropna(subset=["pos"])
    dcg = (found["label"].map(GAIN) / np.log2(found["pos"] + 2)).groupby(found["user_id"]).sum()
    ideal_gains = labels.assign(g=labels["label"].map(GAIN)).sort_values(["user_id", "g"],
ascending=[True, False])
    ideal_gains = ideal_gains.groupby("user_id").head(k)
    ideal_gains["pos"] = ideal_gains.groupby("user_id").cumcount()
    idcg = (ideal_gains["g"] / np.log2(ideal_gains["pos"] +
2)).groupby(ideal_gains["user_id"]).sum()
    ndcg = (dcg.reindex(idcg.index, fill_value=0.0) / idcg).mean()

    return {f"recall@{k}": recall, f"order_recall@{k}": order_recall, f"ndcg@{k}": ndcg}
```

It takes two frames. `recs` holds `user_id, item_id` rows in display order; `labels` is the table from `data.labels`. We go through it in four moves.

Move 1: number the recommendations. `recs.groupby("user_id").head(k)` keeps each user's first k rows, so position is *derived from row order*: an unsorted frame silently scores a random list. `cumcount()` numbers rows `0, 1, 2, …` into `pos`. Positions are zero-based, hence the discount `np.log2(pos + 2)` for the one-based rank i of the formula, $\log_2(i+1)$.

Move 2: attach recommendations to labels (the merge). `labels.merge(recs, ..., how="left")` starts from the *labels*, so every future positive survives the merge, with `pos` filled in if we recommended it and `NaN` otherwise. That makes the denominators right: rows per user is R, and `count()` of `pos` (which ignores `NaN`) is the number of hits. A user for whom we recommend nothing gets recall 0 rather than being dropped, so a method cannot improve its score by skipping hard users. `order_recall` does the same for label-3 rows only, the closest metric to revenue; its average covers only users who ordered.

Move 3: DCG. `found` keeps only the hits, then `GAIN[label] / log2(pos + 2)` is summed per user. `GAIN = {1: 1.0, 2: 3.0, 3: 7.0}` is the table $2^{\text{rel}} - 1$ from above.

Move 4: IDCG and the ratio. The ideal list is built from the same labels: sort each user's labels by gain descending, keep the first k, and discount them identically. DCG (users with no hits get 0) divided by IDCG, averaged over users, is NDCG. Because the ideal list is cut at k too, a perfect score stays reachable for users with more than k positives.

Now the ceiling check:

recommender/recsys/evaluate.py · candidate_recall

```
def candidate_recall(cands: pd.DataFrame, labels: pd.DataFrame) -> float:
    """Upper bound for any ranker: share of future positives present in the candidate set."""
    m = labels.merge(cands[["user_id", "item_id"]].assign(found=1), on=["user_id", "item_id"],
how="left")
    return float(m.groupby("user_id")["found"].apply(lambda s: s.notna().mean()).mean())
```

This is the same left-merge idea: after the merge, `found` is `NaN` for positives missing from the candidate set, so the per-user mean of `found.notna()` is the fraction of that user's positives that are candidates. There is no `k`: candidates are a set, not a ranked list.

The chapters that follow call a shared helper, `evaluate` in `book/code/common.py`, which wraps `metrics` and reports recall at 5, 10, 20 and 50, order recall, NDCG at 20, and coverage at 20 (distinct recommended items divided by the items in stock at the cutoff). Every method is scored by identical code, so a number in chapter 6 compares with one in chapter 3 without footnotes.

Run it

The companion script `book/code/ch02_metrics.py` builds a ten-item list whose outcome we control, and prints every intermediate value so you can follow the arithmetic with a pencil. Run it from the `book/` directory:

```
cd book
uv run python code/ch02_metrics.py
```

The first block shows the list with each item's label (what the user later did with it), its gain, the discount, and the discounted gain:

TERMINAL OUTPUT

```
=== The ranked list (what the user later did with each item) ===
pos item  label  gain  1/log2(pos+2)  discounted gain
  1 A         0   0.0         1.0000          0.0000
  2 B         3   7.0         0.6309          4.4165
  3 C         0   0.0         0.5000          0.0000
  4 D         1   1.0         0.4307          0.4307
  5 E         0   0.0         0.3869          0.0000
  6 F         0   0.0         0.3562          0.0000
  7 G         2   3.0         0.3333          1.0000
  8 H         0   0.0         0.3155          0.0000
  9 I         0   0.0         0.3010          0.0000
 10 J         1   1.0         0.2891          0.2891
```

Item B was ordered (label 3, gain 7), G was added to the cart (label 2, gain 3), and D and J were clicked (label 1, gain 1 each). Note that the column header `1/log2(pos+2)` uses the code's zero-based position; as a one-based rank it is the $1/\log_2(i+1)$ of the formulas.

Precision and recall at every cut-off

The user has five future positives in total, including a fifth item, `Z`, that our list never contained:

TERMINAL OUTPUT

```
=== Precision and recall at each cut-off ===
future positives in total: 5 (one, 'Z', is never recommended)
  k  hits  precision@k  recall@k
  1     0        0.000     0.000
  2     1        0.500     0.200
  3     1        0.333     0.200
  4     2        0.500     0.400
  5     2        0.400     0.400
  6     2        0.333     0.400
  7     3        0.429     0.600
  8     3        0.375     0.600
  9     3        0.333     0.600
 10     4        0.400     0.800
```

Follow one row: at $k = 4$ we have shown items A to D, of which B and D are hits, so precision is $2/4 = 0.500$ and recall is $2/5 = 0.400$. Two patterns are visible. Recall only goes up (more slots can only add hits), maxing out at 0.800 because `Z` is never recommended. Precision zigzags: it jumps whenever a hit lands (positions 2, 4, 7, 10) and decays in between. That zigzag is exactly what average precision averages.

Average precision

TERMINAL OUTPUT

```
=== Average precision ===
hit at pos 2: precision@2 = 0.5000
hit at pos 4: precision@4 = 0.5000
hit at pos 7: precision@7 = 0.4286
hit at pos 10: precision@10 = 0.4000
AP = sum / 5 future positives = 1.8286 / 5 = 0.3657
```

The four hits are at ranks 2, 4, 7 and 10, with precisions 0.5000, 0.5000, 0.4286 and 0.4000, summing to 1.8286. Divide by the *five* future positives (not by four hits), and AP is 0.3657. The fifth positive, `Z`, pulls the score down by contributing a zero.

DCG, IDCG and NDCG

TERMINAL OUTPUT

```
=== DCG, IDCG and NDCG ===
ideal order of the 5 future positives (labels): [3, 2, 2, 1, 1]
  ideal pos 1: label 3, gain 7.0 / log2(2) = 7.0000
  ideal pos 2: label 2, gain 3.0 / log2(3) = 1.8928
  ideal pos 3: label 2, gain 3.0 / log2(4) = 1.5000
  ideal pos 4: label 1, gain 1.0 / log2(5) = 0.4307
  ideal pos 5: label 1, gain 1.0 / log2(6) = 0.3869
DCG  = 6.1362
IDCG = 11.2103
NDCG = DCG / IDCG = 0.5474
```

The ideal ordering of the user's five positives is by gain: the order (label 3), then the two add-to-carts (label 2: item G from our list, plus `Z`, which we never recommended), then the two clicks (label 1). Its terms are 7.0000, 1.8928, 1.5000, 0.4307 and 0.3869, which sum to an IDCG of 11.2103. Our list achieved a DCG of 6.1362, so

$$\text{NDCG} = \frac{6.1362}{11.2103} = 0.5474.$$

We found most of the value (we got the order at rank 2 and a cart at rank 7) but lost some because `Z` was missing and because B was not first.

Order matters: the same hits, a worse list

Now the experiment that justifies NDCG. Keep exactly the same four hits, but move the ordered item B from rank 2 down to rank 10, with the other hits at ranks 2, 5 and 8:

TERMINAL OUTPUT

```
=== Same positives, but the order item B pushed to the last slot ===
DCG = 4.1304   NDCG = 0.3684   (recall@10 is unchanged: 0.800)
```

The set of recommended items is the same, so recall@10 does not change (0.800). But NDCG falls from 0.5474 to 0.3684. A recall-only dashboard would call these two lists identical; a shopper who must scroll to the bottom to find the thing she wants would not.

Results: what does a random recommender score?

Finally, we pipe a deliberately useless method through the shared `evaluate` helper: for every user in the label set, draw 50 items uniformly at random from the in-stock catalogue.

TERMINAL OUTPUT

```
=== Random recommender through common.evaluate (same code every chapter uses) ===
        recall@5  recall@10  recall@20  recall@50  order_recall@20  ndcg@20  coverage@20
method
random    0.0005     0.0025     0.0069     0.0212           0.0066   0.0026          1.0

evaluated users: 947, available items: 2311
```

This is the **floor** of every table in the book. Several things are worth pausing on.

First, the evaluation population: 947 users had at least one positive in the final week, and 2311 items were available at the cutoff. If you pick 20 items out of 2311 for one user with a single positive, the chance of hitting it is $20/2311 \approx 0.0087$. That is the value we expect for recall@20 from a method with no information (almost all positives, about 99.85%, are items that were in stock at the cutoff, so the expected value is barely lower). The measured 0.0069 is a single random draw: most users have only a few positives, so each user's recall is 0 or a small fraction, and the mean over 947 users is noisy. It will move a little between random seeds, but it will always hover around 0.0087. The random recommender also finds 0.0212 of positives at recall@50, rising with K as expected, while NDCG@20 is only 0.0026: random hits land at random depths, and a hit at rank 15 earns very little.

Second, coverage@20 is 1.0: random sampling touches every item, so it has *perfect* breadth and the worst accuracy possible, which is why coverage is a guardrail and never a target.

Third, every later method must beat these numbers by a wide margin, and also beat a one-line popularity baseline, the subject of the next chapter.

> **Did you know?**
>
> Offline evaluators often speed up their metrics by ranking each test positive against a small random sample of items (say 100) instead of the whole catalogue. Krichene and Rendle showed that such **sampled metrics** are inconsistent with the exact ones: they do not even preserve the ordering of recommenders in expectation, so "model A beats model B" under sampling can be false on the full catalogue. Their advice is to avoid sampling when you can afford the full computation, which is exactly what `evaluate.py` does.
>
> Source: Krichene & Rendle, KDD, 2020

The same metrics inside the application

The recommender service evaluates each trained model with the same `metrics` function and stores the results; the "How it works" page shows them.

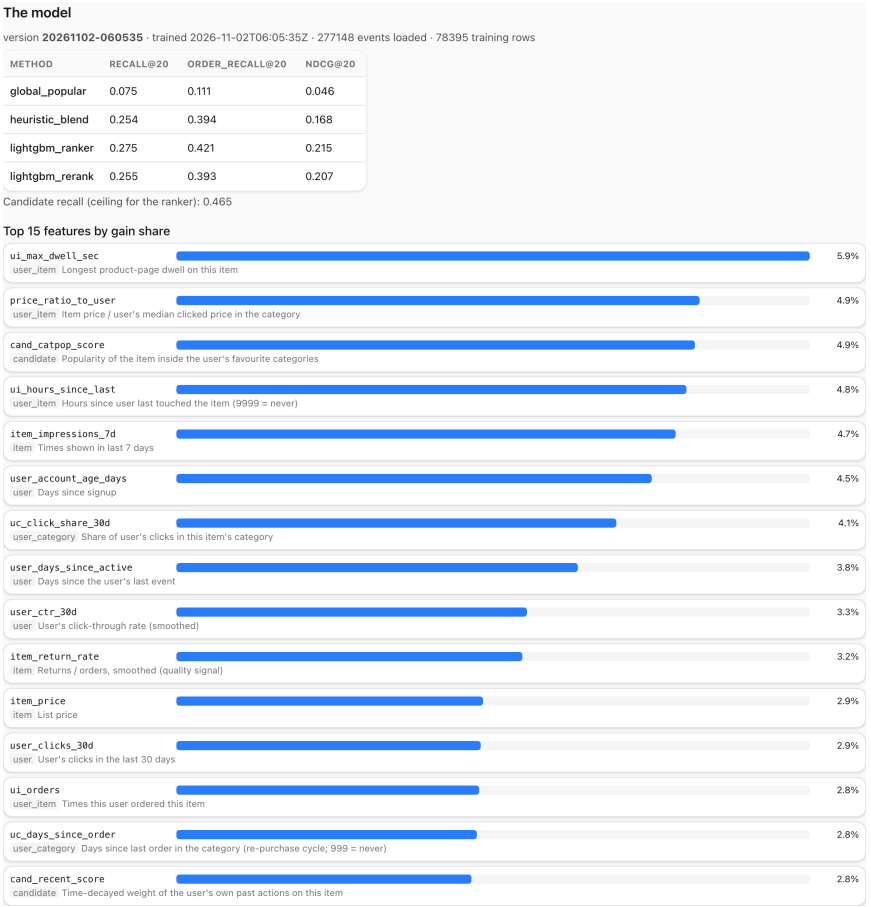


Figure 2.3: *The "The model" section of the How it works page: a table of recall@20, order_recall@20 and ndcg@20 for the four methods, the candidate-recall ceiling, and the top features by gain share.*

Its values differ from this book's because the app trains on the live database when you click "Retrain", whereas the book's scripts use a fixed simulation, cutoff and seeds. Look at the *shape*: a few methods ranked by NDCG, with a ceiling above them.

Pitfalls, and how it is done in production

Leakage through features, not only events. The split above keeps labels out of the history, but a careless feature (for example the item's all-time order count, computed on today's database) quietly

reintroduces the future. Chapter 9 demonstrates a model whose offline NDCG more than doubles because of one leaky column, and then falls apart in honest evaluation. If an offline number looks too good, assume leakage until proven otherwise.

Noise and survivorship. Our label window is one week and 947 users, so a difference in the third decimal place between two methods is probably noise; production teams report bootstrap confidence intervals and average several cutoffs or seeds (chapter 10 does this for ablations). Metrics also only cover users with at least one future positive, so slice them by cohort (new versus returning, city, activity level; chapter 12).

Logging bias. A click happens only on what the old system chose to show, and higher slots get more clicks regardless of quality, so offline labels reward models that imitate the old system. Inverse-propensity weighting and exploration traffic exist for this reason (chapters 11 and 12); the A/B test remains the final arbiter.

Optimising a single metric. NDCG says nothing about diversity, freshness, fairness to small sellers, or the price of what you are pushing. Production teams keep a dashboard of **guardrail metrics** next to the primary one: coverage, share of impressions going to new items, category diversity (chapter 11), latency and error rate. A model that lifts NDCG 2% while collapsing catalogue coverage is not an improvement.

Production practice in short. Keep one evaluation harness and one definition of each metric; run it on every training job (the service in this repo stores the result next to the model file); version the dataset and cutoff; and compare every new method against the *previous production model* and a trivial baseline.

Summary

- Offline evaluation replays history to reject bad ideas cheaply; online A/B testing is the ground truth. Use offline as a gate, not as a promise.
- Always split by **time**: build candidates and features from events before the cutoff, take labels from the following 7 days. Random splits leak the future.
- Labels are graded: click 1, add to cart 2, order 3, with the strongest behaviour per (user, item) kept; only users with at least one positive are scored.
- Precision@K measures waste, recall@K measures how much of what the user wanted you found, and AP adds sensitivity to where the hits sit.
- DCG converts labels to gains $2^{rel} - 1$ (1, 3, 7), discounts by $1/\log_2(i+1)$, and NDCG divides by the ideal DCG so scores of different users are comparable. Moving a hit from rank 2 to rank 10 cut NDCG from 0.5474 to 0.3684 while recall stayed put.
- Coverage is a guardrail, not a goal: a random recommender has perfect coverage and the worst accuracy. Candidate recall is the ceiling on any ranker built on top of a candidate set.
- `evaluate.py` merges labels **left** onto recommendations, so missed positives still count and empty lists score 0.

Quiz

Q2.1 A team evaluates a recommender by randomly shuffling all events and holding out 20% for testing. Which statement best describes the main flaw of this setup?

A. It makes the training set too small to learn anything.
B. Test events can be earlier than training events, so the model can learn from the future.
C. It produces precision values that are always higher than recall values.
D. It prevents the use of graded labels such as click, cart and order.

Q2.2 Two recommenders return exactly the same ten items for a user, but recommender X places the ordered item first and recommender Y places it last. Which metric will distinguish them?

A. recall@10
B. precision@10
C. NDCG@10
D. catalogue coverage

Q2.3 In this book's setup, which statement about *candidate recall* is correct?

A. It is always lower than the recall of the final ranking.
B. It is an upper bound on the recall that any ranker built on those candidates can reach.
C. It measures how many different items the system recommends across all users.
D. It can be raised only by training a better ranker.

Q2.4 In the average precision formula, why do we divide by the total number of future positives R instead of the number of hits?

Q2.5 `evaluate.metrics` merges `labels` with `recs` using `how="left"` starting from the labels. Explain what would change if the merge started from `recs` instead (an inner join between the two frames).

Q2.6 Look at the two statements in the included `metrics` function that build `found` and `dcg` (the ones using `dropna(subset=["pos"])` and `np.log2(... + 2)`). What does `dropna(subset=["pos"])` remove, and why is the constant `+ 2` used rather than `+ 1`?

Q2.7 A user's top-3 list has relevance labels `[0, 1, 3]` at ranks 1, 2, 3 (gains 0, 1, 7). The user's only future positives are those two items. Compute DCG@3, IDCG@3 and NDCG@3, to three decimals.

Q2.8 The same user has $R = 4$ future positives, and a top-5 list hits two of them, at ranks 1 and 3. Compute precision@5 and recall@5.

Q2.9 Why look at coverage and candidate recall in addition to NDCG when comparing two ranking models? Give one failure each would reveal that NDCG would not.

Q2.10 *Hands-on.* In `book/code/ch02_metrics.py`, change the random recommender at the end so that every user receives the same fixed 50 items (for example the first 50 ids from `common.available_items()`). What should coverage@20 become, and what does that say about coverage versus accuracy?

Chapter 3 — Popularity baselines

It is the Monday after a payday, and Dewi, a product manager at a Shopee-like marketplace, has a problem. The new home feed is two weeks from launch, the machine-learning team is still arguing about embeddings, and the CEO has just asked a simple question: "If we ship *nothing clever*, what does the home page show?" Dewi's answer is the one every recommender project starts with: show what everyone else is buying. A single SQL query, a `GROUP BY item_id ORDER BY count(*) DESC LIMIT 20`, and the page is no longer empty.

That query is the oldest recommender in the world. It is also the one every serious system keeps around forever, as the cold-start fallback, as a candidate source, as a feature, and above all as the **yardstick**. If a fancy model cannot beat "show the bestsellers", it is not a model, it is an expensive way to be wrong. In this chapter we build a family of popularity baselines, from pure random to "popular in the categories *you* like", measure them all on the same data with the same metrics from Chapter 2, and learn the small bits of maths (exponential decay, Bayesian shrinkage) that reappear all through the rest of the book.

What you'll learn

- Why a baseline comes first, and what a "good" baseline looks like.
- Global, windowed, and time-decayed popularity, including the half-life maths behind decay.
- The Bayesian average (and its generalisation `_smoothed` in this repository) for not trusting small samples.
- How a tiny amount of personalisation, "popular in the categories you like", more than doubles ndcg@20 over the best global list.
- What popularity bias is, why coverage collapses, and what production systems do about it.

Words you will meet in this chapter (each is explained again where it first matters):

Term	Plain meaning
Baseline	The simplest sensible method. Any cleverer method only counts if it beats it. Think of the time a casual jogger runs a 5 km course: a new training plan is only "better" if you beat that time.
Yardstick	The same thing seen from the other side: the baseline is the ruler we hold every other method against.
SQL query	A one-line question asked to a database, such as "count orders per item and give me the top 20".
Event	One logged action: a click, an add-to-cart, or an order.
Cold start	A new visitor or a new item with no history yet, so there is nothing to personalise with.
Fallback	Plan B: what we show when the main method has nothing to say.
Candidate source	A cheap method that proposes a shortlist of items, which a heavier model then sorts more carefully.
Feature	A number describing a user or an item (for example "how many clicks this week") that a model reads as input.
Hyperparameter	A setting *you* choose by hand and test, instead of one the computer learns from data. The window length in days is one.
ndcg@20, recall@20, coverage@20	Chapter 2 metrics, each looking at a 20-item list. *ndcg*: a score that rewards putting the right items near the top. *recall*: the share of what the user really touched afterwards that our 20 items contain. *coverage*: the share of the whole catalogue that is recommended to at least one user.

Intuition: the shelf at the front of the shop

Imagine a neighbourhood bookstore. The owner knows nothing about you the first time you walk in, so what does she do? She puts the bestsellers on the table right by the door. She does not need to know your taste: a book that hundreds of other people bought is a decent bet for a stranger. That table is a **global popularity recommender**: one ranking, the same for everyone.

Now notice what a clever shopkeeper adds over time.

1. **She forgets slowly.** A book that sold 500 copies last year but only two last week is not on the table any more. That is *time decay*: recent sales count more than old ones. A shop that never forgets ends up decorating the front table with last season's hits.
2. **She does not trust tiny samples.** A book with a perfect five-star score from one reviewer, the author's cousin, should not beat a book with 4.7 stars from two thousand readers. That is *shrinkage*, or the **Bayesian average**: when evidence is thin, we start from what we know about books in general (the "prior guess") and move away from it only as fast as real evidence arrives. We will do this with numbers in Method 4.
3. **She notices who you are.** Once you say you like cooking, she walks you to the cookbook shelf, and *there* she again points at the bestsellers. She did not use any sophisticated model of your taste; she just intersected "what you like" with "what is popular". That is **personalised popularity**, and it is the bridge from this chapter to the rest of the book.

Every one of those moves has a precise numeric form. None of them needs more than a pandas `group-by`. And yet, as we will see in the results, the third move alone gets a score 2.5 times higher than the plain bestseller table.

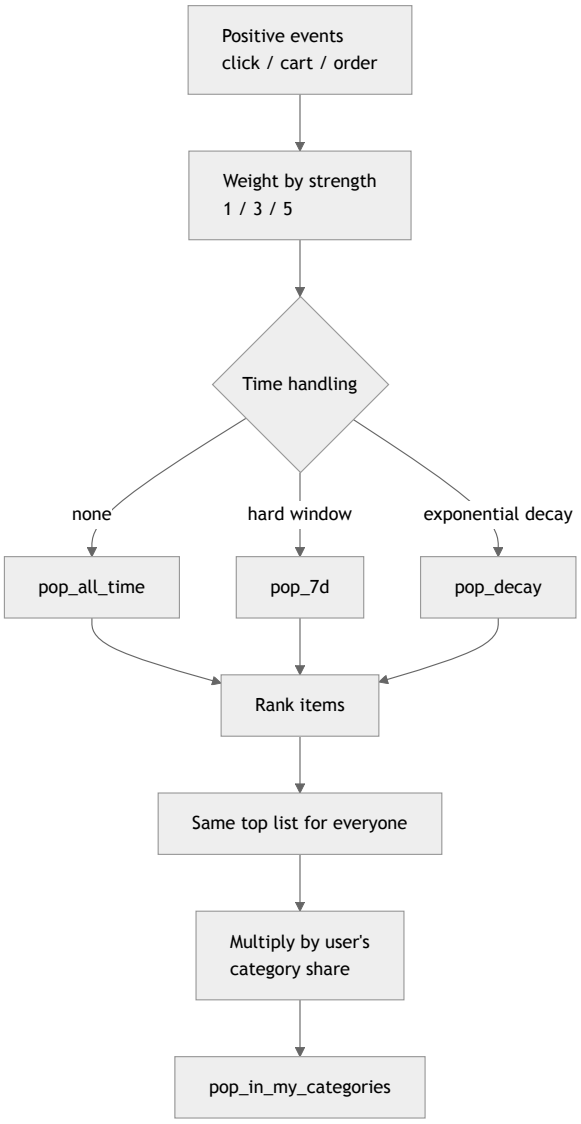


Figure 3.1: *Every popularity baseline starts from the same weighted positive events and differs only in how it treats time and whether it looks at the user.*

The algorithm: five ways to count

All five methods below are, at heart, one idea: *add up evidence, sort, take the top.* They differ in what counts as evidence. We keep the evaluation identical for all of them: they score every in-stock item for each of the evaluation users, take the 50 best, and are judged on the week after the cutoff (Chapter 2 explains why that split is honest).

Step 0: Turning events into weights

The marketplace logs many event types, but only three are *positive* signals: a click, an add-to-cart and an order. The repository gives them strengths in `schema.EVENT_WEIGHT`: click = 1, add-to-cart = 3, order = 5. A purchase is five times more telling than a click. `History.interactions()` is where that hap-

pens: it filters the event log down to those three types, attaches the weight `w`, and carries along `age_days`, the number of days between the event and the cutoff.

Everything in this chapter reads from that one table. Let w_e be the weight of event e, and a_e its age in days.

A tiny example. Suppose shopper Sari clicks a cat-food bag twice and orders it once. That bag collects $1 + 1 + 5 = 7$ points from her. Every shopper's events pile up on the items in the same way. The cutoff is the moment we freeze time (as in Chapter 2): only events before it are used, and `age_days` is how many days before the cutoff each event happened.

Method 1: Global popularity

The simplest scoring rule is to sum the weights for each item over all history:

$$\text{pop}(i) = \sum_{e\,:\,\text{item}(e)=i} w_e.$$

Read the symbols out loud: $\sum$ means "add up"; the small text underneath says *which* numbers to add: the weights w_e of every event e whose item is i. Example: a cat-food bag with 10 clicks, 2 add-to-carts and 1 order has $10 \times 1 + 2 \times 3 + 1 \times 5 = 21$ points. A phone case with 3 orders has 15 points, so the bag ranks above the case.

Sort descending and you have a bestseller list. Its virtue is that it is trivial, robust, and it needs no user information, so it works for visitors who have never logged in. Its vice is that it is blind to time: an item that was a monster hit six weeks ago and is dead today still sits near the top.

Method 2: A hard time window

The standard fix is to count only recent events. For a window of W days:

$$\text{pop}_W(i) = \sum_{e\,:\,\text{item}(e)=i,\ a_e<W} w_e.$$

In words: *look only at the last W days of receipts and ignore everything older.* The condition $a_e < W$ is the "is this event younger than W days?" test. Example with $W = 7$: an item has clicks aged 2, 5 and 9 days. The first two count (2 points), the 9-day-old click is ignored.

With $W = 7$ this is the "trending this week" list, and it is exactly what `candidates.item_popularity(hist, 7)` computes. The window is a blunt tool: an event 6.9 days old counts fully, one 7.1 days old counts for nothing. There is also a statistical price. Fewer events means a noisier count: if an item sells ten units a week, the weekly count fluctuates a lot just by chance.

Method 3: Exponential time decay

Instead of a cliff we can use a smooth slope: **every event loses value as it gets older, and the loss follows a fixed rule.** The rule we use says: *after a fixed number of days, an event is worth exactly half of what it was.* That number of days is the **half-life** h. With $h = 7$ days, a click from today counts 1, the same click a week from now counts 0.5, two weeks from now 0.25, three weeks from now 0.125, and

so on. Without any fading, a pile of old clicks can outweigh fresh ones forever; with fading, last season's hit slowly drops off the list. The formula that does this is

$$d(a) = 2^{-a/h} = e^{-\lambda a}, \qquad \lambda = \frac{\ln 2}{h}.$$

The two forms are the same function. In the first, the exponent $-a/h$ simply counts *how many half-lives have passed* (age 14 with $h = 7$ is two half-lives, so the weight is $\frac{1}{2} \times \frac{1}{2}$). The second form uses the number $e \approx 2.718$ and the natural logarithm $\ln$; it is the same curve written the way calculus likes, and you can skip it. With $h = 7$ days, the repository's `HALF_LIFE_DAYS`, we get $\lambda = 0.693/7 \approx 0.099$ per day. The decayed popularity of an item is simply the sum of *each event's weight times its fading factor*:

$$\text{pop}_{\text{decay}}(i) = \sum_{e \,:\, \text{item}(e) = i} w_e \cdot 2^{-a_e/h}.$$

Let us sanity check that the shape is what we claim. At age 0 the weight is 1. At age 7 it is $2^{-1} = 0.5$. At 14 it is $2^{-2} = 0.25$, at 21 it is 0.125. After a month it is about 5%, and an event from the start of the history is worth almost nothing. The script prints these numbers, and the chart plots them. In the chart, the line falls fast at first and then flattens: each week removes half of *what is left*, never a fixed amount.

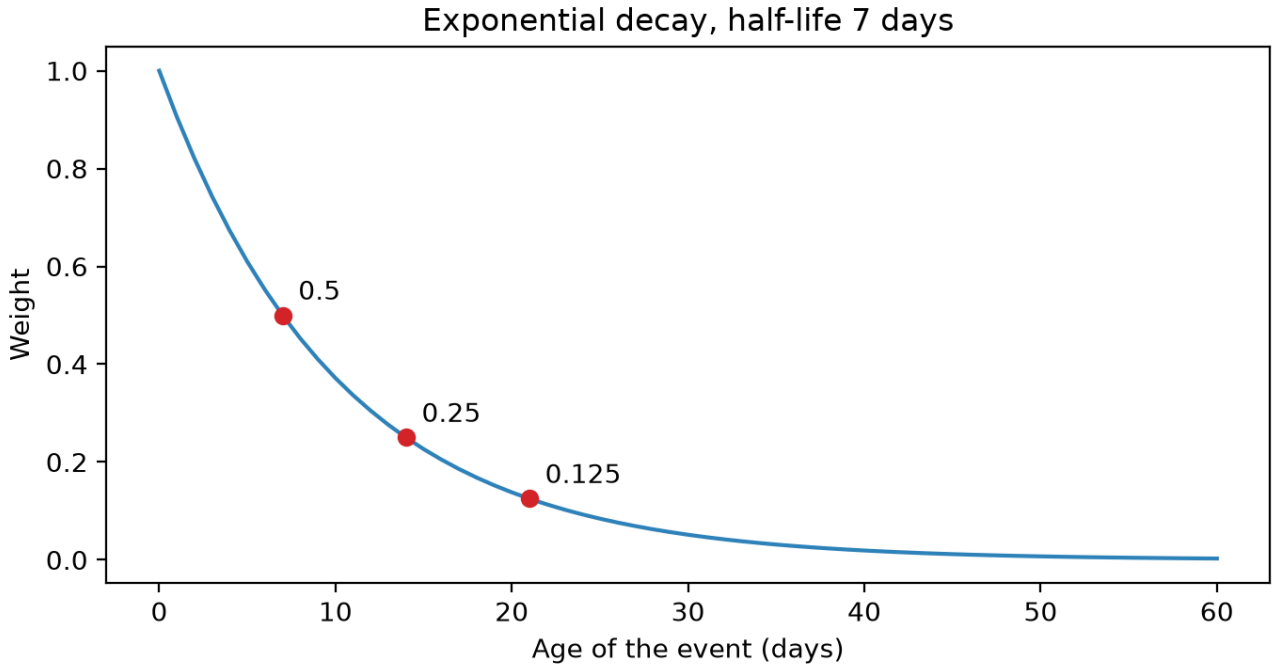


Figure 3.2: *Exponential decay with a 7-day half-life: the weight halves every seven days, from 1 at age zero to 0.5, 0.25 and 0.125 at ages 7, 14 and 21.*

Exponential decay is the production default for three reasons.

1. **Ranking does not flip just because time passes.** Tomorrow, every event is one day older, so every score is multiplied by the *same* factor ($2^{-1/7} \approx 0.906$). All items shrink together and their order stays. (The technical name is *memoryless*: the value of an event relative to another depends only on the gap between them.)

2. **It updates cheaply.** You do not need to re-read the whole history. Keep one running number S per item. When a new event with weight w arrives, first fade the old total for the time since the last event, then add: $S \leftarrow S \cdot 2^{-(t - t_{\text{last}})/h} + w$. That is one multiplication and one addition. (This is called *incremental* updating.) For example, if $S = 2.55$ today and nothing happens tomorrow, then tomorrow $S = 2.55 \times 0.906 \approx 2.31$.
3. **It has one intuitive knob.** You choose how fast interest fades by picking h. To make an event worth 10% after 30 days, solve $2^{-30/h} = 0.1$. Since $2^{3.32} \approx 10$, we need $30/h = 3.32$, giving $h = 30 \ln 2 / \ln 10 \approx 9.0$ days.

A tiny worked example

Take two items and a toy history. Item A got three clicks recently: two yesterday and one three days ago. Item B was a hit three weeks ago, with five clicks all 20 days ago. All clicks have weight 1. The toy numbers below are invented for illustration; the real ones come later.

Method	Item A	Item B	Winner
All-time: sum of weights	3	5	B
7-day window	3	0	A
Decay, $h = 7$	$2 \times 0.906 + 0.743 = 2.55$	$5 \times 2^{-20/7} = 5 \times 0.138 = 0.69$	A

All-time says B because it never forgets. The 7-day window says A, but only because B fell off the cliff; if B's clicks had been 6 days old it would have counted them fully. Decay gives a gentle answer: A wins because its evidence is fresher, yet B is not zero, so if it keeps getting clicks it will climb back. This smoothness is the reason decay shows up in half the candidate generators in the repository.

Method 4: The Bayesian average (not trusting small samples)

So far we ranked by *demand* (how often people act on an item). Another popular signal is *quality*: the average star rating. But averages can lie when they rest on few ratings. Think of a football player who scored in his only match: "1.0 goals per match" looks better than a star striker's 0.8 over 300 matches, but nobody believes it. A fair judge would treat the newcomer as if he had already played a number of extra matches at the league average. That is exactly the trick here. A product with a single 5-star rating has an average of 5.0, higher than a bestseller with 4.8 from thousands of buyers. We want a score that starts at the *global* mean when there is no evidence and gradually moves to the item's own mean as evidence piles up.

Let R be an item's average rating, v its number of ratings, C the average rating of all items (the "prior guess": what we expect for an item we know nothing about), and m a **prior strength**: how many ratings of "imaginary evidence" at the global mean we add. ("Prior" just means "what we believed before looking at this item's data"; "Bayesian" means we mix that belief with the evidence.) The Bayesian average, written WR for *weighted rating*, is

$$\text{WR} = \frac{v}{v+m}\,R + \frac{m}{v+m}\,C.$$

Read it as a weighted mix. The weight on the item's own average, $v/(v+m)$, is 0 when $v = 0$ and tends to 1 as v grows. The weight on the global mean is the remainder. When $v = m$ it is a 50/50

blend, so m is "the number of ratings at which we trust the item as much as the prior".

An equivalent view makes the idea reusable. Multiply the numerator out: $R \cdot v$ is the total stars, so

$$\text{WR} = \frac{(\text{total stars}) + m \cdot C}{v + m}.$$

In words: *pretend every item received m extra ratings, all equal to the global mean.* The general shape "(numerator + prior × strength) ÷ (denominator + strength)" works for any ratio, not only star ratings. Click-through rate is clicks ÷ impressions, conversion is orders ÷ clicks, return rate is returns ÷ orders. The repository wraps it in one tiny function, `features._smoothed`, and uses it many times in Chapter 9.

Here is a quick hand-sized check. An item has $v = 20$ ratings averaging $R = 4.8$. The global mean is $C = 4.0$ and the prior strength is $m = 100$. Then

$$\text{WR} = \tfrac{20}{120} \cdot 4.8 + \tfrac{100}{120} \cdot 4.0 = 0.8 + 3.33 = 4.13.$$

Walk through it. The weight on the item's own average is $20/120 \approx 0.17$ and the weight on the global mean is $100/120 \approx 0.83$. So 4.13 is 17% "this item's 4.8" and 83% "typical item's 4.0".

The item looks great (4.8), but twenty ratings is not much, so the estimate stays close to the prior.

The same arithmetic works on a conversion-style rate. Here a *click* is a user clicking an item, an *impression* is one time the item was shown, and CTR (click-through rate) is clicks ÷ impressions. Take 3 clicks in 10 impressions (30% raw) with a 5% global CTR and strength 50: it smooths to $(3 + 2.5)/60 = 9.2\%$, while 50 clicks in 1,000 impressions stays at 5.0%. Small samples are shrunk hard; large ones barely move.

> **Did you know?**
>
> **The "Top 250" is a Bayesian average.** IMDb publicly documents the formula it uses to rank its Top 250 films: weighted rating = (v ÷ (v+m)) × R + (m ÷ (v+m)) × C, where R is the title's mean rating, v its number of ratings, m the minimum ratings required to be listed (currently 25,000), and C the mean rating across all titles. IMDb itself calls this "a true Bayesian estimate". It is exactly the formula in this section.
>
> Source: IMDb, IMDb Help Center, 2026

Method 5: Popular in the categories *you* like

The first four methods return the *same list for every user*. The last one adds the smallest imaginable dose of **personalisation** (showing different things to different people). The idea in one sentence: *take the bestsellers, but only from the shelves this user spends time at.* It uses two ingredients that the repository already computes.

1. **Category affinity** $s_{u,c}$: how much user u likes category c, measured as the share of their (decayed) activity (clicks, carts, orders *and* searches) that falls in c. For each user the shares add up to 1 (like a pie chart of their interests). It is produced by `candidates.category_affinity`.
2. **Normalised popularity within a category** ("normalised" = rescaled so the best item is 1): for item i in category $c(i)$,

$$\tilde{p}(i) = \frac{\text{pop}_7(i)}{\max_{j \in c(i)} \text{pop}_7(j)} \in [0, 1].$$

Read it as: this item's weekly points divided by the weekly points of the *best* item of the same category ($\max$ = the biggest value). The result lies between 0 and 1 (the notation $\in [0, 1]$).

The best item of each category gets 1, the rest are fractions of it. The final score multiplies them:

$$\text{score}(u, i) = s_{u,\, c(i)} \cdot \tilde{p}(i).$$

Why normalise within the category? Without it, a huge category such as Snacks would drown out everything else: imagine its best item sold 900 points' worth and the best cat-food item only 90. After dividing, both get 1. Every category then has the same "ceiling", and the user's affinity decides *which categories' stars* appear.

A tiny extra term is also added: a millionth of the global popularity. It matters only for *ties*, where two items would otherwise get exactly the same score. Users with *no* history have all-zero affinity, so every score would be 0; the extra term leaves them with the plain `pop_7d` ranking, which is the safety net.

Worked example: Sari spends 60% of her activity on Cat Food, 30% on Dog Toys and 10% on Skincare. The best cat-food item scores $0.6 \times 1 = 0.6$, the runner-up at 80% of its popularity 0.48, the best dog toy 0.3, the best skincare item only 0.1. Her feed is mostly pet items, ordered by popularity within each shelf.

> **Did you know?**
>
> **Popularity bias is not a bug in your implementation; it is a documented property of recommenders.** Abdollahpouri and colleagues showed on a movie dataset that many recommendation algorithms concentrate their output on popular items, even for users whose taste is mostly niche, and they quantified this as a "bias disparity" between what users want and what they get.
>
> Source: Abdollahpouri et al., RMSE Workshop at RecSys, 2019

In plain words: when a system is trained to be right on average, it learns that popular items are the safest bet, so it keeps recommending them, even to people who would have loved something less known.

Code walkthrough

The repository already contains the building blocks. We will read them first, then the script that wraps them in an evaluation.

The interaction table

Everything starts from `History.interactions()`. Remember that a `History` object is the event log *cut off at a moment in time*, so nothing after the cutoff can leak in (Chapter 9 revisits this in depth).

recommender/recsys/data.py · History.interactions

```
def interactions(self) -> pd.DataFrame:
    """Positive actions (click/cart/order) with their strength weight."""
    pos = self.events[self.events["event_type"].isin(schema.EVENT_WEIGHT)].copy()
    pos["w"] = pos["event_type"].map(schema.EVENT_WEIGHT)
    return pos
```

It keeps only event types that appear in `EVENT_WEIGHT` and adds the weight column `w`. The `age_days` column is attached when the `History` is built. Everything below is a different reduction of these rows.

(A `pandas` column of numbers is called a *Series*; a `DataFrame` is a table of such columns. `History.interactions()` returns a DataFrame with one row per event.)

Time decay in one line

recommender/recsys/candidates.py · _decay

```
def _decay(age_days: pd.Series) -> pd.Series:
    return np.power(0.5, age_days / HALF_LIFE_DAYS)
```

That is the entire half-life formula: `0.5 ** (age_days / HALF_LIFE_DAYS)`. Because it is applied to a whole pandas Series, it works on millions of events at once, with no Python loop. The constant `HALF_LIFE_DAYS = 7.0` is declared at the top of `candidates.py`. Notice that the function is private (leading underscore) but used by several retrievers: `recent` multiplies each interaction's weight by it, and `category_affinity` uses it for both clicks and searches. The same curve therefore expresses "how long does interest last" everywhere in the first stage.

In plain words: "multiply by one half for every seven days of age". Because it is applied to a whole column at once ("vectorised"), pandas does the loop in fast compiled code.

Hard-window popularity

recommender/recsys/candidates.py · item_popularity

```
def item_popularity(hist: History, window_days: float) -> pd.Series:
    pos = hist.interactions()
    return pos[pos["age_days"] < window_days].groupby("item_id")["w"].sum()
```

Three operations: take positive interactions, keep rows with `age_days < window_days`, sum weights per item. Note the strict `<`: an event aged exactly `window_days` is *excluded*. The function returns a `pd.Series` indexed by `item_id`, and items with no events in the window are simply absent, so callers must `fillna(0)`. The "trending" retriever (with a 3-day window), the category-popular retriever (7 days) and `similar_items` all call it, so the whole system shares *one* definition of "popular" and only the window changes.

"Series indexed by `item_id`" means a column of numbers where each number is labelled by its item. `fillna(0)` replaces "missing" with 0, so items without events in the window score 0 instead of being absent.

Smoothing

recommender/recsys/features.py · _smoothed

```
def _smoothed(num: pd.Series, den: pd.Series, prior: float, strength: float) -> pd.Series:
    return (num + prior * strength) / (den + strength)
```

Again a one-liner, and again exactly the formula: `(num + prior * strength) / (den + strength)`. Map the Bayesian average onto it by setting `num` to total stars, `den` to the number of ratings, `prior` to C and `strength` to m. Elsewhere in `features.py` it is called with different strengths: 20 pseudo-impressions for a user's CTR, 50 for an item's CTR, 20 for conversion and 10 for return rate. These values are tuned by feel, not learned; the smaller the typical denominator for a statistic, the smaller the strength you can afford.

A "pseudo-impression" is an imaginary extra view added to the data, the same idea as the imaginary extra ratings in Method 4. "Tuned by feel" means a person picked the numbers after trying a few, instead of the computer learning them.

The experiment script

The companion script `book/code/ch03_popularity.py` builds all six scorers over the same items and users and evaluates them with the shared `common.evaluate`. Two helpers do the bookkeeping: `per_item` aligns a per-item Series to the fixed array of available items (zeros for items without events), and `recs_from_scores` turns a score matrix (or a single vector, broadcast to every user) into the top 50 rows per user.

book/code/ch03_popularity.py · per_item

```
def per_item(values: pd.Series) -> np.ndarray:
    return values.reindex(item_ids).fillna(0.0).to_numpy()
```

book/code/ch03_popularity.py · recs_from_scores

```
def recs_from_scores(S: np.ndarray, name: str) -> pd.DataFrame:
    """S: (n_users, n_items) or (n_items,) scores -> top-50 rows per user. Ties broken by item
order (stable)."""
    if S.ndim == 1:
        S = np.broadcast_to(S, (len(users), len(item_ids)))
    top = np.argsort(-S, axis=1, kind="stable")[:, :TOP_N]
    return pd.DataFrame({"user_id": np.repeat(users, TOP_N), "item_id": item_ids[top].ravel()})
```

The stable sort matters: many items tie at zero, and an unstable sort would make runs irreproducible.

"Stable" means that when two items have the same score, they stay in their original order. Without it, ties could be shuffled differently on each run, and you could not repeat your results.

Now the scorers themselves. The first three are nearly one-liners (they sit at module level in the script):

book/code/ch03_popularity.py · ch03_scores

```
pop_all = per_item(pos.groupby("item_id")["w"].sum())
scores["pop_all_time"] = pop_all

pop7 = per_item(candidates.item_popularity(hist, 7))
scores["pop_7d"] = pop7

w_decay = pos["w"] * np.power(0.5, pos["age_days"] / HALF_LIFE)
pop_decay = per_item(w_decay.groupby(pos["item_id"]).sum())
scores["pop_decay"] = pop_decay
```

The decayed variant repeats the `_decay` formula inline instead of importing the private helper, so that the chapter's script is self-contained. The Bayesian average operates on the catalogue's `rating_avg` and `rating_count` columns:

book/code/ch03_popularity.py · ch03_bayes

```
v, R = items["rating_count"].to_numpy(float), items["rating_avg"].to_numpy(float)
m = v.mean()
C = float((R * v).sum() / v.sum())
bayes = v / (v + m) * R + m / (v + m) * C
```

Two choices are worth discussing. The prior strength m is set to the *mean number of ratings per item*, a common heuristic: at an average-sized sample, the evidence and the prior count equally. The global mean C is weighted by v, so it is the mean over all *ratings*, not the mean of item means. Both are cheap to compute and need no tuning.

Finally the personalised version. It builds a user × category matrix `A` from `category_affinity`, an item-wise matrix `P` of within-category normalised popularity, and multiplies them:

book/code/ch03_popularity.py · ch03_personal

```
scores["pop_in_my_categories"] = A @ P + 1e-6 * pop7 / max(pop7.max(), 1e-9)
```

`A` has one row per evaluation user and one column per category; `P` has one row per category and one column per item, and is non-zero only where the item belongs to the row's category. The matrix product therefore gives, for each user and item, exactly $s_{u,c(i)}\,\tilde{p}(i)$. The trailing term is the tie-breaker described earlier.

Concretely, if Sari's row of `A` is 0.6 for Cat Food, 0.3 for Dog Toys and 0.1 for Skincare, then multiplying by `P` gives each item its category's weight times its normalised popularity, exactly the worked example above. One matrix product does this for every user and item at once.

Run it

From the `book/` directory, with the dependencies installed (see the Preface):

```
cd book
uv run python code/ch03_popularity.py
```

The script also prints the top five items of each global method; we show three groups here. First, the Bayesian-average setup and four sample rows showing the shrinkage in action:

TERMINAL OUTPUT

```
=== Building the baselines ===
m (mean rating_count) = 105.2, C (global mean rating) = 4.324

Smoothing examples (few ratings are pulled toward C, many ratings are trusted):
      rating_avg  rating_count  bayes
0            4.5           0.0  4.324
892          4.4           0.0  4.324
1840         4.4          34.0  4.343
610          5.0        2718.0  4.975

eval users with no category signal (fall back to pop_7d): 21 of 947
```

Then the main table: one row per method, evaluated on the same users, the same label week and the same metric code:

TERMINAL OUTPUT

```
=== Results (top 50 per user, same labels and metrics for everyone) ===
                      recall@5  recall@10  recall@20  recall@50  order_recall@20  ndcg@20
coverage@20
method
random                  0.0026     0.0055     0.0090     0.0210           0.0081   0.0035
1.0000
pop_all_time            0.0323     0.0515     0.0914     0.1575           0.1202   0.0527
0.0087
pop_7d                  0.0310     0.0493     0.0786     0.1335           0.1066   0.0475
0.0087
pop_decay               0.0318     0.0519     0.0813     0.1557           0.1091   0.0491
0.0087
bayesian_rating         0.0216     0.0329     0.0648     0.1178           0.0716   0.0325
0.0087
pop_in_my_categories    0.0926     0.1425     0.2173     0.3529           0.3050   0.1302
0.2164
```

Finally the decay weights at a few ages:

TERMINAL OUTPUT

```
=== Decay weights (half-life 7 days) ===
age  0 days -> weight 1.0000
age  1 days -> weight 0.9057
age  3 days -> weight 0.7430
age  7 days -> weight 0.5000
age 14 days -> weight 0.2500
age 21 days -> weight 0.1250
age 30 days -> weight 0.0513
age 53 days -> weight 0.0053
```

The script also writes `results/ch03_popularity.json`, which the figure script reads to draw the two charts below.

Results

Reading the decay curve

The decay curve in Figure 3.2 confirms the arithmetic. The printed weights agree with it: 0.9057 after one day, 0.7430 after three, exactly 0.5000 after seven, 0.2500 after fourteen and 0.1250 after twenty-one. By the 30-day mark the weight has fallen to 0.0513, and an event 53 days old, the oldest in the

history here, is worth only 0.0053 of a fresh one. This means that, for ranking purposes, decay with a one-week half-life behaves like a soft window of roughly two to three weeks.

The leaderboard

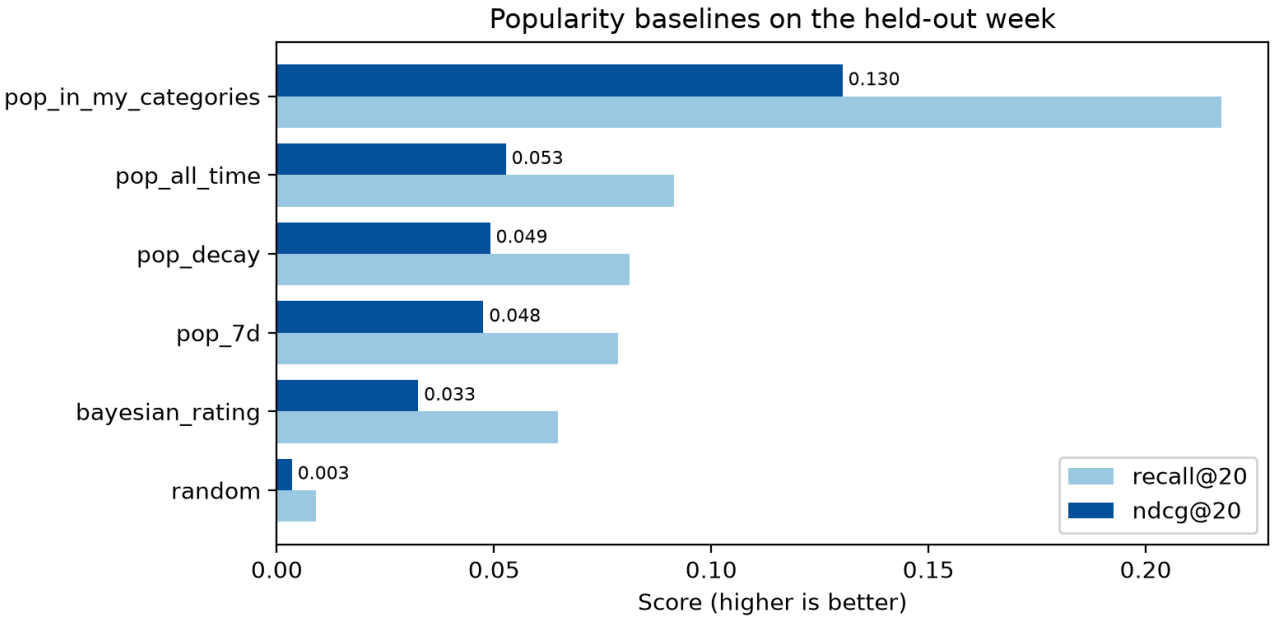


Figure 3.3: *Horizontal bars of recall@20 (light) and ndcg@20 (dark) for six popularity baselines; pop_in_my_categories leads by a wide margin and random is at the bottom.*

Read the main table row by row. We focus on ndcg@20 (the single number that rewards putting hits near the top) and recall@20 (how many of the user's held-out interactions we found in 20 slots).

A quick reminder of how to read these numbers: both go from 0 to 1 and higher is better. Absolute values look small because we only have 20 slots and the catalogue is large, so what matters is the *ratio* between methods.

Random is the floor. `random` reaches ndcg@20 of 0.0035 and recall@20 of 0.0090. Nothing in this chapter is allowed to score lower, and anything that does is broken. Its coverage is 1.0000: every item gets recommended to somebody. We will return to the oddity that the *worst* method has the *best* coverage.

Global popularity is already roughly fourteen to fifteen times better than random in ndcg@20. `pop_all_time` scores ndcg@20 0.0527 and recall@20 0.0914, `pop_7d` scores 0.0475 and 0.0786 and `pop_decay` lands in between with 0.0491 and 0.0813. The gap between random and the cheapest sensible method is far bigger than the gaps among the sensible ones. Most of the achievable "easy" value lives in simply knowing which items people want.

All-time slightly beats the 7-day window, which is the opposite of what intuition suggests. Many practitioners expect "trending" to win. It does not here, and the reason is a property of *this data*: in the simulator, each item has a fixed hidden quality, so an item's hidden quality does not change and an item that was popular a month ago is still popular. Under stable popularity, more history means less noise, and the 53-day count is a better estimate of the same underlying number than a one-week count

is. The decay variant sits between the two for exactly that reason: its effective window is longer than seven days but still downweights the old.

At deeper cut-offs the decayed list is much closer to the all-time list (recall@50 is 0.1557 versus 0.1575, while the hard window falls to 0.1335): a smooth slope keeps the long tail of evidence that a cliff discards.

"Long tail of evidence" means the many older events. A hard window throws them away; decay keeps them but gives them little weight. "Noise" means random ups and downs in a count that come from chance, not from real popularity.

> **Warning**
>
> Do not take "all-time beats 7-day" as a law of nature. It says that the *simulated* catalogue has stationary popularity. On a real marketplace, flash sales, viral products, seasonal categories (Ramadan, 9.9, 11.11) make popularity drift, and the windowed or decayed list typically wins. The lesson is to **measure on your own data**, and to treat the window or half-life as a hyperparameter tuned on a validation week.

Quality is not demand. `bayesian_rating` has ndcg@20 0.0325 and recall@20 0.0648: about nine times better than random, but worse than every demand-based method. In the sample rows, item 892 (stated 4.4, zero ratings) scores 4.324, the global mean C; item 1840 (4.4, 34 ratings) moves a little to 4.343; item 610 (5.0, 2718 ratings) ends at 4.975 because the evidence swamps the prior ($m = 105.2$). Shrinkage works as designed, but a good rating is not a reason to click: a well-rated niche product matters only to the few who need it. Use Bayesian ratings as a feature or a "top rated" shelf, not as a home-feed ranker.

It is also worth noticing the placeholders hiding in raw data. The first printed row has `rating_avg` 4.5 with zero ratings: a default value written to every new item. Without smoothing, every unrated item would carry a fictitious 4.5, which would look *better* than most real products. The Bayesian average silently repairs this by treating zero ratings as "no information".

A little personalisation goes a long way. `pop_in_my_categories` reaches ndcg@20 of 0.1302 and recall@20 of 0.2173. Compared with the best global list that is a factor of about 2.5 in ndcg, and recall@50 climbs to 0.3529 versus 0.1575. Order recall@20, which only counts held-out *orders* (the strongest signal), is 0.3050 versus 0.1202. We did not train anything. The only new information is "which categories does this user touch", and it moves the metric from "mediocre" to "decent". This is the most important number of the chapter: *knowing even a coarse user preference matters more than any amount of refinement of the global list.*

Notice the 21 evaluation users, out of 947, who had no category signal at all and fell back to plain `pop_7d`. That is about 2% of users, and the fallback means they still get a sensible page rather than an empty one: the same graceful degradation the production system relies on when a new visitor arrives.

Pitfalls and how it is done in production

Popularity is self-reinforcing

The most important problem is not accuracy, it is the **feedback loop**. If you show the same twenty items to everybody, people click those twenty items (because those are the ones they can see), the click counts for those items grow, and next week they are even more "popular". Items outside the list cannot gain evidence, no matter how good they are. Economists call it the Matthew effect; recommender folks call it popularity bias.

The loop is simple: exposure produces clicks, clicks produce score, and score decides exposure, so items outside the list cannot enter it.

The table makes it visible. The three global methods all have coverage@20 of 0.0087: across 947 users, they recommend the *same twenty items*. The personalised method, at 0.2164, touches roughly twenty-five times as many distinct items, and random, at 1.0000, touches them all.

Production systems address it in layers, none of which is a baseline's job but all of which build on one:

- **Personalisation** (the rest of this book) reduces the dominance of a single global list.
- **Exploration**: reserve a few slots for items with little data, and show them often enough to measure how good they are. Chapter 11 introduces ε-greedy (with a small probability ε, show a random or little-known item instead of the current best) and Thompson sampling (show each item in proportion to how likely it is to be the best).
- **Position-bias correction** so that "was clicked" is not mistaken for "is relevant" when the item was merely put at position 1, where people click the most just because it is on top (Chapter 12).
- **Calibrated or diversified re-ranking** such as MMR (Maximal Marginal Relevance: build the list step by step, choosing items that are relevant but not too similar to those already chosen), so the top 20 is not twenty near-duplicates (Chapter 11).
- **Normalising by exposure**: use click-through rate with smoothing instead of raw click counts, so that items are rewarded for *conversion per view*, not for how often they were placed on the page. This is where `_smoothed` comes back in.

Choosing the window and half-life

Both W (the window length) and h (the half-life) are hyperparameters: settings we choose by hand and test on a *held-out* week, meaning a week of data we did not use to build the lists. Too short chases noise (a one-hour flash sale decides the list); too long serves yesterday's news. Fashion, electronics and food move at different speeds, so some marketplaces use one half-life per top-level category. Try three values (say 3, 7, 14 days) on a held-out week, as the hands-on exercise asks.

Do not mix scales, count users

Scales of `pop_7d`, `pop_decay` and a $[1, 5]$ rating are not comparable: one is a sum of points that can be large, another lives between 1 and 5. Normalise to $[0, 1]$ (divide by the maximum) before blending, or feed each to a ranker (the model that sorts the final list) as a separate feature. And count distinct users (or cap each user's contribution), since one shopper clicking 40 times inflates a score by 40.

Popularity as a candidate source and as a feature

In this repository baselines survive in three roles: a **fallback** (the trending retriever for users without history, and the Rust API's top-recent-orders query when Python is down), a **candidate source** (`cat-pop` in Chapter 8 is nearly method 5), and a **feature** (smoothed popularity, CTR and conversion columns the ranker weighs itself).

Summary

- A popularity recommender sums weighted positive events per item and sorts. It is cheap, robust, needs no user data, and defines the bar every other method must clear.
- A hard window counts only recent events; exponential decay with half-life h gives each event weight $2^{-a/h}$, so evidence fades smoothly and scores can be updated incrementally with one multiply and one add.
- The Bayesian average $\frac{v}{v+m}R + \frac{m}{v+m}C$ shrinks small samples toward the global mean. The repository generalises it as `_smoothed(num, den, prior, strength)` for CTR, conversion and return rate.
- On this data all-time popularity slightly beat the 7-day window, because simulated popularity is stable. Which window wins depends on how fast real popularity drifts, so tune it on held-out data.
- Ranking by rating alone loses to ranking by demand: quality is not the same as relevance.
- Multiplying category affinity by within-category popularity lifted ndcg@20 from about 0.05 to 0.13 with no model training, the clearest argument for personalisation.
- Global popularity gives every user the same few items (coverage 0.0087) and feeds a self-reinforcing loop; later chapters add personalisation, exploration and diversity to counter it.

Quiz

Q3.1 Why does `pop_all_time` slightly beat `pop_7d` on this book's simulated data?

A. The 7-day window contains no clicks.
B. Simulated item popularity is stable, so a longer history gives a less noisy estimate of the same quantity.
C. `pop_7d` uses a different evaluation week.
D. Decay is not applied to `pop_all_time`, and decay always hurts.

Q3.2 An item has a catalogue rating of 4.4 but `rating_count = 0`. What is its Bayesian-average score?

A. 4.4, because the item's own rating is always used.
B. 0, because there is no evidence.
C. The global mean rating C, because with $v = 0$ all the weight goes to the prior.
D. The mean of m and C.

Q3.3 The three global popularity methods all have `coverage@20 = 0.0087`. What does that mean?

A. They fail for 99% of the users.
B. Every user receives the same 20 items, so the methods only ever recommend a tiny part of the catalogue.
C. They recommend an item once every 115 requests.
D. The metric is broken, since coverage must exceed recall.

Q3.4 In your own words, show how `features._smoothed(num, den, prior, strength)` reproduces the Bayesian average of an item's star rating. State what you pass as each of the four arguments.

Q3.5 Explain the popularity feedback loop in two or three sentences and name one technique from this chapter or the next that weakens it.

Q3.6 Read the code below. For an interaction whose `age_days` is exactly `7.0`, does it contribute to the result? What is the weight of the surviving interactions, and what would you change to count events from the last 7 *and* the 8th day?

`recommender/recsys/candidates.py · item_popularity`

```
def item_popularity(hist: History, window_days: float) -> pd.Series:
    pos = hist.interactions()
    return pos[pos["age_days"] < window_days].groupby("item_id")["w"].sum()
```

Q3.7 Numeric. With a half-life of 7 days, compute the decayed popularity score of an item that has these events (weights from the repository: click = 1, add-to-cart = 3, order = 5): an order 7 days ago, a click 14 days ago and an add-to-cart 3 days ago. Use $2^{-3/7} \approx 0.743$.

Q3.8 Numeric. An item has $v = 20$ ratings averaging $R = 4.8$. The global mean rating is $C = 4.0$ and the prior strength is $m = 100$. What is the Bayesian average? What would it be with $v = 400$ ratings and the same average?

Q3.9 Short answer. You want an event to be worth exactly 25% of a fresh one after 10 days. What half-life do you need? Then explain in one sentence why a *smaller* half-life makes the recommender react faster but also noisier.

Q3.10 Hands-on exercise. Open `book/code/ch03_popularity.py` and change `HALF_LIFE = 7.0` to 2.0, then to 30.0, running `uv run python code/ch03_popularity.py` each time (afterwards restore the file and the results with `git checkout code/ch03_popularity.py results/`). Compare the `pop_decay` row's ndcg@20 with the `pop_7d` and `pop_all_time` rows each time. What do you observe as the half-life becomes very short and very long, and does it match the explanation given in the Results section?

Chapter 4 — Content-based filtering

It is the week before the 9.9 sale and Sari, a product manager on the marketplace's discovery team, is staring at a dashboard with one number she does not like. Every day a few hundred new products are listed by sellers: a ceramic coffee dripper from a shop in Bandung, a pair of sneakers from a brand nobody has searched for yet, a cat-food bundle that went live ten minutes ago. None of them has a single click. The popularity baselines from the previous chapter cannot see them, because popularity is *history*, and these products have none. Yet a buyer who spent last week browsing pour-over coffee gear would very probably love that dripper.

What Sari needs is a recommender that looks at what an item *is*, not only at who touched it. If the dripper's title says "coffee" and the buyer has been clicking on coffee things, the two can be matched without a single interaction on the new product. That idea is **content-based filtering**.

Sari's problem has a name: **cold start**. A *cold* item or user is one with no history yet, so every method that learns from past clicks has nothing to learn from. Content-based filtering is the standard answer to cold start, because it only needs facts that exist from the first second: the title, the brand and the category.

What you'll learn

- How to turn an item's text and metadata into a numeric vector, and why we weight words with TF-IDF instead of just counting them.
- What cosine similarity measures, and why it is the default distance for sparse text vectors.
- How to build a user profile as a decay-weighted average of the item vectors the user touched, and how to score every item against it.
- How the companion script `ch04_content.py` performs on the simulated marketplace, and why it beats global popularity but not category-aware popularity.
- Where content-based methods shine (cold start) and where they fail (over-specialisation, shallow metadata).

Here is a quick map of the words this chapter uses. Each one is explained again, slowly, where it first appears.

Word	Plain meaning
Vector	A list of numbers in a fixed order. We turn each product into one, so a computer can compare products.
TF-IDF	A way to fill that list so that rare, specific words count more than common, generic ones.
Cosine similarity	A score from 0 to 1 for how alike two vectors are. 1 means the same direction, 0 means nothing in common.
User profile	A taste summary of one shopper, built by blending the vectors of the products they touched.
Cold start	Recommending for a new item or new user who has no history yet.
Filter bubble	When a recommender keeps showing more of the same, so the shopper never sees anything different.

Intuition: the librarian who never watches the readers

Imagine a librarian who is not allowed to look at the lending records. She cannot say "people who borrowed this also borrowed that". All she may do is read the covers: the title, the subject shelf, the author. When you hand her three novels you enjoyed, she walks to the shelves, finds books whose covers *sound like* the three you gave her, and hands them over.

That librarian is a content-based recommender. She has two skills:

1. She can describe every book as a short list of descriptive words (an **item representation**).
2. She can summarise your taste as a list of the same kind of words (a **user profile**), and compare the two lists.

Notice what she does not need: other readers. That is both the superpower and the weakness: a book that arrived this morning is recommendable at once, but she will keep walking you to the crime shelf forever.

In our marketplace the "covers" are the `title`, `brand` and category (plus parent category) columns of the `items` table.

The chapter's pipeline looks like this:

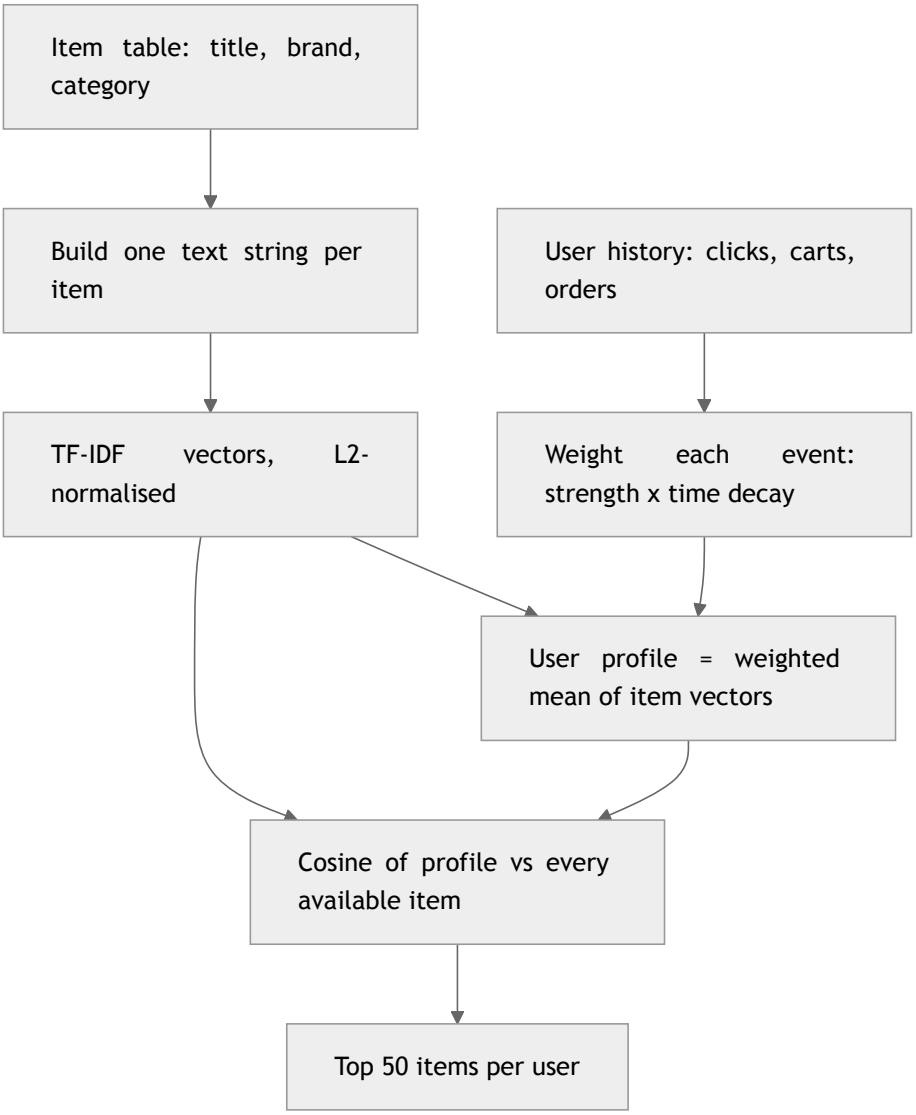


Figure 4.1: *The content-based pipeline. Item vectors are computed once; user profiles are recomputed from the latest history.*

Everything here is cheap, deterministic and explainable, which is why content-based scoring remains a workhorse for "similar items" widgets and cold-start fallbacks.

The algorithm

Step 1: item representation as text

The simplest, and surprisingly effective, way to represent an item is to glue its descriptive fields into one string and treat that string as a short document. In the companion script this is one line:

book/code/ch04_content.py · item_text

```
items["text"] = items["title"] + " " + items["category_name"] + " " + items["brand"] + " " +
items["parent_name"]
```

An item titled "Volt Skincare Lokal" in category "Skincare" with brand "Volt" and parent category "Beauty" becomes the document `Volt Skincare Lokal Skincare Volt Beauty`. Note that category and

brand words are repeated on purpose: titles in this simulator are short, and repeating the structured fields lets them count more than once.

Next we turn each document into a **vector**. A vector is just a **list of numbers in a fixed order**. To build one, we first collect the **vocabulary**: the list of every distinct word that appears in any item. Then the vector holds one number per vocabulary word. The simplest recipe is a **bag of words**: shake the words out of the title as if from a bag, and write down how many times each vocabulary word appears.

For example, take a vocabulary of four words in the order (`aurora`, `coffee`, `sneakers`, `volt`). The title text `volt sneakers sneakers` becomes the vector $(0, 0, 2, 1)$: zero `aurora`, zero `coffee`, two `sneakers`, one `volt`. You will meet this exact vector again as item D in the hand-worked example below.

Why bother? A computer cannot tell that two titles "feel alike", but it can easily compare two lists of numbers. Real vocabularies have thousands of words, and any one title uses only a handful of them, so nearly every entry is zero. Such a mostly-zero vector is called **sparse**, and we store only its non-zero entries to save memory.

Step 2: TF-IDF weighting

Bag-of-words has an obvious flaw. The word "home" appears in every item of the category "Home & Living" (and in the parent-category text of each one), so it tells us almost nothing about *which* home item a buyer wants, while the word "sneakers" is specific and carries much more information. We want **rare words to count more than common words**.

Think of a detective game where you must guess a suspect from clues. "Wears shoes" is a useless clue: nearly everyone does. "Walks with a limp" is a great clue: very few people do. A good clue is one that most suspects do *not* have. TF-IDF scores words the same way, using two parts:

- **TF, term frequency**: how many times the word appears in *this* item's text. More repeats, more important to this item.
- **IDF, inverse document frequency**: how *unusual* the word is across the whole catalogue. Here a "document" is one item's text. A word found in almost every item gets a low IDF; a word found in only a few items gets a high IDF.

Multiply the two and you get the **TF-IDF** (term frequency–inverse document frequency) weight. Without the IDF part, common words such as "home" would drown out the specific ones. In symbols, for a term t in document d out of a collection of N documents:

$$\mathrm{tfidf}(t, d) = \mathrm{tf}(t, d) \times \mathrm{idf}(t)$$

where $\mathrm{tf}(t, d)$ is the number of times t occurs in d, and the inverse document frequency downweights terms that occur in many documents. The version used by scikit-learn's `TfidfVectorizer` (a ready-made TF-IDF tool in the scikit-learn Python library), which our script uses, is the *smoothed* one:

$$\mathrm{idf}(t) = \ln \frac{1 + N}{1 + \mathrm{df}(t)} + 1$$

with $\mathrm{df}(t)$ the **document frequency**: the number of documents that contain t. $\ln$ is the natural logarithm, a function that grows slowly: a bigger ratio inside gives a bigger result, but never explosively. The "+1" terms keep the logarithm finite and guarantee that even a term present in every document keeps a small positive weight.

Let us plug numbers into a catalogue of $N = 4$ items:

The word appears in...	df	idf = ln((1+4)/(1+df)) + 1
all 4 items (generic)	4	$\ln(5/5) + 1 = 1.000$
2 items	2	$\ln(5/3) + 1 \approx 1.511$
1 item (rare)	1	$\ln(5/2) + 1 \approx 1.916$

The rarer the word, the bigger the multiplier. If "sneakers" appears twice in an item (tf = 2) and is rare (idf 1.916), its weight is $2 \times 1.916 = 3.832$.

Finally we **L2-normalise** each document vector: we rescale it so that its length becomes exactly 1. The L2 length is the ordinary straight-line length you know from Pythagoras: square every entry, add them up, take the square root. For the vector $(3, 4)$ the length is $\sqrt{9 + 16} = 5$, so dividing every entry by 5 gives $(0.6, 0.8)$, which has length 1 and points the same way. Formally, each document vector $\mathbf{x}_d$ becomes

$$\hat{\mathbf{x}}_d = \frac{\mathbf{x}_d}{\|\mathbf{x}_d\|_2}$$

where $\|\mathbf{x}_d\|_2$ is that length (the little hat on $\hat{\mathbf{x}}$ means "length 1"). This makes sure a long title does not automatically look more important than a short one: only the *proportions* between its words remain.

Step 3: cosine similarity

Cosine similarity answers one question: **do these two vectors point the same way?** Picture each vector as an arrow from the same starting point. Two arrows pointing in the same direction are "the same taste" (score 1); two arrows at right angles share nothing (score 0). We measure the angle between the arrows, rather than the distance between their tips:

$$\cos(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \, \|\mathbf{b}\|}$$

Two pieces of notation. The **dot product** $\mathbf{a} \cdot \mathbf{b}$ means: multiply the numbers in matching positions, then add them up. And $\|\mathbf{a}\|$ is the length of $\mathbf{a}$ from the previous step. By hand with two small vectors, $\mathbf{a} = (1, 2, 0)$ and $\mathbf{b} = (2, 1, 3)$:

- dot product: $1 \times 2 + 2 \times 1 + 0 \times 3 = 4$.
- lengths: $\|\mathbf{a}\| = \sqrt{5} \approx 2.236$ and $\|\mathbf{b}\| = \sqrt{14} \approx 3.742$.
- cosine: $4 / (2.236 \times 3.742) \approx 0.478$.

Two sanity checks. Vectors $(1, 0)$ and $(0, 1)$ share no word at all, so their dot product is 0 and the cosine is 0. Vectors $(1, 2, 0)$ and $(2, 4, 0)$ are the same direction, one just twice as long: dot product 10, lengths product $2.236 \times 4.472 = 10$, cosine exactly 1.

Because TF-IDF vectors have no negative entries, cosine lies between 0 (no shared terms at all, the arrows are at right angles) and 1 (identical direction). When both vectors are already unit length, as ours are after step 2, the denominator is 1 and **cosine similarity is simply the dot product**. That is why the script can compute the similarity of every pair of items with one sparse matrix multiplication, `X @ X.T`: `X` has one row per item, and the product is a big table with the similarity of every item to every other item.

Why angle and not Euclidean distance (the straight-line distance between the two arrow tips)? A text repeated twice gives a vector twice as long but pointing the same way, as in our $(1, 2, 0)$ and $(2, 4, 0)$ example. The tips are far apart, yet the content is the same, and cosine correctly calls it identical.

Step 4: the user profile

The profile is **a taste summary of one shopper, written as a vector in the same word-space as the items**. Items now live in a **vector space** (just "the place where all these vectors sit"). A user is described by what they did to items, so the natural profile is a **weighted average of the vectors of the items they interacted with**.

A weighted average is like a course mark where the exam counts more than a quiz. If a quiz scores 80 with weight 1 and an exam scores 90 with weight 5, the final mark is $(1 \times 80 + 5 \times 90)/(1 + 5) = 88.3$: pulled towards the exam because it weighs more. In the same way, the items that matter most pull the profile towards themselves. Two kinds of weight apply to each event (a click, an add-to-cart or an order):

- **Strength.** In this repo a click counts 1, an add-to-cart 3 and an order 5 (`EVENT_WEIGHT` in `recommender/recsys/schema.py`). Buying something is stronger evidence of taste than glancing at it.
- **Recency.** Taste drifts; a click from last week matters more than a click from last month. We apply exponential decay with a half-life h of 7 days: an event of age a days is multiplied by $2^{-a/h}$.

What is **exponential decay with a half-life**? It means an event loses *half* of its weight every h days. With $h = 7$:

Age of the event	Multiplier $2^{-a/7}$
today (0 days)	1
7 days	0.5
14 days	0.25
21 days	0.125

So the weight fades smoothly: an order placed a week ago (strength 5) counts as 5 × 0.5 = 2.5, instead of vanishing abruptly or counting at full strength forever.

Now the formula, symbol by symbol. E_u is the set of user u's events. For one event e: w_e is its strength, a_e is its age in days, and $\hat{\mathbf{x}}_{i(e)}$ is the unit-length vector of the item $i(e)$ it touched. The top of the fraction adds up all the item vectors, each scaled by its strength and freshness; the bottom adds up those same scales. The profile is

$$\mathbf{p}_u = \frac{\sum_{e \in E_u} w_e \, 2^{-a_e/h} \, \hat{\mathbf{x}}_{i(e)}}{\sum_{e \in E_u} w_e \, 2^{-a_e/h}}, \qquad \hat{\mathbf{p}}_u = \frac{\mathbf{p}_u}{\|\mathbf{p}_u\|_2}.$$

In words: multiply each touched item's vector by "strength × freshness", add the results, divide by the total of "strength × freshness", then scale the result to length 1 (the second equation).

The denominator makes it a proper average, so a user with 100 events does not get a bigger vector than a user with 5. The decay function is the same one the repo already uses for its "recently viewed" retriever (a retriever is the quick first step of a recommender that picks a shortlist of candidate items):

recommender/recsys/candidates.py · _decay

```
def _decay(age_days: pd.Series) -> pd.Series:
    return np.power(0.5, age_days / HALF_LIFE_DAYS)
```

Step 5: scoring and ranking

Now we **score** items: we give every item a number saying how well it fits this user's taste. We use cosine similarity between the profile and the item. For every available item j (one that is in stock at the cutoff), the score for user u is

$$\text{score}(u, j) = \hat{\mathbf{p}}_u \cdot \hat{\mathbf{x}}_j.$$

Sort descending and keep the top 50 per user. Users with no history have an all-zero profile, so every item scores exactly 0, a complete tie. The script **breaks the tie** with a tiny popularity term (a very small extra number added to the score, which only matters when everything else is equal), so these users get the `pop_7d` list: the globally most popular items of the last 7 days, from chapter 3.

A worked example by hand

Let us run the whole thing on a toy catalogue of four items, small enough to check with a pocket calculator. The vocabulary has four words and each item has just a few of them:

Item	Text	aurora	coffee	sneakers	volt
A	`aurora coffee`	1	1	0	0
B	`volt coffee`	0	1	0	1
C	`aurora sneakers`	1	0	1	0
D	`volt sneakers sneakers`	0	0	2	1

Every word occurs in exactly two of the four items, so $\text{idf} = \ln(5/3) + 1 \approx 1.511$ for all of them and cancels out after normalisation. After L2 normalisation:

- A $= (0.707,\ 0.707,\ 0,\ 0)$ and B $= (0,\ 0.707,\ 0,\ 0.707)$ and C $= (0.707,\ 0,\ 0.707,\ 0)$ (coordinates in the order aurora, coffee, sneakers, volt).
- D has raw TF-IDF $(0,\ 0,\ 2 \cdot 1.511,\ 1.511)$, length $1.511\sqrt{5}$, so $\hat{\mathbf{x}}_D = (0,\ 0,\ 0.894,\ 0.447)$. The word repeated twice dominates the vector.

Cosine similarities follow directly. A and B share only "coffee": $0.707 \times 0.707 = 0.5$. A and C share only "aurora": also 0.5. A and D share nothing: 0. C and D share "sneakers": $0.707 \times 0.894 = 0.632$.

Now a user clicked item A today (strength 1, age 0) and ordered item C seven days ago (strength 5, decay $2^{-7/7} = 0.5$). The effective weights are $1 \times 1 = 1$ and $5 \times 0.5 = 2.5$, sum 3.5. The profile is

$$\mathbf{p} = \frac{1 \cdot \mathbf{x}_A + 2.5 \cdot \mathbf{x}_C}{3.5} = (0.707,\ 0.202,\ 0.505,\ 0),$$

and after normalisation $\hat{\mathbf{p}} = (0.793,\ 0.226,\ 0.566,\ 0)$. The scores are A 0.721, B 0.160, C 0.961 and D 0.506. C wins because it carries both of the user's strongest words; D outranks B because the sneakers signal is strong while B shares only the weak "coffee" signal. (A and C are already seen; a production system filters them out, the simple baseline here does not.)

How to read this result: the profile $(0.793,\ 0.226,\ 0.566,\ 0)$ says "mostly `aurora`, then `sneakers`, a little `coffee`, nothing about `volt`". It is a readable taste summary, and each item's score is just how closely its own word-list lines up with it.

Code walkthrough

The full script is `book/code/ch04_content.py`; the snippets below are included straight from it and run top to bottom.

Preparing the item text

The script starts by loading the shared simulated data through `common.py`, so every algorithm in the book sees exactly the same history, **cutoff** (the moment in time we freeze; the model may only see what happened before it) and eval users (the shoppers on whom we measure the results). It then joins each item to its category and parent category names and builds the text:

book/code/ch04_content.py · data_prep

```
hist = common.history()
avail = common.available_items()
users = common.eval_users()
cats = hist.categories.set_index("category_id")

items = hist.items.copy()
items["category_name"] = items["category_id"].map(cats["name"])
items["parent_name"] = items["category_id"].map(cats["parent_name"])
```

Then the text column is built:

book/code/ch04_content.py · item_text

```
items["text"] = items["title"] + " " + items["category_name"] + " " + items["brand"] + " " +
items["parent_name"]
```

Note that `hist` is `History.at(tables, CUTOFF)`: only the data visible at the cutoff. Items listed after the cutoff do not exist for this model, which keeps the evaluation honest.

Fitting TF-IDF

book/code/ch04_content.py · tfidf

```
vec = TfidfVectorizer(lowercase=True)
X = vec.fit_transform(items["text"])  # rows are L2-normalised already
print(f"\nitems: {X.shape[0]}, vocabulary: {X.shape[1]} terms, non-zeros per item: {X.nnz /
X.shape[0]:.1f}")
idf = pd.Series(vec.idf_, index=vec.get_feature_names_out()).sort_values()
print("\nLowest IDF (common, uninformative):")
print(idf.head(5).round(3).to_string())
print("Highest IDF (rare, discriminative):")
print(idf.tail(5).round(3).to_string())
item_row = {it: i for i, it in enumerate(items["item_id"])}
```

`fit_transform` does three jobs at once: it learns the vocabulary and the document frequencies, computes the IDF for each word, and returns a sparse matrix `X` with one L2-normalised row per item. `item_row` maps an item id to its row number so we can look items up later. A **sparse matrix** is a table that stores only its non-zero cells. Keeping it sparse matters: with a real vocabulary of tens of thousands of words, a table with a cell for every item-word pair would be unmanageable, but each item only touches a handful of columns.

Building profiles with sparse algebra

This is the heart of the script, and it avoids any Python loop over users:

book/code/ch04_content.py · profile_build

```
pos = hist.interactions()
pos = pos[pos["item_id"].isin(item_row)]
pos = pos.assign(s=pos["w"] * np.power(0.5, pos["age_days"] / HALF_LIFE))
uidx = {u: i for i, u in enumerate(users)}
p = pos[pos["user_id"].isin(uidx)]
p = p.groupby(["user_id", "item_id"], as_index=False)["s"].sum()
W = sp.csr_matrix((p["s"], (p["user_id"].map(uidx), p["item_id"].map(item_row))), shape=
(len(users), X.shape[0]))
wsum = np.asarray(W.sum(axis=1)).ravel()
```

The last line of the profile construction is the normalisation step:

book/code/ch04_content.py · profile_norm

```
profile = normalize(sp.diags(1 / np.where(wsum > 0, wsum, 1)) @ W @ X)
```

`hist.interactions()` returns the click, add-to-cart and order events with a column `w` holding the strength weight:

recommender/recsys/data.py · History.interactions

```
def interactions(self) -> pd.DataFrame:
    """Positive actions (click/cart/order) with their strength weight."""
    pos = self.events[self.events["event_type"].isin(schema.EVENT_WEIGHT)].copy()
    pos["w"] = pos["event_type"].map(schema.EVENT_WEIGHT)
    return pos
```

The script multiplies `w` by the decay factor to get the effective weight `s`, then sums the effective weight per (user, item) pair. These sums fill a sparse **user × item weight matrix** `W`. Then the formula for the profile becomes pure matrix algebra:

- `W @ X` computes, for every user, $\sum_e s_e \, \hat{\mathbf{x}}_{i(e)}$, a weighted sum of item vectors.
- `sp.diags(1 / wsum)` divides each user's row by their total weight, giving the weighted average.
- `normalize(...)` rescales each profile to unit length, so the final dot product is a true cosine.
- `np.where(wsum > 0, wsum, 1)` is a guard: a user with no positive events has `wsum == 0`, and dividing by zero would produce `inf`. Replacing zero by one leaves their all-zero row at zero.

Scoring, tie-breaking and evaluation

book/code/ch04_content.py · scoring

```
item_ids = avail["item_id"].to_numpy()
Xa = X[[item_row[i] for i in item_ids]]
S = np.asarray((profile @ Xa.T).todense())
pop7 = candidates.item_popularity(hist, 7).reindex(item_ids).fillna(0).to_numpy()
S = S + 1e-6 * pop7 / max(pop7.max(), 1e-9)  # cold users: tie-break by popularity
top = np.argsort(-S, axis=1, kind="stable")[:, :TOP_N]
```

`profile @ Xa.T` produces the full user × available-item score matrix in one product. Only items that are in stock at the cutoff (`avail`) are scored. The popularity tie-break is scaled to 10^{-6} (one millionth) so it can only separate scores that are otherwise equal: it decides among cold users (all zeros) and among near-duplicate items, but never overrides a real content difference. The final `argsort` returns item positions sorted by score, from which we pick the top 50 per user, and `common.evaluate` computes exactly the same metrics used for all other chapters.

Run it

From the `book/` directory:

```
uv run python code/ch04_content.py
```

The script first prints the vocabulary statistics and IDF extremes:

TERMINAL OUTPUT

```
items: 2495, vocabulary: 64 terms, non-zeros per item: 4.8

Lowest IDF (common, uninformative):
living         2.861
home           2.861
baby           2.896
mother         2.896
electronics    3.008
Highest IDF (rare, discriminative):
men            4.518
smartwatches   4.532
skincare       4.546
shampoo        4.648
sneakers       4.695
```

The documents are short (4.8 non-zero terms per item) and the vocabulary is just 64 terms, because the simulator builds titles from a template; a real marketplace has millions of words, model numbers and typos. The IDF ranking behaves as theory promises: "living" and "home" have the lowest weights, near 2.86, while "sneakers", "shampoo" and "skincare" have the highest, around 4.5 to 4.7.

Results

What the profiles look like

TERMINAL OUTPUT

```
Strongest profile terms for user 894:
diapers    0.809
baby       0.301
mother     0.301
bamboo     0.148
aurora     0.145
garuda     0.117
```

Of the 947 eval users, 919 have a usable profile; 28 have no positive history and fall back to the popularity tie-break. For user 894 the strongest term is "diapers" (0.809), followed by "baby" and "mother" (both 0.301): a parent buying baby supplies, and the profile is directly readable by a human.

How well does it rank?

TERMINAL OUTPUT

```
=== Results (cosine(profile, item), top 50 per user) ===
               recall@5  recall@10  recall@20  recall@50  order_recall@20  ndcg@20  coverage@20
method
content_tfidf    0.0589     0.0837     0.1378     0.2416           0.1834   0.0909       0.9892
```

Here is a reminder of the metrics in that table. **NDCG@20** (normalised discounted cumulative gain) is a score from 0 to 1 that rewards placing the item the shopper really bought near the *top* of their 20 recommendations. **recall@20** is the share of the items a shopper really bought that appear anywhere in the 20. **coverage@20** is the share of the whole catalogue that appears in *anyone's* top 20. For every metric, higher is better.

Chapter 3 reported `pop_7d` (most popular items of the last 7 days) at 0.0475 and `pop_in_my_categories` (most popular items inside the categories the user already shops in) at 0.1302 on the same labels.

Three conclusions:

1. **Content-based beats global popularity.** The NDCG@20 of `content_tfidf` is 0.0909 against 0.0475 for `pop_7d`, nearly double. Personal taste, even from category-and-brand words alone, carries real information.
2. **It loses to category-aware popularity.** `pop_in_my_categories` scores 0.1302, clearly above our 0.0909. Why? Both methods exploit the same fact: users shop within categories. But `pop_in_my_categories` also ranks *inside* the category by popularity, which separates good items from poor ones. Our TF-IDF vector contains only category and brand words, so every item of a category looks almost the same, and the choice among them is a coin flip (decided only by the 10^{-6} popularity tie-break).
3. **Coverage is excellent.** `coverage@20` of 0.9892 means that nearly the whole in-stock catalogue shows up in somebody's top 20, against 0.0087 for popularity, which recommends the same few dozen items to everyone. Content-based methods are a natural antidote to the "rich get richer"

effect we met in chapter 3. The coverage figure is slightly inflated by the random-ish choice among near-ties, but the direction is real.

Nearest neighbours and the "cosine = 1.0" lesson

TERMINAL OUTPUT

```
=== Five nearest items for three example items ===

Query: item 12 'Aurora Coffee & Tea Import' (Coffee & Tea)
   1.000  item 2399  Aurora Coffee & Tea Import  [Coffee & Tea]
   0.953  item 449   Aurora Coffee & Tea Lokal  [Coffee & Tea]
   0.953  item 1016  Aurora Coffee & Tea Lokal  [Coffee & Tea]
   0.952  item 1936  Aurora Coffee & Tea Lite  [Coffee & Tea]
   0.952  item 1476  Aurora Coffee & Tea Premium  [Coffee & Tea]
```

Look at the first query. "Aurora Coffee & Tea Import" has a nearest neighbour with cosine **1.000**: a different item (id 2399) with exactly the same title and category. The next ones, 0.953, differ only in the suffix "Lokal", "Lite" or "Premium", which contributes almost nothing.

This is a teaching point, not a bug. Identical text gives identical vectors, so no content-based method can tell the items apart; a content-only recommender will happily fill the top 5 slots with copies of the same product. Telling them apart needs *behavioural* signals such as price, rating and popularity.

A **heat map** is a table where each cell is coloured by its number: bright means high, dark means low. This one shows the cosine between 15 items, five from each of three categories:

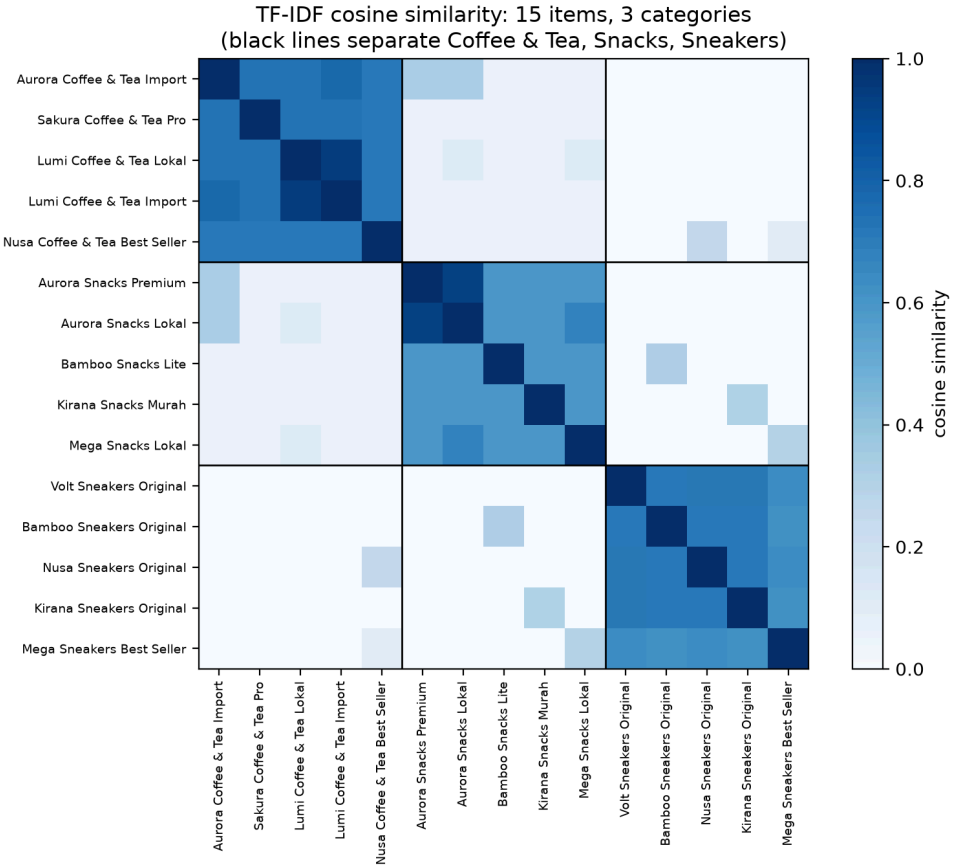


Figure 4.2: *Cosine similarity between 15 items from three categories (Coffee & Tea, Snacks, Sneakers). Black lines separate the categories. Blocks on the diagonal are bright; everything off-diagonal is near zero.*

Items of the same category are similar; items of different categories are not. Numbers confirm it:

TERMINAL OUTPUT

```
=== Mean cosine similarity within / across the three heat-map categories ===
Coffee & Tea    0.752    0.084    0.014
Snacks          0.084    0.633    0.037
Sneakers        0.014    0.037    0.681
```

The rows and columns are Coffee & Tea, Snacks and Sneakers. The mean similarity between two different items of the same category is 0.752, 0.633 and 0.681 on the diagonal; off-diagonal it is at most 0.084. The faint squares off the diagonal come from shared brand words such as "Aurora".

Did you know?

TF-IDF is over fifty years old. The idea of weighting a term by the inverse of how many documents contain it was published by Karen Spärck Jones in 1972, as "a statistical interpretation of term specificity". It has been used in search engines and recommender systems ever since, and it is still a strong baseline today.

Source: Spärck Jones, Journal of Documentation, 1972

Did you know?

Pandora's "Music Genome" is content-based filtering done by hand. Pandora's founders assembled a team of musicologists who analysed and classified songs on roughly 400 attributes, such as vocal style or rhythm, and used those attribute vectors to find songs similar to ones a listener liked, instead of relying on other listeners' histories.

Source: Westergren, NPR interview, 2011

The same idea inside the app

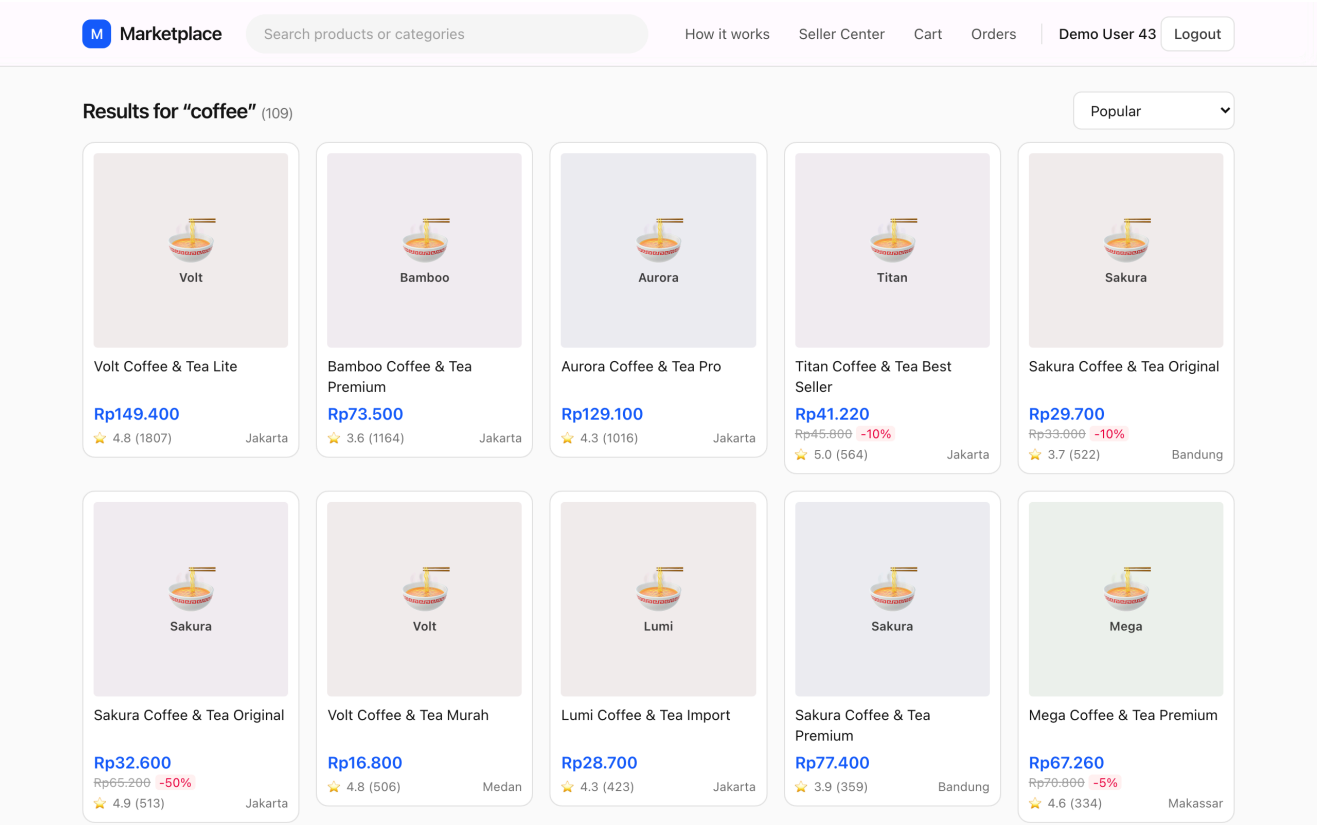


Figure 4.3: *The marketplace's search page for the query "coffee". Results are matched on title and category text and then sorted by popularity.*

The app already contains a tiny content-based component: the backend matches the typed query against product titles and category names (an `ILIKE` filter, meaning a case-insensitive "contains this text" match, in `backend/src/catalog.rs`), and the "Popular" ordering is not content-based at all. The profile approach of this chapter replaces the typed query with a *query built from the user's own behaviour*.

Pitfalls and how it is done in production

Over-specialisation

The librarian who only reads covers keeps recommending more of the same. Our profile for user 894 is dominated by "diapers", so the recommendations will be diaper-adjacent; if that user needs a stroller next, no word overlap suggests it. This is **over-specialisation**, and when it happens to a whole feed it is often called a **filter bubble**: the shopper sees only what resembles their past, and the system never learns whether they would have liked something different. The system has no **serendipity** (the pleasant surprise of finding something you did not know you wanted).

Mitigations: blend with collaborative signals such as co-visitation (items viewed together in the same session; chapter 5), re-rank for diversity with MMR (maximal marginal relevance: a re-ranking rule that penalises an item that is too similar to ones already chosen) and category caps (chapter 11), and reserve a few slots for exploration.

Shallow metadata, noisy text

Content-based methods are only as good as the content. Here titles come from a template; real seller titles are stuffed with keywords ("MURAH!!! Best Seller Sneakers Pria Wanita Original 100%"), misspell brands and mix languages. Production systems clean titles, extract structured attributes, and increasingly replace TF-IDF with learned text and image **embeddings** (vectors a model has learned so that similar things end up close together; chapter 7).

The cold-start strength

A brand-new item has no clicks, so popularity, co-visitation and matrix factorisation (chapters 3, 5, 6) cannot recommend it. But its title, brand and category exist from the first second, so a content-based scorer can place it in front of the right buyers *immediately*; we revisit this in chapter 12. A new *user* with a single click is similarly served: one click already yields a non-zero profile. Remember that content tells the system *where to show* an item; only real clicks tell it *whether the item is good*.

Scalability and tuning

Item vectors are computed offline and stored in an approximate-nearest-neighbour index (a lookup structure that finds items with similar vectors quickly, without comparing against every single item, at the cost of occasionally missing the very best match; chapter 7); the profile is looked up at request time. Tune the half-life, the event weights and the text recipe on the same time-split evaluation as everything else; you will vary the half-life in the hands-on exercise.

How this fits the repo's ranking system

The content-based score is not a retriever in the final repo pipeline; the four retrievers are `recent`, `co-visit`, `catpop` and `global_trending`. Content knowledge enters the ranker (the slower, smarter model that sorts the shortlist) differently: as *features* (input numbers the model learns from) such as `uc_click_share_30d` (the share of the user's clicks that fall in the item's category) and `item_price_vs_category` (price divided by the category median), defined in `schema.py` and computed in `features.py`. That is a common production pattern. A cheap content score becomes one input among many into a learned ranker, rather than the whole recommender. The ranker then learns, from data, how much to trust content relative to popularity or recency.

Summary

- Content-based filtering represents every item by its own attributes (text, category, brand) and recommends items that resemble what the user already liked.
- TF-IDF weights words by term frequency times inverse document frequency, so specific words ("sneakers") matter more than generic ones ("home"); L2 normalisation makes cosine similarity equal to a dot product.
- A user profile is the weighted average of the vectors of items the user touched, with weights equal to event strength times exponential time decay (half-life 7 days here).
- On the simulated marketplace the method reaches an NDCG@20 of 0.0909, almost twice the 0.0475 of `pop_7d`, but below the 0.1302 of `pop_in_my_categories`, because category and brand words cannot separate items inside a category.

- Coverage is very high (0.9892 versus 0.0087 for popularity), but near-duplicate titles have cosine similarity exactly 1.0, so content alone cannot tell copies apart; behavioural signals must.
- The main strength is cold start (new items and new users can be scored immediately); the main weakness is over-specialisation, as the method never suggests anything unlike the history.
- In the repo's final system, content signals are used as ranker features rather than as a standalone retriever.

Quiz

Q4.1 Why does TF-IDF multiply the term frequency by an *inverse* document frequency?

A. To make long documents score higher than short ones.
B. To give more weight to terms that appear in few documents and less to terms that appear in many.
C. To make all vectors orthogonal to each other.
D. To remove stop-words from the vocabulary automatically.

Q4.2 After TF-IDF vectors are L2-normalised, which statement about cosine similarity is correct?

A. It equals the Euclidean distance between the two vectors.
B. It equals the dot product of the two vectors.
C. It can be negative for any two items.
D. It depends on how many words each item's text contains.

Q4.3 A seller lists a new item at 09:00 and nobody has clicked it yet. Which method can still recommend it at 09:01?

A. Global popularity over the last 7 days.
B. Item-to-item co-visitation counted from sessions.
C. A content-based scorer that uses the item's title, brand and category.
D. A recommender using only past orders.

Q4.4 In the results, two different items have a cosine similarity of exactly 1.000. What does this tell you about their text, and why is it a problem for a content-only recommender?

Q4.5 Explain "over-specialisation" in content-based filtering and name two ways to reduce it.

Q4.6 Read this line from the script:

book/code/ch04_content.py · profile_norm

```
profile = normalize(sp.diags(1 / np.where(wsum > 0, wsum, 1)) @ W @ X)
```

What would go wrong without `np.where(wsum > 0, wsum, 1)`, and what would the profile row of a user with no positive events be after the line runs?

Q4.7 A user clicked item A today (strength 1, age 0 days) and added item B to the cart 14 days ago (strength 3). With a half-life of 7 days, what fraction of the final profile's total weight comes from item A? (Give the answer to three decimals.)

Q4.8 Two items have unit-length vectors $\mathbf{a} = (0.6,\ 0.8,\ 0)$ and $\mathbf{b} = (0.8,\ 0,\ 0.6)$. Compute their cosine similarity.

Q4.9 *Hands-on.* In `book/code/ch04_content.py`, change `HALF_LIFE = 7.0` to `1.0`, run the script, and then repeat with `30.0`. Compare the `ndcg@20` and `recall@20` in the results table with the baseline of 0.0909 and 0.1378. What do you expect to observe about how quickly the profile should forget, and does the outcome match? (Restore the value afterwards.)

Chapter 5 — Collaborative filtering

Nobody wrote the word "coffee" on that card

It is a Tuesday evening and a buyer in Surabaya opens the product page of a coffee-and-tea gift set. Underneath the photo, a "Similar items" row fills with five other coffee and tea listings. Nobody wrote a rule saying "set X goes with sets Y and Z", and nobody labelled the products. The row is there because, over the last thirty days, many shoppers who looked at this product in the same browsing session also looked at those five.

That is the central trick of **collaborative filtering** (CF): *the crowd already did the labelling for you*. Every click, cart and order is a vote. If you never look at a single title, price or photo, but you know who touched what and when, you can still say a surprising amount about which items belong together and which people share tastes.

Chapter 4's content-based filtering needed text, categories and brands. Collaborative filtering looks only at the pattern of behaviour, so it connects items a content model never would, and fails in its own way when behaviour data is thin. We build four methods on the same simulated marketplace and evaluation as Chapter 3, so numbers compare directly with the **popularity baseline** (the simplest recommender: show everyone the items most clicked lately; any smarter method must beat it). The four are user-kNN, item-kNN, session co-visitation (what the app's "Similar items" row uses, and one of the repo's four **retrievers**, the cheap first-stage components that pull a few hundred plausible candidates out of thousands of items), and a look at scalability (what happens when the shop grows 1,000 times bigger).

What you'll learn

- How a user–item matrix is built from implicit events.
- User-kNN and item-kNN, with cosine and Jaccard worked out by hand.
- How the repo's `covisitation_matrix` works and why it normalises each row.
- How to read real results: ndcg, coverage and popularity padding.
- Why item-to-item designs scale.

Words we will use

Four everyday words carry most of this chapter. Here they are in one place, with the shopper Sari and her cat-food shop as the running example.

Word	Plain meaning	Example
Event	One logged action: a click, an add-to-cart, or an order.	Sari clicks a 2 kg bag of cat food.
Implicit feedback	Taste we *infer* from what people do, because nobody typed "I like this" or gave stars.	Ordering is a stronger "like" than a click.
Session	One sitting in the shop: a burst of events by one visitor, usually within minutes.	Sari compares three cat-food brands in one evening.
Neighbour	Something (a user or an item) whose behaviour pattern looks like another's.	Two shoppers who clicked almost the same items.

Later we also use **ndcg**, **recall** and **coverage**. They are explained when we first read results, in "Reading the table".

The intuition: asking people with similar taste

Think about picking a restaurant in a new city. You could read the menu (content-based), or ask a friend: "You liked that ramen place, what else should I try?" That needs no explanation of *why* you like it, only some overlap in visited places; the friend's *other* visits are the recommendations.

That is **user-based collaborative filtering**, the original form. It was the idea behind early Usenet recommenders in the 1990s (Usenet was an early internet discussion board where people read and posted articles), and it is easy to explain to a product manager: "customers like you also bought…".

Now flip the direction. Instead of asking which *people* are similar, ask which *items* are similar, where two items are similar if the same people interacted with both. If everyone who bought a French press also bought a bag of coarse-ground coffee, those two items are neighbours, even if their titles share no words. When a user shows interest in the French press, we look up its neighbours and offer the coffee. This is **item-based collaborative filtering**. The roles are swapped, and, as we will see, that swap is what makes it practical at scale.

The third variant changes what "together" means. In user-kNN and item-kNN, two items are connected if the same *person* touched both, possibly months apart. In **session co-visitation**, two items are connected only if they were touched during the same *browsing session*, typically within minutes of each other. That is a much sharper signal of "these two are being compared right now": a shopper choosing between three phone cases looks at all three in one sitting. A shopper who bought a phone case in March and a mug in April has told us almost nothing about whether cases and mugs relate.

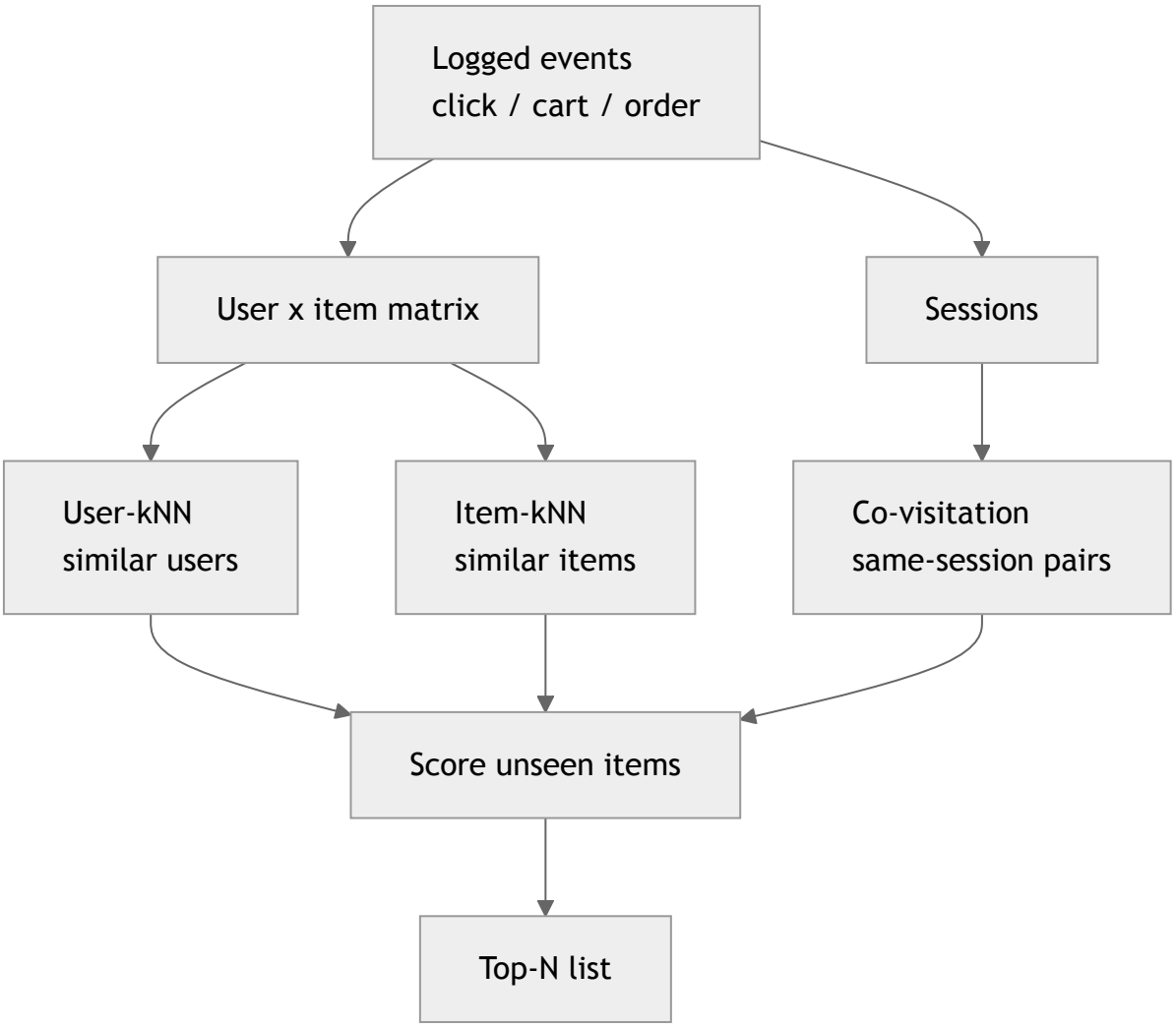


Figure 5.1: *Three ways to turn the same event log into recommendations. The first two read the whole-history matrix; the third reads session boundaries.*

The algorithm

The interaction matrix

A **matrix** is just a table of numbers: one row per user, one column per item. We call it R. The cell R_{ui} (row of user u, column of item i) says how strongly that user interacted with that item. A zero means "no recorded interaction".

Our logs are **implicit**: nobody rated anything with stars, so we turn event types into strengths. The repo does this with `schema.EVENT_WEIGHT`: a click counts 1, an add-to-cart 3 and an order 5. A cell is the sum over all of the user's events on that item. If you clicked an item twice and carted it once, $R_{ui} = 1 + 1 + 3 = 5$.

Here is a tiny matrix with Sari and two other shoppers, three items:

	cat food	cat toy	phone case
Sari	5	1	0
Budi	3	0	1
Dewi	0	0	5

Sari ordered cat food (5) and clicked a toy (1). Budi put cat food in his cart (3) and clicked a phone case (1). Dewi ordered a phone case. Read a **row** to see one person's taste; read a **column** to see who is interested in one item. User-kNN works with rows, item-kNN with columns.

Three properties matter:

- R is **sparse**: almost all cells are zero, because a user touches a few dozen items out of thousands. In our data the real matrix has 1,408 users × 2,120 items and only 0.612% of cells are non-zero (see "Run it"). Sparse is why we store only the non-zero cells.
- R is **non-negative**: a zero usually means "never saw it", not "dislikes it". A missing click is not a complaint.
- R is **built at a cutoff**: we only count events before a frozen date, so the evaluation never sees the answers (Chapter 2).

User-kNN

User-kNN in one sentence: find the shoppers whose history looks most like yours, then recommend what *they* liked and you have not seen. The name stands for *k nearest neighbours*: "the k most similar users", where k is a number we choose (say 50).

Think of asking a few friends with taste like yours for restaurant tips, and counting the closest friend's opinion more than a distant one's. That is the whole algorithm: *find similar people, weight their votes by how similar they are, add up*.

To make "similar" precise we need a number. The standard one is **cosine similarity**. Picture each user's row as a list of numbers. Cosine similarity asks: *do these two lists have the same proportions?* It ignores how much someone shops overall. A light shopper with a row $(1, 1, 0)$ and a heavy shopper with $(2, 2, 0)$ like the same things in the same proportions, so their cosine similarity is 1 (the maximum). Two shoppers with no item in common get 0. Without this "ignore the volume" property, heavy shoppers would look similar to everyone.

For a target user u:

1. Compute the similarity between u and every other user v with cosine similarity of their rows of R:

$$\text{sim}(u, v) = \frac{\sum_i R_{ui}\, R_{vi}}{\sqrt{\sum_i R_{ui}^2}\,\sqrt{\sum_i R_{vi}^2}}$$

In words: the top ($\sum_i$ means "add up over all items i") multiplies the two users' strengths item by item and adds them. Items only one of them touched contribute 0, so only shared items count. The bottom divides by the "length" of each row (square each number, add, take the square root), which removes the volume effect described above.

2. Keep the k users with the highest similarity, excluding u themselves. These are the neighbours $N_k(u)$.
3. Score each item by summing the neighbours' interaction strengths, weighted by how similar each neighbour is:

$$\hat{r}_{ui} = \sum_{v \in N_k(u)} \operatorname{sim}(u, v)\ R_{vi}$$

Here $\hat{r}_{ui}$ ("r-hat") is the predicted score for user u on item i, the sum runs over the neighbours v, and R_{vi} is how strongly neighbour v touched item i. A very similar neighbour's vote counts a lot; a barely similar one's counts little.

4. Remove items the user is not allowed to see (out of stock, already hidden) and return the top N by score.

A tiny worked example. Three items A, B, C and three existing users. The numbers in each row are (R_{vA}, R_{vB}, R_{vC}):

user	A	B	C
u_1	1	1	0
u_2	1	3	0
u_3	0	1	1

A new target user t has a click on A and a heavier interaction with C, say $(1, 0, 2)$. We want to know whether to recommend B. First the similarities. Take $u_1 = (1, 1, 0)$ as a walk-through of the formula: the top is $1 \cdot 1 + 0 \cdot 1 + 2 \cdot 0 = 1$ (only item A is shared). The length of t is $\sqrt{1^2 + 0^2 + 2^2} = \sqrt{5}$ and the length of u_1 is $\sqrt{1 + 1 + 0} = \sqrt{2}$. Dividing gives:

- $\operatorname{sim}(t, u_1) = \frac{1 \cdot 1 + 0 \cdot 1 + 2 \cdot 0}{\sqrt{5}\,\sqrt{2}} = \frac{1}{3.162} = 0.316$
- $\operatorname{sim}(t, u_2) = \frac{1 \cdot 1}{\sqrt{5}\,\sqrt{10}} = \frac{1}{7.071} = 0.141$
- $\operatorname{sim}(t, u_3) = \frac{2 \cdot 1}{\sqrt{5}\,\sqrt{2}} = \frac{2}{3.162} = 0.632$

With $k = 3$ all three are neighbours, and

$$\hat{r}_{tB} = 0.316 \cdot 1 + 0.141 \cdot 3 + 0.632 \cdot 1 = 1.372$$

Item B is the only item t has not touched, so it is our recommendation. Check the sum in words: each neighbour's vote for B (their R_{vB}: 1, 3 and 1) is multiplied by how similar they are to t (0.316, 0.141 and 0.632), then added. Notice who contributed most: u_3, the neighbour who shares the order on C, has the largest similarity, so even though their evidence for B is a single click, their vote adds 0.632. u_2 loves B (strength 3) but looks quite different from t, so the vote is shrunk to $0.141 \times 3 = 0.423$. The ordered item dominates because weights enter the cosine too.

Cosine versus Jaccard

Cosine uses the *strengths*. **Jaccard similarity** is a simpler alternative that throws strengths away and only asks *who showed up*: of everyone who touched either item, what fraction touched both? Think of

two guest lists for two parties: the Jaccard score is the number of people invited to both, divided by the number invited to at least one.

$$J(X, Y) = \frac{|X \cap Y|}{|X \cup Y|}$$

Here X and Y are *sets* (a set is just a collection with no counts). For item similarity they are the users with any interaction; for user similarity they are the items. $X \cap Y$ ("intersection") means members of *both* sets; $X \cup Y$ ("union") means members of *either*; the bars $|\cdot|$ count the members. Take items A and B from the table above. The users who touched A are $\{u_1, u_2\}$ and the users who touched B are $\{u_1, u_2, u_3\}$. The intersection has 2 users and the union has 3, so $J(A, B) = 2/3 = 0.667$. For B and C: the intersection is $\{u_3\}$, the union is $\{u_1, u_2, u_3\}$, so $J(B, C) = 1/3 = 0.333$.

Jaccard suits genuinely binary signals ("in the same session or not"), and it protects against heavy users: someone who clicked an item 500 times counts the same as someone who clicked it once. Cosine is the better default when weights carry information, as ours do: an order is worth five clicks. Both are symmetric (the similarity of A to B equals that of B to A) and blind to how popular an item is.

Item-kNN

Item-kNN in one sentence: look at the items the user already touched, find the items that are most often touched by the *same people*, and recommend those. It is the same machinery as user-kNN, rotated by 90 degrees: compare columns instead of rows. Two items are neighbours if their columns look alike, meaning the same users interacted with both.

Think of a shop assistant who has noticed over the years that customers who buy a French press nearly always come back for coarse-ground coffee. She does not know anything about either product; she only remembers who bought what. When you pick up the press, she points to the coffee. Item-kNN *is* that memory, written as a table.

Define the similarity of items i and j as the cosine of their columns:

$$\text{sim}(i, j) = \frac{\sum_u R_{ui}\, R_{uj}}{\sqrt{\sum_u R_{ui}^2}\,\sqrt{\sum_u R_{uj}^2}}$$

(same formula as before, but now the sum runs over *users* u instead of items), and score a candidate item i for user u by summing its similarity to everything the user has touched, weighted by how strongly they touched it:

$$\hat{r}_{ui} = \sum_j \text{sim}(i, j)\; R_{uj}$$

Here j runs over the items the user has touched, $\text{sim}(i, j)$ says how related the candidate i is to each of them, and R_{uj} says how much the user cares about each.

Using the same toy matrix, the columns over users (u_1, u_2, u_3) are $A = (1, 1, 0)$, $B = (1, 3, 1)$ and $C = (0, 0, 1)$. Then:

- $\text{sim}(A, B) = \frac{1+3}{\sqrt{2}\sqrt{11}} = \frac{4}{4.690} = 0.853$
- $\text{sim}(B, C) = \frac{1}{\sqrt{11}\cdot 1} = 0.302$
- $\text{sim}(A, C) = 0$, since nobody touched both.

For the target $t = (1, 0, 2)$ over (A, B, C) the score for B is $0.853 \cdot 1 + 0.302 \cdot 2 = 1.457$: the click on A contributes 0.853×1 and the heavier interest in C contributes 0.302×2. The shape of the computation differs from user-kNN: there you compare the target against every user at request time, whereas the item–item table does not depend on the target, so it is computed once ahead of time and a request is a lookup of the neighbours of the few items the user touched.

Session co-visitation

Co-visitation in one sentence: count how often two items were looked at in the same *session*, and recommend items that often appear together with what you are looking at now. It drops the user as the unit of grouping and uses the session instead. Think of a shop assistant who tidies up at closing time and notes which products customers handled during the same visit; "co-visitation" is just "visited together".

Imagine three sessions about phone shopping, all just clicks: $S_1 = \{\text{case}, \text{phone}\}$, $S_2 = \{\text{phone}, \text{charger}\}$, $S_3 = \{\text{phone}, \text{cable}\}$. The phone appears next to three different things; the case appears only next to the phone. We want a rule that captures this, and the next steps build it.

The steps are:

1. Keep only positive events (click, cart, order) from a recent window (30 days in this repo), because old habits are weaker evidence.
2. Within each session, keep one weight per item: the strongest event that happened to it. Refreshing a page ten times should not count ten times.
3. For every *ordered* pair of different items (a, b) that share a session ("ordered" means (a, b) and (b, a) are two separate pairs), add up b's weight over all the sessions where both appeared.
4. **Normalise per source item**: divide so that the outgoing weights of every item a sum to 1. In plain words: turn raw counts into *shares*, "of everything that gets looked at next to a, what share is b?":

$$w(a \to b) = \frac{\sum_{s \ni a,b} \omega_{s,b}}{\sum_{c \neq a} \sum_{s \ni a,c} \omega_{s,c}}$$

Symbols: s is a session; $s \ni a, b$ means "sessions that contain both a and b"; $\omega_{s,b}$ ("omega") is the strength of item b in session s (1 click, 3 cart, 5 order); c is any other item. The top is b's total weight next to a; the bottom is the total over *all* items next to a.

Worked example. Three sessions: S_1 has a click on A and a click on B; S_2 has a click on A, an add-to-cart on B and a click on C; S_3 has a click on A and a click on C. Look at the row for A. The raw sum for B is $1 + 3 = 4$ (a click in S_1, a cart in S_2). The raw sum for C is $1 + 1 = 2$. The total is 6, so $w(A \to B) = 4/6 = 0.667$ and $w(A \to C) = 2/6 = 0.333$.

Why normalise per source? Without it, a very popular item a would have large raw counts to everything and would swamp the rankings. Dividing makes every row sum to 1, so every item gets the same

"budget of attention" to split among its neighbours. It also makes the matrix **asymmetric** ($w(a \to b) \neq w(b \to a)$). Check it on the phone sessions: the case's row has only the phone, so $w(\text{case} \to \text{phone}) = 1$. The phone's row splits among three items, so $w(\text{phone} \to \text{case}) = 1/3 = 0.333$. That matches life: people who view a phone case nearly always also view a phone, but people who view a phone mostly do not look at that case.

To personalise, take the user's most recent items as **seeds** (starting points: "what has this person been looking at lately?"), each with a **recency score** from a time-decayed sum. Decay means older events count less; with a *half-life* of 7 days, an event loses half its weight every 7 days (the same decay as Chapter 3). The score of a candidate b is $\sum_{a \in \text{seeds}} \text{recent}(u, a) \cdot w(a \to b)$: for each seed, multiply its recency score by the seed's weight toward b, then add up over the seeds. If a user's seed A has recency score 0.8, then in the toy example B gets $0.8 \times 0.667 = 0.533$ and C gets $0.8 \times 0.333 = 0.267$.

> **Did you know?**
>
> **Did you know?** In 2003 Amazon engineers Greg Linden, Brent Smith and Jeremy York described the item-to-item collaborative filtering algorithm behind Amazon.com's recommendations: instead of comparing a customer with millions of other customers, they compare *items*, and they build the expensive similar-items table offline. The online step then only looks up similar items for the titles the user has purchased or rated, so it scales independently of catalogue size and of the number of customers. The paper contrasts this with traditional collaborative filtering, whose online computation grows with both.
>
> Source: Linden et al., IEEE Internet Computing, 2003

> **Did you know?**
>
> **Did you know?** The word "collaborative" in collaborative filtering goes back to 1994. The GroupLens project (Resnick and colleagues at MIT and the University of Minnesota) built a system that predicted how much readers would like Usenet netnews articles, based on the heuristic that people who agreed in the past will probably agree again. Users entered ratings, rating servers called "Better Bit Bureaus" correlated them, and the predicted scores were displayed in the news reader. That is user-kNN in its original habitat.
>
> Source: Resnick et al., CSCW, 1994

> **Did you know?**
>
> **Did you know?** The user-based approach was the default for years, but in 2001 Sarwar, Karypis, Konstan and Riedl compared it to item-based methods and reported that item-based algorithms provided dramatically better performance than user-based algorithms, while also providing better quality than the best available user-based algorithms (on the data sets they tested). That paper helped move the field's attention from "similar users" to "similar items".
>
> Source: Sarwar et al., WWW, 2001

Code walkthrough

All numbers in this chapter come from `book/code/ch05_cf.py`. It uses the shared setup from `book/code/common.py`: the same simulated data, the same cutoff, the same evaluation. First the ma-

trix. The helper turns the positive events from the history into a **sparse matrix** (a format that stores only the non-zero cells and their positions, instead of millions of zeros) of user × item values, each the sum of `EVENT_WEIGHT`s:

book/code/common.py · interaction_matrix

```
def interaction_matrix(hist: data.History) -> tuple[sp.csr_matrix, pd.Index, pd.Index]:
    """User x item matrix of summed EVENT_WEIGHT (click 1, cart 3, order 5)."""
    pos = hist.interactions()
    user_index = pd.Index(np.sort(pos["user_id"].unique()), name="user_id")
    item_index = pd.Index(np.sort(pos["item_id"].unique()), name="item_id")
    agg = pos.groupby(["user_id", "item_id"])["w"].sum().reset_index()
    m = sp.csr_matrix(
        (agg["w"].to_numpy(), (user_index.get_indexer(agg["user_id"]),
item_index.get_indexer(agg["item_id"]))),
        shape=(len(user_index), len(item_index)),
    )
    return m, user_index, item_index
```

`pos["w"]` is the per-event weight (1, 3 or 5). The `groupby` is the "sum all events of the same user and item" step from the algorithm section, and the `csr_matrix` constructor (CSR = *compressed sparse row*, one particular sparse format) places each sum at the right row and column. The two `pd.Index` objects are translation tables between real user and item ids (which can be any numbers) and matrix positions (0, 1, 2, …).

For cosine we need unit-length rows. "Unit length" means we divide each row by its own length so that the length becomes 1; for example the row $(3, 4)$ has length $\sqrt{9 + 16} = 5$, so it becomes $(0.6, 0.8)$. The cosine of two such rows is then just their **dot product** (multiply matching entries, add them up), which is much cheaper than the full formula:

book/code/ch05_cf.py · row_normalise

```
def row_normalise(m: np.ndarray) -> np.ndarray:
    norm = np.linalg.norm(m, axis=1, keepdims=True)
    return m / np.maximum(norm, 1e-12)
```

The `np.maximum(norm, 1e-12)` guards against dividing by zero for users or items whose row is empty (an all-zero row has length 0). After this, "similarity of everyone to everyone" becomes a single **matrix product**, a standard operation computers do very fast: entry (u, v) of the result is the dot product of the row of u with the row of v.

User-kNN, step by step

The following excerpts from `main()` are the entire user-kNN model. The variable `R` is the dense user × item matrix and `R_eval` the rows of the evaluation users:

book/code/ch05_cf.py · user_knn_sim

```
Rn = row_normalise(R)
sim_users = row_normalise(R_eval) @ Rn.T  # eval users x all users
```

Then, inside the loop over k:

book/code/ch05_cf.py · user_knn_topk

```
idx = np.argpartition(-sim_users, k, axis=1)[:, :k]
w = np.zeros_like(sim_users)
np.put_along_axis(w, idx, np.take_along_axis(sim_users, idx, axis=1), axis=1)
scores = to_avail(w @ R)
```

Read it as the four algorithm steps. The matrix product computes all cosines at once. `argpartition` finds the indices of the k largest similarities per row *without* sorting the entire row (it is linear, not $n \log n$: to find the top 50 of 1,408 you do not need to order all 1,408). The next two lines build a matrix `w` that holds the similarity at the neighbour positions and zero elsewhere. And `w @ R` is exactly the formula $\hat{r}_{ui} = \sum_{v \in N_k(u)} \text{sim}(u, v) R_{vi}$ for all users and items in one go: `@` is matrix multiplication, which performs "multiply each neighbour's row by their weight and add them" for every user. The script also sets the diagonal entry of each evaluation user to zero. The diagonal is each user's similarity to *themselves*, which is always the maximum, 1; leaving it in would make every user their own nearest neighbour and "recommend" their own history to themselves, which scores beautifully and means nothing.

Item-kNN, step by step

Item-kNN is shorter because the heavy lifting is the same product, applied to columns:

book/code/ch05_cf.py · item_knn_sim

```
Cn = row_normalise(R.T)  # items x users, unit rows
S = Cn @ Cn.T
np.fill_diagonal(S, 0.0)
```

and the scoring line:

book/code/ch05_cf.py · item_knn_score

```
scores = to_avail(R_eval @ S)  # score(u, i) = sum_j sim(i, j) * weight(u, j)
```

`R.T` swaps rows and columns (the *transpose*), so `Cn @ Cn.T` is the item–item cosine table `S` (all 2,120 × 2,120 pairs), with the diagonal zeroed so no item is its own neighbour. The scoring line applies the formula $\hat{r}_{ui} = \sum_j \text{sim}(i, j) R_{uj}$ to every user at once. This table is **dense** (every pair stored, even if similarity is 0), fine for our small shop. Production never materialises a dense table like this (see scalability).

The repo's co-visitation matrix

Now the piece that runs inside the real system. The function lives in `recommender/recsys/candidates.py`:

recommender/recsys/candidates.py · covisitation_matrix

```python
def covisitation_matrix(hist: History) -> pd.DataFrame:
    """(item_a, item_b, weight): normalised co-occurrence of positive actions within a session."""
    pos = hist.interactions()
    pos = pos[pos["age_days"] < COVISIT_WINDOW_DAYS]
    per_session = pos.groupby(["session_id", "item_id"], as_index=False)["w"].max()
    pairs = per_session.merge(per_session, on="session_id", suffixes=("_a", "_b"))
    pairs = pairs[pairs["item_id_a"] != pairs["item_id_b"]]
    m = pairs.groupby(["item_id_a", "item_id_b"], as_index=False)["w_b"].sum()
    # Normalise per source item, so very popular seeds don't dominate everything.
    m["w"] = m["w_b"] / m.groupby("item_id_a")["w_b"].transform("sum")
    return m[["item_id_a", "item_id_b", "w"]]
```

Walk through it line by line. `hist.interactions()` returns the positive events (click, cart, order) with their weights `w` and their age in days. The next line keeps only events newer than `COVISIT_WINDOW_DAYS` (30). Taking `["w"].max()` per `(session_id, item_id)` collapses repeated actions on the same item within a session into the single strongest one, so a shopper who refreshed the page ten times does not count ten times. The next step is a **self-join**: the table of events is joined to *itself* on `session_id`, which pairs every event in a session with every other event of the same session. This is how "all pairs of items in one session" are made; the `!=` filter removes the trivial pairs of an item with itself. Summing `w_b` per ordered pair gives the raw co-occurrence mass (the 4 and 2 from the worked example), and the final `transform("sum")` computes each source item's total outgoing mass and divides by it (turning 4 and 2 into 0.667 and 0.333), which is the normalisation from the algorithm section.

The self-join is the cost driver: a session with n items produces $n(n-1)$ rows (each of the n items pairs with the other $n-1$), so a bot with 5,000 items in one session would produce about twenty-five million.

Turning the matrix into per-user candidates is the job of `covisit`:

recommender/recsys/candidates.py · covisit

```python
def covisit(hist: History, seeds: pd.DataFrame) -> pd.DataFrame:
    top_seeds = _top_per_user(seeds, "cand_recent_score", N_SEEDS)
    m = covisitation_matrix(hist)
    j = top_seeds.merge(m, left_on="item_id", right_on="item_id_a")
    j["s"] = j["cand_recent_score"] * j["w"]
    out = j.groupby(["user_id", "item_id_b"], as_index=False)["s"].sum()
    out = out.rename(columns={"item_id_b": "item_id", "s": "cand_covisit_score"})
    return _top_per_user(out, "cand_covisit_score", N_COVISIT)
```

`top_seeds` picks each user's ten most relevant recent items (`N_SEEDS`). The merge pairs every seed with its outgoing row of the matrix (the same "look up the neighbours" step as in the toy example), the product `cand_recent_score * w` weights each suggestion by how much the seed matters to the user (the 0.8×0.667 step), and the group-by sums contributions when several seeds point to the same candidate. Finally `_top_per_user` keeps the best 40 (`N_COVISIT`). Two seeds pointing at the same candidate reinforce each other, which is the right behaviour: an item co-visited with *both* of the things you looked at is a better bet than one co-visited with only one.

The seeds come from `recent`, which weights each event by `_decay`: an event t days old counts $0.5^{t/7}$ of a fresh one (Chapter 3). A 7-day-old event counts $0.5^1 = 0.5$, a 14-day-old one $0.5^2 = 0.25$.

Where the product page gets its row

The "Similar items" row from the opening story is not a different algorithm. The gRPC method `SimilarItems` calls `pipeline.similar_items`, which reads one row of the same co-visitation matrix:

recommender/recsys/pipeline.py · similar_items

```
def similar_items(tables: dict[str, pd.DataFrame], now: pd.Timestamp, item_id: int, k: int) ->
pd.DataFrame:
    """Product page "similar items": co-visited items first, then popular items of the same
category."""
    hist = data.History.at(tables, now)
    empty = pd.DataFrame({"item_id": pd.Series(dtype="int64"), "score":
pd.Series(dtype="float64"), "source": pd.Series(dtype="object")})
    if item_id not in set(hist.items["item_id"]):
        return empty
    in_stock = hist.items.loc[hist.items["in_stock"], ["item_id", "category_id"]]
    m = candidates.covisitation_matrix(hist)
    co = m[(m["item_id_a"] == item_id) & (m["item_id_b"] != item_id) &
m["item_id_b"].isin(in_stock["item_id"])]
    co = co.sort_values("w", ascending=False).head(k)
    out = pd.DataFrame({"item_id": co["item_id_b"].to_numpy(), "score": co["w"].to_numpy(),
"source": "covisit"})

    category = hist.items.loc[hist.items["item_id"] == item_id, "category_id"].iloc[0]
    pop = candidates.item_popularity(hist, 7)
    same = in_stock[(in_stock["category_id"] == category) & (in_stock["item_id"] != item_id) &
~in_stock["item_id"].isin(out["item_id"])]
    same = same.assign(score=same["item_id"].map(pop).fillna(0.0)).sort_values("score",
ascending=False, kind="stable")
    fill = pd.DataFrame({"item_id": same["item_id"].to_numpy(), "score": same["score"].to_numpy(),
"source": "same_category"})
    out = pd.concat([out, fill.head(k - len(out))], ignore_index=True) if len(out) < k else out
    return out.astype({"item_id": "int64", "score": "float64"}).reset_index(drop=True) if len(out)
else empty
```

The first half builds the co-visitation row for the page's item: rows where `item_id_a == item_id`, restricted to in-stock items, sorted by weight. The second half is a **fallback** for rarely seen items (a plan B used when the main method has too little data): if co-visitation produces fewer than `k` neighbours, fill the remaining slots with the most popular in-stock items of the *same category*. The result carries a `source` label (`covisit` or `same_category`) so that the frontend and the log can tell which mechanism produced each card.

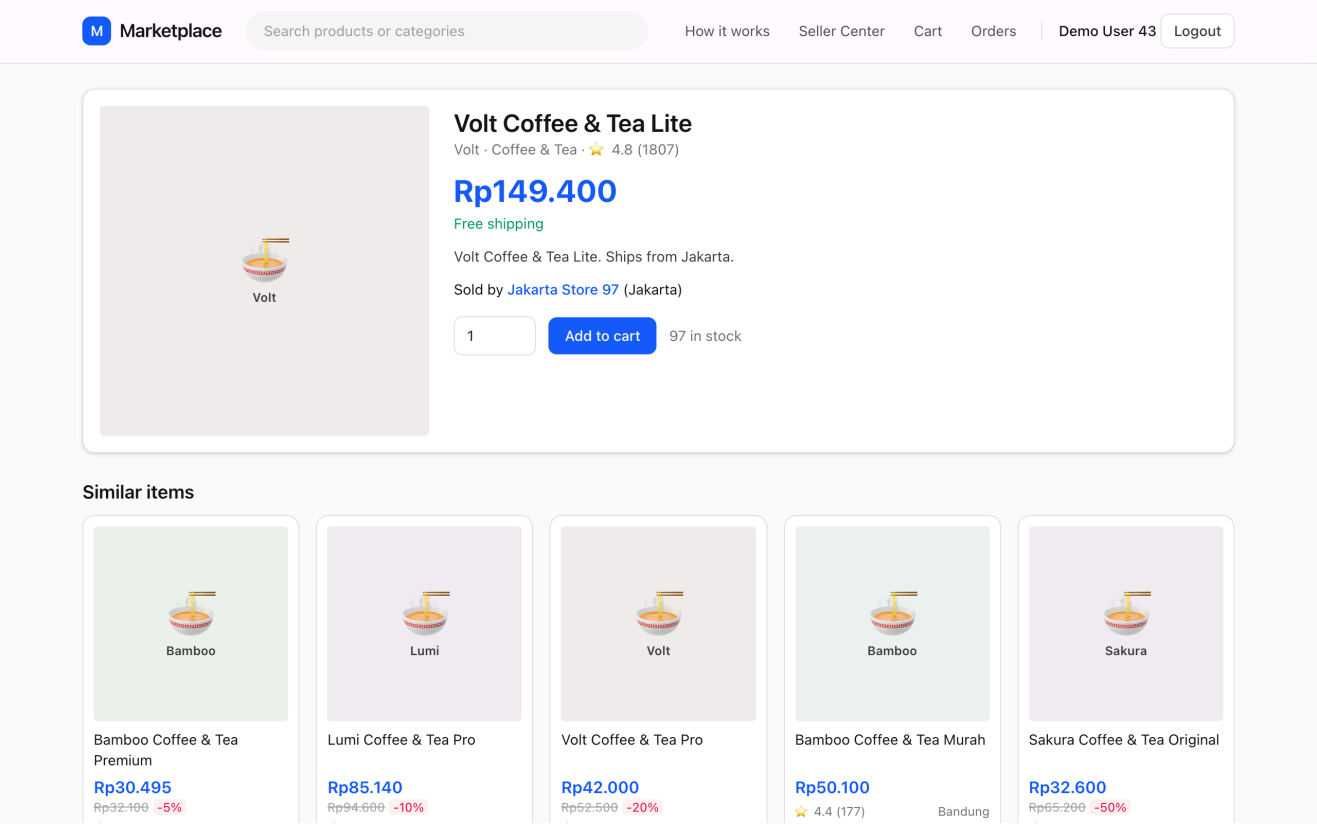


Figure 5.2: *The product page of a coffee-and-tea listing with its "Similar items" row underneath. Each card in the row comes from the co-visitation row of the item above, with same-category popular items filling empty slots.*

Run it

From the `book/` directory:

```
uv run python code/ch05_cf.py
```

The setup:

TERMINAL OUTPUT

```
=== Setup ===
interaction matrix: 1408 users x 2120 items, 18256 non-zero cells, density 0.612%
eval users: 947   recommendable (in-stock) items: 2311
eval users with history before the cutoff: 919  (the rest get 7-day popularity)
```

Next, the user-kNN sweep over the neighbourhood size k, with an example of what a neighbourhood looks like:

TERMINAL OUTPUT

```
=== User-kNN (cosine, neighbours k) ===
k=10   ndcg@20 0.1456  recall@20 0.1740
k=25   ndcg@20 0.1601  recall@20 0.2072
k=50   ndcg@20 0.1662  recall@20 0.2273
k=100  ndcg@20 0.1702  recall@20 0.2419
example: strongest neighbours of the first user with history
  user 1: user 328 (cos 0.703), user 964 (cos 0.380), user 1479 (cos 0.284)
```

Item-kNN, with the nearest items of one concrete item:

TERMINAL OUTPUT

```
=== Item-kNN (cosine between item columns) ===
item-item similarity: 2120 x 2120, mean non-zero similarity 0.0772, max 1.000
example: most similar items to item 1769 ('Sakura Earphones Original'):
  item 1064  cos 0.538  'Volt Instant Noodles Lokal'
  item 956   cos 0.497  'Bamboo Snacks Best Seller'
  item 2322  cos 0.497  'Sakura Instant Noodles Pro'
```

Co-visitation, and how much popularity padding each method needed:

TERMINAL OUTPUT

```
=== Session co-visitation (seeded with the user's decayed recent items) ===
co-visitation pairs: 13632   seeds per user <= 10

=== Padding with 7-day popularity (users/slots the method could not fill) ===
pop_7d     no-score users    0  partially padded    0  slots padded 0.0%
user_knn   no-score users   29  partially padded    1  slots padded 3.1%
item_knn   no-score users   29  partially padded    1  slots padded 3.1%
covisit    no-score users   30  partially padded  117  slots padded 8.6%
```

Finally the summary table, evaluated on the top 50 per user:

TERMINAL OUTPUT

```
=== Results (top-50 lists, evaluated at K=20) ===
           recall@5  recall@10  recall@20  recall@50  order_recall@20  ndcg@20  coverage@20
method
pop_7d       0.0310     0.0493     0.0786     0.1335           0.1066   0.0475       0.0087
user_knn     0.1230     0.1755     0.2273     0.3163           0.3456   0.1662       0.4158
item_knn     0.0460     0.1001     0.1692     0.2934           0.2772   0.0925       0.8399
covisit      0.0795     0.1319     0.2069     0.3220           0.2999   0.1197       0.6460
```

Results

The headline picture: every collaborative method beats the popularity baseline by a wide margin. Before reading numbers, a quick reminder of the three metrics (Chapter 3 has the full story). Each recommender produces a top-20 list per shopper; we then look at what the shopper *really* did after the cutoff.

- **recall@20**: of the items the shopper later interacted with, what fraction appeared in our top 20? Missing nothing = 1, missing everything = 0. (Imagine 4 true items; we caught 1 of them: recall 0.25.)
- **ndcg@20**: like recall, but it also rewards putting the right item *near the top*. A hit at rank 1 is worth more than a hit at rank 20. Scale 0 to 1, higher is better.
- **coverage@20**: what share of the catalogue appears in anyone's list. Low coverage means the same few items go to everybody.

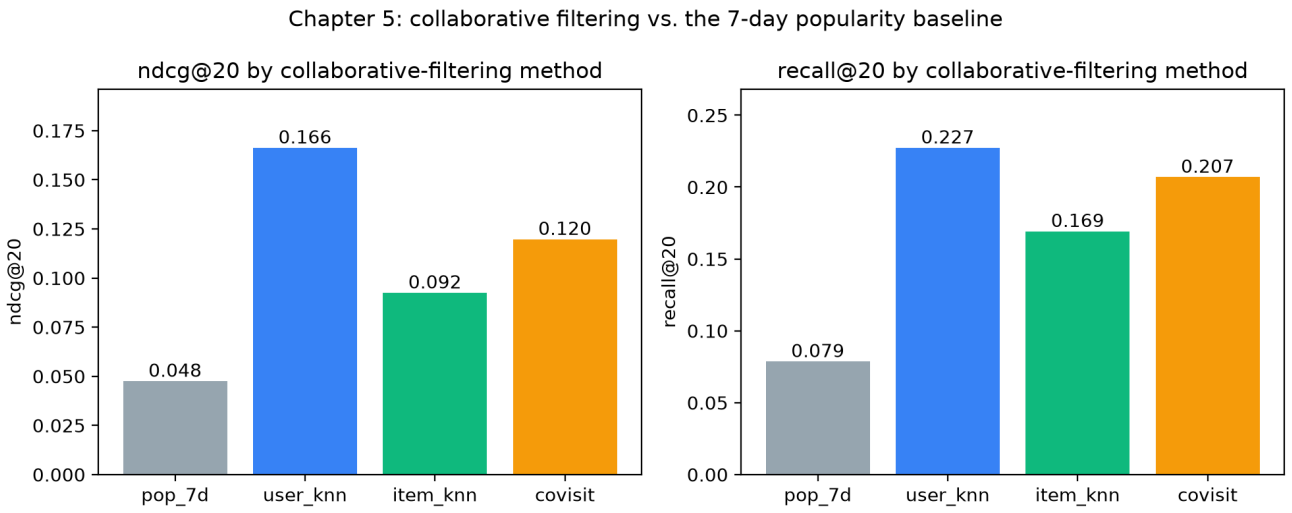


Figure 5.3: *ndcg@20 and recall@20 for the 7-day popularity baseline and the three collaborative methods.*

Reading the table

The baseline `pop_7d` (recommend the most popular items of the last 7 days to everyone) gets ndcg@20 of 0.0475 and recall@20 of 0.0786; its coverage@20 of 0.0087 means only about twenty distinct items out of 2,311 ever appear. User-kNN lifts ndcg@20 to 0.1662, about three and a half times the baseline. Even the weakest method, item-kNN at 0.0925, nearly doubles it.

User-kNN wins on ndcg. With 50 neighbours it reaches 0.1662 ndcg@20 and 0.3456 order_recall@20 (recall on orders only, the strongest signal). The sweep shows diminishing returns: more neighbours help, but each extra batch helps less; from 10 to 100 neighbours ndcg@20 climbs from 0.1456 to 0.1702. In the simulator tastes follow category preference, so users with overlapping histories do share what they do next. With only 1,408 users, scanning all of them is cheap, so this experiment hides what makes user-kNN painful at scale. (We use a fixed $k = 50$; tuning k on the evaluation users would be mild **leakage**, meaning information from the test sneaks into the choices we make, which makes the score look better than it would on new data.)

Item-kNN is weaker on ranking, stronger on breadth. Its ndcg@20 is 0.0925, but its coverage@20 is 0.8399 against user-kNN's 0.4158: item-kNN spreads its recommendations over a very large part of the catalogue. That is partly good (long-tail exposure, Chapter 1: giving rarely clicked items a chance) and partly a symptom of noise, as the example shows:

The nearest items to item 1769, 'Sakura Earphones Original', include 'Volt Instant Noodles Lokal' at cosine 0.538. Earphones and noodles are unrelated; both items have few interactions and a couple of users happened to touch both. When an item has only two or three fans, a single shared fan makes the cosine look high: a weak coincidence looks like a strong link. The mean non-zero similarity is just 0.0772 but the maximum is 1.000, almost certainly two items touched by one and the same user. Production item-kNN therefore applies **support thresholds** or **shrinkage**. A support threshold ignores pairs with too few common users (for example, fewer than 5). Shrinkage multiplies the similarity by $n_{ij}/(n_{ij} + \lambda)$, where n_{ij} is the number of users who touched both items and λ ("lambda") is a constant you choose. With an illustrative $\lambda = 4$, a pair backed by one shared user is multiplied by $1/5 = 0.2$ (cut by 80%), whereas a pair backed by 20 shared users is multiplied by $20/24 = 0.83$ (barely changed). Weak evidence is shrunk toward zero; strong evidence survives.

Co-visitation sits in the middle on ndcg and wins on one metric. It reaches 0.1197 ndcg@20, but its recall@50 is 0.3220, the highest of any method in the table (user-kNN has 0.3163). That is the pattern you want from a *retriever*: it may not order its top 5 as well as the best method, but a wider list catches slightly more of what the user eventually wants. Its recall@5 is 0.0795 against 0.1230 for user-kNN, which is exactly the ordering weakness. In the production pipeline the ordering is the **ranker's** job (a second-stage model that re-orders the retrievers' candidates); the retriever just needs to bring the right items to the table. Chapter 8 measures this on all four retrievers.

Padding: the honest part

Padding means filling empty slots in a list with something safe, here the 7-day popularity list, when the model has nothing to say. It happens for users with no history before the cutoff (the model has no row to compare) or too little. Reporting how much padding each method needed keeps the comparison honest: a method that quietly leaned on popularity for 40% of its slots would look better than it is. The padding lines tell you how often a method had nothing to say:

- user-kNN and item-kNN each had 29 evaluation users without any model score (essentially the users with no usable history before the cutoff), plus one partially padded user, and 3.1% of all slots were filled with the 7-day popularity list.
- Co-visitation had 30 users with no score and 117 who were partially padded, so 8.6% of the slots came from the popularity fallback.

Co-visitation pads more because it needs a user's *recent* items to have been co-visited inside the 30-day window: high precision, low per-user coverage. The repo never uses it alone; `trending` and `catpop` cover the gaps and the ranker sees all four scores.

Pitfalls and how it is done in production

Scalability: why item-to-item won

Count what each approach does at request time (**online**, while the shopper waits, as opposed to **offline**, a batch job run beforehand). User-kNN must compare the target to all U users: on the order of $U \times I$ operations for dense vectors (users times items), or more carefully U sparse dot products. With 1,408 users that takes milliseconds. With tens of millions of users it is impossible per request, and the common workarounds (sampling users, clustering them, reducing dimensions) all give up quality.

Item-kNN moves the expense offline. The item–item table is computed in a **batch job** (a program that runs on a schedule, for example nightly, with nobody waiting) and stored as a map from each item to its top-m neighbours (typically 50–200). At request time the system looks up the neighbour lists of the few items in the user's history and merges them. The work depends on the number of items the *user* touched, not on the catalogue or the user base. This is the property the Amazon paper highlights, and it is why so much of industry lives with "also viewed" tables.

The offline computation still needs care: you exploit sparsity (only pairs with a common user matter), cap history per user and sample blockbuster items. In our dense script the table is 2,120 × 2,120; with ten million items it would be a hundred trillion cells ($10^7 \times 10^7 = 10^{14}$), so production stores a sparse top-m list.

Co-visitation has the same shape, and the repo stores only pairs that actually occurred: 13,632 of them, as the output above shows. The cost driver, as mentioned, is the session self-join. In production you would cap items per session (say, the most recent 50) and window the data, which the repo does with `COVISIT_WINDOW_DAYS`.

Popularity bias and gray sheep

Popular items appear in many neighbour lists, get recommended more, and collect more clicks, which makes them appear even more (a feedback loop called **popularity bias**); normalisation damps this but does not remove it (Chapter 12). Users whose taste overlaps with nobody (the **gray sheep**, a user with unusual taste who matches no group) get weak neighbourhood signals, and popularity padding takes over.

Evaluation traps

- **Self-similarity.** Forgetting to remove the user's own row from their neighbour list makes the model recommend the user's own history: excellent numbers, worthless model.
- **Tuning on the test users.** The neighbourhood size k, the shrinkage constant and the number of seeds should be tuned on a **validation window** (a separate slice of time held out just for choosing settings) before the evaluation window, so the final test stays untouched.
- **Cold start.** A *new* item has no column in R and no session pairs, and a new user has an empty row, so CF has nothing to say (Chapter 12). This is why the repo combines four retrievers.

What production systems actually do

Few large systems run pure user-kNN. Co-visitation or item-to-item tables, plus learned **embeddings** (compact number-lists that represent users and items; Chapters 6 and 7), act as cheap high-recall **retrievers** ("recall" here means they rarely miss good items); a learned ranker orders the union (Chapters 9 and 10). Here `cand_covisit_score` is one of the ranker's input columns.

> **Tip**
>
> When you build a co-visitation table for real traffic, keep the raw counts and the normalised weights as separate columns. Normalised weights are what you rank by, but a pair with a weight of 0.5 backed by two sessions is far less trustworthy than one backed by two thousand. A minimum-count threshold, applied before normalisation, is the cheapest quality upgrade you can make.

> **Try it yourself**
>
> Open `book/code/ch05_cf.py`, find the line `for k in (10, 25, 50, 100):` and change the neighbourhood sizes. Re-run it and look at the ndcg@20 column. How far can you push k before the curve stops improving? See Quiz question Q5.10 for the full exercise.

Summary

- Collaborative filtering recommends from behaviour alone, so it connects items a content model would not; zeros in the implicit matrix mean "unseen", not "disliked".

- User-kNN uses the similarity-weighted votes of the k nearest users; item-kNN uses similarity to items the user touched; both rest on cosine (or Jaccard for binary signals).
- All three beat 7-day popularity (ndcg@20 0.0475): user-kNN 0.1662, co-visitation 0.1197, item-kNN 0.0925. Item-kNN has the broadest coverage (0.8399) but noisy similarities where few users overlap.
- Session co-visitation is item-to-item CF with sessions as the grouping unit; the repo normalises each source item's outgoing weights to sum to 1, which damps popular items and makes the matrix asymmetric.
- Co-visitation has the best recall@50 of all methods in the table (0.3220), which is what a retriever needs, but leans on popularity padding for users with little recent history (8.6% of slots).
- Item-to-item designs scale because the expensive table is computed offline and the online step depends only on the size of one user's history.
- In production, collaborative signals are retrievers and ranker features, not the final word on ordering.

Quiz

Q5.1 In user-based collaborative filtering, which assumption does the recommendation rest on?

A. Items with similar titles are liked by the same people.
B. Users who interacted with many of the same items in the past will probably like similar items in the future.
C. Popular items are always better than unpopular ones.
D. A user's taste does not change over time.

Q5.2 Why does the Amazon-style item-to-item design scale to catalogues and customer bases of tens of millions, while classic user-kNN does not?

A. It uses a more accurate similarity measure.
B. It needs no interaction data at all.
C. It precomputes the expensive similar-items table offline, so the online lookup depends only on the number of items in the user's history.
D. It stores a full dense user × item matrix in memory for fast access.

Q5.3 Which situation is the best fit for Jaccard similarity rather than cosine similarity on the raw weighted matrix?

A. The matrix cells hold carefully tuned weights where an order is worth five clicks and you want them to matter.
B. The signal is binary (a pair is "in the same session" or not) and heavy users should not dominate through large counts.
C. You want a similarity that is not symmetric.
D. You need to compare vectors of different dimensions.

Q5.4 Explain in two or three sentences why `covisitation_matrix` divides each pair's weight by the total outgoing weight of the source item. What would happen to the candidate lists if it did not?

Q5.5 In the results, item-kNN's closest items to 'Sakura Earphones Original' include noodles. Give the likely cause and name one fix that production systems use.

Q5.6 (Code reading.) In `covisitation_matrix`, the line `per_session = pos.groupby(["session_id", "item_id"], as_index=False)["w"].max()` is executed before the self-merge. What does it do, and what would change if `.max()` were replaced by `.sum()`?

Q5.7 (Numeric.) Item X was touched by three users with strengths $(2, 0, 1)$ over users (u_1, u_2, u_3); item Y with strengths $(1, 1, 0)$. Compute the cosine similarity of X and Y, and the Jaccard similarity of the sets of users who touched them. Give both to three decimal places.

Q5.8 (Numeric.) A user has two seed items. Seed A has recency score 0.5 and seed E has recency score 1.0. In the co-visitation matrix, $w(A \to C) = 0.3$ and $w(E \to C) = 0.4$. What is the candidate score of item C for this user, using the formula in `covisit`?

Q5.9 Based on the results table, which statement is correct?

A. user_knn has the highest ndcg@20 and also the highest coverage@20.
B. item_knn has the highest coverage@20, but lower ndcg@20 than both user_knn and covisit.
C. covisit has the highest recall@5.
D. pop_7d beats item_knn on ndcg@20.

Q5.10 (Hands-on.) In `book/code/ch05_cf.py`, change the neighbourhood sweep `for k in (10, 25, 50, 100):` to `for k in (50, 100, 200, 400):` and run the script from a scratch copy (so you do not overwrite the committed results). Compare the ndcg@20 and recall@20 printed for each k. What do you observe about the effect of increasing k beyond 100, and why?

Chapter 6 — Matrix factorisation

The shopper nobody resembles

Rina has clicked a ceramic pour-over set, a hand grinder and paper filters, plus, on a slow Tuesday, wired earphones. The user-based recommender from the previous chapter looks for neighbours (shoppers with similar click histories) who clicked the *same* items, and in a catalogue of two thousand products almost nobody did. This is the sparsity problem of Chapter 1 (the click table is almost entirely empty): neighbourhood methods compare raw rows of that nearly empty table, so two shoppers with the same taste but no shared SKU (stock-keeping unit, a product's unique code) have cosine zero (cosine similarity, a 0-to-1 score of how much two rows overlap, is zero when they share nothing).

Matrix factorisation asks instead: *what small set of hidden tastes could explain all these clicks at once?* Each shopper becomes a short vector of taste strengths, each product a short vector of "how much of each taste I contain", and a pair's score is their dot product. We build it in plain numpy so nothing hides behind a library call.

Think of it this way. Instead of recording every one of Rina's clicks, we write a short "taste card" for Rina, such as "mostly coffee gear, a little gadgets", and a taste card for every product. If Rina's card and a product's card point the same way, we recommend it. Finding those cards automatically is the whole technique.

What you'll learn

- What a *latent factor* is, and why compressing a giant sparse matrix into two thin matrices produces useful recommendations.
- How the classic explicit-rating factorisation (the one that powered the Netflix Prize era) is trained with stochastic gradient descent, and why it is a poor fit for clicks.
- How implicit-feedback ALS reframes clicks as *preference plus confidence*, and how to derive its closed-form per-row solve from scratch.
- How BPR optimises ranking directly with a pairwise loss, with its gradient worked out by hand.
- How to read the loss curves, tune `factors`, `reg`, `alpha`, `lr`, and compare both models against the baselines from Chapters 3 and 5 on exactly the same labels.

Words you will meet. Each is explained again where it first matters; this table is only a first map.

Word	Plain meaning	Tiny example
Vector	A short list of numbers	Sari's taste card $(0.9, 0.1)$
Dot product	Multiply two vectors position by position, then add	$(0.9, 0.1) \cdot (1.0, 0.0) = 0.9 \times 1.0 + 0.1 \times 0.0 = 0.9$
Parameter	A number the model learns by itself	Every number on every taste card
Loss	A single score for "how wrong is the model right now"; training tries to make it small	Predicted 3.5 when the truth is 5
Hyperparameter	A dial *you* set before training starts; the model does not learn it	How many numbers each card has
Epoch / iteration	One full pass over the training data	Reading every shopper's clicks once

Intuition: the taste dimensions nobody wrote down

Imagine describing your regulars to a new barista. You would not hand over a spreadsheet with a tick per menu item; you would say "Dewi likes strong and black, Bayu likes sweet and milky". A few phrases summarise hundreds of purchases and *predict* behaviour.

Matrix factorisation automates exactly that summarising step. Those phrases ("strong-and-black", "sweet and milky") are the **latent factors**: dimensions of taste that nobody labelled in advance, discovered because they make the purchase data easier to explain. The word *latent* just means hidden. The algorithm does not know what a factor means; it only knows that giving every user a value on factor 1 and every item a value on factor 1 lets it reconstruct the click matrix better.

Why does this matter? Without hidden factors, the only way to say two shoppers are alike is to find a product both clicked. Factors let two shoppers be alike because they share a *taste*, even when they share no product.

A toy example makes the mechanics concrete. Suppose there are only two factors, and by luck we can name them: factor A is "gadget-ness" and factor B is "kitchen-and-food-ness". Every product has a value on each factor, and so does every shopper:

	factor A (gadget)	factor B (kitchen)
Items		
wired earphones	1.0	0.0
power bank	0.9	0.1
instant noodles	0.0	1.0
coffee grinder	0.4	0.8
Users		
Gadget lover	1.2	0.2
Foodie	0.1	1.1

The score is the dot product: for the gadget lover and the earphones, $1.2 \times 1.0 + 0.2 \times 0.0 = 1.20$; for the foodie and the noodles, $0.1 \times 0.0 + 1.1 \times 1.0 = 1.10$. Ranked, the gadget lover gets earphones (1.20), power bank (1.10), coffee grinder (0.64), noodles (0.20); the foodie gets noodles, grinder, power bank. The grinder is a *bridge* with a bit of both tastes, which a neighbourhood method cannot express. The foodie also never needs to click the *same* noodles as anyone else: clicks on peanuts, cooking oil and a rice cooker land the vector near $(0.1, 1.1)$, so every item with a large factor B scores high. The unit of transfer is the factor, not the product.

These short lists of numbers also have another name: **embeddings**. An embedding is a short vector of numbers that stands in for a user or an item, placed so that similar things get similar numbers. Here the "gadget lover" row and the "foodie" row are user embeddings, and each item row is an item embedding. Chapter 7 uses the word a lot, so it is worth recognising it now.

The second idea is **low rank**. A click matrix with m users and n items has $m \times n$ cells. A factorisation with f factors replaces it with two matrices that hold only $(m + n) \times f$ numbers. In this chapter's data that is the difference between roughly three million cells ($1408 \times 2120 = 2{,}984{,}960$) and $(1408 + 2120) \times 32 = 112{,}896$ parameters. Squeezing that much information through a narrow bottleneck *forces* the model to find the regularities in the data, which is why compression and generalisation are, here, the same thing.

In plain words: **low rank means few factors.** Instead of remembering every shopper's reaction to every product, we keep one short taste card per shopper and one per product (32 numbers each), and any reaction can be rebuilt from two cards. The word *parameter* here means one of the numbers on those cards, the things training must find.

The algorithm

We will walk through three models in increasing order of relevance to this repository: classic explicit factorisation (to see the shape of the idea), implicit ALS (the workhorse for click data), and BPR (the ranking-aware alternative).

Step zero: set up the notation

Let R be the $m \times n$ matrix of interactions. In our book R_{ui} is the summed event weight of user u on item i: a click counts 1, an add-to-cart 3 and an order 5, which is `schema.EVENT_WEIGHT` from Chapter 1, so a user who clicked and then bought has $R_{ui} = 6$. Cells with no interaction are zero, but, as we will see, *zero does not mean dislike*.

The model assigns every user a vector $x_u \in \mathbb{R}^f$ and every item a vector $y_i \in \mathbb{R}^f$, stored as rows of two matrices X (users by factors) and Y (items by factors). The predicted score is

$$\hat{r}_{ui} = x_u \cdot y_i = \sum_{k=1}^{f} x_{uk}\, y_{ik}.$$

Training means choosing X and Y. The three methods differ only in *what loss they minimise*.

What is a loss function? A loss function is a score for how wrong the model's guesses are; training means changing the numbers until the score is as small as possible. Imagine a dart game where you

lose points for the distance from the bullseye: a perfect throw costs 0, a wild throw costs a lot. The most common recipe is to take the gap between guess and truth and **square** it. If the model guesses 3.5 and the truth is 5, the gap is 1.5 and the loss is $1.5^2 = 2.25$. Squaring does two useful things: over-guessing and under-guessing both cost points (no cancelling), and big misses cost far more than small ones (a miss of 0.5 costs only 0.25). Without a loss there is nothing to improve against: the model would have no way to tell a good set of taste cards from a bad one.

Explicit factorisation and Funk's SGD

Suppose, for a moment, that we had star ratings, the setting of the Netflix Prize. Only the cells in the observed set $\mathcal{K}$ carry information, and we want predictions that match them while keeping the vectors small, to avoid memorising noise:

$$\mathcal{L} = \sum_{(u,i)\in\mathcal{K}} \left(r_{ui} - x_u \cdot y_i\right)^2 + \lambda \left(\|x_u\|^2 + \|y_i\|^2\right).$$

Read this formula piece by piece. $\mathcal{K}$ is the set of cells where we really saw a rating. r_{ui} is the true rating, $x_u \cdot y_i$ is the model's guess, and their gap squared is the loss from the previous paragraph, added up over all observed cells. The second part, with λ (lambda, a dial you choose), is **regularisation**: a small penalty on the *size* of the numbers on the cards, since $\|x_u\|^2$ is just "how big is this vector", squared.

Why penalise size? Because of **overfitting**. A student who memorises last year's exam answers scores perfectly on last year's paper and badly on this year's; the model can do the same, using huge, oddly specific numbers to remember each click exactly, noise included, and then predicting new clicks poorly. The penalty makes big numbers expensive, so the model prefers simple cards that capture real patterns and not one-off accidents. A larger λ means a stronger penalty.

What are a gradient and gradient descent? We still need a way to change the cards so the loss shrinks. The **gradient** says, for each number, "if you nudge this number up a little, does the loss rise or fall, and how fast?" Think of walking down a foggy hill: you cannot see the valley, but you can feel which way the ground tilts under your feet, so you take a step downhill, feel again, and repeat. That is **gradient descent**. The size of each step is the **learning rate**, written γ here. **Stochastic** gradient descent (**SGD**) feels the slope using just one rating at a time rather than all of them, which is cheap and works well in practice. A step too large overshoots the valley; too small takes forever.

Write the prediction error $e_{ui} = r_{ui} - x_u \cdot y_i$. Differentiating one term with respect to x_u gives $-2\,e_{ui}\,y_i + 2\lambda x_u$, and similarly for y_i. Folding the constant 2 into the learning rate γ, one step of stochastic gradient descent on one observed rating is:

$$x_u \leftarrow x_u + \gamma\left(e_{ui}\,y_i - \lambda x_u\right), \qquad y_i \leftarrow y_i + \gamma\left(e_{ui}\,x_u - \lambda y_i\right).$$

Notice how sensible this update is. If the model under-predicts ($e_{ui} > 0$), the user vector moves *toward* the item vector and the item vector toward the user vector, so their dot product grows. If it over-predicts, they move apart.

Worked example 1: one SGD step. Let $x_u = (2.0, 1.0)$, $y_i = (1.5, 0.5)$, $r_{ui} = 5$, $\gamma = 0.05$, $\lambda = 0$. The prediction is 3.5, so $e = 1.5$. Then $x_u' = (2.1125, 1.0375)$ and, using the old x_u, $y_i' =$

$(1.65, 0.575)$. The new prediction is $2.1125 \times 1.65 + 1.0375 \times 0.575 \approx 4.08$. The error shrank from 1.5 to about 0.92 in a single step. Repeat that for millions of observed ratings, several times over, and the vectors settle. This is essentially the procedure that Simon Funk published during the Netflix Prize, and it is the reason you will hear the method called "Funk SVD", although it is not a true singular value decomposition: a true SVD needs every cell present, and here we deliberately ignore the empty ones.

> **Did you know?**
>
> **Skipping the empty cells.** When Simon Funk described his approach in December 2006, the Netflix training data was a matrix of roughly 500,000 users by 17,000 movies, about 8.5 billion cells, of which only one in eighty-five held a rating. His key trick was to skip the empty cells entirely and run gradient descent on the observed ones, which let a laptop make a full training pass over 100 million ratings in seconds.
>
> Source: Funk, "Netflix Update: Try This at Home" (sifter.org blog), 2006

Why explicit factorisation is a poor fit for clicks

Explicit and implicit feedback. *Explicit* feedback is when users tell you directly how much they like something, such as a 1-to-5 star rating. *Implicit* feedback is what you infer from behaviour: clicks, add-to-carts, orders. Shopping data is almost entirely implicit, because shoppers rarely rate what they click.

Implicit feedback has **no observed negatives**: if Rina did not click the gaming mouse, she may dislike it or may never have seen it. Fitting only nonzero cells lets predictions drift upward; treating zeros as ratings of 0 claims most of the catalogue is hated. And the numbers mean something else: a weight of 6 (click plus order) expresses *confidence*, not the size of a preference.

Hu, Koren and Volinsky resolved both problems with one reframing. Split every cell into two quantities:

- A binary **preference** $p_{ui} = 1$ if $R_{ui} > 0$, and 0 otherwise.
- A **confidence** $c_{ui} = 1 + \alpha R_{ui}$.

Here is what that means with this chapter's `alpha = 20` (a dial you set; see the hyperparameter list below) for shopper Sari:

What Sari did to an item	R_{ui}	Preference p_{ui}	Confidence $c_{ui} = 1 + 20R_{ui}$
Never touched it	0	0	1
Clicked it once	1	1	21
Put it in the cart (weight 3)	3	1	61
Ordered it (weight 5)	5	1	101

The preference is the same (1) for all three touched rows; only the confidence grows. A shop assistant analogy fits: a customer who buys a product tells you far more than one who glanced at it, so you trust the purchase more.

The preference answers "did this user show any interest?". The confidence answers "how much should the loss care about getting this cell right?". Observed cells with a lot of evidence get large confidence; unobserved cells keep the floor value $c = 1$: we still *weakly* believe they are zeros, because on a huge

catalogue most unseen things really are uninteresting, but we believe it far less than we believe a purchase.

The new loss runs over **all** $m \times n$ cells:

$$\mathcal{L}_{\text{ALS}} = \sum_{u,i} c_{ui}\,(p_{ui} - x_u \cdot y_i)^2 + \lambda \left(\sum_u \|x_u\|^2 + \sum_i \|y_i\|^2 \right).$$

Because it sums over every cell, SGD would have to touch three million terms per epoch even though only 18,256 are non-trivial. The trick of the next section is that a different optimiser can exploit exactly that imbalance.

> **Did you know?**
>
> **A confidence-weighted factor model with a linear-time solver.** Hu, Koren and Volinsky's ICDM 2008 paper proposed treating implicit data as indications of positive and negative preference "associated with vastly varying confidence levels", together with an optimisation procedure whose cost scales linearly with the size of the data. They applied it to a recommender for television shows.
>
> Source: Hu et al., ICDM, 2008

ALS: deriving the closed-form solve

The loss $\mathcal{L}_{\text{ALS}}$ is not convex in (X, Y) jointly, because it contains the product $x_u \cdot y_i$. But fix one side and it becomes a perfectly ordinary weighted least-squares problem in the other. That observation gives the algorithm its name: **alternating least squares**. Hold Y fixed and solve for every user vector exactly; then hold X fixed and solve for every item vector exactly; repeat.

Two terms in that paragraph need a plain reading. **Least squares** means "find the numbers that make the sum of squared gaps as small as possible"; with a single unknown number you can often write down the best answer directly. **Alternating** is like tuning two knobs on a radio: hold one still, set the other to its best position, then swap roles, and repeat until neither wants to move. We hold Y still so only the user cards are unknown, which turns a hard problem into many easy ones.

Take a single user u with Y fixed. The only terms that involve x_u are

$$J(x_u) = \sum_i c_{ui}\,(p_{ui} - x_u \cdot y_i)^2 + \lambda \|x_u\|^2.$$

Set the gradient to zero:

$$\frac{\partial J}{\partial x_u} = -2 \sum_i c_{ui}\,(p_{ui} - x_u \cdot y_i)\,y_i + 2\lambda x_u = 0.$$

Expand $x_u \cdot y_i = y_i^\top x_u$ and gather the terms that contain x_u:

$$\Big(\sum_i c_{ui}\, y_i y_i^\top + \lambda I \Big) x_u = \sum_i c_{ui}\, p_{ui}\, y_i.$$

In matrix form, with C^u the diagonal matrix of user u's confidences over all items, and $p(u)$ the vector of their preferences:

$$\boxed{x_u = \left(Y^\top C^u Y + \lambda I\right)^{-1} Y^\top C^u \, p(u)}$$

This is the closed-form solve. The matrix in brackets is $f \times f$, symmetric and, thanks to λI, positive definite, so the solution is unique and each such solve can only *decrease* the loss, never increase it. That is why the training curve we will see later falls monotonically with no learning-rate tuning at all. Item vectors follow by symmetry: swap the roles of users and items and use the columns of C and P.

In plain words: this is a **closed-form** answer, a formula that gives the best x_u in one calculation, with no stepping downhill and therefore no learning rate. "Positive definite" and "monotonic" mean the formula always has exactly one answer and that each solve can only keep the loss the same or lower it. The curve in the Results section will show this.

The Gram-matrix trick

Computing $Y^\top C^u Y$ naively costs $O(f^2 n)$ per user, since C^u has n diagonal entries. With $n = 2120$ items and 1408 users that is a lot of wasted work, because almost all of those entries equal 1. Hu et al. split the matrix:

$$Y^\top C^u Y = Y^\top Y + Y^\top \left(C^u - I\right) Y.$$

The first term, the **Gram matrix** $Y^\top Y$, does not depend on the user, so it is computed once per sweep. The second term has non-zero entries only for the n_u items the user actually touched (where $c_{ui} - 1 = \alpha R_{ui} > 0$), so it costs $O(f^2 n_u)$. Likewise the right-hand side collapses to a sum over the user's touched items because $p_{ui} = 0$ elsewhere. The total cost of a sweep is therefore proportional to the number of *observed* cells, not to $m \times n$, which is what makes ALS practical on real data.

A note on the notation $O(f^2 n_u)$: it is a rough count of how many arithmetic steps the work takes, so it tells you how cost grows. $O(f^2 n)$ with $n = 2120$ items per user is a lot more work than $O(f^2 n_u)$ with only the handful of items n_u that one user touched.

Worked example 2: one ALS solve by hand. Take $f = 1$ factor, item factors $y = (1, 2, 3)$, a user who touched items 1 and 2 with $R = (1, 2)$ (item 3 untouched), $\alpha = 2$, $\lambda = 0.5$. Then $p = (1, 1, 0)$ and $c = (3, 5, 1)$. The Gram term is $Y^\top Y = 14$, the correction is $Y^\top (C - I) Y = 2 \cdot 1 + 4 \cdot 4 = 18$, so the left side is $14 + 18 + 0.5 = 32.5$ (summing $c\,y^2 + \lambda$ directly gives the same). The right side is $\sum c\,p\,y = 3 + 10 = 13$, so $x_u = 13 / 32.5 = 0.4$ and the predictions are $(0.4, 0.8, 1.2)$. With one factor the model cannot satisfy everyone, which is why real models use 32; the structure to remember is that one user is a small weighted ridge regression over the items they touched.

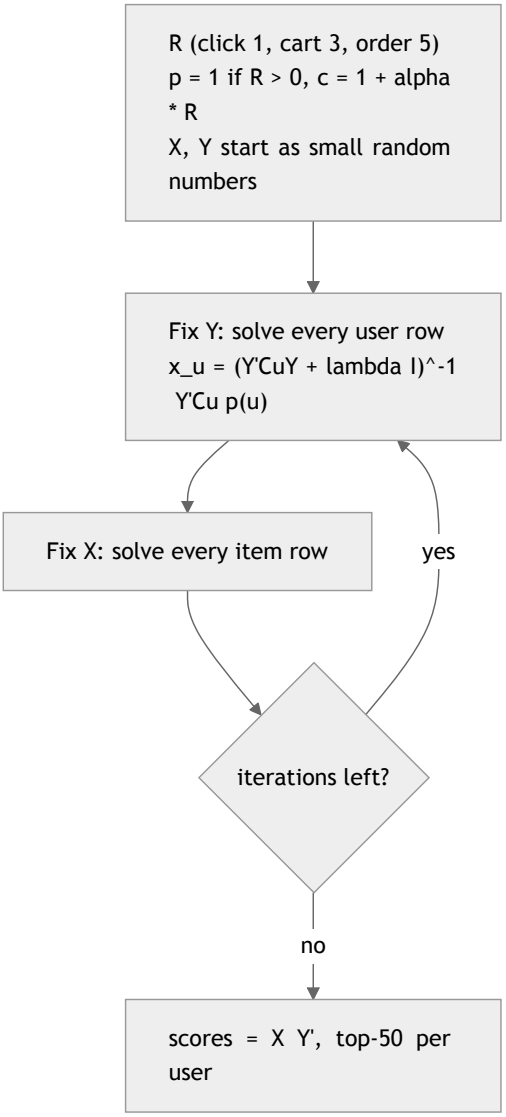


Figure 6.1: *One ALS training run. Each box in the loop is an exact least-squares solve, so there is no learning rate to tune.*

BPR: optimise the ordering, not the values

ALS minimises squared error on 0/1 targets. But we never show a user a 0/1 prediction; we show a *ranked list*. Rendle and colleagues asked: why not optimise the ranking directly? The training data becomes **triples** (u, i, j) where i is an item user u interacted with and j is one they did not. The model should score i above j.

A tiny example: Sari clicked cat food (i) and never clicked a phone case (j). BPR does not ask "what is Sari's exact score for cat food?"; it asks only "is cat food scored *above* the phone case?" and nudges the numbers until the answer is yes. (*BPR* stands for Bayesian Personalised Ranking: learning an order for each person.)

Let the gap be $\hat{x}_{uij} = x_u \cdot y_i - x_u \cdot y_j = x_u \cdot (y_i - y_j)$: how far the clicked item's score is above the random item's. We squash it with the logistic function $\sigma(z) = 1/(1 + e^{-z})$ to get the model's probability of ordering the pair correctly.

The *logistic function* σ (also called the sigmoid) squashes any number into the range 0 to 1, so it can be read as a probability. For example $\sigma(0) = 0.5$ (a coin flip), $\sigma(0.5) \approx 0.6225$ (a mild lead), $\sigma(4) \approx 0.982$ (a big lead, almost certainly right) and $\sigma(-4) \approx 0.018$ (almost certainly wrong).

The BPR objective maximises the log of that probability over sampled triples. A *prior* is a starting belief, here that parameters should be small; BPR's "Gaussian prior" on the parameters simply turns into the same size penalty (L2 regularisation) as before, and it is the "Bayesian" in the name. So read the next formula as: *push the probability that every clicked item beats its random item as high as possible, while keeping the numbers small*. The minus sign turns "maximise" into "minimise a loss":

$$\mathcal{L}_{\text{BPR}} = -\sum_{(u,i,j)} \ln \sigma\big(\hat{x}_{uij}\big) + \lambda \|\Theta\|^2.$$

This is a smooth, differentiable stand-in for AUC, the probability that a random positive outranks a random negative for the same user.

The gradient is pleasantly short. Since $\frac{d}{dz}\ln\sigma(z) = 1 - \sigma(z) = \sigma(-z)$, and $\hat{x}_{uij}$ is linear in each vector:

$$\frac{\partial \hat{x}_{uij}}{\partial x_u} = y_i - y_j, \qquad \frac{\partial \hat{x}_{uij}}{\partial y_i} = x_u, \qquad \frac{\partial \hat{x}_{uij}}{\partial y_j} = -x_u.$$

So one ascent step with learning rate η and $s = \sigma(-\hat{x}_{uij})$ is

$$x_u \mathrel{+}= \eta\big(s\,(y_i - y_j) - \lambda x_u\big), \quad y_i \mathrel{+}= \eta\big(s\,x_u - \lambda y_i\big), \quad y_j \mathrel{+}= \eta\big(-s\,x_u - \lambda y_j\big).$$

The scalar s is the key. When the pair is already ordered correctly by a wide margin, $\hat{x}_{uij}$ is large, $s \approx 0$, and the triple barely changes anything. When the pair is wrong or tied, s is near 0.5 or higher and the update is large. The model spends its effort on the pairs it currently gets wrong, which is the same intuition as the LambdaMART gradients you will meet in Chapter 10.

Worked example 3: one BPR step. Let $x_u = (0.5, -0.2)$, $y_i = (1.0, 0.5)$ (clicked), $y_j = (0.2, 1.0)$ (random), $\eta = 0.1$, $\lambda = 0$. The difference is $y_i - y_j = (0.8, -0.5)$, so the gap is $0.5 \cdot 0.8 + (-0.2)(-0.5) = 0.5$. Then $\sigma(0.5) \approx 0.6225$, the loss is $-\ln 0.6225 \approx 0.474$ and $s = 0.3775$. The updates are $x_u \approx (0.5302, -0.2189)$, $y_i \approx (1.0189, 0.4924)$ and $y_j \approx (0.1811, 1.0076)$. The new gap is about 0.557 and the loss about 0.453: one tiny step moved the pair the right way.

One practical detail. At initialisation all vectors are close to zero, so every gap is close to 0 and $\sigma(0) = 0.5$, meaning every triple has loss $-\ln 0.5 = \ln 2 \approx 0.693$. When you train BPR, a loss that *starts* near 0.693 is a good sign that the code is wired correctly.

Hyperparameters in one place

A **hyperparameter** is a dial you set *before* training; **parameters** (the numbers on the taste cards) are what training learns. **Underfitting** is the opposite of overfitting: the model is too simple to capture real patterns (a taste card with too few numbers to tell coffee from gadgets). One **epoch** is one full pass over the training data.

- `factors` f: too low underfits (tastes cannot be separated), too high overfits and slows training.
- `reg` λ: too low memorises sparse users, too high shrinks every score toward 0.
- `alpha` α (ALS): too low makes observed and unobserved cells look alike; too high lets one click dominate and collapses recommendations onto popular items.

- `iters` / `epochs`: stop when the loss plateaus; BPR also needs a sensible `lr` (too high oscillates) and enough sampled negatives.

Our scripts use `factors = 32`, `reg = 0.1`, `alpha = 20`, 15 iterations for ALS; and `lr = 0.05`, `reg = 0.01`, 30 epochs, one negative for BPR. These are defaults, not tuned optima.

Code walkthrough

All code lives in `book/code/ch06_mf.py`. It trains both models on the very same user-by-item matrix as Chapters 3 and 5 and hands the scores to the shared evaluation helper. We start with the matrix.

Building the interaction matrix

book/code/common.py · interaction_matrix

```
def interaction_matrix(hist: data.History) -> tuple[sp.csr_matrix, pd.Index, pd.Index]:
    """User x item matrix of summed EVENT_WEIGHT (click 1, cart 3, order 5)."""
    pos = hist.interactions()
    user_index = pd.Index(np.sort(pos["user_id"].unique()), name="user_id")
    item_index = pd.Index(np.sort(pos["item_id"].unique()), name="item_id")
    agg = pos.groupby(["user_id", "item_id"])["w"].sum().reset_index()
    m = sp.csr_matrix(
        (agg["w"].to_numpy(), (user_index.get_indexer(agg["user_id"]),
item_index.get_indexer(agg["item_id"]))),
        shape=(len(user_index), len(item_index)),
    )
    return m, user_index, item_index
```

`hist.interactions()` returns one row per positive event (click, cart or order) with its weight, taken straight from the `EVENT_WEIGHT` dictionary. The groupby sums them per user-item pair, so a user who clicked an item twice and bought it once has $R = 1 + 1 + 5 = 7$. We then map raw ids to dense row and column positions with `pd.Index.get_indexer` and build a CSR sparse matrix. The two returned indexes are essential later: they translate "row 17" back into "user 61".

The ch06 script densifies this with `.toarray()`: 1408 users by 2120 items of float64 is only a few tens of megabytes, so it is fine for a teaching script. At production scale you never do this.

ALS: the whole algorithm in one function

book/code/ch06_mf.py · als_implicit

```python
def als_implicit(R: np.ndarray, factors=FACTORS, reg=0.1, alpha=20.0, iters=15, seed=0):
    """Minimise  sum_ui c_ui (p_ui - x_u . y_i)^2 + reg (|X|^2 + |Y|^2)
    with p = 1[r > 0] and c = 1 + alpha * r, by alternating exact ridge solves per row."""
    rng = np.random.default_rng(seed)
    n_u, n_i = R.shape
    P = (R > 0).astype(np.float64)
    C = 1.0 + alpha * R
    X = rng.normal(0, 0.1, (n_u, factors))
    Y = rng.normal(0, 0.1, (n_i, factors))
    eye = reg * np.eye(factors)

    def solve(fixed, conf, pref):
        gram = fixed.T @ fixed  # computed once per sweep: the HKV trick
        out = np.empty((conf.shape[0], factors))
        for r in range(conf.shape[0]):
            nz = np.flatnonzero(conf[r] > 1.0)
            Fn = fixed[nz]
            cu = conf[r, nz]
            A = gram + Fn.T @ ((cu - 1.0)[:, None] * Fn) + eye
            b = Fn.T @ (cu * pref[r, nz])
            out[r] = np.linalg.solve(A, b)
        return out

    losses = []
    for _ in range(iters):
        X = solve(Y, C, P)
        Y = solve(X, C.T, P.T)
        err = P - X @ Y.T
        losses.append(float((C * err**2).sum() + reg * ((X**2).sum() + (Y**2).sum())))
    return X, Y, losses
```

Read the function top to bottom, and match each piece to the derivation.

- `P` and `C` are the preference and confidence matrices; `X` and `Y` start as small Gaussian noise (all-zero would keep every solve identical).
- `solve(fixed, conf, pref)` is the closed-form solve for every row of one side. `gram = fixed.T @ fixed` is $Y^\top Y$, computed once per sweep: that line *is* the Hu-Koren-Volinsky trick.
- Per row, `nz` finds the touched items, `A = gram + Fn.T @ ((cu - 1.0)[:, None] * Fn) + eye` is $Y^\top Y + Y^\top (C^u - I) Y + \lambda I$, and `b` is $Y^\top C^u p(u)$. `np.linalg.solve(A, b)` avoids forming an inverse.

The main loop then calls `solve(Y, C, P)` for the users and `solve(X, C.T, P.T)` for the items. Transposing C and P is all it takes to reuse the same function for the other side. After each sweep the code records the full objective with dense numpy operations, `(C * err**2).sum() + reg * (...)`, which is the loss plotted later.

One honest caveat: `C` and `err` here are dense. They are fine for a lab with three million cells but not for millions of users. Production ALS keeps R sparse throughout and never builds C.

BPR: sampling, gap and the three gradients

book/code/ch06_mf.py · bpr

```python
def bpr(R: np.ndarray, factors=FACTORS, lr=0.05, reg=0.01, epochs=30, batch=256, seed=0):
    """Bayesian Personalised Ranking: for each positive (u, i) draw a negative j and push
    x_u.y_i above x_u.y_j (maximise log sigmoid of the gap). Mini-batch SGD in numpy."""
    rng = np.random.default_rng(seed)
    n_u, n_i = R.shape
    pos_mask = R > 0
    users, items = np.nonzero(pos_mask)
    X = rng.normal(0, 0.1, (n_u, factors))
    Y = rng.normal(0, 0.1, (n_i, factors))
    losses = []
    for _ in range(epochs):
        order = rng.permutation(len(users))
        total = 0.0
        for s in range(0, len(order), batch):
            b = order[s : s + batch]
            u, i = users[b], items[b]
            j = rng.integers(0, n_i, len(b))
            for _retry in range(5):  # resample negatives that are actually positives
                bad = pos_mask[u, j]
                if not bad.any():
                    break
                j[bad] = rng.integers(0, n_i, bad.sum())
            xu, yi, yj = X[u], Y[i], Y[j]
            gap = (xu * (yi - yj)).sum(1)
            sig = 1.0 / (1.0 + np.exp(gap))  # = 1 - sigmoid(gap) = -d loss / d gap
            total += float(np.log1p(np.exp(-gap)).sum())
            gu = sig[:, None] * (yi - yj) - reg * xu
            gi = sig[:, None] * xu - reg * yi
            gj = -sig[:, None] * xu - reg * yj
            np.add.at(X, u, lr * gu)
            np.add.at(Y, i, lr * gi)
            np.add.at(Y, j, lr * gj)
        losses.append(total / len(users))
    return X, Y, losses
```

The function maps to the BPR equations one-to-one.

- `np.nonzero(pos_mask)` lists every positive pair; one epoch visits each once in mini-batches of 256. `j` is a uniform random item, redrawn up to five times if it is a real positive.
- `gap` is $\hat{x}_{uij}$ for the batch; `sig = 1.0 / (1.0 + np.exp(gap))` is $\sigma(-\text{gap})$, the step multiplier s; `np.log1p(np.exp(-gap))` accumulates the loss.
- `gu`, `gi`, `gj` are the three gradients (the minus sign on `gj` pushes the negative away). `np.add.at` is needed because a user can appear several times in a batch.

From scores to a top-50 list

book/code/dense_recs.py · scores_to_recs

```python
def scores_to_recs(scores: np.ndarray, users: np.ndarray, item_ids: np.ndarray, n: int =
common.TOP_N):
    """scores: (len(users), len(item_ids)); only score > 0 counts as 'the model has an opinion'.

    Returns (recs DataFrame, stats dict) where stats describes how much padding was needed.
    """
    scores = np.where(np.isfinite(scores), scores, 0.0)
    flag = scores > 0
    row_max = np.maximum(scores.max(axis=1, keepdims=True), 1e-12)
    pop = pop7_vector(item_ids)[None, :]
    key = flag * 2.0 + np.where(flag, scores / row_max, 0.0) + np.where(flag, 0.0, pop)
    top = np.argsort(-key, axis=1, kind="stable")[:, :n]
    n_model = np.minimum(flag.sum(axis=1), n)
    recs = pd.DataFrame({"user_id": np.repeat(users, n), "item_id": item_ids[top].ravel()})
    stats = {
        "users": int(len(users)),
        "users_without_model_scores": int((n_model == 0).sum()),
        "users_padded_partially": int(((n_model > 0) & (n_model < n)).sum()),
        "share_of_slots_from_popularity_padding": float(1 - n_model.sum() / (n * len(users))),
    }
    return recs, stats
```

Both models produce a dense `users × items` score matrix, `X[users] @ Y[items].T`; this helper turns it into the top-50 lists `common.evaluate` expects, padding users without usable history with 7-day popularity. It does not remove items the user already touched; that filter belongs to Chapter 11.

Run it

From the `book/` directory:

```
uv run python code/ch06_mf.py
```

The run takes a couple of seconds because the matrix is tiny. ALS first. Each line prints the full objective after one alternating sweep:

TERMINAL OUTPUT

```
=== ALS (implicit): factors 32, lambda 0.1, alpha 20, 15 iterations ===
trained in 1.7s
iteration  1  loss       225190.5
iteration  2  loss       147262.9
iteration  3  loss       131616.4
iteration  4  loss       124535.6
iteration  5  loss       120478.2
iteration  6  loss       117899.3
iteration  7  loss       116156.7
iteration  8  loss       114928.5
iteration  9  loss       114035.7
iteration 10  loss       113371.0
iteration 11  loss       112865.4
iteration 12  loss       112473.0
iteration 13  loss       112162.2
iteration 14  loss       111911.5
iteration 15  loss       111705.9
```

And then the opening and closing epochs of BPR. The loss reported is the mean of $-\ln \sigma(\text{gap})$ over the epoch's sampled triples, so it is a training-time quantity, not a ranking score:

TERMINAL OUTPUT

```
=== BPR: factors 32, lr 0.05, reg 0.01, 30 epochs, 1 negative per positive ===
trained in 0.8s
epoch  1  mean -log sigmoid(gap) 0.6937
epoch  2  mean -log sigmoid(gap) 0.6864
epoch  3  mean -log sigmoid(gap) 0.6771
epoch  4  mean -log sigmoid(gap) 0.6667
epoch  5  mean -log sigmoid(gap) 0.6530
epoch  6  mean -log sigmoid(gap) 0.6331
epoch  7  mean -log sigmoid(gap) 0.6061
epoch  8  mean -log sigmoid(gap) 0.5707
epoch  9  mean -log sigmoid(gap) 0.5294
epoch 10  mean -log sigmoid(gap) 0.4835
epoch 11  mean -log sigmoid(gap) 0.4398
```

TERMINAL OUTPUT

```
epoch 20  mean -log sigmoid(gap) 0.1918
epoch 21  mean -log sigmoid(gap) 0.1770
epoch 22  mean -log sigmoid(gap) 0.1681
epoch 23  mean -log sigmoid(gap) 0.1555
epoch 24  mean -log sigmoid(gap) 0.1471
epoch 25  mean -log sigmoid(gap) 0.1365
epoch 26  mean -log sigmoid(gap) 0.1320
epoch 27  mean -log sigmoid(gap) 0.1222
epoch 28  mean -log sigmoid(gap) 0.1188
epoch 29  mean -log sigmoid(gap) 0.1132
epoch 30  mean -log sigmoid(gap) 0.1094
```

Finally, how many evaluation users had no usable vector and how the two models score on the held-out week:

TERMINAL OUTPUT

```
=== Padding with 7-day popularity ===
als_implicit  users without scores   28  slots padded 3.0%
bpr           users without scores   28  slots padded 3.0%

=== Results (top-50 lists, evaluated at K=20) ===
              recall@5  recall@10  recall@20  recall@50  order_recall@20  ndcg@20  coverage@20
method
als_implicit    0.1145     0.1816     0.2715     0.3798           0.4122   0.1581       0.5431
bpr             0.0970     0.1542     0.2218     0.3274           0.3203   0.1379       0.4267
```

Results

Reading the loss curves

A **loss curve** plots the loss after each iteration or epoch. If it goes down, the model is learning; when it stops going down and flattens, we say it has **converged** (settled) and more training adds little. The "knee" is the point where a steep drop turns into a flat line.

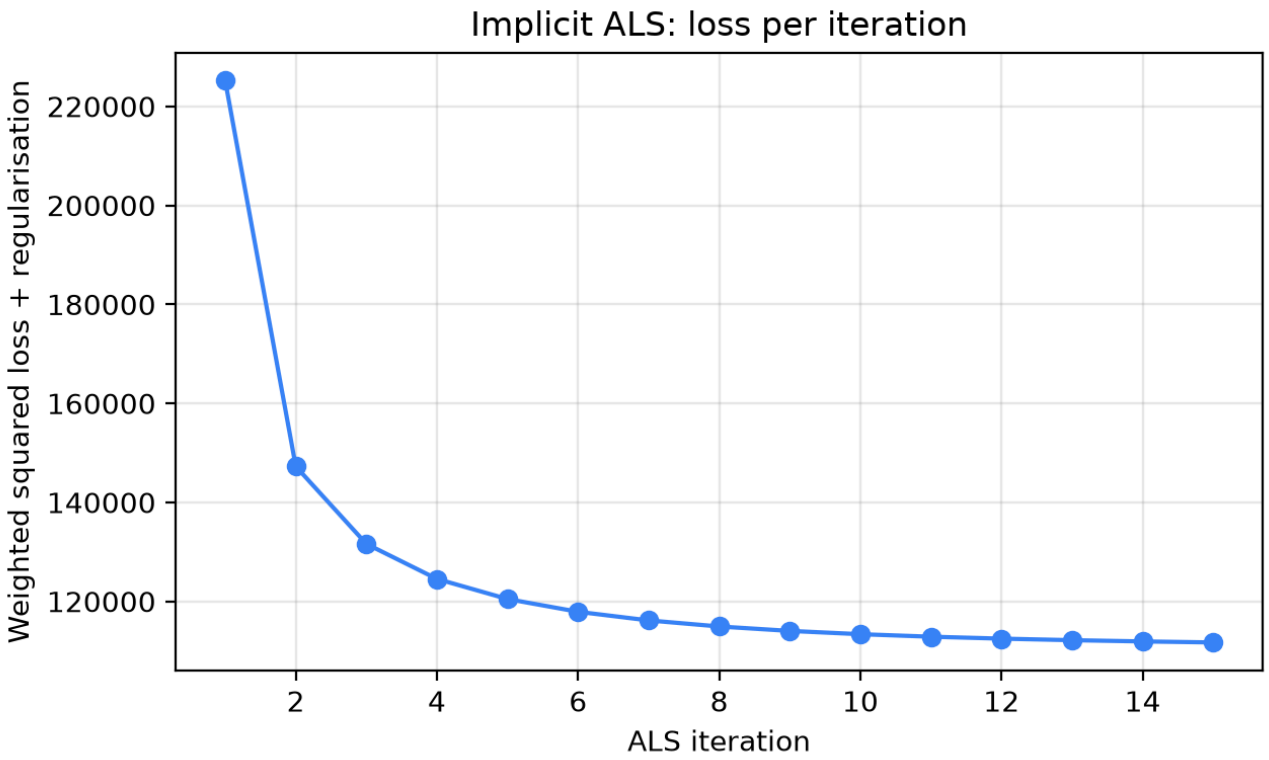


Figure 6.2: *Implicit ALS: weighted squared loss plus regularisation per iteration. The first alternating sweep removes most of the loss, then the curve flattens.*

The ALS loss falls on **every** iteration, as the derivation promised: from 225190.5 at iteration 1 to 147262.9 at iteration 2, 120478.2 at iteration 5, and 111911.5 then 111705.9 at iterations 14 and 15 (about 0.2% apart). The curve has flattened, so 15 iterations is past the knee.

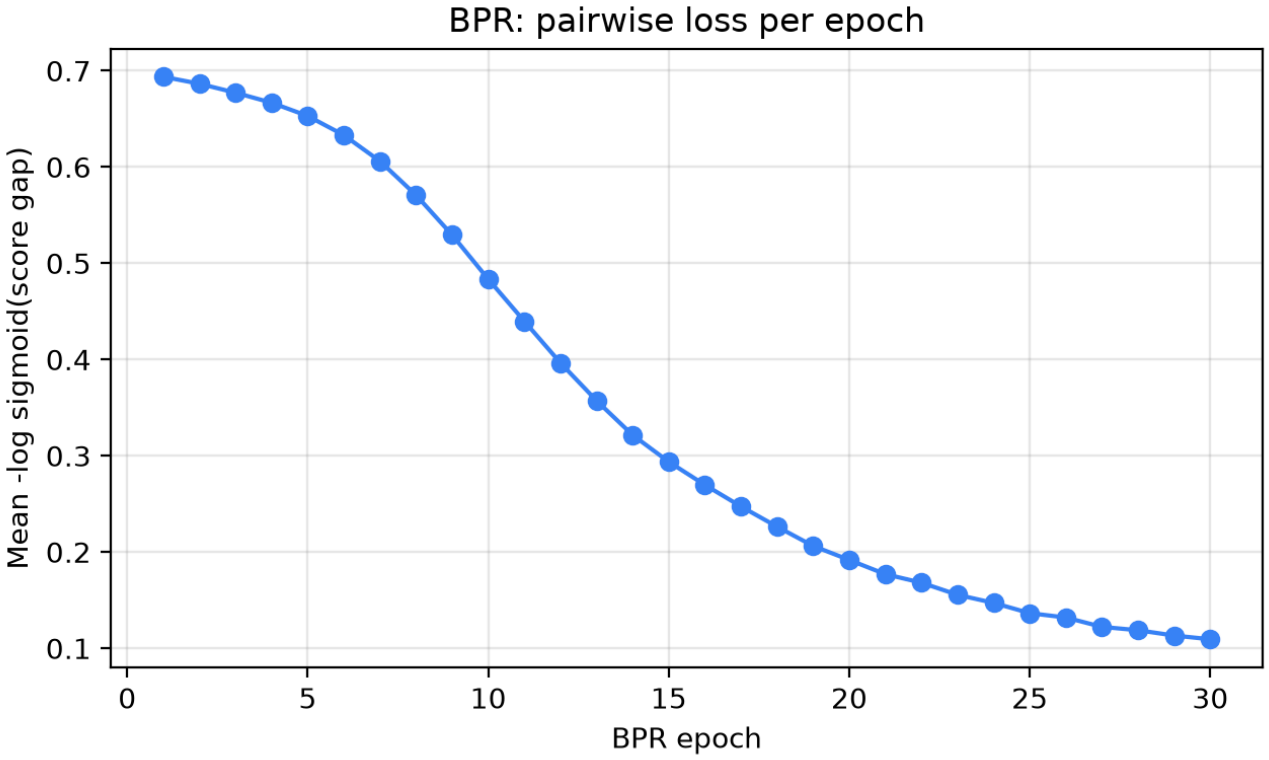


Figure 6.3: *BPR: mean pairwise loss per epoch. It starts near ln 2 and decreases slowly at first, then quickly, then slowly again.*

The BPR curve starts at 0.6937, essentially $\ln 2 \approx 0.693$: at initialisation every gap is near zero. Almost nothing happens for five epochs (0.6530 at epoch 5) because tiny vectors give tiny gradients; then the loss drops fast (0.4835 at epoch 10, 0.2937 at epoch 15) and reaches 0.1094 by epoch 30. It was **still falling** at epoch 30, so the model is not converged.

Ranking quality against everything so far

Training loss is not ranking quality, so we evaluate both models on the same held-out week and metrics as earlier chapters (the ch06 rows are in the table above). First, quick reminders of the metrics. **recall@20**: of the items the shopper really interacted with in the held-out week, what share appear in our top 20. **ndcg@20**: a score that also rewards putting those items near the top of the list rather than at position 20. **coverage@20**: what share of the catalogue appears in at least one user's top 20; low coverage means everyone gets the same items. A **candidate generator** is a model that cheaply finds a few hundred plausible items; a **ranker** then orders them carefully.

For comparison, the Chapter 5 methods and the best Chapter 3 baseline on identical labels:

TERMINAL OUTPUT

```
=== Results (top-50 lists, evaluated at K=20) ===
          recall@5  recall@10  recall@20  recall@50  order_recall@20  ndcg@20  coverage@20
method
pop_7d      0.0310     0.0493     0.0786     0.1335           0.1066   0.0475       0.0087
user_knn    0.1230     0.1755     0.2273     0.3163           0.3456   0.1662       0.4158
item_knn    0.0460     0.1001     0.1692     0.2934           0.2772   0.0925       0.8399
covisit     0.0795     0.1319     0.2069     0.3220           0.2999   0.1197       0.6460
```

TERMINAL OUTPUT

```
pop_in_my_categories    0.0926     0.1425     0.2173     0.3529           0.3050   0.1302
0.2164
```

A few conclusions follow directly from the numbers.

1. Factorisation crushes popularity. ALS has ndcg@20 0.1581 against 0.0475 for `pop_7d`, more than three times higher.

2. ALS beats BPR here, but not conclusively. ALS gets 0.1581 ndcg@20 and 0.2715 recall@20; BPR gets 0.1379 and 0.2218. BPR's loss was still falling and it has more knobs, so read this as "with these defaults ALS is better and cheaper".

3. ALS does not beat user-kNN on ndcg. User-kNN with 50 neighbours reaches 0.1662, and even `pop_in_my_categories` reaches 0.1302, close to BPR's 0.1379. On a small catalogue, neighbourhood and category signals are strong baselines.

4. The metrics disagree informatively. ALS has higher recall@20 (0.2715 vs 0.2273) and order_recall@20 (0.4122 vs 0.3456) than user-kNN but lower ndcg@20. It finds more of the right items and sorts them less sharply, which is why a candidate generator is judged by recall and a ranker by ndcg.

5. Coverage and cold users. Coverage@20 is 0.5431 for ALS and 0.4267 for BPR against 0.0087 for `pop_7d`. But 28 evaluation users had no vector (no pre-cutoff interactions) and were padded with 7-day popularity, about 3.0% of slots: factorisation cannot score a user it has never seen (Chapter 12).

Did you know?

A million dollars for ten percent. The Netflix Prize launched in October 2006 and asked the world to improve Netflix's rating-prediction accuracy by 10%. It was awarded in September 2009 to the team "BellKor's Pragmatic Chaos", whose winning submission improved the existing system by 10.06%, reported as arriving less than half an hour before the contest deadline. The team's published description of its method highlighted two families of techniques: neighbourhood models and latent-factor models of the kind in this chapter.

Source: Sarno, Los Angeles Times, 2009

Did you know?

Optimising the ranking directly. In their 2009 UAI paper, Rendle and colleagues noted that popular methods for item recommendation from implicit feedback, such as matrix factorisation and adaptive k-nearest-neighbour, "are designed for the item prediction task of personalized ranking" yet "none of them is directly optimized for ranking". Their BPR-Opt criterion, derived as a maximum posterior estimator, is trained with stochastic gradient descent using bootstrap sampling and can be applied to both model families.

Source: Rendle et al., UAI, 2009

Pitfalls and how it is done in production

Tuning on your test set. Sweeping `alpha`, `factors` and `reg` against the evaluation labels fits the evaluation data. Tune on an earlier validation window and evaluate once on the test week; we left the defaults un-tuned for this reason.

Unobserved is not negative. ALS gives blanks weight 1 and BPR samples blanks uniformly, but a great item the user never saw is still penalised. Production systems use *exposure-aware* negatives (items shown but ignored, i.e. `impression` events, which this repo logs) and popularity-weighted sampling, since uniform sampling picks rare items as negatives far more often than popular ones.

Dense matrices do not scale. Our code builds *dense* C, P and error matrices (every cell stored, even the zeros). At Shopee scale production keeps R *sparse* (only non-zero cells stored), processes users in GPU-sized blocks and *shards* X and Y (splits them across many machines); the open-source `implicit` library and Spark MLlib implement the sparse form.

Fold-in. Fold-in means computing a vector for a user *without retraining*. With Y frozen, a user's vector is a closed-form solve, recomputable in well under a millisecond from their latest clicks, so a nightly-trained model can still react to fresh behaviour (compare this repo's on-request design).

Dot product, not distance. Vector length matters for $x_u \cdot y_i$; ANN libraries (approximate nearest-neighbour search: fast, slightly inexact lookup of the closest vectors) need a maximum-inner-product variant (Chapter 7).

Where it sits in this repository. The recommender service does not use ALS or BPR today: its sources are the four retrievers of Chapter 8 and its ranker is LightGBM (Chapter 10). A natural extension is to train ALS during retrain and use the dot product as an extra candidate source and as a ranker feature (see the checklist in Chapter 14).

Summary

- Matrix factorisation gives each user and item a short vector of **latent factors** and scores a pair by their dot product, compressing about three million cells into roughly a hundred thousand parameters.
- Explicit factorisation (Funk SVD) fits observed ratings by SGD; it is a poor fit for clicks, where blanks are not zeros and weights express confidence.
- **Implicit ALS** splits each cell into preference $p = 1[R > 0]$ and confidence $c = 1 + \alpha R$ and minimises the weighted squared error over all cells.
- With one side fixed each row has a closed-form solve, $x_u = (Y^\top C^u Y + \lambda I)^{-1} Y^\top C^u p(u)$; the Gram trick makes a sweep cost proportional to observed cells, and the loss falls on every iteration.
- **BPR** pushes a clicked item above a random one, with gradient scaled by $\sigma(-\text{gap})$; a healthy run starts near $\ln 2 \approx 0.693$.
- ALS reached ndcg@20 of 0.1581 and BPR 0.1379 against 0.0475 for `pop_7d`; user-kNN's 0.1662 is still higher, although ALS has higher recall@20, so factorisation is a retrieval model, not a sharp ranker.
- 28 evaluation users had no vector and needed popularity padding; the lasting value of factors is the representation, which Chapter 7 takes further.

Quiz

Q6.1 In a factorisation model with 32 factors, what is a *latent factor*?

- A. A column of the raw interaction matrix, such as the clicks on one product.
- B. One of the learned dimensions of the user and item vectors; it is not given a name in advance and is discovered because it helps reconstruct the interactions.
- C. A hyperparameter chosen by cross-validation, such as the learning rate.
- D. A feature such as price or category that the seller types in when creating the product.

Q6.2 In the Hu-Koren-Volinsky implicit ALS model, what roles do p_{ui} and c_{ui} play?

- A. p_{ui} is the predicted score and c_{ui} is the true rating.
- B. p_{ui} is a binary "showed interest" target and $c_{ui} = 1 + \alpha R_{ui}$ says how much the loss should care about getting that cell right.
- C. p_{ui} is the popularity of the item and c_{ui} is the category of the item.
- D. p_{ui} is the learning rate and c_{ui} is the regularisation strength.

Q6.3 Why is the Gram-matrix trick $Y^\top C^u Y = Y^\top Y + Y^\top (C^u - I) Y$ important for ALS?

- A. It makes the loss convex in X and Y jointly, so ALS always finds the global minimum.
- B. It removes the need for regularisation.
- C. $Y^\top Y$ can be computed once per sweep and shared by all users, so the per-user cost depends only on the items that user touched rather than on the whole catalogue.
- D. It allows ALS to handle explicit star ratings.

Q6.4 Explain in two or three sentences why treating every empty cell as a true rating of 0 is a bad idea for click data, and how implicit ALS softens this.

Q6.5 BPR training loss in this chapter starts at about 0.693 in epoch 1. Why is that the value you expect when the vectors are nearly zero?

Q6.6 (Calculation) One user has touched two items. A single-factor model has item factors $y = (2, 1)$ for those two items (and no other items). The user's raw interaction weights are $R = (1, 3)$. Use $\alpha = 4$ and $\lambda = 1$. Compute the user's factor x_u with the ALS closed-form solve (both items have preference 1).

Q6.7 (Calculation) In BPR, suppose for one triple $\hat{x}_{uij} = -\ln 3 \approx -1.0986$. Compute $\sigma(\hat{x}_{uij})$, the loss $-\ln \sigma(\hat{x}_{uij})$ and the gradient multiplier $s = \sigma(-\hat{x}_{uij})$. Is the triple ordered correctly? Will the update be large or small?

Q6.8 (Code reading) In the `bpr` function shown in the walkthrough, look at `sig = 1.0 / (1.0 + np.exp(gap))` and at `gu`, `gi` and `gj`. What does `sig` equal mathematically, why does it multiply all three gradients, and why does `gj` carry a negative sign?

Q6.9 ALS has a higher recall@20 than user-kNN (0.2715 against 0.2273) but a lower ndcg@20 (0.1581 against 0.1662). What does the difference between the two metrics tell you about what each method does well? Where in a multi-stage system would you therefore use ALS?

Q6.10 (Hands-on) Open `book/code/ch06_mf.py`. In `main()`, change the ALS call to `als_implicit(R, alpha=1.0)`, and in a second run to `als_implicit(R, factors=8)`. Run the script after each change. Write down the `ndcg@20` for `als_implicit` and the final ALS loss each time, and compare them with the results in this chapter. State which change you expect to hurt more and why, then check. What does this experiment suggest about validating hyperparameters on the same labels you report?

Chapter 7 — Embeddings and two-tower retrieval

It is 9.9 sale night and the marketplace team is watching a dashboard. Item-to-item co-visitation, the retriever you met in Chapter 5, has been serving the "similar items" strip for months, and it is good at one thing: telling you what people who looked at *this* phone also looked at. But Dewi, the newest engineer on the team, has a different complaint. A user who spent the whole week browsing running shoes, a sports watch and an electrolyte drink opens the home page and sees phone cases. "The system knows nothing about what she *is*," says Dewi. "It only knows which product pages she touched. There is no notion of a taste that connects shoes, watches and electrolyte powder."

What Dewi wants is a *representation*: a short list of numbers for every user and every item, learned so that "fits this user" turns into a single arithmetic operation. If the numbers are good, the sports-minded user lands near sports items even when none of those exact items ever co-occurred with her history. And if the operation is cheap, the team can score millions of items without a heavy ranker. That is the promise of **embeddings** and the **two-tower model**, the architecture family that large-scale retrieval systems such as YouTube's (see the fact box below) are built on.

In this chapter we build one from scratch in PyTorch on the same simulated marketplace as every other chapter, train it, and evaluate it against the popularity baseline. The result is not what the textbooks make you expect: the model that follows the recipe faithfully scores *below* the trivial "popular in the last 7 days" baseline. Finding out why, and fixing it with a one-line correction from a real production paper, is the most instructive part of the chapter. We finish by building an approximate nearest-neighbour index (IVF) with nothing but numpy, so you see why retrieval over a huge catalogue does not need to touch every item.

Two words we use throughout. A **retriever** is the fast first stage: it looks at the whole catalogue and picks a few hundred *candidates* that might suit the user. A **ranker** is the careful second stage: it scores only those candidates, using more information, and puts them in the final order. Think of a shop assistant who first pulls twenty books off the shelves, and then you decide which three to buy. This chapter is about building a better retriever.

What you'll learn

- What an embedding is, and why "closeness in a vector space" is a useful replacement for hand-written similarity rules.
- How a two-tower model is wired: a user tower, an item tower, a dot product, and why the split matters for serving.
- How the in-batch softmax loss with a temperature trains the towers, and why in-batch negatives are biased against popular items.
- How the logQ correction removes that bias, and what it did to our offline numbers.
- How approximate nearest-neighbour search (IVF from scratch; HNSW, ScaNN and FAISS in the real world) trades recall for speed.
- How to read a PCA picture of embeddings without over-interpreting it.

Building blocks you need first

The two-tower model uses a handful of ideas from machine learning. We introduce each one here with a tiny example, so the rest of the chapter reads smoothly. If you already know them, skip to the next section.

A vector is a list of numbers

A **vector** is an ordered list of numbers, like `[0.6, 0.8]`. How many numbers it holds is its **dimension**. Two numbers can be drawn as a point on a flat map. Thirty-two numbers cannot be drawn, but the idea is the same: a point in a space, and points that are close together are similar. Our model gives every user and every item such a list. We call that list an **embedding** (more on that in a moment).

The dot product measures agreement

The **dot product** of two vectors is: multiply the numbers in matching positions, then add the results. Suppose shopper Sari's vector is $(0.6, 0.8)$. Compare her with three items:

Item vector	Calculation	Dot product
Running shoes $(0.6, 0.8)$	$0.6 \times 0.6 + 0.8 \times 0.8$	1.0
Phone case $(0.8, -0.6)$	$0.6 \times 0.8 + 0.8 \times (-0.6)$	0.0
Opposite taste $(-0.6, -0.8)$	$0.6 \times (-0.6) + 0.8 \times (-0.8)$	-1.0

If you draw each vector as an arrow from the origin, the dot product is large when the arrows point the same way, zero when they are at right angles, and negative when they point opposite ways. So one cheap multiplication-and-addition gives a "how well does this item fit this user?" score. That is the whole reason embeddings are useful for recommendation.

Unit length and cosine similarity

The **length** (also called **norm**) of a vector is the square root of the sum of its squares. For $(3, 4)$ it is $\sqrt{9 + 16} = 5$. To **L2-normalise** a vector means to divide every number by its length, so the new length is exactly 1: $(3, 4)$ becomes $(0.6, 0.8)$. ("L2" is just the name of this ordinary length.) For two normalised vectors the dot product is called the **cosine similarity**: 1 means the same direction, 0 unrelated, -1 opposite, exactly as in the table above. Without normalising, an item could win a high score simply by having a long vector, whatever its direction.

A neural network is adjustable arithmetic

A **neural network** is a function built by stacking simple steps, with many adjustable numbers called **parameters** (or *weights*). You need only these pieces:

- A **linear layer** computes weighted sums. For example, one output could be $0.9 \times (\text{shoe views}) + 0.7 \times (\text{watch views}) - 0.2 \times (\text{phone-case views})$. The 0.9, 0.7 and -0.2 are parameters. A layer "64 wide" computes 64 such sums at once.
- **ReLU** is a one-line rule: keep positive numbers, turn negative numbers into 0 ($\mathrm{ReLU}(3) = 3$, $\mathrm{ReLU}(-2) = 0$). Without it, stacking linear layers would collapse into a single linear layer; ReLU lets the network learn curved, more flexible patterns.

- An **MLP** (multi-layer perceptron) is the simple stack *linear layer → ReLU → linear layer*. When the diagram says "MLP 64 → 32", the middle layer is 64 numbers wide and the output is 32 numbers.
- An **embedding table** is a spreadsheet with one row per id (one row per user, per brand, and so on). "Looking up the embedding" of brand 7 means reading row 7. The rows start as random numbers and are parameters, so training adjusts them.

Training means finding good parameters. We start with random ones, and the network makes a prediction on some examples. A **loss** is a single number saying how wrong the predictions were (lower is better). The **gradient** tells us, for every parameter, which small nudge would reduce the loss. An **optimiser** (we use one called Adam) applies those nudges, and the **learning rate** is the size of each nudge. It is like a hiker in fog who cannot see the valley but can feel the slope under their feet and keeps stepping downhill. Examples go through in small groups called **batches**, and one full pass over all the training data is an **epoch**.

A final word on **hyperparameters**: these are settings *we* choose before training (the embedding size, the learning rate, the temperature we meet below), as opposed to parameters, which training learns.

Intuition: from names to coordinates

Think about how a good bookshop assistant works. She does not memorise a table of "customers who bought X also bought Y". She has a *map in her head*: cookbooks on one side of the shop, thrillers on the other, with "food memoirs" somewhere between. A new customer walks in, says two sentences about what she likes, and the assistant places her on the map. Recommending is then just "walk to the nearest shelf that hasn't been picked over".

An **embedding** is that map made explicit. Every user and every item becomes a point in a vector space, say 32 dimensions. Nobody tells the model what the dimensions mean. Training pushes users towards the items they interact with and away from items they ignore, and the coordinates that make this possible are the embedding. Matrix factorisation (Chapter 6) already produced such vectors: one row of P per user, one row of Q per item. So what is new?

Three things.

1. **The vectors are computed by a function, not looked up in a table.** In matrix factorisation each item has its own freely adjustable row of numbers. An item with no interactions never has its row trained, so it gets no useful vector. In a two-tower model, the item vector is the *output of a small neural network*. Its inputs include **side information**, extra facts about the item such as category, brand and price bucket (which fifth of the category's prices it falls in). A new item with a known category and brand gets a sensible vector on its first day. The same goes for users: the user tower can read the last items a user touched and their category affinities, so the vector reacts to what she did five minutes ago, as long as the history features are fresh when you compute them.
2. **The loss is a ranking-flavoured softmax, not a squared error.** Matrix factorisation in Chapter 6 minimised *squared error*: the gap between a predicted number and the true number, squared. Here we do not predict a rating. We ask a multiple-choice question: among the items in this

batch, which one did the user actually interact with? That is a classification problem (pick one answer out of many), and it scales to large data.

3. **Serving is a nearest-neighbour lookup.** *Serving* means answering live requests after training is done. The final score is a dot product between a user vector and an item vector, and the item vectors do not depend on the user. So we can compute every item vector once, put them in an index, and answer "top 50 items for this user" without running the neural network once per item (a *forward pass* is one run of an input through the network).

> **Did you know?**
>
> **Vectors that do arithmetic.** The 2013 Google paper on efficient word-vector training recalls an earlier finding: with simple algebra on word vectors, vector("King") − vector("Man") + vector("Woman") gives a vector closest to the vector for "Queen". Nobody programmed that regularity into the vectors. Recommender embeddings follow the same idea with items and users in place of words.
>
> Source: Mikolov et al., arXiv, 2013

The two-tower architecture

A *tower* is just a function that turns raw inputs into a vector. The two-tower model has two of them that never see each other's inputs.

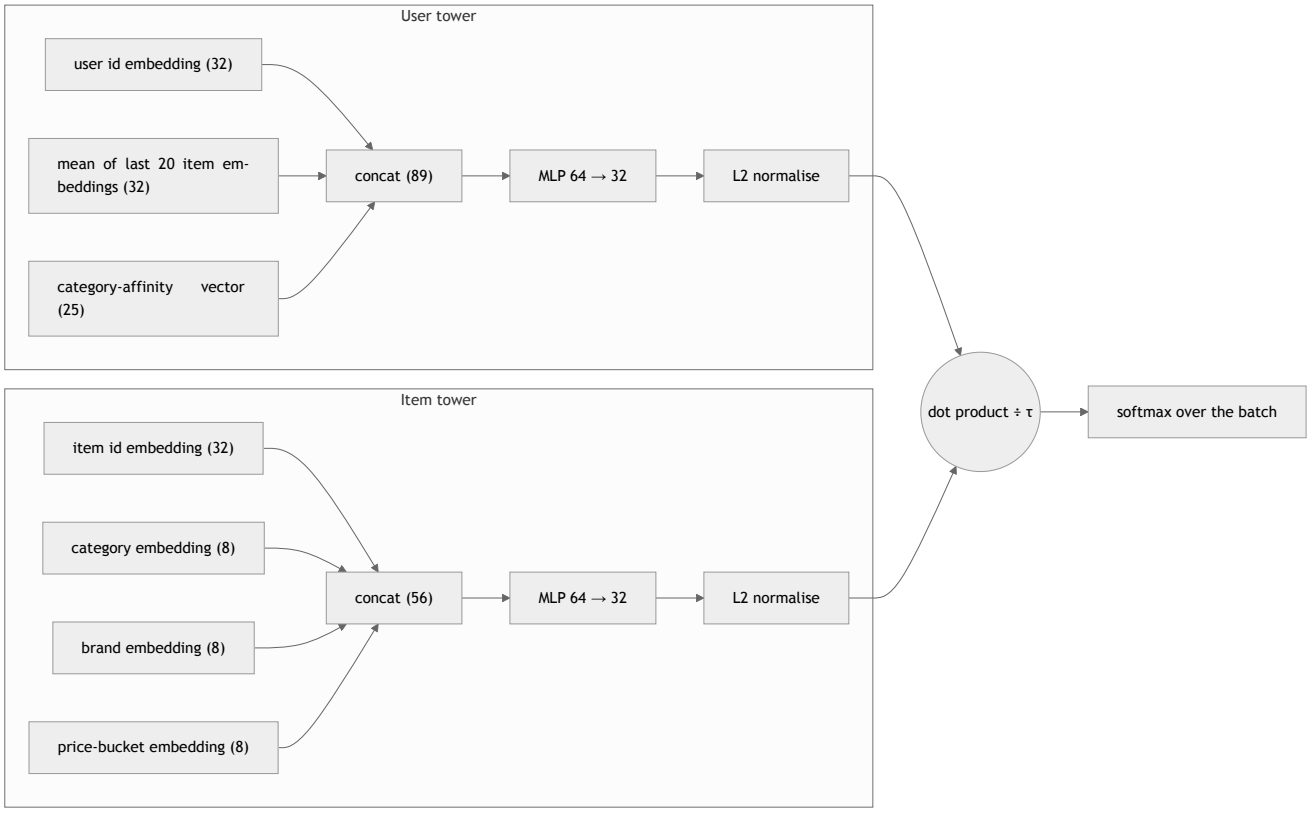


Figure 7.1: *The two towers of our model. The only place the user side and the item side meet is the final dot product.*

Two small operations appear in the diagram. **Concat** (concatenate) glues vectors end to end: $[0.2, 0.8]$ and $[0.5]$ become $[0.2, 0.8, 0.5]$; that is why $32 + 32 + 25 = 89$. The **mean of the last 20 item embeddings** averages position by position: the mean of $(0.2, 0.8)$ and $(0.6, 0.4)$ is

$(0.4, 0.6)$. It summarises the user's recent history as one vector. The **category-affinity vector** has one number per category (25 here), saying what share of the user's recent interest went to that category.

The **user tower** concatenates a learned user-id embedding (32 numbers), the *mean* of the embeddings of the last 20 items the user interacted with (32, from the same item-id table the item tower uses), and the 25-dimensional category-affinity vector from `recsys.candidates.category_affinity`; a two-layer MLP maps these 89 numbers to 32. The **item tower** concatenates the item-id embedding (32), category (8), brand (8) and price-bucket (8, the quintile of price relative to the category median) embeddings; another MLP maps these 56 numbers to 32.

Both outputs are **L2-normalised** (scaled to length 1, as in the building blocks above), so the dot product of a user vector and an item vector is exactly their *cosine similarity*, between -1 and 1. This stops the model inflating the length of popular items to win high scores, and it bounds the score, which is what lets us talk about a temperature next.

Why two towers rather than one network that sees user and item together? A joint (*cross*) model can learn interactions such as "this user likes cheap items *only* in Beauty" and is usually more accurate, but it must run once per (user, item) pair: feasible for 2,311 items, not for 50 million. The two-tower design gives up expressive interactions to gain the ability to precompute and index one side, the right trade for stage one, because the ranker of Chapters 9 and 10 re-scores the survivors with all the cross features you like.

> **Did you know?**
>
> **Two stages at YouTube.** The 2016 YouTube paper splits its recommender into two networks: candidate generation, which winnows a huge corpus to hundreds of videos, and ranking, which scores them. Candidate generation is posed as extreme multiclass classification over millions of videos using learned user and video embeddings. Because scoring millions of videos within tens of milliseconds needs an approximate scheme that is sublinear in the corpus size, serving reduces to a nearest-neighbour search in the dot-product space. The structure of this book's system follows the same split.
>
> Source: Covington et al., RecSys, 2016

The algorithm: training with an in-batch softmax

Step by step

Training data are **positive pairs**: a user and an item the user interacted with before the cutoff (click, cart or order, from `common.interaction_matrix` and `History.interactions`). We have no labelled **negatives**, meaning items the user rejected; "the user did not click" is not the same as "the user dislikes it". The standard trick is to manufacture negatives for free from the batch itself.

Softmax and cross-entropy in plain words

Two small tools do the work, so we meet them first.

Softmax turns any list of scores into probabilities: positive numbers that add up to 1, where a bigger score gets a bigger share. The recipe: raise $e \approx 2.718$ to the power of each score (this makes every

number positive and makes big scores grow faster), then divide each result by the total. Say Sari's raw scores (called **logits**) for three items are shoes 2, watch 1 and phone case 0:

Item	Logit	e^{logit}	Probability
Running shoes	2	7.39	$7.39/11.11 = 0.67$
Sports watch	1	2.72	$2.72/11.11 = 0.24$
Phone case	0	1.00	$1.00/11.11 = 0.09$

Cross-entropy is the loss that grades those probabilities against the truth. If the right answer is the running shoes, the loss is $-\ln(0.67) \approx 0.40$ ($\ln$ is the natural logarithm). Picture a student taking a multiple-choice quiz who must write a confidence percentage next to the right answer. Writing 90% costs almost nothing, writing 1% costs a lot. Likewise, loss is near 0 when the model gives the right item probability near 1, and grows large as that probability approaches 0.

What in-batch negatives are

Now the quiz is: "this is Sari; which of the items in the batch is the one she actually touched?" The other items in the batch are the wrong options, and they cost nothing, since they are already in memory. With $B = 3$ pairs (Sari with a cat-food bag, Budi with a phone case, Ani with running shoes), the scores form a 3×3 grid: each row is a user, each column an item. The diagonal is the right answer; every other cell in the row is a negative.

	cat food	phone case	running shoes
Sari	right answer	negative	negative
Budi	negative	right answer	negative
Ani	negative	negative	right answer

Everything below is this grid, with a batch of 512 instead of 3.

1. Sample a batch of B positive pairs $(u_1, i_1), \ldots, (u_B, i_B)$.
2. Run the user tower on every u_b to get $\mathbf{u}_b$, and the item tower on every i_b to get $\mathbf{v}_b$. Both are unit vectors.
3. Build the $B \times B$ matrix of scaled scores $s_{bj} = \mathbf{u}_b \cdot \mathbf{v}_j / \tau$, where τ ("tau") is the **temperature**, explained next.
4. Row b is a B-way classification problem: the right answer is column b (the item that user actually touched); the other $B - 1$ items in the batch are treated as negatives.
5. Minimise the cross-entropy of that classification, averaged over the batch. In words: for each row, take the softmax probability of the right column, take minus its log, and average over the rows:

$$\mathcal{L} = -\frac{1}{B} \sum_{b=1}^{B} \log \frac{\exp(s_{bb})}{\sum_{j=1}^{B} \exp(s_{bj})}$$

Here $\exp(x)$ is e^x, s_{bb} is the score of user b with their true item, and the sum in the denominator adds e^{score} over every item in the batch: the same recipe as the softmax table above.

This is the **sampled softmax** idea: the full softmax over the whole catalogue would be the right objective but is far too expensive (it needs a score for all 2,495 items, or 50 million in a big shop), so we approximate the denominator with the other items in the batch.

The temperature

The **temperature** τ is a dial that divides every score before the softmax, controlling how sharp the probabilities are. We need it because cosines live in $[-1, 1]$, so the raw gap between a good item (0.8) and a mediocre one (0.3) is tiny to a softmax: $e^{0.8}$ versus $e^{0.3}$ is a ratio of only about 1.6. Dividing by a small τ stretches the scale. With $\tau = 0.05$, as in our script, the scores become $0.8/0.05 = 16$ and $0.3/0.05 = 6$, a gap of 10 logit units, which is a ratio of about 22,000, and the softmax becomes decisive. A low temperature makes the loss care about getting the *top* of the ranking right and punishes hard negatives (wrong items that look very similar to the right one) severely; a high temperature spreads probability more evenly and learns more gently. It is a hyperparameter you tune, usually between 0.05 and 0.2.

A worked example with hand-sized numbers

Take $B = 3$ and, to keep arithmetic friendly, $\tau = 0.5$. Suppose user 1's cosines with the three batch items are $0.8, 0.3, 0.1$, and item 1 is her true positive.

- Scaled logits: $1.6,\ 0.6,\ 0.2$.
- Exponentials: $4.95,\ 1.82,\ 1.22$, sum ≈ 8.00.
- Probability of the right item: $4.95/8.00 \approx 0.62$. Loss $= -\ln 0.62 \approx 0.48$.

If training moves her cosine with item 1 to 0.9, the probability rises to about 0.67 and the loss falls. The gradient pulls $\mathbf{u}_1$ towards $\mathbf{v}_1$ and pushes it away from $\mathbf{v}_2, \mathbf{v}_3$; since every item is a positive for some rows and a negative for others, item vectors are pulled and pushed throughout training.

The hidden bias: who ends up as a negative?

In our quiz, the wrong options are not drawn fairly. A batch is a random sample of *positive pairs*, so an item appears in it in proportion to its interaction count. An item with 2% of all interactions is the positive of roughly one row in fifty, and a *negative* in every other row it appears in; a rare item is almost never in the batch. Popular items are therefore over-represented as negatives relative to a uniformly random catalogue item, and the model learns "popular items are usually the wrong answer", shifting their scores down. But at serving time we score every item uniformly, so the learned popularity penalty is a systematic error that suppresses best-sellers that are genuinely relevant to many users.

The logQ correction

Yi et al. described the fix at Google. Let q_j be the **sampling probability** of item j: how likely it is to be drawn into a batch, which we estimate as its share of all training pairs (a bag of cat food with 2% of the pairs has $q = 0.02$). The fix is to subtract $\log q_j$ from item j's logit:

$$s^c_{bj} = \frac{\mathbf{u}_b \cdot \mathbf{v}_j}{\tau} - \log q_j$$

and to use these corrected logits s^c in the softmax. Here $\log$ is the natural logarithm. For a probability below 1 it is negative, and more negative the smaller the probability: $\log 0.5 = -0.69$ and $\log 0.25 = -1.39$. So $-\log q_j$ is a small positive number for a popular item and a big positive number for a rare one.

The idea is the same as **survey weighting**. If a pollster phones 50 people in a big city and 5 in a village, she counts each village answer for more and each city answer for less, to cancel the unequal sampling. Here the popular items are the "city": they are sampled too often as wrong options, so their weight in the denominator is reduced relative to rare items. This cancels the extra pushing-down that popular items would otherwise receive, so the dot product itself learns "fit", not "fit minus a popularity tax". In the paper the frequencies are estimated online from a stream of data; in our offline script we use the empirical item frequencies (count divided by total) among the training pairs.

A numeric version of the earlier example. Let item 1 be popular ($q = 0.5$) and items 2 and 3 rarer ($q = 0.25$). User 1's cosines are still $0.8, 0.3, 0.1$ with $\tau = 0.5$:

	Item 1 (true, popular)	Item 2	Item 3
Scaled logit	1.6	0.6	0.2
$-\log q$	0.69	1.39	1.39
Corrected logit	2.29	1.99	1.59
$e^{\text{corrected}}$	9.90	7.34	4.92

The sum is about 22.16, so the probability of the true item falls from 0.62 to about 0.45 (loss from 0.48 to 0.80). The popular true item no longer gets a free pass just for being popular, while the rare negatives count for more. The model must now separate the positive more cleanly in cosine space instead of leaning on popularity. At serving time the $-\log q$ term is dropped.

> **Did you know?**
>
> **Correcting sampling bias in YouTube retrieval.** Yi et al. consider two-tower retrieval models trained with in-batch negatives, the items sampled in a random batch. They argue that this batch loss is subject to sampling bias which can severely restrict performance, particularly under a power-law item distribution. They propose an algorithm that estimates item frequency from streaming data without a fixed item vocabulary, and apply the bias-corrected approach to Neural Deep Retrieval, a YouTube system that retrieves from a corpus of tens of millions of videos.
>
> Source: Yi et al., RecSys, 2019

Code walkthrough

All code below is from `book/code/ch07_two_tower.py`, which trains four variants (10 or 30 epochs, with and without logQ) and then runs the ANN demo.

The towers

book/code/ch07_two_tower.py · Towers

```
class Towers(nn.Module):
    def __init__(self, n_users, n_items, n_cats, n_brands, n_buckets):
        super().__init__()
        self.user_emb = nn.Embedding(n_users, 32)
        self.item_emb = nn.Embedding(n_items + 1, 32, padding_idx=n_items)  # shared; last row =
padding
        self.cat_emb = nn.Embedding(n_cats, 8)
        self.brand_emb = nn.Embedding(n_brands, 8)
        self.price_emb = nn.Embedding(n_buckets, 8)
        self.pad = n_items
        self.user_mlp = nn.Sequential(nn.Linear(32 + 32 + n_cats, 64), nn.ReLU(), nn.Linear(64,
32))
        self.item_mlp = nn.Sequential(nn.Linear(32 + 8 + 8 + 8, 64), nn.ReLU(), nn.Linear(64, 32))

    def user_vec(self, uid, hist_items, aff):
        mask = (hist_items != self.pad).float().unsqueeze(-1)
        h = self.item_emb(hist_items)
        mean = (h * mask).sum(1) / mask.sum(1).clamp(min=1.0)
        x = torch.cat([self.user_emb(uid), mean, aff], dim=-1)
        return F.normalize(self.user_mlp(x), dim=-1)

    def item_vec(self, iid, cat, brand, bucket):
        x = torch.cat([self.item_emb(iid), self.cat_emb(cat), self.brand_emb(brand),
self.price_emb(bucket)], dim=-1)
        return F.normalize(self.item_mlp(x), dim=-1)
```

Read this class in three parts.

- **Embedding tables.** `user_emb` has one 32-dimensional row per user; `item_emb` has one row per item *plus one extra* row used as **padding** (`padding_idx=n_items`): a dummy "empty slot" row, so users with fewer than 20 items in their history can still fill a fixed-width grid of numbers (a *tensor*). `cat_emb`, `brand_emb` and `price_emb` are small 8-dimensional tables. Note that `item_emb` is **shared**: the item tower looks up an item's own row, and the user tower looks up the rows of the user's history items from the same table. Sharing means the vector for "running shoe A" has one meaning across both towers.
- **The MLPs.** `user_mlp` takes `32 + 32 + n_cats` inputs (32 for the user id, 32 for the history mean, 25 for the affinity), which is the 89 in the diagram. `item_mlp` takes `32 + 8 + 8 + 8 = 56`. Each is `Linear → ReLU → Linear` with a 64-wide hidden layer and a 32-wide output.
- **`user_vec` and `item_vec`.** In `user_vec`, `mask` is a list of 1s and 0s that switches slots on or off: 1 where a history slot holds a real item and 0 where it holds padding. The embeddings are masked before summing and divided by the number of real items (clamped at 1 so an empty history divides by 1), which gives the average over real items only. Without the mask, padding would drag every short-history user towards the same "empty" direction. The mean is concatenated with the user-id embedding and the affinities, passed through the MLP and normalised with `F.normalize` (the L2 normalisation from the building blocks). `item_vec` is simpler: concatenate four embeddings, MLP, normalise. Crucially it takes only item attributes, with no user input, which is what lets us compute the vectors for the whole catalogue once, offline, and store them.

The training loop

The training function is nested inside `main`; the build supports nested symbols, so we can show it on its own.

book/code/ch07_two_tower.py · main.train

```
def train(epochs: int, logq: bool, tag: str):
    torch.manual_seed(0)
    model = Towers(len(all_users), len(items), n_cats, len(brands), 5)
    opt = torch.optim.Adam(model.parameters(), lr=1e-3)
    g = torch.Generator().manual_seed(0)
    counts = torch.bincount(p_i, minlength=len(items)).float()
    log_q = torch.log(counts.clamp(min=1.0) / len(p_i))  # item sampling probability in a batch
    losses = []
    for ep in range(1, epochs + 1):
        model.train()
        perm = torch.randperm(len(p_u), generator=g)
        total, nb = 0.0, 0
        for s0 in range(0, len(perm), BATCH):
            b = perm[s0 : s0 + BATCH]
            u, i = p_u[b], p_i[b]
            h = t_hist[u].clone()
            # book:start leak_mask
            h[h == i[:, None]] = model.pad  # hide the target item from its own history (no
leakage)
            # book:end leak_mask
            uv = model.user_vec(u, h, t_aff[u])
            iv = model.item_vec(i, t_cat[i], t_brand[i], t_bucket[i])
            logits = uv @ iv.T / TEMP
            if logq:
                logits = logits - log_q[i][None, :]  # logQ correction (Yi et al., RecSys 2019)
            same = i[:, None] == i[None, :]
            logits = logits.masked_fill(same & ~torch.eye(len(b), dtype=torch.bool), -1e9)  #
duplicates are not negatives
            loss = F.cross_entropy(logits, torch.arange(len(b)))
            opt.zero_grad()
            loss.backward()
            opt.step()
            total += loss.item()
            nb += 1
        losses.append(total / nb)
        if ep in (1, 5, 10, 20, 30) or ep == epochs:
            print(f"[{tag}] epoch {ep:>2d}  loss {losses[-1]:.4f}")
    model.eval()
    with torch.no_grad():
        iv_all = model.item_vec(t_item, t_cat, t_brand, t_bucket).numpy()
        uv_eval = model.user_vec(torch.tensor(eval_rows), t_hist[eval_rows],
t_aff[eval_rows]).numpy()
    return model, iv_all, uv_eval, losses
```

Walk through the inner loop.

1. `perm = torch.randperm(...)` shuffles the positive pairs each epoch; `b` is a slice of 512 of them (`BATCH`).
2. `h = t_hist[u].clone()` fetches each user's last-20-items matrix (`.clone()` makes a private copy so the stored histories are not changed), and the next line, `h[h == i[:, None]] = model.pad`, **blanks out the target item inside its own user's history**. Without it the user tower would see the answer it is supposed to predict, the same kind of leakage you will study in Chapter 9. The model would reach a low training loss by copying, and then fail on the held-out week where no such shortcut exists.
3. `uv` and `iv` are the user and item vectors for the batch. `logits = uv @ iv.T / TEMP` is the $B \times B$ matrix s_{bj} of the algorithm.
4. `if logq:` subtracts `log_q[i][None, :]`, where `log_q` was computed at the top of `train` as `log(count / total)` of each item among training pairs. *Broadcasting* (the rule that applies one

row of numbers to every row of a matrix) subtracts $\log q_j$ from column j. That one line is the entire logQ correction.

5. The `same` mask handles duplicates: if two users in a batch clicked the same item, that column would be a "negative" that is really a positive, so those off-diagonal entries are set to -10^9 and vanish from the softmax.
6. `F.cross_entropy(logits, torch.arange(len(b)))` is the loss above (the correct column for row b is b), followed by an Adam step (the nudge to the parameters described in the building blocks) with learning rate 10^{-3}, that is 0.001.

When the loop ends, `train` switches to eval mode and computes the item vectors for the whole catalogue and the user vectors for the evaluation users.

Scoring and evaluation

Scoring (`main.evaluate_variant` in the script) is a single matrix product, `uv_eval @ V.T`, of 947 evaluation users against the in-stock items. Users with no history before the cutoff get no model scores and are padded with 7-day popularity, because the model has "no opinion". Then `common.evaluate` computes the metrics every chapter uses: recall@K (what share of the items the user really touched in the held-out week appear in the top K), order recall (the same, counting only items that were bought), NDCG@20 (a score that gives more credit for placing right items near the top of the list), and coverage@20 (what share of the catalogue appears in anyone's top 20).

The ANN index, from scratch

The second half of the script has no deep learning in it. IVF, the *inverted file index*, needs only a clustering routine: a way to sort points into groups of neighbours.

book/code/ch07_two_tower.py · kmeans

```
def kmeans(x: np.ndarray, k: int, iters: int, seed: int = 0):
    rng = np.random.default_rng(seed)
    cent = x[rng.choice(len(x), k, replace=False)].copy()
    for _ in range(iters):
        assign = np.argmax(x @ cent.T - 0.5 * (cent**2).sum(1), axis=1)  # nearest centroid
(euclidean)
        for c in range(k):
            pts = x[assign == c]
            if len(pts):
                cent[c] = pts.mean(0)
    assign = np.argmax(x @ cent.T - 0.5 * (cent**2).sum(1), axis=1)
    return cent, assign
```

This is Lloyd's **k-means**, a method that sorts points into k groups. Pick k starting centres (**centroids**). Assign every point to its nearest centroid. Move each centroid to the average of the points assigned to it. Repeat. A tiny example: points $1, 2, 9, 10$ on a line with $k = 2$ and starting centroids 1 and 10. The assignment is $\{1, 2\}$ and $\{9, 10\}$; the centroids move to 1.5 and 9.5; nothing changes after that, so we stop. The code adds a small trick. Finding the nearest centroid by Euclidean (ordinary straight-line) distance is `argmax(x @ cent.T - 0.5 * ||cent||²)`: expanding $\|x - c\|^2 = \|x\|^2 - 2x \cdot c + \|c\|^2$, the $\|x\|^2$ term is the same for every centroid and can be dropped, and flipping the sign turns "minimise" into "maximise". One matrix multiplication assigns all points at once. Then each centroid moves to the mean of its points, and we repeat for 20 iterations.

Approximate nearest neighbours

Why brute force stops working

Nearest-neighbour search means: given a query vector (the user), find the item vectors with the highest dot product with it. **Brute force** computes the dot product of the query with every item vector and keeps the top K, like walking every shelf in the shop. For our 2,311 in-stock items that is trivial; for 50 million items it is about $50\text{M} \times 32 \approx 1.6$ billion multiply-adds per request, far beyond a latency budget (the time we allow per request) of tens of milliseconds. But we do not need the *exact* top 50, since the ranker re-scores everything anyway; we only need a candidate list containing *most* of the true neighbours. Searching that accepts a few misses in exchange for speed is **approximate nearest-neighbour (ANN)** search.

IVF in four steps

IVF works like the shop's sections. Instead of walking every shelf, you ask "which two or three sections are most likely to hold it?" and search only those.

1. **Train.** Run k-means on the item vectors to get C centroids ($C = 32$ here). Assign every item to its nearest centroid. The set of items for one centroid is its *inverted list* (the "inverted file" in the name): a list of the items in that section.
2. **Query: choose clusters.** For a query vector $\mathbf{q}$, score it against the C centroids (cheap: C dot products) and keep the `n_probe` best.
3. **Query: scan.** Concatenate the inverted lists of those clusters and compute exact dot products only on those items.
4. **Return the best** K among the scanned items.

The cost is $C + n_{\text{probe}} \cdot N/C$ dot products on average, instead of N. For $N = 2{,}311$ and $C = 32$, probing 4 clusters scans about $4 \times 72 \approx 289$ items plus 32 centroids, versus 2,311. The price is that a true neighbour sitting in a cluster you did not probe is lost; that is where *recall versus brute force* comes from. This recall asks: of the true top 20 that brute force would return, what share did the index also return? A recall of 0.9 means 18 of the 20.

Picture two clouds of four items each, around $(1, 1)$ and $(5, 5)$. A query at $(1.5, 1.2)$ with `n_probe = 1` scans only the first cloud and is exact. A query at $(3, 3)$ has neighbours in both clouds, so probing one loses some; that boundary effect is what more probes repair.

The demo in code

The IVF query loop is written inline in `main`. For each query it scores the centroids, picks the top `n_probe`, gathers their members and ranks only those exactly:

book/code/ch07_two_tower.py · ivf_search

```
for qi, ex in zip(q, exact):
    probe = np.argsort(-(cent @ qi))[:n_probe]
    cand = np.concatenate([members[c] for c in probe])
    top = cand[np.argsort(-(V[cand] @ qi))[:20]]
    rec_hits.append(len(set(top) & set(ex)) / 20)
    scanned.append(len(cand))
```

The `probe` line is the cluster choice (`cent @ qi`, only 32 dot products), `cand` concatenates the inverted lists, `top` is the exact re-rank on the scanned items, and the last two lines record recall against the brute-force answer `ex` and the number of items scanned. Below we show what the loop prints, with 400 evaluation users as queries.

We run the ANN demo on the spec model (`two_tower`); index quality depends on the geometry of the vectors, not on recommendation quality.

> **Did you know?**
>
> **Graph indexes.** Besides cluster-based indexes such as IVF, approximate nearest-neighbour search can use proximity graphs. HNSW (Hierarchical Navigable Small World graphs) incrementally builds a multi-layer structure of proximity graphs over nested subsets of the stored vectors. Starting the search from the upper layer lets the algorithm exploit scale separation between links, and the authors report logarithmic complexity scaling.
>
> Source: Malkov et al., IEEE TPAMI, 2020

Graph indexes are not the only option. The FAISS library (a widely used open-source vector-search library), described by Johnson et al. (2021), provides optimised designs for brute-force, approximate and **product-quantisation** search (storing short compressed codes in place of the full vectors, to save memory) on GPUs; their paper reports building a k-NN graph (each vector linked to its k nearest neighbours) over 1 billion vectors in under 12 hours on four GPUs. Guo et al. (2020) target *maximum inner product search* (finding the largest dot products; "inner product" is another name for dot product) with an *anisotropic* quantisation loss, which penalises the error that matters for inner-product ranking rather than plain reconstruction error; Google's ScaNN library is built around this line of work. Our 2,311-item demo needs none of this, but these are the names to search for when the catalogue outgrows a single matrix product.

Run it

```
cd book
uv run python code/ch07_two_tower.py
```

The script is deterministic (all seeds fixed) and takes a few minutes on a laptop CPU.

TERMINAL OUTPUT

```
users 1500  items 2495  categories 25  brands 10  price buckets 5
training pairs (distinct user–item positives before cutoff): 18256
user history window: last 20 items; temperature 0.05; batch 512; epochs 10
```

We have 18,256 distinct (user, item) positive pairs from before the cutoff and 1,500 users. Before blaming a model for a weak score, the script also runs diagnostics (consistent index mappings, history strictly before the cutoff, non-empty affinity vectors, out-of-stock items excluded); all pass. Then training starts.

TERMINAL OUTPUT

```
=== Training two_tower: 10 epochs, logQ correction off ===
[two_tower] epoch  1  loss 6.8075
[two_tower] epoch  5  loss 5.8995
[two_tower] epoch 10  loss 5.1991
sanity: true training item is in the user's top-20 of 2495 items for 14.7% of training pairs
```

The loss of the spec model falls steadily from 6.8075 to 5.1991, so optimisation works. The sanity line tells us that for 14.7% of the *training* pairs, the true item ranks in the user's top 20 out of 2,495 items, far above chance (about 0.8%), so the model has learned real structure. The logQ run's loss is not comparable, because its logits include the extra $-\log q_j$ term.

Results

TERMINAL OUTPUT

```
=== Evaluation: brute-force top 50 over in-stock items ===
users without history (padded with 7-day popularity): 28 of 947
                     recall@5  recall@10  recall@20  recall@50  order_recall@20  ndcg@20
coverage@20
method
two_tower              0.0183     0.0399     0.0818     0.1932           0.0912   0.0361
0.9766
two_tower_logq         0.0747     0.1141     0.1766     0.2846           0.2243   0.1053
0.4344
two_tower_30ep         0.0182     0.0352     0.0754     0.1804           0.0893   0.0357
0.9965
two_tower_logq_30ep    0.0608     0.1074     0.1779     0.2936           0.2129   0.0982
0.6525
pop_7d                 0.0310     0.0493     0.0786     0.1335           0.1066   0.0475
0.0087
best two-tower variant: two_tower_logq; the ANN demo below uses the spec model 'two_tower'
```

Here is the table that surprised us. Read it row by row. As a reminder, `pop_7d` is the baseline that recommends the items with the most interactions in the last 7 days to everyone, and the metrics are the ones defined under "Scoring and evaluation" above: higher is better for recall and NDCG, while coverage says how much of the catalogue gets recommended at all.

The spec model underperforms popularity. `two_tower` reaches an NDCG@20 of 0.0361, below the 7-day popularity baseline's 0.0475. Its recall@20 of 0.0818 is roughly on par with `pop_7d` (0.0786), and its recall@50 is better (0.1932 vs. 0.1335), so it does retrieve useful items deeper in the list, but it ranks the right ones low. On order recall, which counts only items that were actually bought, it is behind: 0.0912 against 0.1066. Training for 30 epochs (`two_tower_30ep`) does not help: NDCG@20 of 0.0357, and the loss keeps dropping, so more optimisation is not the answer.

The `coverage@20` column gives away the cause. The spec model recommends a very wide spread of the catalogue, with coverage@20 of 0.9766, and `pop_7d` only 0.0087, since it shows everyone nearly the same few items. The spec model has learned *not* to recommend popular items, exactly the pathology the logQ section predicted. In this marketplace popularity is a strong and legitimate signal, as the `pop_7d` row shows: one shared list of a few items beats the model's NDCG. A model that deliberately avoids popular items throws away its best prior.

The logQ correction fixes it. `two_tower_logq`, identical in every respect except for the one subtraction line, reaches NDCG@20 of 0.1053, more than double the popularity baseline's 0.0475, and rough-

ly three times the uncorrected model's 0.0361. Recall@20 jumps from 0.0818 to 0.1766 and order recall from 0.0912 to 0.2243. Coverage drops to 0.4344: the corrected model concentrates on a more reasonable subset, still far broader than the popularity baseline's 0.0087, yet no longer scattering recommendations across the whole catalogue. That is the trade-off you want in a retriever: personalised *and* aware of popularity.

More epochs are not the cure either way. With the correction, 30 epochs gives an NDCG@20 of 0.0982, slightly *lower* than the 10-epoch 0.1053, the usual start of overfitting with only 18,256 training pairs. Evaluate on a time-held-out set at several epoch counts and stop early.

An honest caveat: these are single-seed numbers on one simulated marketplace. The gap between the spec and corrected models (about three times in NDCG) is large enough to trust the direction, but we do not claim `two_tower_logq` beats `two_tower_logq_30ep` robustly. Chapter 10 shows how to quantify seed noise.

What the embeddings look like

Next, the item vectors from the spec model, squashed from 32 dimensions to two with **PCA** (principal component analysis). PCA finds the direction in which the points are most spread out, then the next most spread-out direction at right angles to it, and draws each point using only those two. Think of the shadow of a 3-D object on a wall: if you choose the angle well, the shadow keeps most of the object's shape, though some depth is lost. Technically, we take the singular value decomposition of the centred matrix, in numpy.

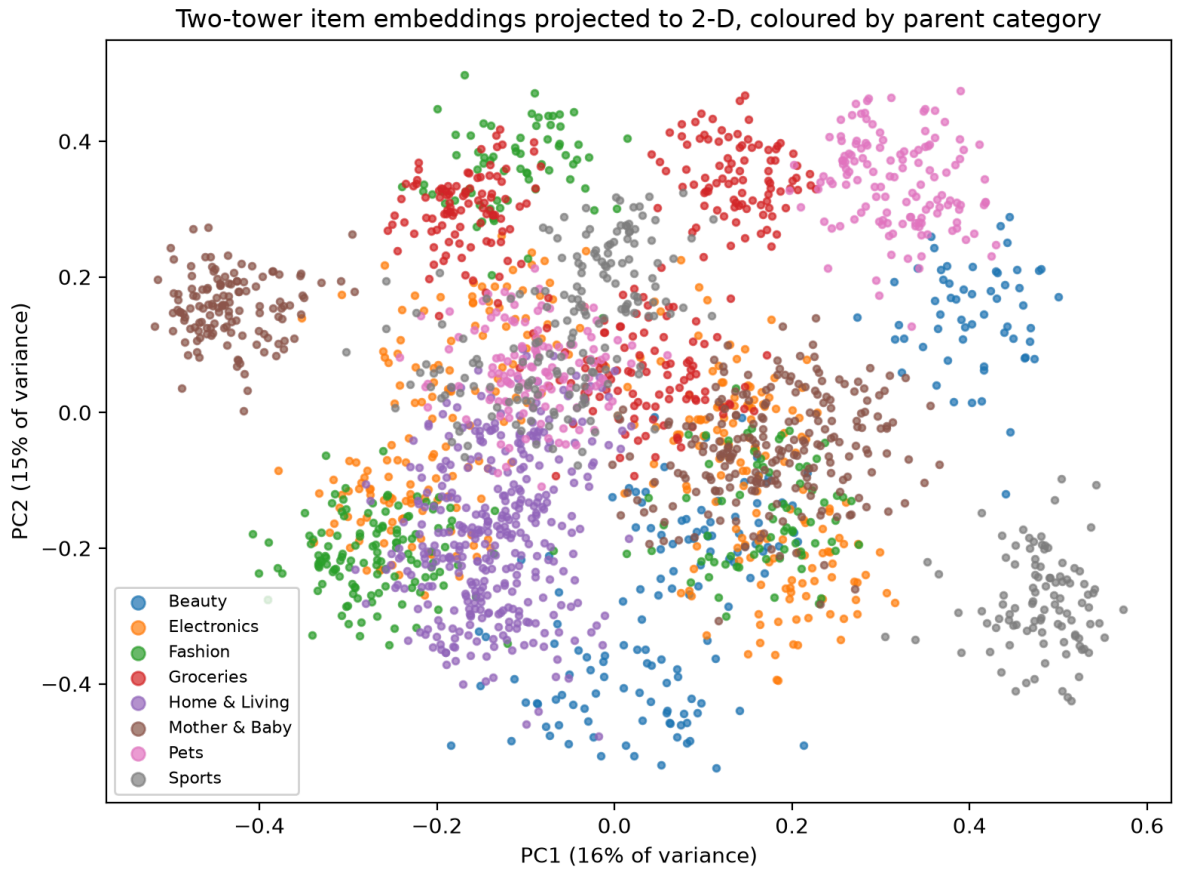


Figure 7.2: *Two-tower item embeddings projected onto their first two principal components and coloured by parent category. Pets, Sports and Mother & Baby form separate islands; Fashion, Electronics, Home & Living and others overlap in the middle.*

TERMINAL OUTPUT

```
PCA: first two components explain 30.6% of item-vector variance
```

The model was never given the parent category, only leaf category, brand and price bucket, yet items cluster by parent: Pets sit in the upper right, Sports form an island on the lower right, Mother & Baby a cloud on the left, and categories with similar shoppers overlap in the middle. But be careful: two components explain only 30.6% of the **variance** (a measure of how spread out the points are), so roughly 70% of the structure is invisible here, and points that look close in 2-D may be far apart in 32-D. PCA is a sanity check, not a measurement.

The recall-versus-speed trade-off

TERMINAL OUTPUT

```
=== Approximate nearest neighbours: IVF with 32 k-means clusters (on two_tower) ===
cluster sizes: min 24  median 68  max 127
brute force: 0.140 ms/query over 2311 items  (400 queries)
probe  1 clusters  recall@20 vs brute force 0.729  0.015 ms/query  scanned 3.3% of items
probe  2 clusters  recall@20 vs brute force 0.899  0.020 ms/query  scanned 6.3% of items
probe  4 clusters  recall@20 vs brute force 0.975  0.028 ms/query  scanned 12.5% of items
probe  8 clusters  recall@20 vs brute force 0.995  0.046 ms/query  scanned 24.9% of items
probe 16 clusters  recall@20 vs brute force 1.000  0.083 ms/query  scanned 49.9% of items
```

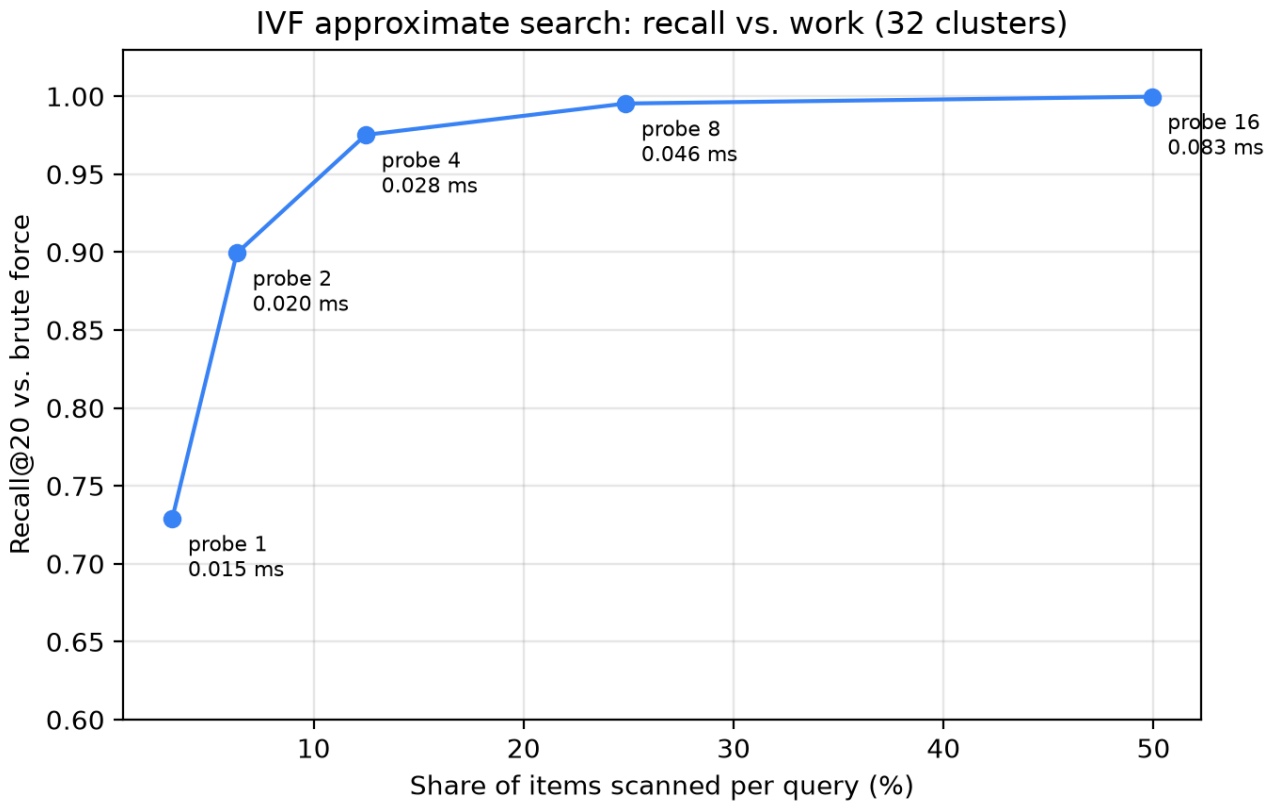


Figure 7.3: *IVF approximate search on the spec model's item vectors: recall@20 relative to brute force against the share of the catalogue scanned, for 1, 2, 4, 8 and 16 probed clusters of 32.*

The 32 clusters have between 24 and 127 items (median 68), uneven as k-means clusters usually are. Probing a single cluster scans just 3.3% of the items and already recovers 0.729 of the exact top 20. Two clusters bring recall to 0.899 at 6.3% of the work, four to 0.975 at 12.5%, and eight to 0.995 at

24.9%; at 16 probes, half the catalogue, recall is 1.000. The first few probes buy almost all of the recall, which is why production systems rarely probe more than a few percent of their clusters.

The times column needs care: at 2,311 items brute force costs only 0.140 ms per query, and Python overhead dominates the IVF timings. The *shape* is what transfers to a catalogue of millions: work grows with the share of items scanned while recall saturates quickly, so extrapolate from the "share scanned" column, not the milliseconds.

Pitfalls and how it is done in production

Training-serving skew. The model sees inputs built one way during training and another way when live. The user tower consumes the last 20 items and the affinity vector. In production a feature pipeline must produce identical features online and offline; a small mismatch in how "last 20 items" is defined silently degrades the model (Chapter 9 treats point-in-time correctness).

Embeddings drift; rebuild indexes together. An item vector from model version 17 and a user vector from version 18 live in unrelated spaces, so the user tower and the item index ship as one versioned unit, and the index is rebuilt offline whenever the model changes.

Popularity bias hides in many places. Any non-uniform negative sampling, or a training set that over-represents hot items, produces the same effect. The cures are similar: reweight, correct the logits, or mix in uniformly sampled negatives.

False negatives. Our `same`-item mask removes the obvious case, but an item can still appear as a "negative" for a user who would have loved it and simply never saw it; large batches make this more likely. Hard negative mining (deliberately choosing items the user was shown but ignored, which look similar to the right answer) needs the same care.

Cold start is only half solved. *Cold start* means a brand-new item or user with no history. A new item gets a sensible vector from its category, brand and price bucket, but its item-id embedding is untrained noise. Teams either drop the id embedding below an interaction threshold or **hash** rare ids into shared buckets (many rare ids share one embedding row). Chapter 12 returns to cold start.

Choosing an index. IVF is the easy baseline and is cheap to build and update. HNSW (the graph index from the fact box) usually gives better recall at the same latency but uses more memory and handles deletions awkwardly; product quantisation compresses vectors. At our scale a single matrix product is the right answer: do not add an ANN dependency before brute force actually hurts.

How this repo does it. The Shopee-style system of the later chapters has no two-tower stage. Its first stage is four hand-written retrievers combined in Chapter 8, and the LightGBM ranker of Chapter 10 does the heavy lifting, a sensible choice for 2,495 items. A learned retriever earns its place when the catalogue outgrows rule-based retrievers or when you want candidates beyond what co-visitation reaches; Chapter 14 shows where it would slot in. Measure it by its marginal recall over the existing sources, not by NDCG alone.

Summary

- An embedding is a learned vector; closeness replaces hand-written similarity rules, and side information gives new items and users meaningful vectors.
- A two-tower model computes user and item vectors with separate networks; the score is their dot product (cosine after L2 normalisation).
- The in-batch softmax treats the other items in a batch as negatives; the temperature τ scales the cosines, and a small τ sharpens the softmax.
- In-batch negatives are drawn in proportion to popularity, so the loss pushes popular items down: the spec model scored an NDCG@20 of 0.0361, below the 7-day popularity baseline's 0.0475.
- Subtracting $\log q_j$ from the logits (Yi et al., 2019) lifted NDCG@20 to 0.1053 and recall@20 from 0.0818 to 0.1766; longer training did not help.
- Item vectors do not depend on the user, so they can be precomputed and indexed; IVF scans only a few k-means clusters per query, and scanning 12.5% of the items kept 0.975 of the brute-force top 20.
- At this catalogue size brute force is enough; ANN, versioned indexes and cold-start handling matter at production scale.

Quiz

Q7.1 What does lowering the temperature τ in the in-batch softmax do?

A. It makes the softmax flatter, so all items in the batch are treated alike.
B. It stretches the gaps between cosine scores, so the loss focuses on getting the top of the ranking right and punishes hard negatives more.
C. It increases the dimensionality of the embeddings.
D. It removes popular items from the batch.

Q7.2 Why do in-batch negatives bias a two-tower model against popular items?

A. Popular items have longer titles, which confuses the item tower.
B. Popular items are always out of stock.
C. Items enter the batch in proportion to their interaction count, so popular items appear as negatives for many other users' rows and are pushed down more.
D. The L2 normalisation gives popular items a smaller norm.

Q7.3 Why does the two-tower design allow approximate nearest-neighbour search at serving time?

A. Because item vectors depend only on item attributes, so they can be precomputed once and indexed, and the user-item score is an inner product.
B. Because the user tower runs on the GPU.
C. Because the temperature is always 1 at serving.
D. Because the item tower takes the user's history as input.

Q7.4 In one or two sentences, explain what the logQ correction subtracts from each logit, and why subtracting it removes the popularity penalty.

Q7.5 Our IVF demo measures "recall@20 versus brute force" rather than NDCG. What question does that metric answer, and why is it the right one for an ANN index?

Q7.6 Read this line from the training loop:

book/code/ch07_two_tower.py · leak_mask

```
h[h == i[:, None]] = model.pad  # hide the target item from its own history (no leakage)
```

What does it do, and what would likely happen to training loss and to evaluation results if it were removed?

Q7.7 An item catalogue has 10,000,000 items. You build an IVF index with 4,096 clusters (assume equal-sized clusters) and probe 16 clusters per query. Roughly how many dot products does one query need (centroid scoring plus scanning), and what fraction of brute force is that?

Q7.8 Two items have sampling probabilities $q = 0.04$ and $q = 0.0004$. By how much does the logQ correction raise the logit of the rarer item relative to the more popular one?

Q7.9 Hands-on. In `book/code/ch07_two_tower.py`, change `TEMP = 0.05` to `TEMP = 0.2`, run the script, and compare the `two_tower` and `two_tower_logq` rows (ndcg@20 and coverage@20) with the table in this chapter. Then repeat with `TEMP = 0.02`. State what you observe about the effect of temperature and whether the logQ correction still helps. (The script overwrites `results/ch07_two_tower.*`; restore them with `git checkout book/results` afterwards.)

Chapter 8 — Multi-source candidate generation

It is 12:05 on a payday, and the home page of a Shopee-like marketplace is about to be opened by a few hundred thousand people in the same minute. Each needs twenty products in a fraction of a second, from a catalogue of millions of items. Somebody proposes: "Let's run our best ranking model (the program that scores how well an item suits a user) on every item for every user and sort." A back-of-the-envelope calculation ends the meeting: even 20 microseconds (millionths of a second) per (user, item) pair means tens of seconds of CPU (processor) time per request, and the model we really want, with dozens of features (input facts) per pair, is far slower.

So the team splits the problem in two. A cheap, broad stage picks a hundred or so items that *might* interest this user; an expensive, precise stage ranks only those. This chapter is about the first stage, **candidate generation** or **retrieval**. It decides what the ranker is even allowed to see, and it sets a hard ceiling on everything downstream.

In this repository the first stage is `recommender/recsys/candidates.py`: four small retrievers, each a different reason to believe a user may like an item, plus a couple of hard filters. The simulator lets us put numbers on every claim.

Four words to know before we start

Think of shopper Sari opening the app. The catalogue holds millions of items; she can look at twenty.

- A **candidate** is an item that *might* be shown to Sari. It is a "maybe", not a decision. The **candidate pool** is the list of all maybes for her, about a hundred items.
- A **retriever** (also called a *source*) is a small, fast piece of code that proposes candidates for one reason, for example "Sari looked at these cat-food bags yesterday". Several retrievers each add items to the pool.
- The **ranker** is the slower, smarter model of a later chapter. It takes the pool and puts the items in order for Sari. It can only reorder what the pool contains.
- **Recall** is the share of the items Sari will really want that made it into the pool. Suppose she will buy 4 things next week and 2 of them are in the pool: recall is $2/4 = 50\%$. If the other 2 are missing, no ranker can ever show them, however clever it is.

Compare this with a shop assistant who first fetches a trolley of 100 products from the warehouse (retrieval) and only then lays out the best 20 on the counter (ranking). The assistant cannot lay out something that is still in the warehouse. That is why this chapter matters: the pool decides what is *possible*.

What you'll learn

- Why recommendation is organised as a funnel of stages, and why recall rather than precision is the goal of the first stage.
- How the four retrievers in the repo work (`recent`, `covisit`, `catpop`, `global_trending`) and which real-life behaviour each one imitates.

- How to combine retrievers with an outer join, keep each retriever's score as a feature, and count how many sources proposed an item.
- How to measure a retriever with recall and with *unique* recall, and what the "recall ceiling" means for the whole system.
- Why hidden and out-of-stock items must be removed with hard filters at this stage and not left to the ranker.

Intuition: the librarian and the shortlist

Imagine asking a librarian for "something good to read tonight". She does not read every book in the building. She shortlists using several instincts: what you borrowed last week, what readers of those books also borrowed, which shelves you browse and which of their books are most borrowed this month, and, if you are a first-time visitor, what everybody is borrowing right now. Only with twenty or thirty books on her trolley does she sit down, look carefully at each one and decide the order.

Notice that the librarian uses *several* instincts, because each fails differently: your last loan is useless if you are new, and "popular this month" is useless for niche tastes. Combining them covers blind spots. That is **multi-source candidate generation**, and the module docstring of `candidates.py` calls each retriever "a different reason to like an item".

Why two stages: the funnel

Suppose the catalogue has N items and we serve U requests per second. Scoring every item with a ranker that costs c seconds per pair needs $U \cdot N \cdot c$ seconds of compute every second. With N in the millions, even a cheap c is unaffordable, because a good ranker needs features computed *per pair*: the user's history with this item, its recent click-through rate (the share of people who saw it and clicked), the shop's quality.

The funnel solves this by spending compute where it pays. Each stage looks at fewer items and may therefore afford more computation per item:

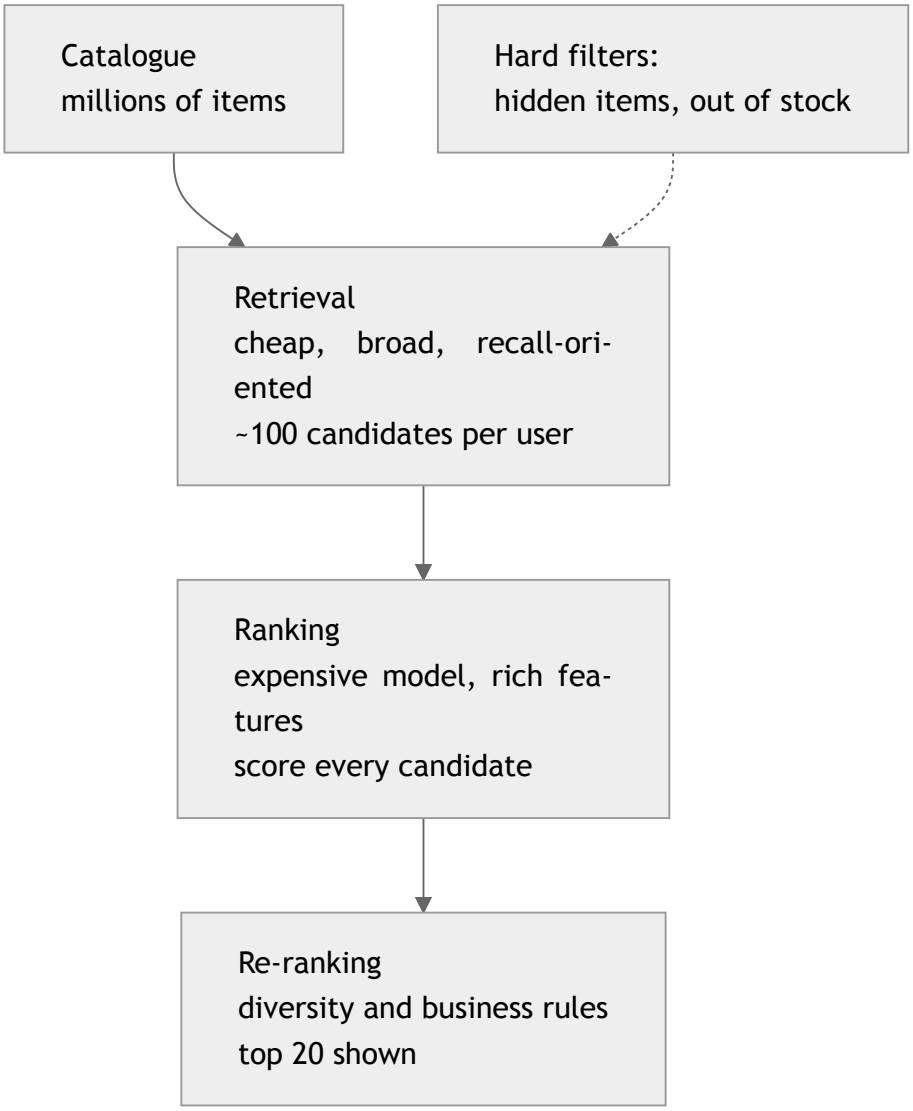


Figure 8.1: *The recommendation funnel. Each stage sees fewer items and can spend more compute per item. Hard filters act at the earliest possible point.*

Three properties follow. **Recall is the metric of stage one:** of the items the user will go on to like, how many are inside my shortlist? Order inside the shortlist is irrelevant and precision is secondary, because the ranker cleans up false positives. **The shortlist is an upper bound:** if a future click is not in the candidate set, no ranker can recommend it. This is the *recall ceiling*. **Cheapness beats elegance:** a retriever should be an index lookup, a group-by or a nearest-neighbour query; anything needing per-pair features belongs in stage two.

Here is the recall idea with numbers you can check by hand. Say Sari goes on to click items $\{P, Q, R, S\}$ the next week. Retriever 1 proposes $\{P, X, Y\}$, retriever 2 proposes $\{P, Q, Z\}$. The pool is the union (everything either one proposed): $\{P, Q, X, Y, Z\}$. Sari's clicks found in the pool are P and Q, so recall is $2/4 = 50\%$. Items R and S are the *ceiling* in action: nothing downstream can bring them back. Retriever 2 alone also found P and Q, so retriever 1 added nothing new that Sari wanted. We will measure exactly this later, as *unique recall*.

> **Did you know?**
>
> **Two networks, one funnel.** YouTube's recommender, as published by Covington, Adams and Sargin, is split into exactly two neural networks: one for candidate generation and one for ranking. The candidate generation network takes the user's activity history and retrieves a small subset, "hundreds" of videos, from a corpus of millions; the ranking network then scores that small set using a much richer set of features. The authors point out that this two-stage design lets them recommend from a very large corpus while still being sure that the few videos shown are personalised, and that it also lets them blend candidates generated by other sources.
>
> Source: Covington et al., RecSys, 2016

The same paper makes a second observation that this repo copies: information from the candidate stage should be passed to the ranker as features, such as which sources nominated a candidate and with what scores. `generate` does this with `cand_recent_score`, `cand_covisit_score`, `cand_catpop_score`, `cand_global_pop_rank` and `cand_n_sources`.

The four retrievers

Each retriever answers a different question and is silent for a different group of users.

Retriever	Question it answers	Analogy	Silent for
`recent`	What did *you* touch lately?	"Your last loans"	Users with no history
`covisit`	What do people who touched *your* items also touch?	"Readers of this also read…"	Users with no history, or whose items nobody else visited
`catpop`	What is popular in the categories *you* like?	"The best-selling shelf you browse"	Users with no clicks *and* no searches
`global_trending`	What is hot for everybody right now?	"The front-of-store display"	Nobody. It is the same list for everyone

All of them share an exponential time decay (the idea from Chapter 3: older actions count less) and a "top-n per user" cut so no source floods the pool:

Two ideas appear in the code below. **Time decay** means that an old action counts less than a fresh one, like yesterday's news versus last month's. We use a **half-life**: after 7 days an action counts half as much, after 14 days a quarter, and so on. A **top-n per user** cut means that each retriever keeps only its best n items for each user, so no single source can flood the pool with hundreds of items. (If a "recent" list had 500 items, it would crowd out the other sources.)

`recommender/recsys/candidates.py · _decay`

```python
def _decay(age_days: pd.Series) -> pd.Series:
    return np.power(0.5, age_days / HALF_LIFE_DAYS)
```

`recommender/recsys/candidates.py · _top_per_user`

```python
def _top_per_user(df: pd.DataFrame, score: str, n: int) -> pd.DataFrame:
    df = df.sort_values(["user_id", score], ascending=[True, False])
    return df.groupby("user_id").head(n)
```

The formula below has three parts: w is how strong the action was (a click is weak, an order is strong), a is how many days ago it happened, and $2^{-a/7}$ is the fading factor. At $a = 0$ it equals 1 (full strength), at $a = 7$ it equals $\frac{1}{2}$.

With `HALF_LIFE_DAYS = 7.0`, for an action of strength w that happened a days before the cutoff:

$$s = w \cdot 2^{-a/7}.$$

An order ($w = 5$) from a week ago counts 2.5, five times a week-old click ($w = 1$ gives 0.5). The weights come from `schema.EVENT_WEIGHT`: click 1, add-to-cart 3, order 5.

Retriever 1: `recent`

The simplest retriever is memory. It sums the decayed weights of all the user's past actions on each item and keeps the top 20 items per user.

recommender/recsys/candidates.py · recent

```
def recent(hist: History) -> pd.DataFrame:
    pos = hist.interactions()
    pos["s"] = pos["w"] * _decay(pos["age_days"])
    out = pos.groupby(["user_id", "item_id"], as_index=False)["s"].sum()
    return _top_per_user(out.rename(columns={"s": "cand_recent_score"}), "cand_recent_score",
N_RECENT)
```

This looks too trivial to be useful, but shoppers revisit things: they look at a phone, leave, and come back two days later. It is also the seed list for `covisit`.

Retriever 2: `covisit`

Co-visitation is the session-based item-to-item collaborative filtering of Chapter 5. `covisitation_matrix` counts how often two items received a positive action in the same session within 30 days and normalises per source item, so each row sums to 1: a distribution $w(a \to b)$ over the items b that usually accompany a.

A **session** is one visit to the app, a stretch of continuous activity. The idea of co-visitation is simple: if many shoppers who looked at a phone case in one session also looked at a screen protector in the same session, the two items are "neighbours". "Normalises per source item" means we turn counts into shares. Say that after phone case A, 30 sessions touched item C and 70 touched item D: the row for A becomes $C = 0.3$, $D = 0.7$, which sums to 1.

The `covisit` retriever then walks from the user's best recent items to their neighbours:

recommender/recsys/candidates.py · covisit

```
def covisit(hist: History, seeds: pd.DataFrame) -> pd.DataFrame:
    top_seeds = _top_per_user(seeds, "cand_recent_score", N_SEEDS)
    m = covisitation_matrix(hist)
    j = top_seeds.merge(m, left_on="item_id", right_on="item_id_a")
    j["s"] = j["cand_recent_score"] * j["w"]
    out = j.groupby(["user_id", "item_id_b"], as_index=False)["s"].sum()
    out = out.rename(columns={"item_id_b": "item_id", "s": "cand_covisit_score"})
    return _top_per_user(out, "cand_covisit_score", N_COVISIT)
```

Think of it as a vote. Each item the user touched recently (a **seed**, as in "a seed that grows into a list of neighbours") casts votes for its neighbours. A stronger and fresher seed has a louder vote, and a neighbour that several seeds vote for gets the most votes in total.

The score of a neighbour b is a weighted vote of the user's seeds:

$$\text{covisit}(u, b) = \sum_{a \in \text{seeds}(u)} \text{recent}(u, a)\; w(a \to b),$$

where `seeds(u)` is the user's ten strongest recent items (`N_SEEDS = 10`) and we keep the top 40 neighbours (`N_COVISIT = 40`). Notice the dependency: `covisit` takes the *output of* `recent` as its input. If the user has no history, `recent` is empty, so `covisit` is empty too.

Retriever 3: `catpop`

The third retriever works on *categories* rather than items. First it builds a user-by-category interest table, `category_affinity`:

An **affinity** is a measure of how much someone likes something, here how much Sari likes each product category (a group such as Pets or Beauty). For Sari, if 6 of her 10 weighted actions were about Pets and 4 about Beauty, her affinity is 60% Pets and 40% Beauty.

recommender/recsys/candidates.py · category_affinity

```
def category_affinity(hist: History) -> pd.DataFrame:
    """User x category interest from clicks/carts/orders AND searches (both decayed), as a
share."""
    pos = hist.interactions().merge(hist.items[["item_id", "category_id"]], on="item_id")
    pos["s"] = pos["w"] * _decay(pos["age_days"])
    se = hist.searched_categories()
    se["s"] = 2.0 * _decay(se["age_days"])  # a search ~ two clicks of intent
    both = pd.concat([pos[["user_id", "category_id", "s"]], se[["user_id", "category_id", "s"]]])
    aff = both.groupby(["user_id", "category_id"], as_index=False)["s"].sum()
    aff["share"] = aff["s"] / aff.groupby("user_id")["s"].transform("sum")
    return aff
```

Two details matter. Searches count: a search for "coffee" becomes a category hit by naive substring matching and weighs 2.0 ("about two clicks of intent"), so a shopper who searched but never clicked still has a profile. And the result is a *share*: each user's weights sum to 1, so a heavy user does not look "more interested" than a light one.

`catpop` takes each user's top three categories and joins them with the most popular available items in those categories:

recommender/recsys/candidates.py · catpop

```
def catpop(hist: History, available: pd.DataFrame) -> pd.DataFrame:
    aff = category_affinity(hist)
    aff = aff.sort_values(["user_id", "share"], ascending=[True,
False]).groupby("user_id").head(N_TOP_CATEGORIES)
    pop = item_popularity(hist, POPULARITY_WINDOW_DAYS).rename("pop")
    items = available[["item_id", "category_id"]].join(pop, on="item_id").fillna({"pop": 0.0})
    items = items.sort_values("pop", ascending=False).groupby("category_id").head(N_PER_CATEGORY)
    items["pop_norm"] = items["pop"] / items.groupby("category_id")
["pop"].transform("max").clip(lower=1e-9)
    j = aff.merge(items, on="category_id")
    j["cand_catpop_score"] = j["share"] * (0.1 + j["pop_norm"])
    return j[["user_id", "item_id", "cand_catpop_score"]]
```

For each category only the 20 most popular items in the last 7 days are kept (`N_PER_CATEGORY = 20`), and popularity is normalised by the category's maximum, giving `pop_norm` in $[0, 1]$. The score is

$$\text{catpop}(u, i) = \text{share}(u, c(i)) \cdot \big(0.1 + \text{pop_norm}(i)\big).$$

The constant 0.1 matters: without it an item with zero recent popularity would score exactly zero and look "not proposed". With it, even a brand-new listing in the user's favourite category stays eligible.

Retriever 4: `global_trending`

The last retriever is deliberately dull:

`recommender/recsys/candidates.py · global_trending`

```python
def global_trending(hist: History, available: pd.DataFrame) -> pd.DataFrame:
    pop = item_popularity(hist, 3.0)
    pop = pop[pop.index.isin(available["item_id"])].nlargest(N_GLOBAL)
    return pd.DataFrame({"item_id": pop.index, "cand_global_pop_rank": np.arange(len(pop))})
```

It takes the popularity of each item over the last 3 days (a shorter window than `catpop`'s 7, since "trending" means *right now*), keeps only in-stock items, and returns the top 30 with their rank. The list is the same for every user. Its value is not personalisation; it is *coverage*: it guarantees every user, including a brand-new visitor with no events, receives at least 30 candidates. The order of operations is important: the in-stock filter happens **before** `nlargest(N_GLOBAL)`. If it were applied afterwards, any sold-out item in the top 30 would leave a hole, and the list would silently shrink.

A tiny worked example

Let us calculate one user by hand. At the cutoff, user u has this history (decay with half-life 7 days):

- Clicked item A seven days ago: $1 \cdot 2^{-1} = 0.5$.
- Ordered item B fourteen days ago: $5 \cdot 2^{-2} = 1.25$.

`recent` therefore proposes B with score 1.25 and A with score 0.5. Now suppose the co-visitation rows are:

seed	neighbour	w
B	C	0.4
B	D	0.6
A	C	0.5
A	E	0.5

Then `covisit` scores are $C = 1.25 \cdot 0.4 + 0.5 \cdot 0.5 = 0.75$, $D = 1.25 \cdot 0.6 = 0.75$ and $E = 0.5 \cdot 0.5 = 0.25$. Item C received a vote from two seeds, which is exactly the kind of agreement we want to reward.

Finally, say the user's affinity is 60% Pets and 40% Beauty. A pet-food item with `pop_norm = 0.5` gets $0.6 \cdot (0.1 + 0.5) = 0.36$, while a pop-0.9 beauty item gets $0.4 \cdot (0.1 + 0.9) = 0.40$. A less popular item in the more-loved category can lose to a popular one elsewhere; the ranker decides later.

Combining the sources: `generate`

We now have four tables with different shapes: `recent` and `covisit` and `catpop` are keyed by (user, item) with their own score column, while `global_trending` is a user-independent list. `generate` stitches them together.

Before the code, a short note on the tool it uses. A **join** glues two tables together by matching rows on a key. An **outer join** keeps rows even when only one table has them, and fills the gaps with "missing" (null). A tiny example for Sari, whose key is the item:

item	`recent` score	`covisit` score
A	0.5	*missing*
B	1.25	0.4
C	*missing*	0.75

Item B, proposed by both sources, appears in **one** row with both scores filled in, not in two rows. That merging of several proposals into one row is **deduplication** (removing duplicates). The *missing* cells record **source attribution**: which sources nominated the item (A came only from `recent`, C only from `covisit`). An *inner* join, by contrast, would keep only B, the items proposed by every source, and throw away almost everything.

recommender/recsys/candidates.py · generate

```
def generate(hist: History, user_ids: np.ndarray) -> pd.DataFrame:
    """Union of all retrievers -> one row per (user_id, item_id) with per-source scores."""
    available = hist.items[hist.items["in_stock"]]
    rec = recent(hist)
    sources = [rec, covisit(hist, rec), catpop(hist, available)]
    users = pd.DataFrame({"user_id": user_ids})
    trending = users.merge(global_trending(hist, available), how="cross")
    sources.append(trending)

    cands = users.merge(sources[0], on="user_id")
    for s in sources[1:]:
        cands = cands.merge(s, on=["user_id", "item_id"], how="outer")
    cands = cands[cands["user_id"].isin(user_ids) & cands["item_id"].isin(available["item_id"])]

    # Negative feedback is a hard filter: never re-show an item the user hid.
    hidden = hist.events.loc[hist.events["event_type"] == "hide", ["user_id",
"item_id"]].drop_duplicates()
    cands = cands.merge(hidden.assign(_hidden=True), on=["user_id", "item_id"], how="left")
    cands = cands[cands["_hidden"].isna()].drop(columns="_hidden")

    score_cols = ["cand_recent_score", "cand_covisit_score", "cand_catpop_score"]
    cands["cand_n_sources"] = cands[score_cols].notna().sum(axis=1) +
cands["cand_global_pop_rank"].notna()
    cands[score_cols] = cands[score_cols].fillna(0.0)
    cands["cand_global_pop_rank"] = cands["cand_global_pop_rank"].fillna(999)
    return cands.reset_index(drop=True)
```

Walk through it in four movements.

1. Union by outer join. The global list is cross-joined with the user list (a **cross join** pairs every row of one table with every row of the other) so that every user gets the same 30 rows. Then the sources are merged with `how="outer"` on `(user_id, item_id)`. An item proposed by any source survives. An item proposed by two sources becomes *one row with two non-null score columns*, not two rows. This is the

single most important design decision in the function: the unit of the ranking stage is the (user, item) pair, and the sources are *features of that pair*.

2. Availability filter. `cands["item_id"].isin(available["item_id"])` removes anything out of stock *at the cutoff*. `History.at` drops the future stock-out time, so the filter uses only information available at request time.

3. The hidden filter. The "Not interested" button logs a `hide` event. `generate` left-joins every `(user, item)` pair with such an event as a marker column (a **left join** keeps every row of the pool and attaches a marker where a match exists) and drops the matches: a *hard filter* on the pool, not a feature.

4. Source bookkeeping. Finally the function computes `cand_n_sources`, the number of sources that proposed the pair, *before* it replaces missing scores with 0.0 (and the missing global rank with 999). Order matters here, and we will use it as a quiz question: once `fillna` has run on the three score columns, they are never null, so each would count as a source for every row.

A tiny example of why that order matters. Say an item was proposed by `recent` and `covisit` but not by `catpop` (and ignore `global_trending` for simplicity). `fillna(0.0)` means "replace every missing cell with 0.0":

Moment	`recent`	`covisit`	`catpop`	counted sources
Before `fillna`	1.25	0.75	*missing*	2 (count the non-missing cells)
After `fillna`	1.25	0.75	0.0	3 (wrong: `catpop` never proposed it)

After `fillna` we can no longer tell "proposed with score 0" from "not proposed", so we count first.

Measuring retrievers: recall, pair recall and unique recall

How do we know that a retriever is good? By the rule from Chapter 2 we need a time split: we build candidates from history before the cutoff, and look at what the user actually did in the following seven days (the *labels*, here 947 evaluation users with 3,357 positive (user, item) pairs). The question for each retriever is then: what share of those future positives does it contain?

The word **label** means the correct answer we hide from the retriever, here "what Sari actually did next". **Pooled** means we put everyone's pairs into one big heap and count together, instead of averaging per person.

The companion script `book/code/ch08_candidates.py` computes three numbers per source:

- **recall**: for each user, the share of their future positives found in the source's candidates, averaged over users. This is `evaluate.candidate_recall`.
- **pair recall**: the same share, but pooled over all positive pairs, so users with many positives count more.
- **unique pair recall**: the share of pooled pairs found by *this* source and by *no other*. This is the number that justifies the source's existence.

Unique recall answers the engineering question "what would we lose if we deleted this retriever?" A retriever with a high recall but zero unique recall is a redundant retriever, and we pay for it in latency

and in feature noise for nothing.

Run it

From the `book/` directory:

```
uv run python code/ch08_candidates.py
```

The first part of the output is the per-source table:

TERMINAL OUTPUT

```
=== Retrievers one by one (eval users: 947, label pairs: 3357) ===
                  cands_per_user  users_with_cands  recall  pair_recall  unique_pair_recall
source
recent                   12.1880            0.9704  0.1833       0.2049              0.0298
covisit                  35.8321            0.9683  0.2818       0.3125              0.0524
catpop                   54.9102            0.9778  0.3274       0.3545              0.0968
global_trending          30.0000            1.0000  0.0969       0.0983              0.0253
UNION (generate)        103.9366            1.0000  0.4645       0.5019                 NaN

recall = mean over users of (their future positives that are in the set);
pair_recall = same, pooled over all (user, item) positives; unique = found by this source only.
```

`cands_per_user` is the average size of each source's list. `users_with_cands` shows `recent` and `covisit` at about 0.97, so a few percent of users have no history, while `global_trending` reaches everyone. The union averages 103.94 candidates per user, the "hundred or so" of the funnel.

Results

The picture below shows the per-source recall against the union and the recall of pairs found *only* by that source.

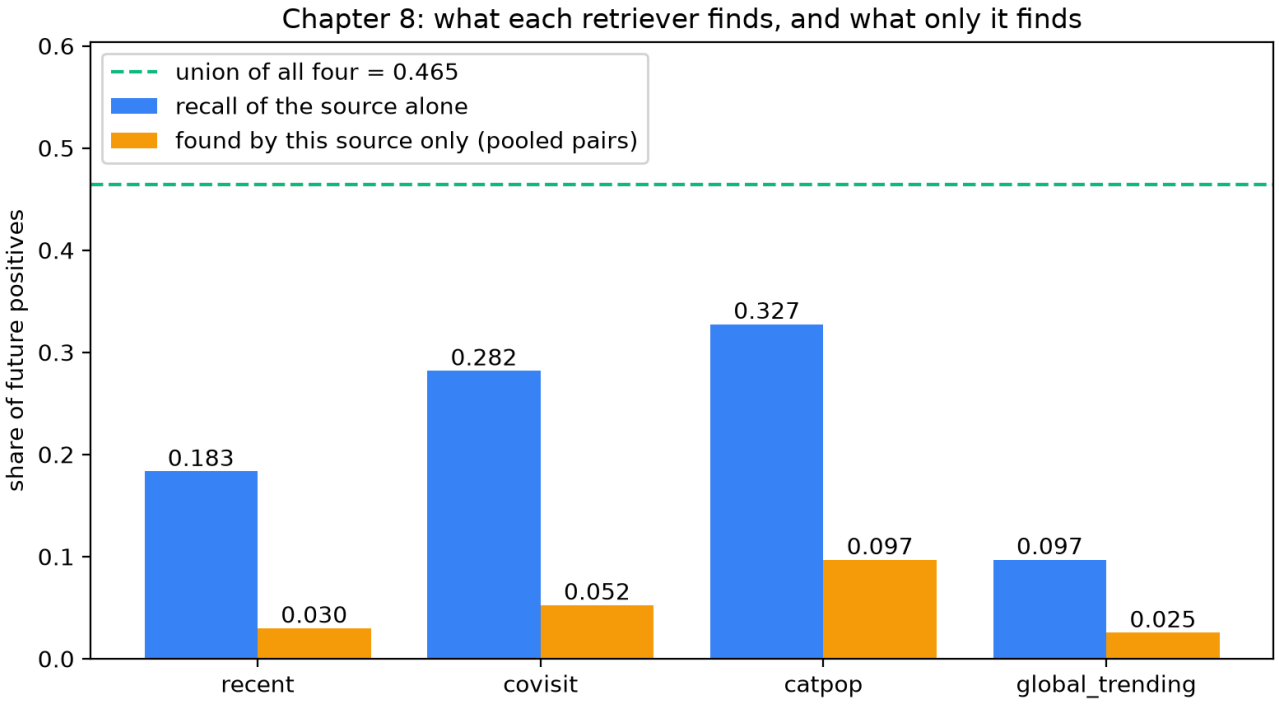


Figure 8.2: *What each retriever finds on its own (blue) and what only it finds (orange), against the recall of the union of all four (dashed line).*

Four observations, tied to the numbers.

No single retriever is enough. The best single source, `catpop`, reaches a recall of 0.3274; the union reaches 0.4645, about 14 points (roughly 42% relative) higher. **Overlap is information, not waste.** The four `cands_per_user` values sum to $12.19 + 35.83 + 54.91 + 30.00 \approx 132.9$, but the union holds 103.94, so about 29 candidates per user (a fifth) were proposed by more than one source.

Every source carries unique value, but unequally. `catpop` finds 9.68% of all positive pairs that nobody else finds, `covisit` 5.24%, `recent` 2.98% and `global_trending` 2.53%. The trending list has the lowest recall yet almost as much unique value as `recent`: it is the cheap safety net for positives no personalised source reached.

Even the union misses over half of the future. A ceiling of 0.4645 means 53.55% of what users click, add or order within the week is never offered to the ranker. Some of it is irreducible (discovery through search, ads, friends); some is addressable with more sources, such as Chapter 7's two-tower retrieval (a model that places users and items in the same "map" and fetches the items nearest to the user).

Recall as a function of how many candidates we keep is plotted below (the values are in `results/ch08_candidates.json`). Each source is ranked by its own score and the union by the repo's heuristic blend.

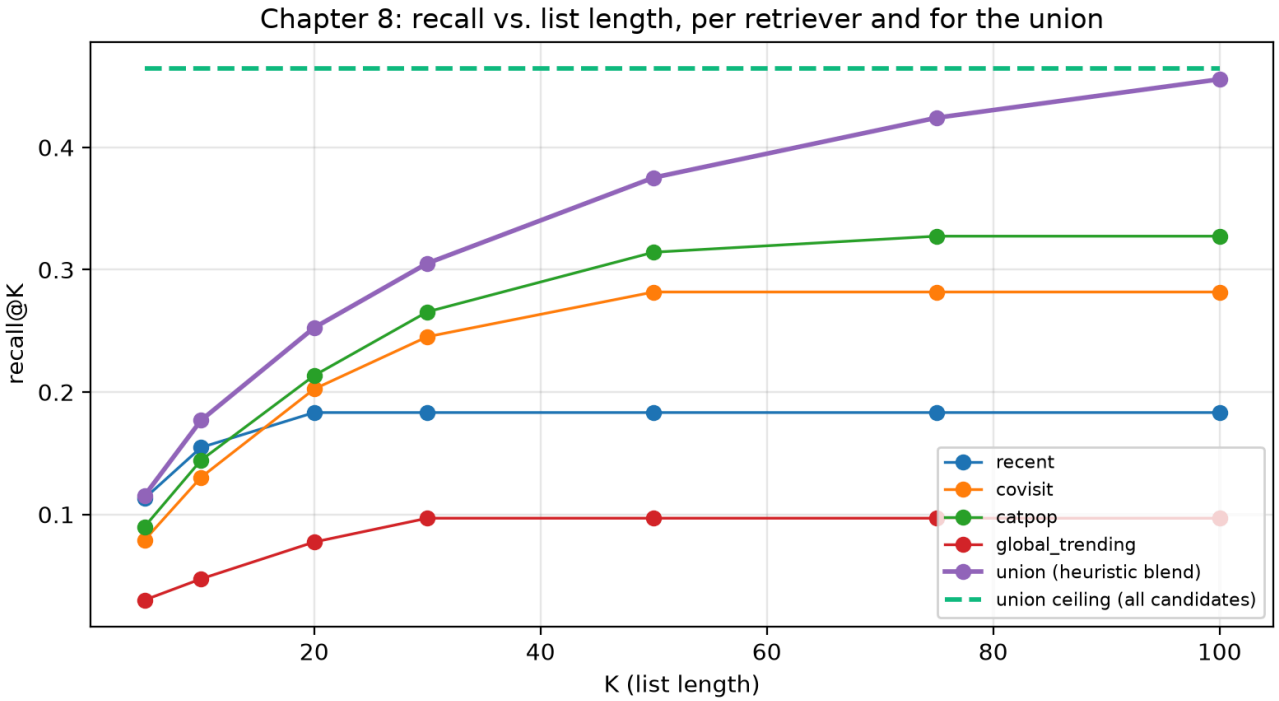


Figure 8.3: *Recall@K for each retriever (by its own score) and for the union ranked by the heuristic blend. Single sources flatten when they run out of candidates; the union keeps climbing toward the ceiling.*

The flat tails are a signature of retrievers with a fixed size. `recent` is at 0.1833 from $K = 20$ on because it never proposes more than 20 items per user. `covisit` stops at 0.2818 from $K = 50$, and `catpop` at 0.3274 from $K = 75$. The union ranked by the heuristic blend continues to climb, and at $K = 100$ it reaches 0.4557, close to the ceiling of 0.4645. It does not reach it exactly because the average pool holds 103.94 candidates per user, so a cut at 100 still drops a few positives that sit at the end of some users' lists. The ceiling is the recall you get when nothing is cut at all.

At $K = 20$ the heuristic blend achieves 0.2526 recall against 0.4645 available. That 0.21 gap is the headroom a better ranker (Chapter 10) can try to capture; it cannot exceed the ceiling, but it can move much closer.

> **Did you know?**
>
> **Why more sources help: agreement is evidence.** Candidates proposed by several retrievers are better bets. In our run, 76,612 candidates were proposed by exactly one source and only 0.9% of them turned out to be positive in the next week. For candidates proposed by all four sources (622 of them), the positive rate was 18.2%, about twenty times higher. This is the reason `cand_n_sources` exists as a ranking feature, mirroring the practice Covington et al. describe of propagating the nominating sources and their scores from candidate generation into the ranker.
>
> Source: Covington et al., RecSys, 2016

The rate rises monotonically with the number of sources (the per-count table is in `results/ch08_candidates.txt`): 0.0090, 0.0319, 0.0806 and 0.1817. Each extra retriever that independently proposes an item raises the odds that the user wants it, and a tree-based ranker uses this signal heavily.

Ranked by the repo's hand-made heuristic blend (a fixed weighted sum of the source scores, with no learning at all), the union already scores an NDCG@20 of 0.1674 (a ranking-quality score from

Chapter 2: higher when relevant items sit near the top of the top 20) with a coverage of 0.5682 (the share of the in-stock catalogue appearing in somebody's top 20). Chapter 10's learned ranker improves on this on the same pool, always *inside the ceiling this chapter draws*.

> **Did you know?**
>
> **Built for scale and for real time.** Linden, Smith and York describe Amazon's item-to-item collaborative filtering as a response to a retailer with "tens of millions of customers and millions of distinct catalog items" whose pages must be returned in real time, in no more than half a second. Their algorithm's online computation scales independently of the number of customers and the number of catalogue items, because the expensive item-to-item work is not done per request. The `covisit` retriever has the same spirit: the co-visitation matrix summarises 30 days of sessions, and serving only joins it with the user's ten seed items.
>
> Source: Linden et al., IEEE Internet Computing, 2003

Hard filters: what must never reach the ranker

Some rules are absolute, not "learn a weight". `generate` applies two while the pool is assembled.

Out-of-stock items

An item that cannot be bought should not be shown, so `generate` restricts every source to `hist.items[hist.items["in_stock"]]`. A filter beats a feature here because of the asymmetry of the error: a sold-out item at position 3 teaches the shopper that the recommendations are unreliable, while losing it from the pool costs nothing, since it could not have produced a sale.

Hidden items

When a user presses "Not interested", the frontend sends a `hide` event; `generate` drops every row carrying one. Why not give the ranker a feature? A learned model is statistical and might still show the hidden item once in a hundred requests, and the user sees exactly that one. Users forgive an irrelevant recommendation but not one they explicitly rejected.

Code walkthrough: how the evaluation script is built

The recall numbers come from `book/code/ch08_candidates.py`. It calls each retriever, restricts the result to evaluation users and in-stock items, and builds a set of `(user_id, item_id)` pairs per source:

book/code/ch08_candidates.py · unique_recall

```
covered = {}
for name, df in per_source.items():
    covered[name] = set(zip(df["user_id"], df["item_id"]))
label_pairs = set(zip(pairs["user_id"], pairs["item_id"]))
```

Unique recall is a set difference. For a source s, with L the set of positive label pairs and C_s the pairs covered by s:

$$\text{unique}(s) = \frac{\left|(L \cap C_s) \setminus \bigcup_{o \neq s} C_o\right|}{|L|}.$$

The *union* row calls the real `candidates.generate`, so the script measures the production path. Global trending has no per-user score, so the script negates its rank to turn "rank 0 is best" into "larger is better". Every source is evaluated on the same 947 users, labels and availability filter; comparing retrievers on different populations is a common mistake.

Pitfalls and how it is done in production

Pitfall 1: optimising precision in retrieval. The ranker removes false positives better than a retriever can, and a discarded candidate is gone for good. Tune for recall at a fixed candidate budget.

Pitfall 2: the candidate budget is a latency budget. Every extra candidate costs feature computation and inference. The constants at the top of `candidates.py` (`N_RECENT`, `N_COVISIT`, `N_PER_CATEGORY`, `N_GLOBAL`) are the dials; plot recall against pool size (our second figure) and stop where the curve flattens.

> **Warning**
> A candidate-stage bug is hard to see: recommendations stay plausible but come from a poorer pool. Monitor offline candidate recall and online `candidates_considered` like any health metric; a sudden drop is usually a broken pipeline, not user behaviour.

How production systems do it. Real marketplaces run many more retrievers in parallel, each with its own latency budget: embedding retrieval with an approximate nearest-neighbour index (Chapter 7), session-based retrieval, "same shop as items you ordered", "frequently bought with", campaign lists. The merge step applies per-source quotas and deduplicates by item; hard filters run against fast stores such as a key-value blocklist; and offline recall at the pool size is a daily monitored job.

Two terms from that paragraph. A **quota** is a fixed number of slots reserved for each source, for example "at most 30 from `catpop`, at most 20 from `recent`", so one chatty source cannot take the entire pool. **Deduplicating by item** means that if three sources propose the same cat-food bag, the pool keeps one copy, not three, but remembers all three nominators (the attribution from earlier). An **approximate nearest-neighbour (ANN) index** is a data structure that finds "items similar to this one" quickly by accepting that it may occasionally miss the exact best match.

Summary

- A two-stage funnel exists because ranking every item for every user with a rich model is unaffordable.
- Stage one is judged on recall and sets a ceiling: the union of all sources contains 46.45% of a user's future positives, and no ranker can exceed that.
- The repo uses four retrievers: `recent` (decayed own history), `covisit` (items viewed in the same sessions), `catpop` (popular in favourite categories, from clicks and searches) and `global_trending` (user-independent fallback).

- Sources are merged with an outer join on (user, item); each score is kept as a feature and `cand_n_sources` counts agreeing sources (positive rate 0.9% for one source, 18.2% for four).
- Judge a retriever by recall *and* unique recall; overlap is common (about 133 proposals per user vs 103.94 in the union).
- Hidden and out-of-stock items are hard filters on the pool, not features.

Quiz

Q8.1 Why is recall, rather than precision, the primary metric for the candidate generation stage?

A. Because precision cannot be computed on implicit feedback.
B. Because an item that is missing from the candidate set can never be recovered by the ranker, while false positives can be removed by it.
C. Because recall is cheaper to compute than precision.
D. Because the ranker only works when its input contains no irrelevant items.

Q8.2 According to the per-source table in `results/ch08_candidates.txt`, which retriever finds the largest share of positive pairs that no other retriever finds?

A. `recent`
B. `covisit`
C. `catpop`
D. `global_trending`

Q8.3 In the output, the positive rate increases from 0.0090 for candidates with one source to 0.1817 for candidates with four sources. What is the most appropriate use of this finding in the system?

A. Remove candidates that only one source proposed.
B. Pass `cand_n_sources` (and the per-source scores) to the ranker as features.
C. Return only the candidates that all four sources propose.
D. Replace the ranker by counting sources.

Q8.4 A visitor with no history opens the home page. Which retriever(s) still produce candidates for them?

A. All four, because the outer join fills missing values.
B. `recent` and `covisit` only.
C. Only `global_trending`.
D. `catpop` and `global_trending`, because `catpop` falls back to popular items overall.

Q8.5 The union of the sources has a recall ceiling of 0.4645, yet the union ranked by the heuristic blend reaches only 0.4557 at K=100. Explain why the figure at K=100 is not exactly the ceiling.

Q8.6 Why does the repo treat "hidden" items as a hard filter in `generate` instead of giving the ranker a `was_hidden` feature and letting it learn to avoid them?

Q8.7 Read the end of `generate`: the line `cands["cand_n_sources"] = ...` comes before `cands[score_cols] = cands[score_cols].fillna(0.0)`. Suppose you swapped the two statements.

What value would `cand_n_sources` take for a candidate that only `covisit` proposed and that is not in the trending list, and why is that a problem?

Q8.8 Read `global_trending`. The code applies `pop[pop.index.isin(available["item_id"])]` before `.nlargest(N_GLOBAL)`. What would be the observable problem if the order were reversed?

Q8.9 Using the `cands_per_user` column of the retriever table, compute the average number of (user, item) pairs per user that were proposed by more than one source, i.e. the sum of the four per-source sizes minus the size of the union. Give the value to one decimal place and as a percentage of the sum.

Q8.10 Hands-on. Open `book/code/ch08_candidates.py` and, at the very top of `main()`, add the line `candidates.N_TOP_CATEGORIES = 5` (the default is 3). Run the script again, then restore the original (`git checkout book/code/ch08_candidates.py book/results`). What changes do you expect to see in the `catpop` row (`cands_per_user`, `recall`) and in the union row, and what happens to the average number of candidates per user? Explain in one or two sentences why the ranker's job gets harder even if recall improves.

Chapter 9 — Feature engineering for ranking

Dewi works on the growth team of a marketplace that looks a lot like Shopee. On Monday morning she shows her manager a ranking model that scores an offline NDCG of 0.50, more than double the previous model. (*Offline* means we scored the model on saved historical data, not on live shoppers. NDCG is a score from 0 to 1 for how well the top of a ranked list matches what people really did; higher is better.) The manager is delighted and asks for a launch date. On Thursday the model goes into a 5% A/B test (a live experiment: 5% of shoppers get the new model, the rest keep the old one), and by Friday the dashboard shows that it performs no better than the old one. In some segments it is worse. Dewi spends the weekend reading her feature pipeline line by line, and at 2 a.m. on Sunday she finds it: one of the columns, "clicks on this item this week", was computed over the whole week *including the days the model was supposed to predict*. The model had not learned to recommend. It had learned to read the answer sheet.

This story, in one form or another, happens at almost every company that builds a recommender. It happens because features are the place where a ranking system touches the real world, and the real world has a time axis that a spreadsheet does not. In Chapter 8 we built the machinery that proposes a few hundred plausible items per user. In this chapter we turn each of those (user, item) pairs into a row of numbers that a ranking model can learn from. How you build those numbers matters more than which model you feed them to.

Before we start, here is the vocabulary of this chapter in one place. Each term gets a full explanation later; this table is only a map.

Term	Plain meaning
Feature	One number that describes a (user, item) pair, for example "how many times this user clicked this item last week".
Feature row	All the features of one pair, side by side. The model reads one row per candidate.
Label	The answer we want the model to predict: did the user click, add to cart or order the item *afterwards*?
Cutoff (written T)	The moment we freeze time. Features may use only what happened before it; labels come from after it.
Leakage	A feature that secretly contains information from after the cutoff, so the model "cheats".
Window	"The last 3 days", "the last 30 days": we count only events inside that stretch before the cutoff.
Smoothing	Pulling a rate that rests on very little evidence back toward the average.
Cross feature	A feature about a *pair* of things, such as this user × this item, rather than about one thing alone.
Sentinel value	A deliberately extreme number (such as 9999) that means "this never happened".

What you'll learn

- Why a feature is only valid if it is computed *as of* a cutoff time, and how `History.at` enforces that rule for the entire code base.
- The eight feature groups used in this repo (46 features in total), what each group is for, and which ones carry personalisation versus quality.
- How time windows (3 / 7 / 14 / 30 days) and Bayesian smoothing turn raw counts into stable rates.
- How cross features such as user × item, user × category and user × shop are built with a few `groupby` calls.
- How to read one complete feature row for a real candidate, and how to compare positives with negatives.
- How to write an automated leakage test, and how big the damage is when you do leak (offline NDCG 0.50 versus an honest 0.21).

Intuition: features are a briefing note

Imagine you are a personal shopper hired to pick ten products for a customer. Before you start, an assistant hands you a one-page briefing note for each candidate product. The note says things like: "This customer looked at this exact product three times last week", "this product is 40% cheaper than similar ones", "the shop answers messages within an hour", "this category is 31% of what the customer has been browsing this month". You do not see raw logs of millions of clicks. You see the note. If the notes are good, even a mediocre shopper makes good picks. If the notes are poor, the cleverest shopper in the world is guessing.

A ranking model is that shopper, and a feature row is the briefing note. Gradient-boosted trees (Chapter 10) are good at combining a few dozen informative numbers, but they cannot invent information that is not in the row. This is why practitioners say that the model is the easy part and the features are where the work is.

There is one more rule for the briefing note, and it is the rule that Dewi violated: **the note must only contain what was knowable at the moment the decision was made**. A personal shopper on Tuesday cannot be told what the customer will buy on Wednesday. In machine learning terms, the feature for a training example must be computed at a cutoff time T (the moment we freeze time), using only events with timestamp strictly before T, while the label comes from events at or after T. We call that property *point-in-time correctness*: every feature is computed "as of" the cutoff, exactly as it could have been at that moment. A violation is called *leakage*: information from the future leaks into the note.

Here is a tiny example with shopper Sari. Suppose we freeze time at day 53:

Day	What Sari did	May a feature use it?
1–52	Clicked three cat-food brands, added one to her cart	Yes, it is before the cutoff
53 = cutoff T	*we freeze time here*	
55	**Orders a 2 kg bag of cat food**	**No**: this is the label, the answer we want to predict

A feature such as "Sari's cat-food clicks in the last 30 days, counted from day 53" is fine. A feature such as "Sari's cat-food orders this week" would include day 55. It would tell the model the answer, and the model would look brilliant offline and useless in real life, because on a real Tuesday nobody knows Wednesday's orders.

The algorithm: from raw events to a feature row

Step 0: fix the cutoff and split the world in two

Let T be a cutoff timestamp. Everything the system may use is the *history* H_T:

$$H_T = \{\, e \in \text{events} : e.\text{ts} < T \,\} \cup \{\, i \in \text{items} : i.\text{listed_at} \le T \,\}$$

In words: the history H_T is "every event whose timestamp (`ts`) is before T, plus every item that was listed in the catalog at or before T". The symbol $\in$ means "is one of", and $\cup$ means "together with". Nothing after T is in the set.

The labels come from the future window $[T, T + 7\ \text{days})$. The square bracket means "including T" and the round bracket means "up to but not including $T + 7$ days". The same code is used at training time (with T in the past, so the future is observable and we can label) and at serving time (with $T =$ now, so the future is empty). *Serving* means answering a real shopper's request live. This symmetry is the single most important design choice in the feature pipeline. If training and serving share one code path, a whole class of bugs known as *training/serving skew* cannot occur. Skew means the model was trained on numbers computed one way and is fed numbers computed another way when it goes live, like practising for a test with one answer key and sitting the real exam with a different one.

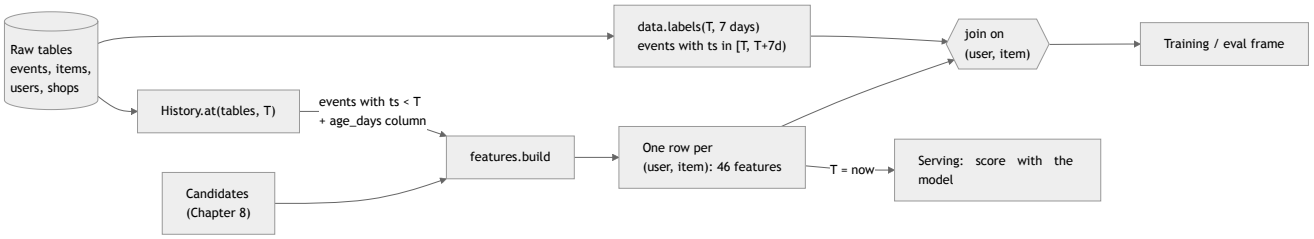


Figure 9.1: *Features are computed from the past of the cutoff, labels from its future; at serving time the future part is simply empty.*

Step 1: add an age to every event

Instead of storing absolute timestamps, `History.at` precomputes `age_days = (T − ts) / 1 day` for every surviving event: how many days before the cutoff it happened. An event from 2 days before the cutoff has age 2; one from 12 hours before has age 0.5. A window then becomes a simple filter: "clicks in the last 7 days" is `age_days < 7`. The age column also makes it impossible to write a feature that depends on the current wall clock, because the only clock is T. (If a feature used today's date, the same training row would change value every day you re-ran it.)

Step 2: aggregate over windows

To *aggregate* means to squash many events into one number, usually by counting. A *windowed count* counts only the events inside a recent stretch of time. For example, "how many times was this item

clicked in the last 7 days?" is one windowed count. Formally, for an entity e (a user, an item, or a pair) and an event type t (click, cart, order…), with window w in days:

$$n_{e,t}(w) = \big|\{\,\text{events of type } t \text{ for } e \text{ with age} < w\,\}\big|.$$

The vertical bars $|\cdot|$ just mean "how many things are in this set".

We use several windows because they answer different questions. A 3-day window says "what is hot right now", a 30-day window says "what is steadily good", and the ratio of the two says whether something is heating up or cooling down. Think of a restaurant: 20 customers a day for a year is steadily good; 5 customers a day last month but 20 a day this week is heating up. The repo's trend feature is

$$\text{trend} = \frac{n_{\text{click}}(3)/3 + 0.1}{n_{\text{click}}(14)/14 + 0.1}.$$

Both terms are *click rates per day*: clicks divided by the number of days in the window. A value above 1 means the last three days are busier than the last two weeks on average. Take an item with 6 clicks in the last 3 days and 14 clicks in the last 14 days. Its recent rate is $6/3 = 2$ per day, its two-week rate is $14/14 = 1$ per day, so the trend is $(2 + 0.1)/(1 + 0.1) \approx 1.91$: heating up. An item with 1 click in the last 3 days and 14 in 14 days gives $(0.33 + 0.1)/(1 + 0.1) \approx 0.39$: cooling down. The $+0.1$ is a small constant that stops a division by zero (an item with no clicks at all gets $0.1/0.1 = 1$, "no change", instead of an error) and keeps tiny counts from producing silly ratios.

Step 3: smooth the rates

Smoothing means pulling a rate that rests on very little data back toward the overall average, so one lucky event cannot fake a star. A raw click-through rate (CTR, the share of times an item was shown that it was clicked) is $\text{clicks}/\text{impressions}$. An *impression* is one time the item was shown to a user. An item that was shown once and clicked once has a CTR of 100%, which is nonsense. Think of a basketball player who sinks his first and only shot: you do not believe he never misses. You would guess "about average" until he has taken many shots, and then trust his own record more and more.

That is exactly the Bayesian recipe (named after the statistician who formalised updating a first guess with evidence). The *prior* p_0 is the first guess, here the global average CTR. We pretend every item starts with s imaginary impressions at that average rate, and let real data gradually take over:

$$\widehat{\text{CTR}} = \frac{\text{clicks} + s \cdot p_0}{\text{impressions} + s}.$$

The hat on CTR means "estimated". s is how many imaginary impressions we add; p_0 is the average rate they carry. With $s = 50$ and a global CTR of $p_0 = 0.14$, the first imaginary 50 impressions contribute $50 \times 0.14 = 7$ imaginary clicks. Take two items:

Item	Clicks	Impressions	Raw CTR	Smoothed CTR
A (brand new, lucky)	1	1	1.000	$(1 + 7)/(1 + 50) = 0.157$
B (established)	30	100	0.300	$(30 + 7)/(100 + 50) = 0.247$

The brand-new item is pulled almost all the way back to the average, while the established item keeps most of its advantage. The parameter s is the "strength of belief" in the prior: the larger it is, the more

evidence you need before the item's own data wins. Without smoothing, a ranker would put every item with one lucky click at the top of the list. The same idea appeared in Chapter 3 for ratings.

Step 4: build cross features

An item feature describes the item for everybody, and a user feature describes the user for every item. Neither one says whether *this* user will like *this* item. That information lives in *cross features* (the name comes from "crossing" two columns, like a row and a column of a table meeting in one cell), which are aggregates over a pair of keys. Sari browsing cat food is a user fact; "Whiskas 2 kg is popular" is an item fact; "Sari clicked Whiskas 2 kg three times" is a cross feature, and it is the one that predicts her next move.

- **user × item**: how often did this user click, cart or order this exact item? How many hours ago? How many times was it shown to them and ignored?
- **user × category**: what share of this user's recent clicks fall in this item's category? How many orders did they place there, and how many days ago?
- **user × shop**: has this user bought from this seller before?
- **user price level × item price**: is the item cheap or expensive *for this user*?

Cross features are where personalisation happens. Most of the raw predictive power in a marketplace ranker comes from the combination "this user has a history with something close to this item". Without them, every user would get the same list, ordered by general quality.

Step 5: handle missing values on purpose

A pair that never interacted has no row in the aggregated tables, so the join (the step that attaches the aggregates to the candidate list) produces missing values. A missing value can mean two different things: "zero" (no clicks) or "unknown or never" (no previous order, so the number of days since the last order does not exist). The repo makes this choice per feature. Counts become 0. Recency features, where 0 would mean "just now", receive large sentinel values: 9999 hours for "never touched" or 999 days for "never ordered". A tree model learns by asking yes/no questions about a feature, such as "were there fewer than 30 days since the last order?". With a sentinel of 999, "no" cleanly means "long ago or never", while a fake 0 would wrongly say "just ordered". Trees split on thresholds, so a sentinel far away from real values separates the two cases cleanly.

> **Did you know?**
>
> **Do top features dominate a ranking model?** In a widely cited study of click prediction at Facebook, the authors boosted trees over hundreds of features and found that the top 10 features were responsible for about half of the total feature importance, while the last 300 features contributed less than 1%. They also found that *historical* features (such as cumulative clicks on an ad) carried considerably more explanatory power than contextual ones: the top 10 features were all historical. Contextual features mattered mostly for cold start.
>
> Source: He et al., ADKDD (KDD workshop), 2014

Code walkthrough

The gatekeeper: `History.at`

All of the point-in-time logic of the project lives in one class method. Read it carefully, because every feature in the system depends on it.

recommender/recsys/data.py · History.at

```python
@classmethod
def at(cls, tables: dict[str, pd.DataFrame], cutoff: pd.Timestamp) -> History:
    ev = tables["events"]
    ev = ev[ev["ts"] < cutoff].copy()
    ev["age_days"] = (cutoff - ev["ts"]) / pd.Timedelta(days=1)
    se = tables["searches"]
    se = se[se["ts"] < cutoff].copy()
    se["age_days"] = (cutoff - se["ts"]) / pd.Timedelta(days=1)
    cats = tables["categories"]
    items = tables["items"]
    items = items[items["listed_at"] <= cutoff].merge(
        cats[["category_id", "is_consumable", "median_price"]], on="category_id"
    )
    # Stock status is known at cutoff; the *future* stock-out time is not.
    items["in_stock"] = items["out_of_stock_at"].isna() | (items["out_of_stock_at"] > cutoff)
    items = items.drop(columns="out_of_stock_at")
    return cls(cutoff, ev, se, items, tables["users"], tables["shops"], cats)
```

Walk through it line by line:

1. `ev["ts"] < cutoff` keeps events strictly *before* the cutoff. The inequality is strict, and that matters: an event stamped exactly at T belongs to the label window, not to the history. The label function uses `ts >= cutoff`, so every event lands on exactly one side.
2. `age_days` is computed against the cutoff, never against `now()`.
3. Searches get the same treatment, because a search query is as much a behavioural signal as a click.
4. The catalog is filtered with `listed_at <= cutoff`. An item listed next Tuesday does not exist for a model standing on Friday. This is the catalog version of the same rule, and it is why a brand-new product cannot appear in training candidates before its launch.
5. `in_stock` is true when the item has no stock-out time *or* the stock-out happens after the cutoff, and the future stock-out time is dropped, so the model cannot learn "this item will sell out in two days". Price, rating and rating count are shown as they are *now*; in this simulated data they never change, which keeps the lab honest. A real catalog changes, which is why production keeps snapshots (see the pitfalls).

The `History` dataclass is frozen and nothing else reads the raw `events` table, so a feature function that takes a `History` cannot see the future. The only code that looks forward is `data.labels`: it selects events in $[T, T + 7d)$, maps click / add_to_cart / order to graded labels 1 / 2 / 3 via `schema.LABEL_GAIN`, and keeps the maximum per (user, item).

Building blocks

Two tiny helpers in `features.py` do most of the aggregation. `_count` filters one event type, optionally to a window with `age_days < max_age`, then groups by any key list (user, item, or pair); `groupby` is

the pandas operation "put rows with the same key together and count them", like sorting receipts into one pile per customer. `_smoothed` is the formula of Step 3 on whole columns: `num` is clicks, `den` impressions, `prior` is p_0, `strength` is s. User features (`user_features`) use the same helpers: four windowed counters, a CTR smoothed with strength 20, and recency and account age measured against the cutoff, never the wall clock.

Item features

recommender/recsys/features.py · item_features

```
def item_features(hist: History) -> pd.DataFrame:
    ev = hist.events
    it = hist.items.copy()
    it["item_price"] = it["price"]
    it["item_discount_pct"] = it["discount_pct"]
    it["item_price_vs_category"] = it["price"] / it["median_price"]
    it["item_free_shipping"] = it["free_shipping"].astype(int)
    it["item_rating_avg"] = it["rating_avg"]
    it["item_rating_count"] = np.log1p(it["rating_count"])
    it["item_age_days"] = (hist.cutoff - it["listed_at"]) / pd.Timedelta(days=1)
    it["item_is_consumable"] = it["is_consumable"].astype(int)

    counts = [
        ("item_impressions_7d", "impression", 7),
        ("item_clicks_7d", "click", 7),
        ("item_orders_7d", "order", 7),
        ("_impr_30d", "impression", 30),
        ("_clicks_30d", "click", 30),
        ("_orders_30d", "order", 30),
        ("_clicks_3d", "click", 3),
        ("_clicks_14d", "click", 14),
        ("_orders_all", "order", None),
        ("_returns_all", "return", None),
    ]
    for name, et, age in counts:
        it = it.merge(_count(ev, ["item_id"], et, age, name), on="item_id", how="left")
    it = it.fillna({name: 0 for name, _, _ in counts})

    ctr = it["_clicks_30d"].sum() / max(it["_impr_30d"].sum(), 1)
    cvr = it["_orders_30d"].sum() / max(it["_clicks_30d"].sum(), 1)
    ret = it["_returns_all"].sum() / max(it["_orders_all"].sum(), 1)
    it["item_ctr_30d"] = _smoothed(it["_clicks_30d"], it["_impr_30d"], ctr, 50)
    it["item_cvr_30d"] = _smoothed(it["_orders_30d"], it["_clicks_30d"], cvr, 20)
    it["item_return_rate"] = _smoothed(it["_returns_all"], it["_orders_all"], ret, 10)
    it["item_trend"] = (it["_clicks_3d"] / 3 + 0.1) / (it["_clicks_14d"] / 14 + 0.1)

    sh = hist.shops.rename(columns={"is_mall": "shop_is_mall", "cancel_rate": "shop_cancel_rate",
"response_rate": "shop_response_rate"})
    sh["shop_is_mall"] = sh["shop_is_mall"].astype(int)
    it = it.merge(sh[["shop_id", "city", "shop_rating", "shop_is_mall", "shop_cancel_rate",
"shop_response_rate"]], on="shop_id")
    return it.rename(columns={"city": "shop_city"})
```

The first block of assignments copies static catalog attributes into feature columns. `item_rating_count` goes through `np.log1p`, which computes $\log(1+x)$ and squeezes large numbers together, because counts are heavy-tailed (a few items have huge counts, most have small ones, and 5 versus 50 ratings matters more than 5,005 versus 5,050). `item_price_vs_category` divides by the category median (the middle price when all prices are sorted) so a phone case and a laptop are compared on "cheap or premium for what it is": 0.60 means the item costs 60% of its category's typical price. The middle block declares ten counters as `(name, event_type, window)` triples; underscore names are temporary inputs

to the smoothed rates (strengths 50 for CTR, 20 for conversion rate (CVR, orders per click), 10 for returns, lower for rarer events). Then come the trend formula from Step 2 and a join with shop quality.

Cross features

Three functions build the cross features. They look different but share one pattern: *join events to the second key, then group by the pair*.

recommender/recsys/features.py · user_item_features

```
def user_item_features(hist: History) -> pd.DataFrame:
    ev = hist.events
    keys = ["user_id", "item_id"]
    ui = ev[keys].drop_duplicates()
    for name, et in [("ui_clicks", "click"), ("ui_carts", "add_to_cart"), ("ui_orders", "order"),
("_impr", "impression")]:
        ui = ui.merge(_count(ev, keys, et, None, name), on=keys, how="left")
    ui = ui.fillna(0)
    pos = ev[ev["event_type"].isin(schema.EVENT_WEIGHT)]
    last = (pos.groupby(keys)["age_days"].min() * 24).rename("ui_hours_since_last")
    dwell = ev.groupby(keys)["dwell_sec"].max().rename("ui_max_dwell_sec")
    ui = ui.join(last, on=keys).join(dwell, on=keys)
    ui["ui_ignored_impressions"] = np.where(ui["ui_clicks"] == 0, ui["_impr"], 0)
    return ui.drop(columns="_impr")
```

`user_item_features` counts clicks, carts, orders and impressions per pair. `ui_ignored_impressions` is the number of times the pair was shown *only if the user never clicked it*, which is the fatigue signal: an item that was displayed ten times without a click is probably not interesting. `ui_hours_since_last` uses only positive actions (click, cart, order) and converts days to hours; it measures *recency*, how long ago the last interaction was. `ui_max_dwell_sec` is the longest product-page stay (*dwell time*): a user who spent seven minutes on a product page is telling us more than one who bounced after two seconds.

The user × category block computes the click *share* (`transform("sum")` divides by the user's total, making heavy and light clickers comparable: 3 of 10 clicks and 30 of 100 clicks are both a 30% share), plus searches mapped to categories and order recency. The user × shop block is the same pattern. `user_price_level` is the median of `price / category_median_price` over the user's clicks, that is, how expensive the things this user clicks are compared with their category's norm; dividing the item's own ratio by it gives `price_ratio_to_user`, so 0.62 means the item costs 62% of what this user usually pays relative to category norms. A ratio like this is useful because it removes the unit: it says "cheap for this shopper", whether the item costs 20,000 or 2,000,000 rupiah.

Putting it together: `build`

recommender/recsys/features.py · build

```
def build(cands: pd.DataFrame, hist: History) -> pd.DataFrame:
    """Attach every feature in schema.FEATURES to the candidate rows."""
    df = cands.merge(item_features(hist), on="item_id", how="left")
    df = df.merge(user_features(hist), on="user_id", how="left")
    df = df.merge(user_item_features(hist), on=["user_id", "item_id"], how="left")
    df = df.merge(user_category_features(hist), on=["user_id", "category_id"], how="left")
    df = df.merge(user_shop_features(hist), on=["user_id", "shop_id"], how="left")
    df = df.join(user_price_level(hist), on="user_id")
    df = df.merge(hist.users[["user_id", "city"]].rename(columns={"city": "user_city"}),
on="user_id", how="left")

    df["price_ratio_to_user"] = df["item_price_vs_category"] / df["_user_price_level"].fillna(1.0)
    df["same_city"] = (df["user_city"] == df["shop_city"]).astype(int)
    fill = {
        "ui_hours_since_last": NEVER_HOURS,
        "uc_days_since_order": NEVER_DAYS,
        "user_days_since_active": NEVER_DAYS,
    }
    df = df.fillna(fill)
    zero = ["ui_clicks", "ui_carts", "ui_orders", "ui_ignored_impressions", "ui_max_dwell_sec",
"uc_click_share_30d",
            "uc_searches_30d", "uc_orders", "us_clicks", "us_orders", "user_clicks_7d",
"user_clicks_30d",
            "user_orders_30d"]  # fmt: skip
    df[zero] = df[zero].fillna(0)

    # Keep ids for grouping/re-ranking, then exactly the registered features, in order.
    out = df[["user_id", "item_id", "category_id", "shop_id"] + schema.FEATURE_NAMES]
    assert list(out.columns[4:]) == schema.FEATURE_NAMES
    return out.reset_index(drop=True)
```

`build` takes the candidate frame from Chapter 8 (`user_id`, `item_id`, the five candidate-source scores) and does left joins: candidates stay, features are attached. Two details are easy to miss. The `assert` at the end guarantees that the output columns equal `schema.FEATURE_NAMES` in the same order, so a feature cannot be added to the code and forgotten in the schema, or vice versa. And the explicit `fill` and `zero` dictionaries are where the missing-value policy of Step 5 is written down in one place. Because the same function runs offline and inside the gRPC service, the model sees identical column semantics in both.

The eight feature groups

The schema registers every feature with a name, a group, and a description. The groups and their sizes:

Group	Count	What it describes	Example features
`candidate`	5	Which retrievers proposed the item and how strongly	`cand_recent_score`, `cand_covisit_score`, `cand_n_sources`
`user`	8	Activity level and profile of the user	`user_clicks_30d`, `user_ctr_30d`, `user_days_since_active`
`item`	15	Price, quality, popularity, freshness, trend	`item_ctr_30d`, `item_cvr_30d`, `item_trend`, `item_age_days`
`shop`	4	Seller reliability	`shop_rating`, `shop_is_mall`, `shop_cancel_rate`
`user_item`	7	History of this user with this exact item	`ui_clicks`, `ui_hours_since_last`, `ui_max_dwell_sec`, `price_ratio_to_user`
`user_category`	4	User's relation to the item's category	`uc_click_share_30d`, `uc_days_since_order`
`user_shop`	2	User's relation to the seller	`us_clicks`, `us_orders`
`context`	1	Request or pair context	`same_city`

The counts add up to 46. Notice the proportions. Fifteen features describe the item alone, and it is the largest group. But the `user_item` (7), `user_category` (4), `user_shop` (2) and `context` (1) groups together hold 14 pair-level features. Those 14 are the ones that make two users see different lists. The other groups mostly measure *quality*: how good the item, the shop or the user's general activity is, the same for everyone who sees the item.

> **Did you know?**
>
> **Leakage has a formal definition.** Kaufman, Rosset and Perlich gave leakage a precise statement in data mining: leakage is the introduction of information about the target that should not legitimately be available to the model at prediction time. They catalogued how it sneaks in, through features that are proxies of the label, through the way a data set is sampled, and through time, and used competition data sets to show how often it decides winners and losers.
>
> Source: Kaufman et al., ACM Transactions on Knowledge Discovery from Data, 2012

The leakage test

Discipline is good, but an automated test is better. The repo ships a test that rewrites the future and checks that nothing in the features changes.

`recommender/tests/test_point_in_time.py · _build`

```
def _build(tables, cutoff):
    hist = data.History.at(tables, cutoff)
    users = tables["users"]["user_id"].to_numpy()
    feats = features.build(candidates.generate(hist, users), hist)
    return feats.sort_values(["user_id", "item_id"]).reset_index(drop=True)
```

The helper builds the entire feature frame at a cutoff, candidates included, and sorts it so row order cannot cause false failures.

recommender/tests/test_point_in_time.py · test_features_ignore_events_after_cutoff

```
def test_features_ignore_events_after_cutoff(tables):
    cutoff = pd.Timestamp(SimConfig().start_date) + pd.Timedelta(days=CUTOFF_DAY)
    before = _build(tables, cutoff)

    # Rewrite the future: every post-cutoff event becomes an order of item 0, and
    # every post-cutoff search becomes a query for a real category.
    future = tables["events"]["ts"] >= cutoff
    assert future.any()
    tampered = dict(tables)
    ev = tables["events"].copy()
    ev.loc[future, ["event_type", "item_id"]] = ["order", 0]
    tampered["events"] = ev
    se = tables["searches"].copy()
    se.loc[se["ts"] >= cutoff, "query"] = "diapers"
    tampered["searches"] = se

    pd.testing.assert_frame_equal(before, _build(tampered, cutoff))
```

The test runs in four steps. It builds features at day 30 of the simulation. Then it tampers with the future: *every* event after the cutoff becomes an order of item 0, and every later search becomes a query for "diapers". Then it rebuilds, and `assert_frame_equal` demands that the two frames are identical, cell by cell. If a new feature reads even one future event, the tampered run differs and the test fails. The test is deliberately brutal. A subtle leak, such as using a 30-day window that accidentally extends forward through `abs(age_days)`, would produce a tiny difference in a handful of cells, and it would still be caught.

Run it

The companion script `ch09_features.py` builds the frames, prints one complete feature row, compares positives and negatives and then runs the leakage experiment. From the `book/` directory:

```
uv run python code/ch09_features.py
```

First the frames. The training frame uses a cutoff one week earlier than the evaluation frame, with labels from the following seven days in each case, mirroring the real training pipeline.

TERMINAL OUTPUT

```
=== Frames ===
train frame:   78793 rows, 781 users, 1221 positive rows  (cutoff 2026-09-16 00:00:00)
eval  frame:   98428 rows, 947 users, 1685 positive rows  (cutoff 2026-09-23 00:00:00)
number of features: 46 in 8 groups
```

About 78,800 training rows and 98,400 evaluation rows come from 781 and 947 users respectively. The positive rate is a little over 1.7%: out of every 100 candidate rows, fewer than 2 are followed by a click, cart or order. That is typical, because for the vast majority of candidates the user will do nothing in the next week. We will return to this class imbalance (far more negatives than positives) in Chapter 10.

Now one candidate row with all 46 features: the best heuristic-scored candidate for user 42, abbreviated here to the candidate and user groups (the full row is in the results file).

TERMINAL OUTPUT

```
=== One candidate row, every feature (user 42, best heuristic score) ===
user_id=42 item_id=2381 category_id=9 shop_id=10 label=0
  [candidate]
    cand_recent_score      =     11.629   Time-decayed weight of the user's own past actions on
this item
    cand_covisit_score     =      3.564   Co-visitation: how often this item is clicked in the
same session as the user's recent items
    cand_catpop_score      =      0.381   Popularity of the item inside the user's favourite
categories
    cand_global_pop_rank   =      0.000   Rank in global trending list (999 = not in list)
    cand_n_sources         =      4.000   How many retrievers proposed this item
  [user]
    user_clicks_7d         =      9.000   User's clicks in the last 7 days (activity level)
    user_clicks_30d        =     45.000   User's clicks in the last 30 days
    user_orders_30d        =     14.000   User's orders in the last 30 days
    user_ctr_30d           =      0.132   User's click-through rate (smoothed)
    user_days_since_active =      1.341   Days since the user's last event
    user_account_age_days  =    336.000   Days since signup
    user_device_tier       =      3.000   Phone tier 1-3
    user_age_bucket        =      3.000   Age bucket 0-3
  [item]
```

Results: reading a feature row

Read the row like a detective. The candidate is item 2381, and the briefing note says: the user has interacted with it directly (`cand_recent_score` is 11.63, a large score from the user's own recent actions, where newer actions weigh more), the item was proposed by all four retrievers (`cand_n_sources` = 4) and it is number 1 in the global trending list (`cand_global_pop_rank` = 0, ranks count from 0). The user is active (45 clicks in 30 days, 14 orders) and was last seen 1.3 days ago. The item is cheap for its category (`item_price_vs_category` = 0.60), has the best possible rating (5.0) with $e^{5.673} - 1 \approx 290$ ratings (the table stores $\log(1 + \text{count})$, so we undo it with $e^x - 1$), and it is popular right now (29 clicks in seven days). The cross features tell the personal story: six clicks, six add-to-carts and five orders on this exact item, and 31% of the user's category clicks land in this category.

Do positives look different from negatives?

A feature is only useful if its distribution differs between candidates that were later engaged with and candidates that were not. The script prints means and medians for six representative features.

TERMINAL OUTPUT

```
=== Positives vs negatives (eval frame): mean of selected features ===
                     mean_positive  mean_negative  median_positive  median_negative
feature
cand_recent_score            0.847          0.099            0.000            0.000
item_ctr_30d                 0.144          0.141            0.135            0.129
uc_click_share_30d           0.447          0.245            0.462            0.143
item_clicks_7d               5.893          5.204            4.000            3.000
item_rating_count            4.555          4.414            4.920            4.682
price_ratio_to_user          1.073          1.118            0.980            0.982
base rate of positives among candidates: 0.0171
```

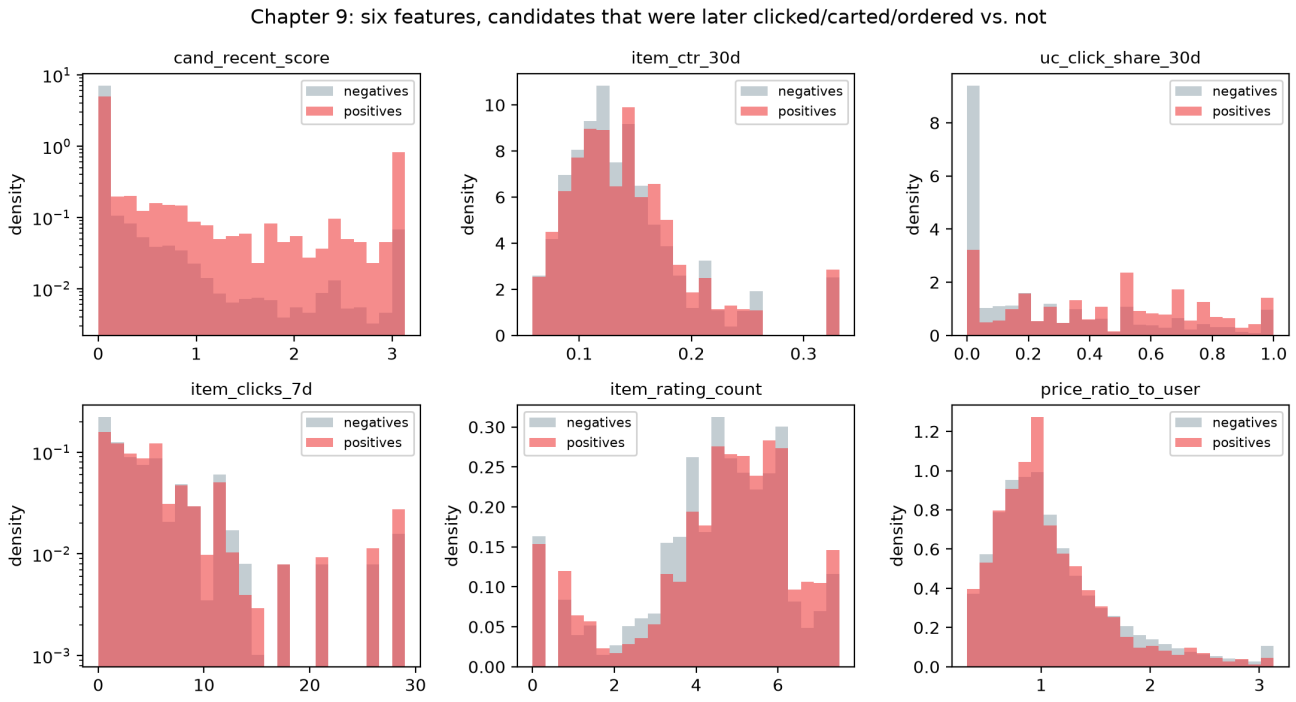


Figure 9.2: *Distribution of six features for positives and negatives*

`cand_recent_score` separates the classes sharply (mean 0.847 for positives versus 0.099; the user's own recent actions are the strongest predictor), and `uc_click_share_30d` does the same in a personalised way (0.447 versus 0.245; negatives pile up at zero). `item_ctr_30d` (0.144 versus 0.141), `item_rating_count` (4.555 versus 4.414) and `price_ratio_to_user` (1.073 versus 1.118) barely differ, and `item_clicks_7d` is only mildly higher (5.893 versus 5.204). Weak-looking features can still help a tree in interaction with others, so never delete a feature purely from a histogram.

What a leak looks like

Now the experiment that explains Dewi's weekend. The script adds *one* extra column, `leaky_future_clicks`, which counts the user's clicks on the item inside the label window. Here is the exact line that creates the leak:

book/code/ch09_features.py · label_window_clicks

```
def label_window_clicks(frame: pd.DataFrame, start: pd.Timestamp) -> pd.Series:
    """The LEAK: how often the user clicked this item inside [start, start+7d) - the future."""
    ev = common.load_tables()["events"]
    win = ev[(ev["ts"] >= start) & (ev["ts"] < start + pd.Timedelta(days=7)) & (ev["event_type"]
== "click")]
    n = win.groupby(["user_id", "item_id"]).size().rename("n").reset_index()
    m = frame[["user_id", "item_id"]].merge(n, on=["user_id", "item_id"], how="left")
    return pd.Series(m["n"].fillna(0).to_numpy(), index=frame.index)
```

It looks innocent: an ordinary `groupby` whose only unusual element is the time filter `ts >= start`, the opposite of what `History.at` enforces. The same LightGBM ranker is trained twice, with the 46 honest features and with 47 (honest plus the leak), and both are evaluated on the same frame.

TERMINAL OUTPUT

```
=== Leakage demo: add the user's clicks on the item *inside the label window* ===
               recall@5  recall@10  recall@20  recall@50  order_recall@20  ndcg@20  coverage@20
method
honest ranker    0.1503     0.2015     0.2725     0.3824            0.421   0.2116       0.4245
leaky ranker     0.4538     0.4641     0.4645     0.4645            0.616   0.5035       0.5907

leaky_future_clicks share of total gain: 88.5% (rank 1 of 47)
ndcg@20 inflation: 2.38x
The leaky model looks brilliant offline and would be useless live: in production the future is not
available.
```

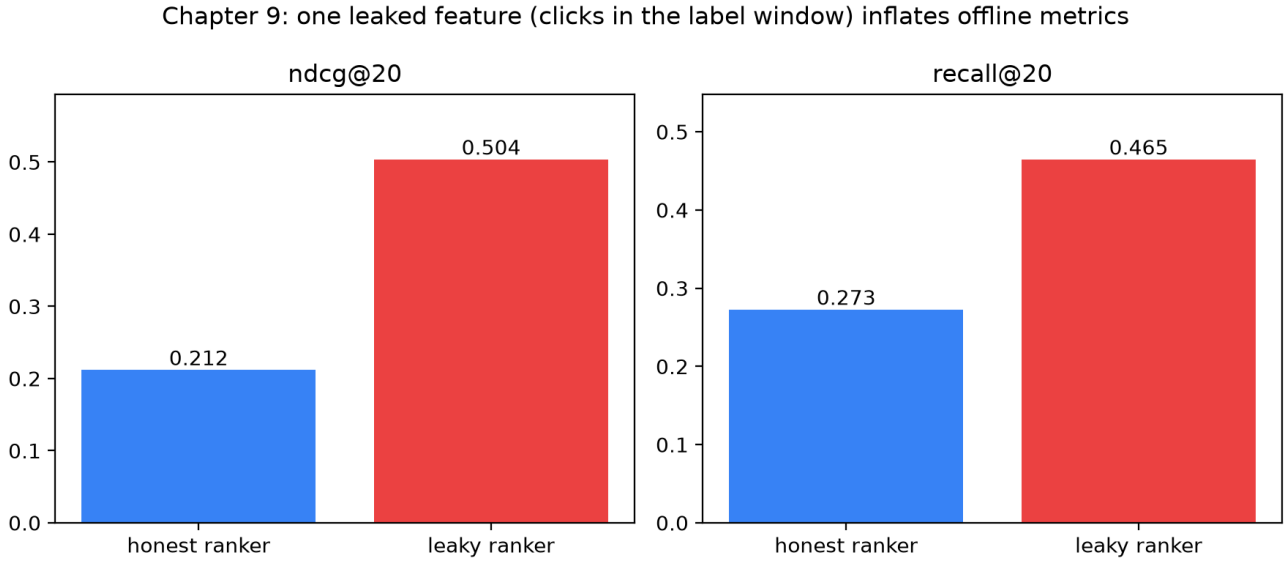


Figure 9.3: *Offline metrics with and without a leaked feature*

The numbers are dramatic. NDCG@20 (the ranking-quality score, looking at the top 20) jumps from 0.2116 to 0.5035, a factor of 2.38, and recall@20 (the share of things the user really engaged with that appear in the top 20) from 0.2725 to 0.4645. The leaked column takes 88.5% of all the gain in the model, ranking first of 47 features. (*Gain* is how much a feature helps the model's trees reduce their errors, summed over all their splits; a feature with 88.5% of the gain is doing almost all the work.) Look at the recall columns for the leaky ranker: recall@10, @20 and @50 are all about 0.464, so essentially every positive that is reachable was already found within the first ten positions. A ranker can only reorder the candidates it is given, so the plateau is the candidate recall ceiling we discussed in Chapter 8: the positives beyond it were never proposed by any retriever. A model that has the answers pushes straight up against that ceiling.

A 2.4× improvement from one column should make you suspicious, not happy. Red flags: a single new feature takes most of the gain; offline metrics jump far more than any earlier change; recall saturates at the candidate ceiling at tiny K; or importance concentrates on features that describe the outcome (`total_orders`, anything computed `all_time` over the whole table).

Pitfalls and how it is done in production

1. Snapshotting slowly changing attributes. In this repo the catalog is a single table, and price or rating changes would overwrite history. A real system needs *point-in-time joins*: the value that was cur-

rent at T. Feature stores (Chapter 14) record every feature value with an event timestamp and implement an "as-of join" that picks the latest value not after T. Without it, the rating of an item at training time includes reviews that were written *because* people bought it after the cutoff. That is the leakage Dewi would have found next.

2. Label-adjacent aggregates. "Total orders of this item" over the full table is the classic leak, because it includes the orders you are trying to predict. The same trap hides in rates, in "last purchase date", in any `max` or `min` over an unbounded column. The `age_days < w` filter inside `_count` is the guard: every windowed aggregate is computed over `History` only.

3. Training/serving skew. The best defence is one code path: `features.build` serves training, evaluation and the live gRPC service. Systems with separate batch and online pipelines need parity tests that run both on the same inputs, because a different rounding, time zone or missing-value rule silently degrades the model.

4. Feature creep. Every feature is a dependency that costs serving time and can break when its source changes. Add a group, run an ablation (Chapter 10), and remove groups that do not help.

> **Did you know?**
>
> **Data dependencies are technical debt.** In "Hidden Technical Debt in Machine Learning Systems", Sculley and colleagues describe *unstable* and *underutilized* data dependencies: input signals that change behaviour over time, or that provide little incremental modelling benefit yet make the system vulnerable to upstream changes. They recommend detecting underutilized dependencies with regular, exhaustive leave-one-feature-out evaluations, which is exactly the group ablation we run in Chapter 10.
>
> Source: Sculley et al., NeurIPS, 2015

5. Cost at serving time. This chapter's code recomputes every feature table on each `Recommend` request, which is why a request takes roughly a second (measured in Chapter 13). Production precomputes the heavy parts (item statistics by a streaming job, user features in a key-value store) and computes only cheap pair features such as `same_city` at request time. The *definitions* stay identical.

Summary

- A feature is valid only if it is computed from events strictly before the cutoff T; labels come from $[T, T + 7d)$. Violating this is leakage.
- `History.at` is the single gate: it filters events, searches and the catalog by the cutoff, adds `age_days`, and hides future stock-outs. All feature code reads only a `History`.
- Windows (3/7/14/30 days) separate "hot now" from "steadily good"; the trend feature compares per-day rates and uses a small constant to stay stable.
- Smoothing, $(\text{num} + s\,p_0)/(\text{den} + s)$, keeps rates from extreme values when data is thin; s is how much evidence you demand.
- The 46 features fall into eight groups: candidate, user, item, shop, user_item, user_category, user_shop and context. The 14 pair-level cross features carry the personalisation.
- Missing values are a design decision: counts become 0, recency features get sentinel values (9999 hours, 999 days) that trees can split on.

- A leaked column took 88.5% of the model's gain and raised offline NDCG@20 from 0.2116 to 0.5035; a result that good from one feature is a warning sign, not a success.
- Automate the guard: a test that rewrites the future and demands identical features catches leaks before they reach a dashboard.

Quiz

Q9.1 What does point-in-time correctness require of a feature used to train a ranker?

A. It must be normalised to zero mean and unit variance.
B. It must be computed only from information that was available before the cutoff time of the training example.
C. It must be computed from the same number of events for every user.
D. It must be recomputed every hour in production.

Q9.2 In `History.at`, why is the filter `ev["ts"] < cutoff` strict rather than `<=`?

A. Because floating-point timestamps cannot be compared with `<=`.
B. Because events exactly at the cutoff belong to the label window `[cutoff, cutoff + days)`, so each event is used on exactly one side.
C. Because the database stores timestamps with one-second resolution.
D. Because impressions are not logged at exactly the cutoff.

Q9.3 An item shown once and clicked once has a raw CTR of 100%. Which statement about the smoothed CTR $(\text{clicks} + s\,p_0)/(\text{impressions} + s)$ is correct?

A. It stays at 100% because the item has no negatives.
B. It moves close to the global average p_0, and moves away from it only as the item collects more impressions.
C. It becomes exactly zero until the item has at least s impressions.
D. It no longer depends on the item's own clicks.

Q9.4 In one or two sentences: why does the code use a sentinel such as 9999 for `ui_hours_since_last` rather than 0 when the user never touched the item?

Q9.5 Explain the difference between an item feature, a user feature and a cross feature, and name one example of each from this chapter. Which kind carries most of the personalisation?

Q9.6 Read the test below (included earlier in this chapter as `test_features_ignore_events_after_cutoff`). Suppose someone adds a feature `item_orders_all_time` that counts orders per item from a module-level variable holding the full, unfiltered events table, instead of from `hist.events`. Would the test fail, and why? What exactly in the test makes the failure visible?

```
ev.loc[future, ["event_type", "item_id"]] = ["order", 0]
pd.testing.assert_frame_equal(before, _build(tampered, cutoff))
```

Q9.7 An item has 3 clicks and 10 impressions. The global CTR is 0.10 and the smoothing strength is $s = 20$. Compute the smoothed CTR, and give the raw CTR for comparison.

Q9.8 An item had 6 clicks in the last 3 days and 14 clicks in the last 14 days. Using the repo's formula with the constant 0.1, compute `item_trend`. Is the item heating up or cooling down?

Q9.9 *Hands-on.* In `book/code/ch09_features.py`, change the window in `label_window_clicks` from `pd.Timedelta(days=7)` to `pd.Timedelta(days=1)` (only the leaky feature, not the labels), then run the script again. Before you run it, predict what will happen to the leaky ranker's NDCG@20 and to the share of gain taken by `leaky_future_clicks`. What do you observe, and what does it tell you about how much of the future a feature needs to leak before the offline metric becomes untrustworthy?

Chapter 10 — Learning to rank

Dina is a ranking engineer on a marketplace team, and on Monday morning she opens a ticket that reads: "Home feed is weird. We show a kettle first to a user who bought a kettle yesterday, and a phone case in slot 9 to someone who has been staring at phone cases for a week." Nothing is wrong with retrieval. The candidate generators from the previous chapters did their job: the right products are somewhere in the pool of roughly a hundred candidates for that user. The problem is the *order*. The pool arrives sorted by whichever retriever happened to find the item first, and for every user the page only has room for a handful of slots at the top.

Her team's stopgap is a hand-written formula, still in the repo as `pipeline.heuristic_score`: normalise the three retriever scores per user and add them up. This chapter replaces that shrug with a learning algorithm: 46 features per `(user, item)` pair (Chapter 9), graded labels from the next seven days, and a model that puts the best items first. By the end you will have opened the model behind the app's "Why this?" popover.

What you'll learn

- The three families of learning-to-rank objectives (pointwise, pairwise, listwise) and when each one makes sense.
- How gradient-boosted decision trees work from first principles, with a hand-sized example of one split and one round of residuals.
- What LambdaMART adds on top of boosting: the *lambda* gradients and the NDCG swap delta.
- How the repo's `ranker.train` configures LightGBM, including graded labels and the `label_gain` table.
- How to read feature importance, SHAP contributions and a feature-group ablation, and how to avoid fooling yourself with seed noise.
- An honest result: on this small simulated dataset, a plain pointwise model scores *slightly higher* than LambdaMART, and why that happens.

Intuition: sorting by a score someone learned

Ranking means putting the same set of items into a better *order*. **Learning to rank** means the order comes from a model that learned from past outcomes, not from a hand-written formula.

Think of two shop assistants arranging the eye-level shelf, where only a few items fit. The new assistant follows a points sheet: "add the three scores, put the highest first" (our heuristic blend). The experienced assistant has watched thousands of shoppers and has a feel for combinations, for example "a kettle is a bad bet for someone who bought one yesterday". Learning to rank extracts that feel from historical outcomes.

Two different questions can be learned from the same history. One is "how likely is this shopper to buy this item?" (a probability). The other is "between these two items, which goes first?" (an ordering). A shelf with five slots only needs the second one. The ranking literature has three families of methods, depending on which question the model is trained to answer:

Family	The model is trained to…	Loss looks at	Example
Pointwise	predict the label of each row on its own	one row	logistic regression, `objective="binary"`
Pairwise	put the more relevant item above the less relevant one	pairs of rows from the same user	RankNet, Ranking SVM
Listwise	directly improve a list-level metric such as NDCG	the whole list for one user	LambdaMART, `objective="lambdarank"`

Since NDCG@20 (Chapter 2) is a list-level metric that discounts the lower slots, a listwise method should be the natural fit. Practice will turn out to be less tidy.

To see the difference, take shopper Sari and three candidates: a cat-food bag she ordered (label 3), a cat toy she only clicked (label 1) and a phone case she ignored (label 0).

- A **pointwise** model looks at each row alone and tries to guess its number: "bag: 3? toy: 1? case: 0?". It never compares items.
- A **pairwise** model looks at pairs and asks "is the bag above the toy? the toy above the case? the bag above the case?". Three pairs from three items.
- A **listwise** model looks at the whole list at once, scores how good the full ordering is (with a metric such as NDCG), and adjusts itself to raise that score.

Pointwise is the easiest to build. Listwise is the closest to what we are graded on. Pairwise sits in between.

Vocabulary: row, feature, label, query group

Four words come up constantly in this chapter.

- A **row** is one `(user, item)` pair, for example "Sari and the cat-food bag".
- **Features** are the numbers describing a row, such as "how long Sari stayed on this item's page" (Chapter 9 built 46 of them).
- The **label** is the right answer for the row, taken from what happened in the next seven days: 0 = nothing, 1 = click, 2 = add-to-cart, 3 = order.
- A **query group** (or just *group*) is all rows that belong to one user. Ranking only makes sense *inside* a group: nobody asks whether Sari's cat food should beat Ben's phone case. In this chapter every user is one group, so the model only compares items that compete for the same user's page.

Without groups, the model would try to compare rows from different users, and the "pairs" it learns from would be meaningless.

The overall flow

The overall flow of this stage in the repo looks like this.

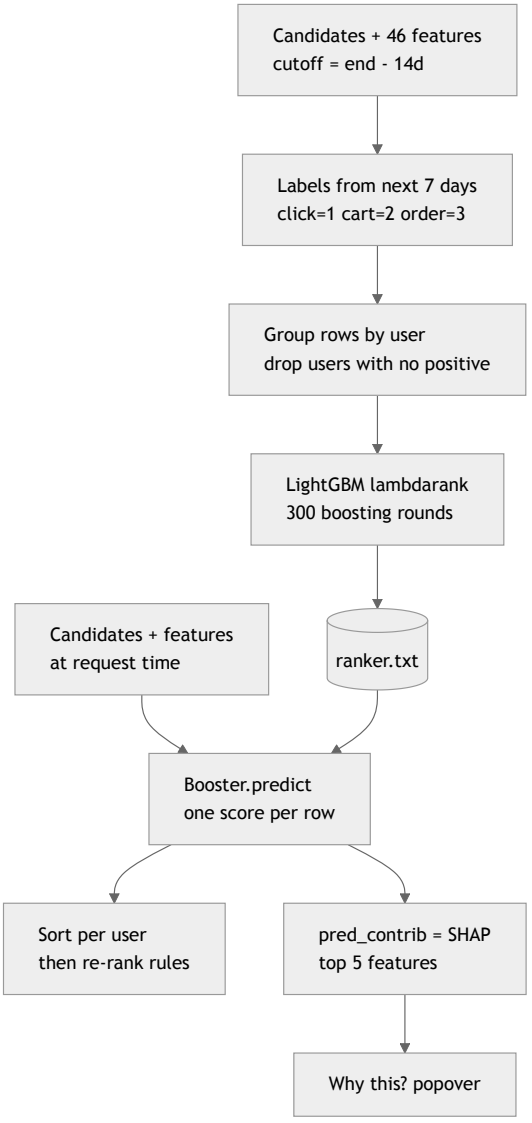


Figure 10.1: *how the ranker is trained offline and used at serving time, including the SHAP explanation path.*

The algorithm

We will build the pieces in order: first plain gradient boosting, then the lambdas that turn it into a ranker, then the graded labels that tell it what "good" means.

Step 1: regression trees and one split

A **decision tree** is a flowchart of yes/no questions, like the game of twenty questions. You enter a row of feature values at the top, answer "is `ui_max_dwell_sec` at most 45?", follow the yes or no branch, and keep going until you reach a **leaf** (an end box). A *regression* tree stores a number in each leaf, and that number is the prediction for every row that lands there.

Training a tree means choosing, at each branching point (a **node**), the question (which feature, which threshold) that sorts the rows into two groups that are each as *similar* as possible. We measure "similar" with the **sum of squared errors (SSE)**: take each row's value, measure how far it is from its group's average, square that distance (so that being too high and too low both count as bad), and add everything up. An SSE of 0 means every row in the group is identical.

Take a toy training set of six candidate rows for one feature, `ui_max_dwell_sec` (the longest the user stayed on this item's page), with their graded labels.

(dwell seconds, label) = (0, 0), (0, 0), (5, 0), (30, 1), (60, 2), (90, 3), one row each, numbered 1 to 6. Read it as "the longer the shopper stayed, the stronger the action they took later".

Boosting starts from the most boring guess possible: the same constant for every row, the mean label, $F_0 = (0+0+0+1+2+3)/6 = 1.0$. (With no information, the average is the safest guess.) The **residual** of each row is *what is still wrong*: the label minus the current prediction, $r_i = y_i - F_0$. For our six rows that gives $-1, -1, -1, 0, 1, 2$. The first three rows are overpredicted, the last two are underpredicted. Their sum of squares is $1+1+1+0+1+4 = 8$, our starting "untidiness".

Now we try candidate **splits** (a split is one yes/no question, here "is dwell at most this number?") and score each by how much SSE it removes. For a split into a left set L and a right set R with residuals r,

$$\mathrm{SSE}(S) = \sum_{i \in S} \left(r_i - \bar{r}_S\right)^2, \qquad \text{gain} = \mathrm{SSE}(\text{all}) - \mathrm{SSE}(L) - \mathrm{SSE}(R).$$

In words: S is any group of rows, $\bar{r}_S$ is the average residual of that group, and SSE adds up the squared distances from that average. The **gain** is how much untidiness the split removes: SSE before the split minus SSE of the two sides after it. Walk through the winning split in the table below, `dwell ≤ 45`. The left side holds rows 1–4 with residuals $-1, -1, -1, 0$. Their mean is -0.75, so SSE(L) $= 3 \times (0.25)^2 + (0.75)^2 = 0.75$. The right side holds rows 5 and 6 with residuals $1, 2$, mean 1.5, SSE(R) $= 0.25 + 0.25 = 0.5$. The gain is $8 - 0.75 - 0.5 = 6.75$.

split	left rows	right rows	SSE(L)	SSE(R)	gain
dwell ≤ 2.5	1, 2	3–6	0	5.0	3.0
dwell ≤ 17.5	1–3	4–6	0	2.0	6.0
dwell ≤ 45	1–4	5, 6	0.75	0.5	**6.75**
dwell ≤ 75	1–5	6	3.2	0	4.8

The winning split is `dwell ≤ 45`. The leaf value is the mean residual on each side: -0.75 on the left, $+1.5$ on the right. A tree like this is a **weak learner**: one question, two numbers, and obviously wrong for rows 4 and 5 individually. That is fine. Boosting does not need one great tree; it needs hundreds of mediocre ones that each fix part of the previous errors, like a team of people who each correct a small slip in the draft they receive.

The *gain* column is also where **feature importance** comes from later. It answers "which features did the trees lean on most?". Every time a feature is chosen for a split, the gain of that split is credited to it. Summing the credit over all splits of all trees and dividing by the total gives the "share of total gain" you will see in the results.

Step 2: boosting = repeated residual fitting

Each new tree is not trusted fully. Instead, we move each prediction only a *fraction* of the way toward what the tree says. That fraction is the **learning rate**, written η (LightGBM calls it `learning_rate`). It works like turning a shower tap: a big turn overshoots, small turns let you check and correct. In sym-

bols, $F_m(x)$ is the model's prediction after m trees, $h_m(x)$ is the number tree m outputs for row x, and

$$F_m(x) = F_{m-1}(x) + \eta\, h_m(x).$$

Take $\eta = 0.5$ for a one-round example. Rows 1–4 fall left and move by $0.5 \times (-0.75) = -0.375$, from 1.0 to 0.625. Rows 5 and 6 move by $0.5 \times 1.5 = +0.75$, to 1.75. The new residuals (label minus new prediction) are:

Row	Label	New prediction	New residual
1–3	0	0.625	−0.625
4	1	0.625	+0.375
5	2	1.75	+0.25
6	3	1.75	+1.25

Total squared error fell from 8 to $3 \times 0.3906 + 0.1406 + 0.0625 + 1.5625 \approx 2.94$. The next tree is trained on these new residuals, and so on. After 300 rounds with a small η the model has taken 300 small corrective steps, and its final prediction is the starting guess plus the sum of all the small steps.

Why "gradient" boosting, then? A **loss** is a number that says how bad the current predictions are (smaller is better); squared error is one loss. The **gradient** of the loss tells you, for each prediction, which direction to nudge it and how strongly to make the loss shrink fastest. For squared error, the residual $y - F$ is exactly that nudge: it is the negative gradient of $\frac{1}{2}(y - F)^2$ with respect to the prediction F. Friedman's insight was that you can plug in *any* smooth (differentiable) loss and fit each new tree to its negative gradient instead. Swap squared error for log-loss (a loss for yes/no labels) and you get a classifier; swap it for a ranking loss and you get a ranker.

> **Did you know?**
>
> **Boosting is gradient descent in function space.** Jerome Friedman's 2001 paper "Greedy function approximation: A gradient boosting machine" framed boosting as stage-wise gradient descent on an arbitrary differentiable loss, where each new tree is fit to the negative gradient of the loss at the current predictions. That single idea is why the same library can train regressors, classifiers and rankers by changing only the objective.
>
> Source: Friedman, Annals of Statistics, 2001

Step 3: from per-row error to per-list quality

A pointwise objective (say log-loss on "was this row clicked or better?") treats each row as an independent example. It will happily spend model capacity on getting the 95th item's probability right. But Dina's page only shows the top 20 positions, and NDCG discounts the later of those heavily. We would like the model to spend its effort where the metric cares.

Recall NDCG (normalised discounted cumulative gain) from Chapter 2. It scores a ranked list from 0 to 1: items with a strong label earn a lot, items placed lower earn less, and the result is compared with the best possible ordering. With graded relevance rel_i at position i:

$$\mathrm{DCG@}K = \sum_{i=1}^{K} \frac{2^{rel_i} - 1}{\log_2(i+1)}, \qquad \mathrm{NDCG@}K = \frac{\mathrm{DCG@}K}{\mathrm{IDCG@}K},$$

where IDCG (ideal DCG) is the DCG of the perfect ordering. The numerator $2^{rel} - 1$ is the **gain**, the reward for having that item in the list: a relevance of 0, 1, 2, 3 gives gains 0, 1, 3, 7. An order is worth seven clicks. The denominator is the **discount**, how much a slot is worth because of its position: position 1 has weight 1, position 2 has 0.631, position 3 has 0.5, and so on. Top slots count most, as on any shelf.

NDCG depends on the *sorted order*, and sorting is not differentiable: nudging a score by a tiny amount either changes nothing or suddenly swaps two items, so there is no smooth slope to follow downhill. We therefore cannot take its gradient. LambdaRank's trick (Burges, 2010) is pragmatic: *skip the loss, define the gradient directly*.

Step 4: lambdas and the NDCG swap

We start with a **pairwise loss**, as in RankNet. Take two items i and j from the same user, where i is the more relevant one. The model's current scores are s_i and s_j; ideally s_i is clearly bigger. The **logistic function** turns the score difference into a probability between 0 and 1 ("how sure is the model that i goes above j?"): near 0.5 when the scores are equal, near 1 when s_i is far above. The pairwise loss pushes $s_i - s_j$ to grow. The push on i is called **lambda** (λ): its sign says up or down, its size says how hard. For a pair, it is

$$\lambda_{ij} = \frac{-\sigma}{1 + e^{\sigma(s_i - s_j)}}.$$

Here σ (sigma) is just a constant that sets how steep the logistic curve is; e is the usual exponential constant. In words: the push is strong when the pair is in the wrong order and fades to nothing when the pair is already well separated.

When i is already far above j the denominator is huge and the push vanishes. When i is wrongly below j, $s_i - s_j$ is negative, the denominator is near 1, and the push is close to its maximum σ. So far this is plain pairwise logic: all mis-ordered pairs are equally urgent, whether they sit at the top of the page or on page nine.

The LambdaRank step is to **multiply by the change in NDCG that swapping the two items would cause**. Intuitively, "how much would the metric care if we fixed this pair?":

$$\lambda_{ij} = \frac{-\sigma}{1 + e^{\sigma(s_i - s_j)}} \, \left|\Delta \mathrm{NDCG}_{ij}\right|.$$

Swapping the items at positions p_i and p_j changes the DCG by $(g_i - g_j)\,(d_{p_i} - d_{p_j})$ in absolute value, where g is the gain and $d_p = 1/\log_2(p+1)$ is the discount. Dividing by IDCG gives $|\Delta\mathrm{NDCG}|$ (the bars mean "ignore the sign"; Δ means "change in"). A pair that is both mis-ordered *and* sits at the top of the list with a big gain gap gets a big lambda. A pair that is mis-ordered at positions 80 and 81, or whose gains are equal, gets almost none. That is how the algorithm "pays attention" to the top of the list without ever differentiating NDCG.

Each row's lambda is the sum over all its pairs, signed up for the more relevant member and down for the less relevant. These lambdas replace the negative gradient (the residual in our toy example) when fitting the next tree: that is **LambdaMART** (**MART**, multiple additive regression trees, is an older name for gradient-boosted trees). So LambdaMART is just gradient boosting where "what is still wrong" is measured in lambdas rather than residuals.

A worked example of the swap delta

Take three candidates for one user, currently ranked A, B, C with graded labels 0, 3, 1. Their gains are $0, 7, 1$.

- $\mathrm{DCG} = 0 \cdot 1 + 7 \cdot 0.631 + 1 \cdot 0.5 = 4.917$.
- The ideal order is B, C, A, so $\mathrm{IDCG} = 7 \cdot 1 + 1 \cdot 0.631 = 7.631$.
- $\mathrm{NDCG} = 4.917 / 7.631 = 0.644$.

Swapping A and B (positions 1, 2) changes NDCG by $7 \times (1 - 0.631)/7.631 = 0.339$; swapping A and C (positions 1, 3) only by $1 \times 0.5/7.631 = 0.066$. The first mistake, a never-clicked item above the order-level one, is five times more important. Now suppose the model's raw scores are $s_A = 2.0$ and $s_B = 1.0$ and $\sigma = 1$. For the pair "B should be above A", $s_B - s_A = -1$, so the logistic factor is $1/(1 + e^{-1}) = 0.731$. The magnitude of the lambda for that pair is $0.731 \times 0.339 = 0.248$: a firm upward push on B and an equal downward push on A. For the pair (C, A) with the same logistic factor the magnitude would be only $0.731 \times 0.066 = 0.048$. The next tree will therefore spend most of its splitting effort on features that tell B and A apart.

In plain terms: the model currently puts the never-clicked A first and the ordered item B second. Fixing that pair would lift NDCG by 0.339, so B gets a strong push up and A a strong push down. Fixing the A/C pair would only add 0.066, so it gets a gentle push. Mistakes that matter more to the metric get louder instructions.

> **Did you know?**
>
> **LambdaMART won the Yahoo! Learning to Rank Challenge.** The system that won Track 1 of the Yahoo! Learning to Rank Challenge (2010) was an ensemble of lambda-gradient models, built on boosted trees trained with LambdaRank-style gradients. Chris Burges's earlier overview paper, "From RankNet to LambdaRank to LambdaMART", is the standard reference for how the three algorithms fit together.
>
> Source: Burges et al., Proceedings of the Learning to Rank Challenge (PMLR), 2011

Step 5: graded labels

In this repo a label is not just 0 or 1. `schema.LABEL_GAIN` says a click is worth 1, an add-to-cart 2 and an order 3. If a user did several things with the same item in the label window, `data.labels` keeps the *strongest* one: someone who clicked and then ordered counts as an order. This matters in two ways:

1. The model is rewarded for putting the item the user ordered above the item the user only clicked.
2. The `label_gain=[0, 1, 3, 7]` parameter in LightGBM converts the integer labels to the gain $2^{rel} - 1$ used in NDCG (label 0 earns 0, label 1 earns 1, label 2 earns 3, label 3 earns 7). LightGBM's default table happens to be the same $2^{rel} - 1$ for these labels, but spelling it out in

`PARAMS` keeps the objective and the offline evaluator from silently drifting apart; it means the objective being optimised and the metric being reported are the same function.

Users with no positive in the window carry no ordering information: every item has label 0, so swapping any two changes nothing (all their $|\Delta\mathrm{NDCG}|$ are 0). `ranker.train` drops them.

Code walkthrough

The ranker itself

The whole LambdaMART configuration lives in the `PARAMS` dictionary at the top of `recommender/recsys/ranker.py` (plus the `NUM_ROUNDS` constant). A **hyperparameter** is a setting you choose before training, rather than something the model learns. The table gives the plain meaning of each:

Parameter	Value	Plain meaning
`objective`	`lambdarank`	use the lambda gradients (Step 4)
`learning_rate`	0.05	the step size η: each tree moves predictions only 5% of the way
`num_leaves`	31	each tree may have at most 31 end boxes (leaves), so it stays small
`min_data_in_leaf`	50	a leaf must hold at least 50 rows, so it cannot memorise one shopper
`bagging_fraction` / `bagging_freq`	0.8 / 1	every tree sees a fresh random 80% of the rows
`feature_fraction`	0.8	every tree sees a random 80% of the columns (features)
`label_gain`	`[0, 1, 3, 7]`	reward for label 0, 1, 2, 3
`lambdarank_truncation_level`	20	only pairs involving the top 20 positions matter
`NUM_ROUNDS`	300	number of trees

Small steps and many rounds generalise better than few large ones (they give the model more chances to correct itself without overreacting to one tree). LightGBM grows trees **leaf-wise**, always splitting the leaf that would gain the most, so `num_leaves` (not depth) controls complexity, and `min_data_in_leaf` stops a leaf memorising one user. **Bagging** (training each tree on a random sample of rows) and feature sampling inject randomness so that the trees do not all copy each other; this is also why the **seed**, the starting number of the random-number generator, changes results (see the ablation). Truncation at 20 matches `K = 20`: pairs deep in the list have negligible $|\Delta\mathrm{NDCG}|$.

> **Did you know?**
>
> **LightGBM is fast because it does less work per split.** The LightGBM paper introduced Gradient-based One-Side Sampling, which skips most rows with small gradients when estimating split gain, and Exclusive Feature Bundling, which merges sparse features that are rarely non-zero together. On the public datasets the authors tested, this sped up training of conventional gradient-boosted trees by up to over 20 times with almost the same accuracy.
>
> Source: Ke et al., NeurIPS, 2017

Training takes eight lines. Read this function slowly: its structure encodes everything about how LightGBM wants ranking data.

recommender/recsys/ranker.py · train

```python
def train(frame: pd.DataFrame, feature_names: list[str] = schema.FEATURE_NAMES, seed: int = 0) ->
lgb.Booster:
    """`frame` = features + `label`. Groups without any positive teach nothing; drop them."""
    has_pos = frame.groupby("user_id")["label"].transform("max") > 0
    frame = frame[has_pos].sort_values("user_id", kind="stable")
    groups = frame.groupby("user_id", sort=False).size().to_numpy()
    ds = lgb.Dataset(frame[feature_names], label=frame["label"], group=groups)
    return lgb.train({**PARAMS, "seed": seed}, ds, num_boost_round=NUM_ROUNDS)
```

Three things to notice. First, the `has_pos` mask keeps only users who have at least one positive in the label window; the rest are dropped. Second, `sort_values("user_id", kind="stable")` puts every user's rows next to each other, because LightGBM's `group=` argument is a list of *consecutive group sizes*, not user ids. Example: if the sorted frame holds 3 rows of user A followed by 2 rows of user B, the group list is `[3, 2]`, and LightGBM reads "the first 3 rows are one user, the next 2 are another". If the rows were not contiguous, the group boundaries would silently cut through the wrong users and the model would train on garbage without raising an error. Third, the seed is injectable: `train(frame, names, seed)` is what lets the ablation retrain the same model with different random seeds.

Training and evaluating end to end

`pipeline.train_and_evaluate` ties together the data split from Chapter 2, the frame builder from Chapter 9 and the ranker above:

recommender/recsys/pipeline.py · train_and_evaluate

```
def train_and_evaluate(tables: dict[str, pd.DataFrame], end: pd.Timestamp, ablation: bool =
False):
    train_cutoff = end - pd.Timedelta(days=2 * LABEL_DAYS)
    eval_cutoff = end - pd.Timedelta(days=LABEL_DAYS)
    train_frame, _ = build_frame(tables, train_cutoff)
    eval_frame, eval_labels = build_frame(tables, eval_cutoff)
    model = ranker.train(train_frame)

    eval_scored = eval_frame.assign(score=ranker.score(model, eval_frame))
    popular = eval_frame[eval_frame["cand_global_pop_rank"] < 999]
    results = {
        "global popular": evaluate.metrics(_top(popular, -popular["cand_global_pop_rank"]),
eval_labels, K),
        "heuristic blend": evaluate.metrics(_top(eval_frame, heuristic_score(eval_frame)),
eval_labels, K),
        "lightgbm ranker": evaluate.metrics(_top(eval_frame, eval_scored["score"]), eval_labels,
K),
        "lightgbm + rerank": evaluate.metrics(rerank.rerank(eval_scored, K), eval_labels, K),
    }
    report = {
        "train_rows": len(train_frame),
        "eval_rows": len(eval_frame),
        "eval_users": eval_labels["user_id"].nunique(),
        "candidate_recall": evaluate.candidate_recall(eval_frame, eval_labels),
        "results": pd.DataFrame(results).T,
        "importance": ranker.importance(model),
    }
    if ablation:
        report["ablation"] = group_ablation(train_frame, eval_frame, eval_labels,
report["results"].loc["lightgbm ranker"])
    return model, report
```

Walk through it in order. The two cutoffs sit 14 and 7 days before `end`. The training frame is built as of `end - 14d`, its labels come from the following 7 days. The evaluation frame is built as of `end - 7d`, with labels from the final 7 days; the model never sees them. The ranker is trained on the first frame only, and scored on the second.

The `results` dictionary compares four methods on the same evaluation frame, using the same `evaluate.metrics` function. "global popular" is the Chapter 3 baseline. "heuristic blend" is Dina's hand-written formula. "lightgbm ranker" is what we just trained. "lightgbm + rerank" applies the business rules from Chapter 11 on top. Without these rows in the same table an ML model has no yardstick, and a model can easily ship that is no better than the blend if nobody checks.

The service, in `recommender/recsys/service.py`, calls the same `retrain` machinery on a timer and on the admin button, so what you read here is the code that actually runs in production for this repo.

Explaining a score with SHAP

A boosted model is a sum of 300 trees. A single prediction passes through all of them, so reading the trees directly is hopeless. We want something like a **shop receipt**: a list of lines, each saying "this feature added this much, that one subtracted that much", that sums exactly to the final total. **SHAP** values (Lundberg and Lee, 2017) give us that receipt. The total is the model's score, the starting amount is the **base score** (what an unremarkable row would get), and each line is one feature's **contribution**.

How do we decide a fair size for each line? SHAP borrows the **Shapley value** from cooperative game theory: imagine the features joining the prediction one at a time, and measure how much the score moves when each joins. The order of joining matters (features can overlap), so we average over all

possible orders. Tiny illustrative example with two features, starting from base 0. Knowing dwell time alone moves the score to $+1.0$; knowing "is a consumable" alone moves it to $+0.4$; knowing both gives $+1.6$. If dwell joins first it adds 1.0; if it joins second it adds $1.6 - 0.4 = 1.2$. Its average is 1.1. For "consumable": 0.4 when first, $1.6 - 1.0 = 0.6$ when second, average 0.5. The contributions $1.1 + 0.5 = 1.6$ add up *exactly* to the score. For one row, the contributions of all features plus the base score always add up exactly to the model's raw output.

Computing Shapley values naively requires trying an exponential number of feature subsets (with 46 features, far too many). For tree ensembles there is a fast exact algorithm, and LightGBM implements it behind a single flag, `pred_contrib=True`. The repo wraps it:

recommender/recsys/ranker.py · contributions

```
def contributions(model: lgb.Booster, frame: pd.DataFrame, top: int = 5) -> list[list[tuple[str,
float, float]]]:
    """Per row: the `top` features by |SHAP contribution|, as (feature, value, contribution).

    LightGBM's pred_contrib returns one column per feature plus a final bias column;
    contributions + bias sum to the raw score.
    """
    names = model.feature_name()
    x = frame[names]
    contrib = model.predict(x, pred_contrib=True)[:, :-1]
    values = x.to_numpy(dtype=float)
    out = []
    for row_c, row_v in zip(contrib, values):
        order = np.argsort(-np.abs(row_c))[:top]
        out.append([(names[j], float(row_v[j]), float(row_c[j])) for j in order])
    return out
```

`model.predict(x, pred_contrib=True)` returns a matrix with one row per candidate and one column per feature, plus one final column holding the **bias**, the model's average output when it knows nothing about the row (the "starting amount" of the receipt). The slice `[:, :-1]` means "all rows, all columns except the last", so it drops the bias column and what remains are the pure per-feature contributions. For each row we then sort features by $|\text{contribution}|$ (largest push first, whether up or down) and keep the top `top` of them as `(name, value, contribution)` triples. The gRPC service (the repo's internal server for other programs to call) calls this in `recommend_rpc` and puts the top five in each response; the Rust API forwards them untouched, and `WhyThis.vue` draws them as bars.

> **Did you know?**
>
> **Exact SHAP for trees runs in polynomial time.** The TreeExplainer paper presented the first polynomial-time algorithm to compute optimal game-theoretic explanations for tree-based models such as random forests and gradient-boosted trees. Before it, exact Shapley values required enumerating exponentially many feature subsets, which made them impractical for models with dozens of features and hundreds of trees.
>
> Source: Lundberg et al., Nature Machine Intelligence, 2020

Run it

The companion script trains three rankers on exactly the same candidate frames, runs a four-seed check, prints importances and a SHAP breakdown, and runs the group ablation. From the `book/`

directory:

```
uv run python code/ch10_ltr.py
```

The script builds the training frame (cutoff 14 days before the end) and the evaluation frame (7 days before the end) once, then reuses them for every model. (A **cutoff** is the moment we freeze time: the model only sees the past of that moment, and the following days serve as the answer key.) The first block compares the three orderings.

TERMINAL OUTPUT

```
=== Three ways to order the same candidates ===
                    recall@5  recall@10  recall@20  recall@50  order_recall@20  ndcg@20
coverage@20
method
heuristic blend       0.1153     0.1769     0.2526     0.3753           0.3978   0.1674
0.5682
pointwise LightGBM    0.1577     0.2176     0.2836     0.3915           0.4250   0.2146
0.4777
LambdaMART            0.1503     0.2015     0.2725     0.3824           0.4210   0.2116
0.4245

candidate recall ceiling: 0.4645
```

Read the table from top to bottom. The heuristic blend reaches ndcg@20 of 0.1674. The two trained models reach 0.2146 (pointwise) and 0.2116 (LambdaMART): roughly 26–28% higher, from the same candidates and the same labels. The last line, 0.4645, is the *candidate recall ceiling*: of all the things a user actually did in the answer window, the fraction that were in the candidate pool at all. A ranker can only reorder the pool, so it can never find an item the pool lacks; recall@20 (the share of the user's true positives that land in the top 20) of any ranker is bounded by this ceiling. LambdaMART's recall@20 of 0.2725 is 59% of what an oracle reordering of the same pool could reach (an *oracle* is a perfect ranker that peeks at the answers).

Results

Pointwise versus LambdaMART: an honest comparison

The second block of the output is the one many ranking tutorials omit. A pointwise LightGBM (`objective="binary"`, label = "was there any positive action?") was trained on the same rows with the same hyperparameters, and both models were retrained with four different random seeds. Why seeds? Training uses random choices (which rows and columns each tree sees), so two runs with different seeds give slightly different models, like baking the same cake four times and getting four slightly different cakes. If two methods differ by less than that run-to-run wobble, we cannot say one is truly better. The wobble is measured by the **standard deviation** (std): roughly, the typical distance of a run's result from the average of all runs.

TERMINAL OUTPUT

```
=== Seed check: pointwise vs LambdaMART over 4 seeds (ndcg@20) ===
      pointwise  lambdamart
seed
0        0.2146      0.2116
1        0.2134      0.2108
2        0.2167      0.2155
3        0.2142      0.2121
mean: {'pointwise': 0.2147, 'lambdamart': 0.2125}  std: {'pointwise': 0.0014, 'lambdamart':
0.0021}
At this data size pointwise is marginally ahead (about one seed-std): with ~100 candidates per
user and few
positives, the list-aware objective has little extra to exploit. Both beat the heuristic blend by
a wide margin.
```

The pointwise model is ahead on every seed. The mean ndcg@20 is 0.2147 against 0.2125, a gap of 0.0022, and the standard deviation across seeds is 0.0014 for pointwise and 0.0021 for LambdaMART. The gap is therefore about one seed-standard-deviation: consistent in sign across the four seeds, but small. Pointwise also leads on recall@5 (0.1577 against 0.1503) and recall@10 (0.2176 against 0.2015). LambdaMART's coverage@20, the share of the catalogue that appears in somebody's top 20 (a measure of how varied the recommendations are), is the lowest of the three (0.4245, against 0.4777 for pointwise and 0.5682 for the heuristic blend).

This contradicts the principle that a listwise objective suits a list-level metric. Plausible reasons:

1. **Few positives per list.** About a hundred candidates and a handful of positives give few informative pairs; a binary pointwise label is a simpler signal the trees learn directly.
2. **A low ceiling.** With candidate recall at 0.4645, the room for ranking gains is small, so two good rankers end up close.
3. **Noisy graded labels.** Orders are rare and depend on mood and stock; `label_gain` amplifies exactly that noise for LambdaMART.

None of this makes LambdaMART the wrong choice. The gap is about one seed standard deviation, on a far smaller dataset than the published results, and you would not launch on it. What is beyond argument is the gap to the heuristic blend: both learned models beat it by roughly 0.045 ndcg@20. The repo keeps `lambdarank` because it should scale with data and handles graded labels; if your A/B test disagrees, `objective="binary"` is a one-line change. The lesson is the process: *keep a simple baseline in the table and rerun with several seeds before declaring a winner.*

What did the model learn? Feature importance

The simplest question to ask a boosted model is which features it used the most. LightGBM can report the total gain per feature (the sum of the split gains from Step 1). `ranker.importance` normalises the gains to shares that sum to one, so "0.063" means "6.3% of all the error-removal credit". The chart shows the top 15 features; the output gives the share of gain by group (a *group* here means a family of features, such as all item features, not a user's query group).

TERMINAL OUTPUT

```
Gain share by feature group:
group
item             0.309
user_item        0.198
user             0.189
candidate        0.134
user_category    0.086
shop             0.065
user_shop        0.014
context          0.004
```

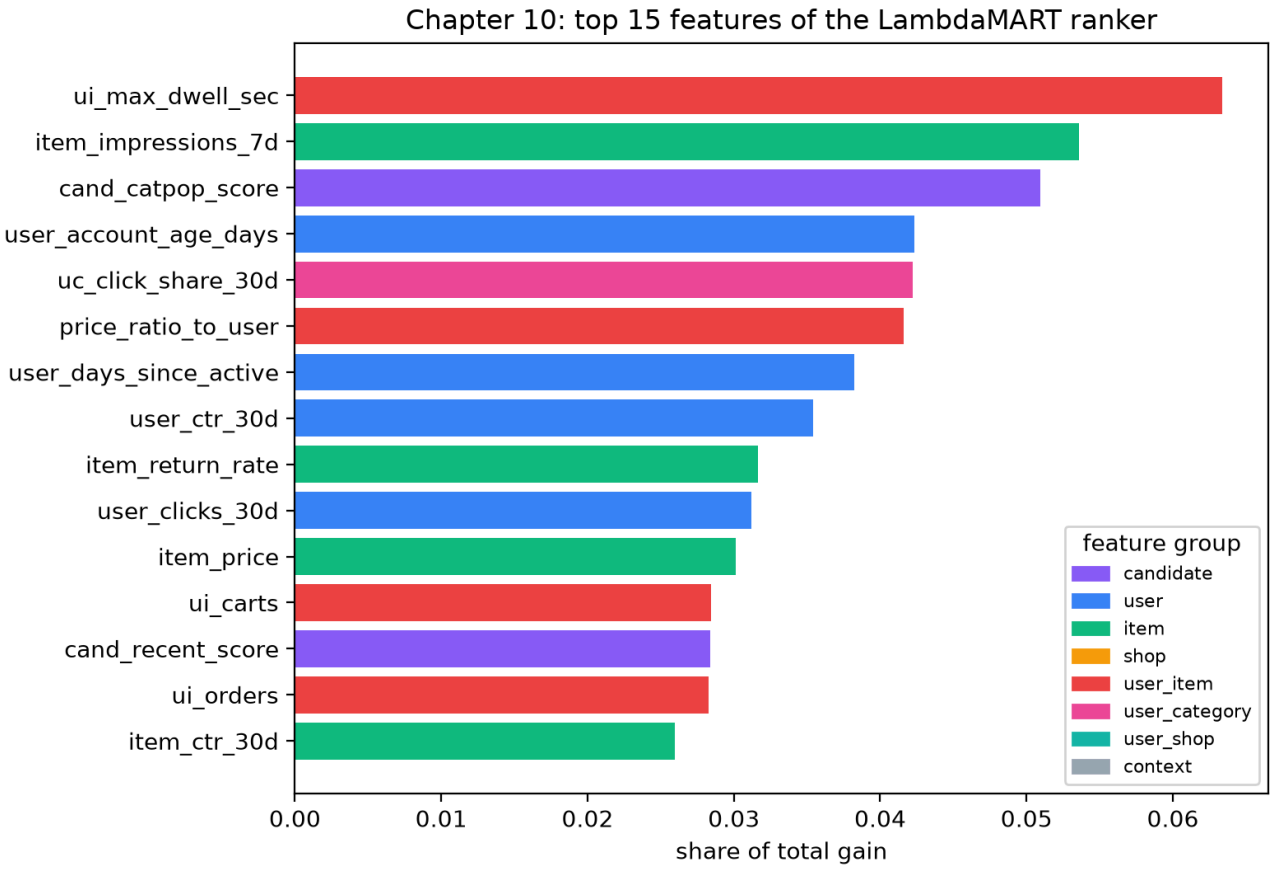


Figure 10.2: *Top 15 features of the LambdaMART ranker by share of total gain, coloured by feature group.*

The importance is *diffuse*: the top feature, `ui_max_dwell_sec`, has only 6.3% of gain. By group, **item** features lead (0.309), then user-item (0.198), user (0.189) and candidate (0.134); **context** gets 0.004. A caution: gain importance describes how the trees were built, not how much the model would suffer without a feature. Correlated features (features that carry similar information) share credit, and features with many distinct values get more chances to be picked for a split. The ablation below shows the discrepancy.

Why this? One prediction under the microscope

Importance averages over every prediction; SHAP explains one. The script scores the 106 candidates for user 42 and prints the contributions of the top item (id 2381), the one the ranker placed first:

TERMINAL OUTPUT

```
=== SHAP breakdown: first recommendation for user 42 ===
candidates scored: 106; top item_id=2381 score=1.277
  ui_max_dwell_sec         value=   449.900  contribution=+0.735   Longest product-page dwell on
this item
  item_is_consumable       value=     1.000  contribution=+0.666   Category is repeat-purchase
  cand_covisit_score       value=     4.028  contribution=-0.505   Co-visitation: how often this
item is clicked in the same session as the user's recent items
  item_impressions_7d      value=    93.000  contribution=+0.425   Times shown in last 7 days
  ui_orders                value=     5.000  contribution=+0.407   Times this user ordered this
item
  item_return_rate         value=     0.002  contribution=+0.348   Returns / orders, smoothed
(quality signal)
  ui_carts                 value=     6.000  contribution=+0.336   Times this user carted this
item
  uc_days_since_order      value=     6.465  contribution=+0.314   Days since last order in the
category (re-purchase cycle; 999 = never)
  (bias / base score: -2.917; contributions + bias = raw score)
  sum of all contributions + bias = 1.277  (model score 1.277)

Top 10 recommended item ids for user 42: [2381, 2200, 1872, 2015, 87, 873, 1743, 30, 2482, 414]
```

The raw score is 1.277 and the bias (the baseline for an unremarkable candidate) is -2.917. The raw score is not a probability and has no unit; only the ordering of scores matters. The last lines confirm additivity: contributions plus bias equal the model score. The eight largest contributions add up to about +2.73, so the other 38 features add the remaining +1.47.

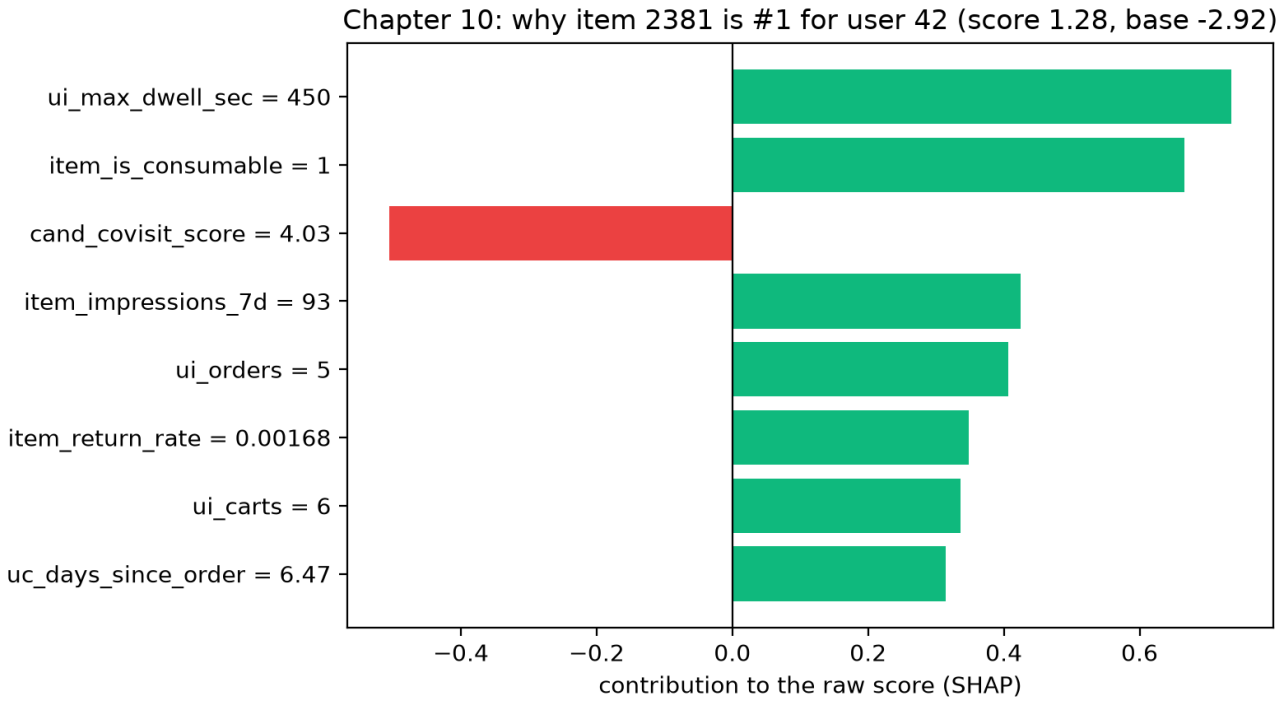


Figure 10.3: *Why item 2381 is ranked first for user 42: the eight largest SHAP contributions, positive in green and negative in red.*

Reading the chart as a story: user 42 dwelled on this item for 450 seconds (+0.735), it is a consumable (+0.666), has many recent impressions (+0.425), and the user ordered it five times and carted it six (+0.407, +0.336). The one negative bar is instructive: `cand_covisit_score` = 4.03 *lowers* the score by 0.505, because an item found only via co-visitation is a weaker bet than direct evidence from this user. A hand-written sum of retriever scores cannot express that.

The app's "Why this?" popover (`WhyThis.vue`, screenshot in Chapter 13) shows these contributions for a live model; its source line comes from `pipeline.source_of`.

Ablation: which groups of features matter, and compared to what?

Importance asks about the trained model. **Ablation** asks a counterfactual: if we had never had these features, would the model be worse? It is like removing one ingredient from a recipe and baking again to see whether the cake gets worse. You answer it by retraining. `pipeline.group_ablation` removes one feature group at a time, retrains with the same settings, and reports the change in each metric against the full model.

`pipeline.group_ablation` retrains without each feature group, and also retrains the *full* model with seeds 1, 2 and 3. Their standard deviation is the **noise floor**: baking the full recipe repeatedly gives slightly different cakes, so any change smaller than that wobble tells you nothing, because a different seed could flip its sign. A **delta** is simply the change in a metric (ablated minus full): negative means the model got worse without the group.

TERMINAL OUTPUT

```
                      recall@20  order_recall@20  ndcg@20
without item            -0.0321          -0.0492  -0.0328
without user_item       -0.0069          -0.0235  -0.0114
without user_category   -0.0159          -0.0148  -0.0034
without context          0.0011          -0.0022   0.0018
without user            -0.0037           0.0032   0.0023
without shop            -0.0017           0.0023   0.0042
without candidate        0.0045           0.0144   0.0046
without user_shop        0.0055           0.0043   0.0047
seed noise (±1 std)      0.0021           0.0059   0.0021
```

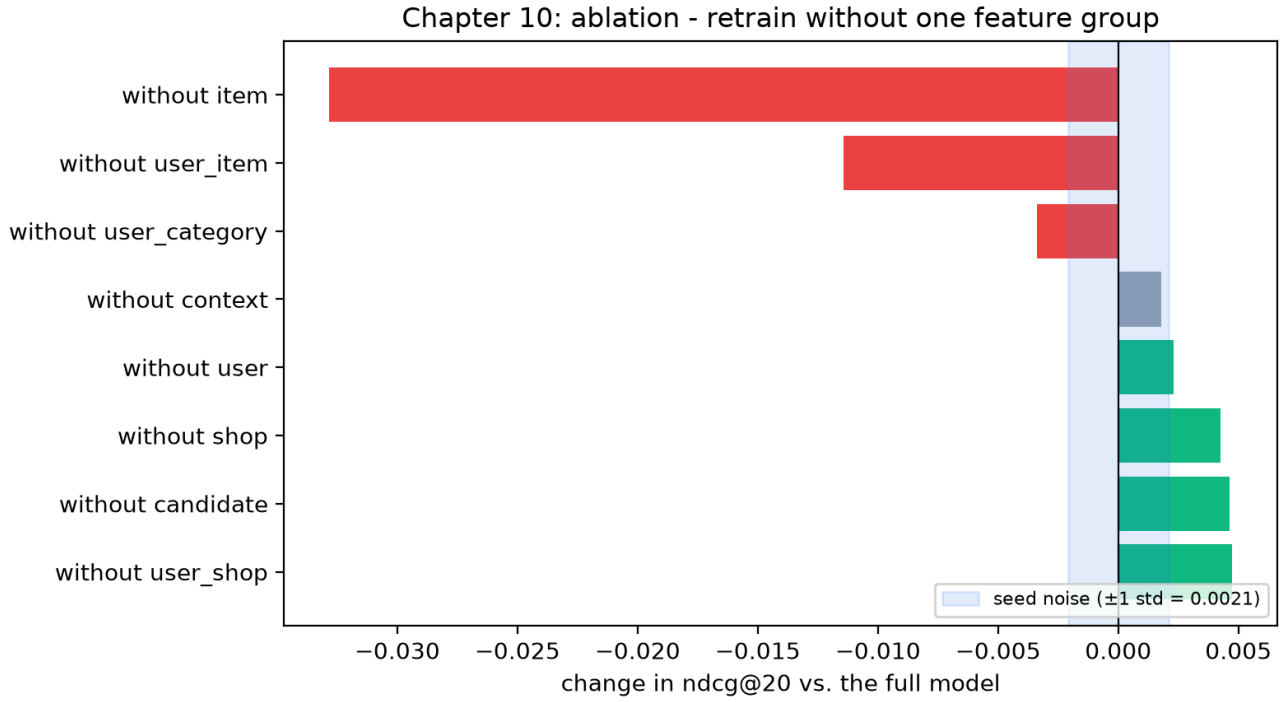


Figure 10.4: *Change in ndcg@20 when each feature group is removed, with a shaded band of one seed-standard-deviation.*

The seed noise on ndcg@20 is 0.0021 (the shaded band). Compare each row's ndcg@20 change to that band:

- **without item: −0.0328.** Fifteen times the noise. The item group (price, popularity, CTR, return rate, stock, and so on) is the most valuable evidence, and no other group can substitute for it.
- **without user_item: −0.0114.** About five times the noise. What this user did with this item is important, and the other groups do not replace it.
- **without user_category: −0.0034.** Beyond the noise band on ndcg, and clearly beyond it on recall@20 (−0.0159 against a noise of 0.0021), so we count it as a real, if smaller, contribution.
- **context, user, shop, candidate, user_shop:** deltas between +0.0018 and +0.0047. These are *positive*, meaning the model without these groups scored *higher*, and they are all within about two noise standard deviations of zero. The honest reading is "no evidence that they help in this data", not "removing them improves the model".

Compare with gain importance: the candidate group has 13.4% of the gain and the user group 18.9%, yet removing either does not hurt, because correlated features (the user-item evidence echoes the retriever scores) take over. Gain importance never reveals that.

Pitfalls and production practice

- **Leakage.** A feature that secretly contains information from the future (the answer key) makes offline scores look great and production scores collapse. A leaky feature took offline ndcg@20 from the honest 0.2116 to 0.5035 in Chapter 9. Build features as of a cutoff via `History.at` and keep `tests/test_point_in_time.py`.
- **Group boundaries.** LightGBM's `group=` is a list of consecutive sizes; unsorted rows run silently and train garbage. Sort first, as `ranker.train` does.
- **Scores have no scale.** A LambdaMART score is only meaningful for ordering. Do not cut it at a threshold ("show if above 0.5") or mix it with probabilities; use a pointwise model or **calibration** (adjusting scores so that "0.3" really means a 30% chance) where a probability is needed.
- **Label design is product design.** The click 1 / cart 2 / order 3 table is a business opinion about what a good outcome is; production teams often train separate **heads** (one model per outcome, such as click and order) and combine them (Chapter 14).
- **Retrain as deploy.** The service retrains every six hours and from the admin button; refuse to swap in a model worse than the heuristic baseline, otherwise a bad training run goes straight to users.
- **Explanations are not causal.** A SHAP bar says a value pushed the score up relative to the baseline, nothing more. It does not say that changing that feature would make the shopper buy.

Summary

- Learning to rank turns a pool of candidates and their features into an ordering. Pointwise methods predict each row's label, pairwise methods order pairs, listwise methods target a list metric like NDCG.

- Gradient boosting builds a model as a sum of small trees, each fit to the residuals (negative gradients) of the current ensemble; a split's gain is the SSE it removes.
- LambdaMART replaces the unusable gradient of NDCG with *lambdas*: pairwise logistic pushes scaled by the NDCG change $|\Delta\text{NDCG}|$ that swapping the pair would cause, so errors near the top with big gain gaps dominate.
- The repo's `ranker.PARAMS` uses `lambdarank`, 300 rounds at a 0.05 learning rate, `label_gain=[0, 1, 3, 7]` for graded click/cart/order labels, and truncation at 20. Rows must be grouped contiguously by user, and users with no positives are dropped.
- Both learned rankers beat the heuristic blend by wide margins (ndcg@20 0.2146 and 0.2116 versus 0.1674), but here the pointwise model is marginally ahead of LambdaMART across all four seeds, by about one seed-standard-deviation. List-aware objectives pay off with more data per query.
- Gain importance is diffuse (the top feature has 6.3%); it tells you how the trees were built, not what the model needs.
- SHAP gives exact, additive per-prediction explanations (contributions plus bias equal the score) and powers the app's "Why this?" popover through `ranker.contributions`.
- A feature-group ablation answers the counterfactual question, but only deltas larger than the seed noise count: here only the item, user–item and user–category groups cleared it.

Quiz

Q10.1 Which statement best distinguishes pointwise, pairwise and listwise learning to rank?

A. Pointwise uses trees, pairwise uses neural networks, listwise uses linear models.
B. Pointwise losses score each row on its own, pairwise losses compare two items of the same query, listwise losses look at the whole ranked list.
C. Pointwise methods need graded labels; pairwise and listwise methods need binary labels.
D. Pointwise methods are trained online, the others offline.

Q10.2 In gradient boosting with squared error, what does each new tree try to predict?

A. The original label of each row.
B. The average label of the whole training set.
C. The residual of the current ensemble, which is the negative gradient of the loss.
D. The feature with the highest gain.

Q10.3 In LambdaMART, why is each pairwise push multiplied by $|\Delta\text{NDCG}|$?

A. To make the loss differentiable.
B. To make scores comparable between users.
C. To speed up training by skipping pairs.
D. To make mis-orderings that would change NDCG a lot (near the top, with large gain gaps) count more than ones that change it little.

Q10.4 `ranker.train` sorts the frame by `user_id` with a stable sort before building the LightGBM dataset. Why must the rows of one user be contiguous, and what goes wrong if they are not?

Q10.5 In our results, pointwise LightGBM scores slightly higher than LambdaMART on ndcg@20. Give two plausible reasons, and say what evidence would make you switch the production objective.

Q10.6 Read this code from `recommender/recsys/ranker.py`:

recommender/recsys/ranker.py · contributions

```
def contributions(model: lgb.Booster, frame: pd.DataFrame, top: int = 5) -> list[list[tuple[str,
float, float]]]:
    """Per row: the `top` features by |SHAP contribution|, as (feature, value, contribution).

    LightGBM's pred_contrib returns one column per feature plus a final bias column;
    contributions + bias sum to the raw score.
    """
    names = model.feature_name()
    x = frame[names]
    contrib = model.predict(x, pred_contrib=True)[:, :-1]
    values = x.to_numpy(dtype=float)
    out = []
    for row_c, row_v in zip(contrib, values):
        order = np.argsort(-np.abs(row_c))[:top]
        out.append([(names[j], float(row_v[j]), float(row_c[j])) for j in order])
    return out
```

Why does the function slice the prediction matrix with `[:, :-1]`? What would happen to the "explanation" if that slice were forgotten, assuming the list is then sorted by $|\text{contribution}|$ for each row?

Q10.7 A user's candidates are ranked with graded labels $1, 0, 3$ at positions 1, 2, 3 (gains $2^{rel} - 1 = 1, 0, 7$). Using $d_p = 1/\log_2(p+1)$, compute $|\Delta\text{NDCG}|$ for swapping the items at positions 1 and 3. Give three decimals.

Q10.8 For the item explained in the chapter, the model score is 1.277 and the bias is -2.917. The eight contributions shown sum to about $+2.726$. What is the combined contribution of the remaining features?

Q10.9 An ablation reports "without user_category: -0.0034 ndcg@20" and the seed noise is ± 0.0021 (one standard deviation). Which conclusion is best supported?

A. Removing the group has a definite effect of exactly -0.0034.
B. The group is useless because the effect is below 0.01.
C. The effect is larger than one noise standard deviation, so there is mild evidence the group helps; checking recall@20 or more seeds would strengthen the conclusion.
D. The result is meaningless because the delta is negative.

Q10.10 *Hands-on.* In `book/code/ch10_ltr.py`, just before the line `model = ranker.train(train)`, add `ranker.PARAMS["label_gain"] = [0, 1, 1, 1]` so that an order is worth no more than a click, then rerun the script. Observe `ndcg@20` and `order_recall@20` for LambdaMART in the first table and compare with the values in this chapter, using the seed standard deviation (0.0021) as the yardstick for whether any change is real. Revert the change afterwards.

Chapter 11 — Re-ranking, diversity and business rules

The page that was all baby formula

Imagine you open the home page of a large marketplace app on a Tuesday evening. Last week you bought a pack of diapers and clicked on three tins of baby formula to compare prices. Your recommender is a good one: a gradient-boosted ranker from the previous chapter, trained on tens of thousands of examples, with a healthy offline NDCG. It looks at your history and concludes, correctly, that you are a new parent. So the first screen is baby formula, baby formula, baby formula, a baby bottle, more baby formula, and one lipstick that slipped in at position five.

Nothing is *wrong* with any single card; each is a defensible bet. But the *page* is a bad page. If you wanted to browse, there is nothing to browse, and if you bought a refrigerator yesterday, the page might offer three more. Your ranker scores items one at a time, and it has no idea that twenty similar good bets are not twenty times as good as one.

This is the job of the last stage of the pipeline: **re-ranking**. It takes the scored list and fixes properties that only exist at the level of the *whole list*: variety, fairness between sellers, things the user must never see again, things the business must show, and a little deliberate randomness so the system keeps learning. We look at the repo's re-ranker (a few lines of pandas), then Maximal Marginal Relevance (MMR), measure what diversity costs in offline accuracy, and finish with epsilon-greedy and Thompson sampling.

Four terms come up again and again, so here they are in plain words:

- **Ranker**: the model from Chapter 10 that gives every candidate item a score. Higher score means "more likely this shopper wants it".
- **ndcg@20** (normalised discounted cumulative gain, "how high are the right items in the top 20?"): a score from 0 to 1. It is higher when the items the shopper really went on to buy sit near the top of the 20-card list.
- **recall@K** ("how many of the right items made the list?"): of the items the shopper really bought, the share that appear in the top K.
- **Business rule**: a rule that comes from what the company needs, not from patterns in the data. "Never show a refrigerator the shopper already bought" and "no more than four items from one shop" are business rules. The ranker cannot learn them from clicks, so we apply them by hand.

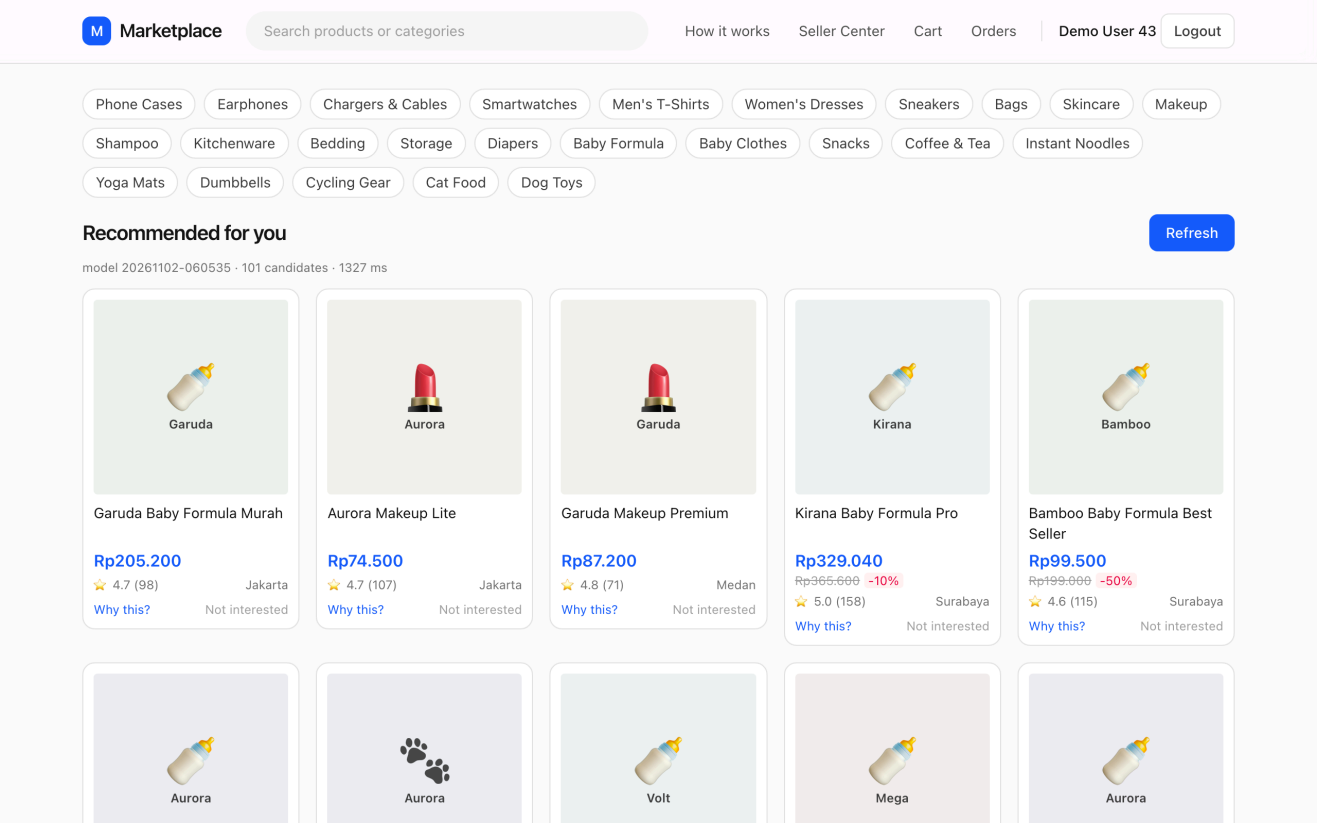


Figure 11.1: *The "Recommended for you" home page in our app: the first row is dominated by baby formula, with makeup mixed in. This is the page a re-ranker is designed to improve.*

What you'll learn

- Why a model that scores items independently produces lists that are bad as lists, and what "list-level" objectives are.
- How Maximal Marginal Relevance trades relevance against redundancy, with the formula and a hand-computed example.
- How the repo's `rerank.rerank` implements an already-bought filter and per-category and per-shop caps in five lines of pandas.
- How much offline accuracy each of these rules costs on our data, using real numbers from the companion script.
- How sellers' fairness, sponsored items and manual boosts fit into the re-ranking stage without corrupting the ranker.
- How epsilon-greedy and Thompson sampling choose which items to explore, with a worked Beta-distribution example.

Intuition: building a shopping window, not picking the best products

Think about how a shop owner dresses a window. She does not sort her stock by profit margin and then put the top twelve items in a row. If the twelve best sellers are all running shoes, the window says "we sell running shoes", and passers-by who need a coat walk on. She chooses a *set* of items that together say something: a shoe, a coat, a bag, a gift. Each item in the window is slightly worse than the

single best item she could have chosen for its slot, but the *window* is better, because the items complement rather than duplicate each other.

A recommender home page is a window. The ranker of Chapter 10 is the owner who sorts by margin. It answers "how likely is this user to engage with this item?" for each item independently, and independence is exactly the problem: after you have shown one baby-formula tin, the chance that a second one is useful is lower than the chance the first one had. Think of cake. The first slice is a treat, the second is nice, the fifth is a chore. Economists call this *diminishing marginal utility* (each extra copy is worth less than the last); in information retrieval it is called *redundancy* (the new item repeats what the page already says).

There are three families of re-ranking logic and we will meet all of them:

1. **Filters** remove items that must not appear (already bought, hidden, out of stock). Plain meaning: a yes/no check on each item.
2. **List-level objectives** such as diversity: the value of an item depends on what is already on the page. Plain meaning: we judge the page as a whole, not each card alone.
3. **Exploration**: sometimes put an item on the page because the system is *uncertain* about it, not because it is the best bet. Plain meaning: occasionally try something new so we can learn whether it is good.

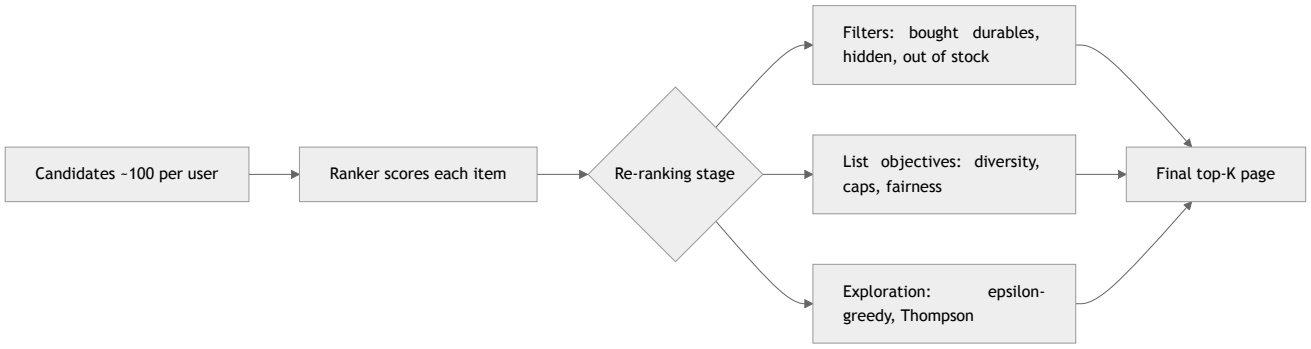


Figure 11.2: *Re-ranking is the last stage. It sees scored candidates and produces the page; it never retrains the model.*

The algorithm

Filters: the already-bought rule

The simplest rule is the *already-bought filter*: **do not show something the shopper already owns, unless it is the kind of thing people buy again.** Shopper Sari bought a refrigerator last month; showing her that same refrigerator is wasted space. Sari also bought a bag of coffee last month; she very probably wants coffee again, so coffee stays. Formally, we drop an item if the user ordered it before and the category is not consumable (a *consumable* is something used up and re-bought, like coffee or diapers). In the simulator (and in the schema from Chapter 1) each category has an `is_consumable` flag, which the feature table exposes as `item_is_consumable`, and the history feature `ui_orders` counts how many times this user ordered this item. The rule is

$$\text{drop}(u, i) \iff \text{ui_orders}(u, i) > 0 \ \wedge \ \text{item_is_consumable}(i) = 0.$$

Read it as: "drop item i for user u exactly when (the user has ordered it at least once) **and** (its category is not consumable)". For Sari's refrigerator, `ui_orders` is 1 and `item_is_consumable` is 0, so it is dropped. For her coffee, `item_is_consumable` is 1, so it stays.

Hidden items (the user pressed "Not interested") and out-of-stock items are removed earlier, in `candidates.generate`, which is cheaper because we do not compute features for rows that will never be shown.

Caps: a poor man's diversity

The second rule limits how many items from one category and one shop may appear. It is like a shelf label that says "maximum two per customer": the best ones still get through, but nobody can take them all. Here is a tiny example with a cap of 2 per category (illustrative; the repo uses larger caps). Sari's candidates, best score first, are: phone case (shop X), phone case (shop X), phone case (shop Y), cat food (shop Z). The first two phone cases are the 0th and 1st of their category, so they stay. The third phone case is the 2nd, which is not below the cap of 2, so it is dropped, even though it scores higher than the cat food. Cat food is the 0th of its category and stays.

Formally: sort the user's items by score. Number the items of each category $0, 1, 2, \ldots$ in score order and do the same for each shop. Keep an item only if its position within its category is below C_{cat} and its position within its shop is below C_{shop}. In the repo $C_{\text{cat}} = 8$ and $C_{\text{shop}} = 4$. Then take the first K survivors (K is the page length, here 20).

Caps are blunt but powerful. They give a hard guarantee (no page ever has more than four items from one shop) at the cost of being completely insensitive to how *similar* the items are. Two phone cases from two different shops, both in the "Phone Cases" category, are to a human near-duplicates, yet the caps allow eight of them.

Maximal Marginal Relevance

MMR builds the page one slot at a time, and each pick must be both good and different from what is already on the page. Picture ordering dishes with friends. You first order the best-sounding dish. For the second, you do not want something nearly identical, so a similar dish loses points even if it sounds great; a different dish with slightly lower appeal wins. MMR does exactly this with a score for every remaining item. The idea was proposed by Carbonell and Goldstein for reordering search results and summaries, and it is the classic answer to "how do we penalise similarity to what is already on the page?". The method is *greedy*: at each step it takes the best-looking choice right now and never goes back to change earlier picks. Start with the empty list S. At each step pick the item i from the remaining candidates R that maximises

$$\text{MMR}(i) = \lambda \cdot \text{rel}(i) - (1 - \lambda) \cdot \max_{j \in S} \text{sim}(i, j),$$

where $\text{rel}(i)$ is the relevance (how much the shopper is likely to want the item; here, the ranker score rescaled to $[0, 1]$ per user) and $\text{sim}(i, j) \in [0, 1]$ measures how similar two items are (1 = near-identical, 0 = unrelated). The $\max_{j \in S}$ part means "look at the page so far and take the *most* similar item already there". The parameter λ (Greek letter "lambda", between 0 and 1) is a dial between relevance and variety. At $\lambda = 1$ the second term vanishes and MMR reproduces the original ranking. At

$\lambda = 0$ relevance is ignored and you get a maximally spread list. For the first pick the set S is empty, so the penalty is zero by convention, and the top-ranked item always comes first.

The algorithm in numbered steps:

1. Scale each user's scores to $[0, 1]$: $\text{rel}(i) = (s_i - s_{\min})/(s_{\max} - s_{\min})$.
2. Precompute the pairwise similarity matrix for this user's candidates (a table with one row and one column per candidate, where each cell says how similar those two items are). In our companion script, $\text{sim} = 1$ for the same category, 0.5 for the same parent category (for example Dog Toys and Cat Food are both under Pets), and 0 otherwise.
3. Maintain, for every remaining candidate, the largest similarity to any selected item so far.
4. Repeat K times: compute the MMR score of every remaining item, select the argmax (the item with the highest score), mark it taken, and update the running maximum of similarities.

The cost is $O(K \cdot n)$ per user after the similarity matrix exists (roughly "K times the number of candidates n" steps), and building the matrix is $O(n^2)$ (every candidate compared with every other). With $n \approx 100$ candidates that is a few thousand operations; trivial.

A worked example with four items

Take $\lambda = 0.7$ and four candidates with normalised relevance and categories:

item	rel	category
A	1.0	Phones
B	0.9	Phones
C	0.6	Shoes
D	0.5	Beauty

Items in the same category have similarity 1; all other pairs have similarity 0 (we ignore the parent level here for simplicity).

- **Pick 1.** Nothing selected yet. Scores are $0.7\times$ rel: A = 0.70, B = 0.63, C = 0.42, D = 0.35. We pick **A**.
- **Pick 2.** B is a Phone, so its similarity to A is 1: $0.7 \times 0.9 - 0.3 \times 1 = 0.33$. C: $0.7 \times 0.6 - 0.3 \times 0 = 0.42$. D: 0.35. We pick **C**, although B is much more relevant.
- **Pick 3.** B still has similarity 1 to A (the maximum over the selected set $\{A, C\}$): 0.33. D has similarity 0 to both: 0.35. We pick **D**.
- **Pick 4.** Only B is left.

The final order is A, C, D, B. The Phones twin B fell from second place to last. With $\lambda = 0.9$ the penalty term is 0.1, so B would score $0.81 - 0.10 = 0.71$ against C's 0.54, and B would keep second place. This is the whole story of the parameter: λ decides how large a relevance gap a duplicate can survive.

Exploration: giving uncertain items a chance

Everything so far assumes the ranker's scores are right, but the ranker only knows what it has seen. An item never shown has no clicks, so the model scores it low, so it is never shown: a *feedback loop* we

study in Chapter 12. Without exploration, a great new item can stay hidden forever.

Think of choosing a lunch place. If you always go to the best restaurant you know, you will never discover the new place that might be even better. But trying a random place every day wastes lunches. The textbook version of this dilemma is the **multi-armed bandit** problem (named after slot machines, "one-armed bandits"): each item is an "arm" with an unknown click probability, and every *impression* (one showing of the item to a shopper) both may earn a click (the reward) and teaches us about that item.

Epsilon-greedy is the simplest policy: with probability $1 - \varepsilon$ (ε is the Greek letter epsilon, a small number like 0.1) show the model's best item, otherwise a random one from a pool. In lunch terms: nine days out of ten you go to your favourite, one day in ten you pick a random place. With $\varepsilon = 0.1$ on a 20-slot page, two slots are exploratory on average. It is trivial to implement and wasteful: the random slot is as likely to pick an item with 50 impressions and a dismal click rate as one with 2 impressions and unknown potential.

Thompson sampling is smarter about *where* to explore. Here is the idea with a coin. Each item behaves like a coin whose chance of "heads" (a click) we do not know. Show the item 10 times and see 3 clicks: the true click rate is probably near 0.3, but 0.2 or 0.45 would not surprise us. Show it 0 times: any rate from 0 to 1 is plausible. So for each item we keep a *curve that says how plausible each click rate is*. A narrow curve means we are fairly sure; a wide curve means we do not know yet.

The curve has a name: the **Beta distribution**, written $\mathrm{Beta}(a, b)$. You can think of a as "clicks so far, plus 1" and b as "non-clicks so far, plus 1". The "plus 1" comes from our starting assumption, called the **prior**: before seeing any data, every click rate from 0 to 1 is equally plausible, which is the flat curve $\mathrm{Beta}(1, 1)$. After we observe data, the updated curve is called the **posterior**. This update is simple (the technical word is *conjugate*, meaning the new curve is the same kind of curve as the old one, just with new numbers): after c clicks and $n - c$ non-clicks,

$$\theta \mid \text{data} \sim \mathrm{Beta}(1 + c,\ 1 + n - c), \qquad \mathbb{E}[\theta] = \frac{1 + c}{2 + n}, \qquad \mathrm{Var}[\theta] = \frac{ab}{(a+b)^2(a+b+1)}.$$

In words: θ (theta) is the item's unknown true click rate; $\mathbb{E}[\theta]$ is the curve's average, our best single guess of θ; $\mathrm{Var}[\theta]$ (the variance) measures how wide the curve is, and its square root is the *standard deviation* (the typical distance from the average). Here $a = 1 + c$ and $b = 1 + n - c$. For an item with 10 impressions and 3 clicks, $a = 4$, $b = 8$, so the average is $4/12 \approx 0.333$.

To choose what to show, *draw one random sample from each item's Beta distribution and rank by the draws*. An item with lots of evidence has a narrow curve, so its draw lands close to its average. An item with little evidence has a wide curve and sometimes draws a high number, which earns it an impression. As evidence accumulates, exploration dies out by itself, with no tunable ε.

A worked Thompson example

Three items, uniform prior:

item	impressions	clicks	posterior	mean	std dev
A (established)	10	3	Beta(4, 8)	0.333	0.131
B (shown twice, no clicks)	2	0	Beta(1, 3)	0.250	0.194
C (brand new)	0	0	Beta(1, 1)	0.500	0.289

The means are $4/12$, $1/4$ and $1/2$. The variances follow from the formula: for A, $ab/((a+b)^2(a+b+1)) = 32/(144 \times 13) \approx 0.0171$, so the standard deviation is about 0.131; for B it is $3/(16 \times 5) = 0.0375$ (std 0.194); for C it is $1/(4 \times 3) = 0.0833$ (std 0.289). Notice that the standard deviation grows as the evidence shrinks: A (10 impressions) is narrowest, C (none) is widest. That width is what lets C win a draw now and then.

Suppose one round of sampling yields the (illustrative) draws A = 0.31, B = 0.18, C = 0.77. The order is C, A, B: the unknown newcomer is shown first. If C then gets 1 click in 1 impression it becomes Beta(2, 1) with mean 0.667; if it gets zero clicks in 5 impressions it becomes Beta(1, 6), mean 0.143, and rarely wins another draw. The system explores exactly as much as it is uncertain.

Did you know?

Maximal Marginal Relevance is a 1998 idea that outlived its original task. MMR was introduced by Carbonell and Goldstein at SIGIR 1998 for reordering retrieved documents and for producing summaries, combining a query-relevance term with a novelty term that penalises similarity to documents already selected. The same greedy formula is still the default starting point for diversifying recommendation slates.

Source: Carbonell et al., SIGIR, 1998

Did you know?

Diversity can hurt average accuracy and still improve satisfaction. Ziegler and colleagues introduced "topic diversification" and the intra-list similarity metric for recommendation lists. They reported that, although it was detrimental to average accuracy, the method improved user satisfaction, which they tested in an online study with more than 2,100 subjects, in particular for lists produced by item-based collaborative filtering.

Source: Ziegler et al., WWW, 2005

Did you know?

Thompson sampling is older than computers, and surprisingly strong. William R. Thompson described the idea of choosing between treatments in proportion to their posterior probability of being best in a 1933 Biometrika paper. In 2011, Chapelle and Li reported that on display advertising and news article recommendation it was highly competitive with other bandit methods, and argued it should be a standard baseline.

Source: Chapelle et al., NeurIPS, 2011

Code walkthrough

The repo's re-ranker

The production re-ranker in this repo is the entire file `recommender/recsys/rerank.py`. Here is its only function.

recommender/recsys/rerank.py · rerank

```python
def rerank(scored: pd.DataFrame, k: int) -> pd.DataFrame:
    """`scored` needs user_id, item_id, category_id, shop_id, ui_orders, item_is_consumable,
score."""
    bought_durable = (scored["ui_orders"] > 0) & (scored["item_is_consumable"] == 0)
    scored = scored[~bought_durable].sort_values(["user_id", "score"], ascending=[True, False])
    # Keep an item only if it is within the user's top-N for its category AND top-M for its shop.
    cat_rank = scored.groupby(["user_id", "category_id"]).cumcount()
    shop_rank = scored.groupby(["user_id", "shop_id"]).cumcount()
    kept = scored[(cat_rank < MAX_PER_CATEGORY) & (shop_rank < MAX_PER_SHOP)]
    return kept.groupby("user_id").head(k)
```

Read it line by line. `scored` has one row per `(user_id, item_id)` candidate with the ranker's `score` plus the columns the rules need. The first statement builds the boolean mask `bought_durable`; the second drops those rows and sorts by user and descending score, which the following lines depend on.

Then comes the trick. `groupby(["user_id", "category_id"]).cumcount()` numbers the rows inside each group in their current order, starting from zero. Because the frame is already sorted by score, the number is exactly "how many higher-scored items of this category does this user already have". The same is done per shop. A row survives when both counters are under their limits (`MAX_PER_CATEGORY = 8`, `MAX_PER_SHOP = 4`, defined at the top of the module). Finally `groupby("user_id").head(k)` truncates each user's list to k items. The function is fully *vectorised* (it works on whole columns at once instead of looping over rows in Python), which matters: during training evaluation it runs for thousands of users at once, and in the live `Recommend` RPC (remote procedure call, the request the app sends to the recommender service) it runs on one user's frame.

The MMR companion script

The companion script `book/code/ch11_rerank.py` implements MMR on top of the LambdaMART scores from Chapter 10 (it re-trains the same ranker through a small shared helper, so the numbers are comparable with the previous chapter). First the similarity function.

book/code/ch11_rerank.py · similarity

```python
def similarity(cat: np.ndarray, parent: np.ndarray) -> np.ndarray:
    """1 if same category, 0.5 if same parent category, else 0."""
    same_cat = cat[:, None] == cat[None, :]
    same_par = parent[:, None] == parent[None, :]
    return np.where(same_cat, 1.0, np.where(same_par, 0.5, 0.0))
```

It broadcasts the category and parent-category arrays into an $n \times n$ matrix with `cat[:, None] == cat[None, :]`; nested `np.where` gives 1 for the same category, 0.5 for the same parent, else 0. Partial credit for the parent matters because "Dog Toys" and "Cat Food" are different categories but both Pets, and a page full of Pets items is still repetitive.

Now the greedy loop itself.

book/code/ch11_rerank.py · mmr

```python
def mmr(scored: pd.DataFrame, parent_of: dict[int, str], lam: float, n: int = common.TOP_N) ->
pd.DataFrame:
    """Greedy MMR per user: pick argmax  lam * rel - (1 - lam) * max_sim_to_selected."""
    rows = []
    for uid, g in scored.groupby("user_id", sort=True):
        s = g["score"].to_numpy()
        rel = (s - s.min()) / max(s.max() - s.min(), 1e-9)
        cat = g["category_id"].to_numpy()
        par = np.array([parent_of[c] for c in cat])
        sim = similarity(cat, par)
        chosen: list[int] = []
        max_sim = np.zeros(len(g))
        avail = np.ones(len(g), dtype=bool)
        for _ in range(min(n, len(g))):
            mmr_score = np.where(avail, lam * rel - (1 - lam) * max_sim, -np.inf)
            j = int(np.argmax(mmr_score))
            chosen.append(j)
            avail[j] = False
            max_sim = np.maximum(max_sim, sim[j])
        sel = g.iloc[chosen]
        rows.append(sel[["user_id", "item_id", "category_id"]])
    return pd.concat(rows, ignore_index=True)
```

Work through it in order. Scores are normalised to $[0, 1]$ per user (`1e-9` guards against all-equal scores). `max_sim` holds, for every candidate, the largest similarity to anything chosen so far; zeros mean "empty set, no penalty". Inside the loop, `mmr_score` is exactly $\lambda \cdot \text{rel} - (1 - \lambda) \cdot \text{max_sim}$, with taken items set to $-\infty$. After choosing `j`, one row of the similarity matrix updates the running maximum with `np.maximum(max_sim, sim[j])`, which is what keeps the algorithm cheap.

The loop returns only `user_id`, `item_id` and `category_id` in chosen order, the format `common.evaluate` expects. A helper, `diversity`, counts distinct categories in each user's top 20 and averages over users; this is the *intra-list diversity* plotted below.

The `main` function loops over $\lambda \in \{1.0, 0.9, 0.7, 0.5, 0.3\}$, then evaluates two more variants: the repo's `rerank.rerank`, and the already-bought filter alone, so we can see which rule costs what.

Run it

From the `book/` directory:

```
uv run python code/ch11_rerank.py
```

The script prints a table for all methods. These are the captured results: the first block shows the five MMR settings and the repo's rules.

TERMINAL OUTPUT

```
                          recall@5  recall@10  recall@20  recall@50  order_recall@20  ndcg@20
coverage@20  distinct_categories@20
method
MMR lambda=1.0                0.1503     0.2015     0.2725     0.3824           0.4210   0.2116
0.4245                  5.3263
MMR lambda=0.9                0.1459     0.1921     0.2648     0.3788           0.4149   0.2081
0.4063                  6.7782
MMR lambda=0.7                0.1163     0.1674     0.2398     0.3688           0.3927   0.1884
0.3726                 11.0317
MMR lambda=0.5                0.0930     0.1169     0.1907     0.3582           0.3164   0.1603
0.3354                 16.2777
MMR lambda=0.3                0.0927     0.1030     0.1430     0.3580           0.2351   0.1401
0.3129                 18.6864
rerank.rerank caps            0.1496     0.1971     0.2505     0.2956           0.3955   0.2044
0.3825                  6.5787
```

And the script's own summary of the effect of each λ, relative to the unmodified ranker:

TERMINAL OUTPUT

```
LambdaMART alone: ndcg@20 0.2116, 5.33 distinct categories in the top 20
  MMR lambda=0.9     ndcg change -0.0035, distinct categories +1.45
  MMR lambda=0.7     ndcg change -0.0232, distinct categories +5.71
  MMR lambda=0.5     ndcg change -0.0513, distinct categories +10.95
  MMR lambda=0.3     ndcg change -0.0715, distinct categories +13.36
```

Results: what does diversity cost?

A reminder of the measures in the table, in plain words. **ndcg@20** and **recall@5/20/50** ask whether the items the shopper really bought later appear in the top 5, 20 or 50 (higher is better). **distinct_categories@20** counts how many different categories appear in a user's top 20, averaged over users: our measure of variety (higher means more varied). **coverage@20** is the share of the whole catalogue that shows up in *anyone's* top 20: it tells us whether the system uses many products or keeps showing the same few.

Start with the first row. `MMR lambda=1.0` is the LambdaMART ranking untouched (with $\lambda = 1$ the penalty is multiplied by zero): ndcg@20 of 0.2116 and recall@20 of 0.2725. The top 20 contains on average 5.33 distinct categories. That is our baseline: roughly five different kinds of product on a 20-card page. Not terrible, but not a window either.

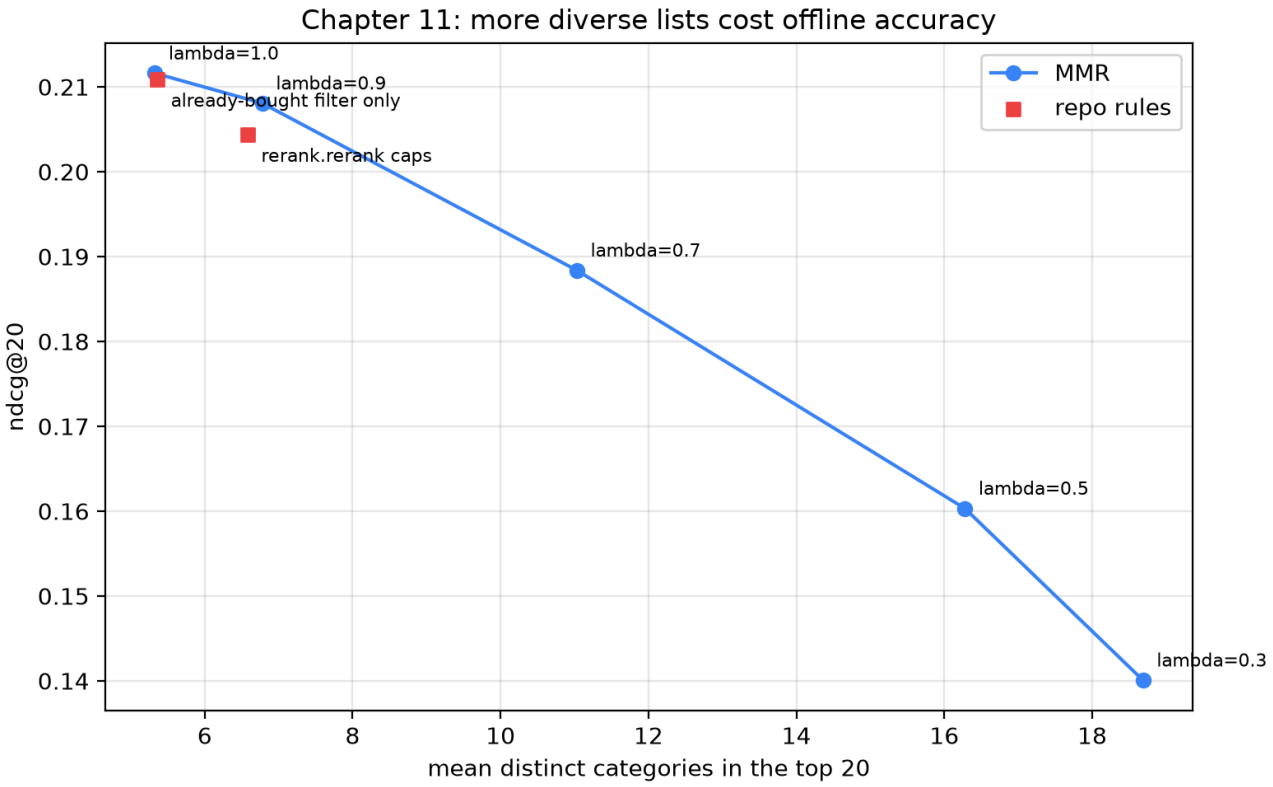


Figure 11.3: *Trade-off between offline ndcg@20 and the mean number of distinct categories in the top 20, for five MMR settings and for the repo's rule-based variants.*

The chart plots the same table. Moving down the MMR line from the top left is moving from "pure relevance" to "pure variety". Reading the numbers:

- $\lambda = 0.9$: ndcg falls from 0.2116 to 0.2081, a change of -0.0035, a relative loss of under 2%. Distinct categories rise from 5.33 to 6.78, an increase of 1.45. This is by far the best exchange rate on the curve: one and a half additional categories for a third of a point of NDCG.
- $\lambda = 0.7$: ndcg drops 0.0232 to 0.1884, while the page now spans 11.03 categories: 5.71 more than the baseline. A little over two additional categories for every 0.01 of ndcg lost, and the curve begins to steepen.
- $\lambda = 0.5$ **and** 0.3: ndcg is 0.1603 and 0.1401, and the distinct-category count reaches 16.28 and 18.69. A 20-card page with 18.69 categories is almost entirely unique categories. We have given up roughly a quarter and a third of the ranker's ndcg to get it.

The deep end is instructive. `recall@5` goes from 0.1503 at $\lambda = 1.0$ to 0.0927 at $\lambda = 0.3$, because MMR pushes down strong items similar to ones already placed, so the *top* of the list suffers most. But `recall@50` barely moves, from 0.3824 to 0.3580: the same items are still in the list, merely lower. Re-ranking changes order and selection from a fixed pool; it never invents new good candidates.

Coverage tells a related story: coverage@20 drops from 0.4245 to 0.3129 as λ falls. MMR diversifies *categories within one user's list*, and to do so it dips into each category's best items, which are the same popular items for everyone. The page is more varied, yet the system shows a narrower slice of the catalogue. Per-list diversity and system-level coverage are different goals needing different tools, such as per-item exposure caps.

The repo's own rules

Now the two red squares in the chart. `already-bought filter only` scores ndcg@20 of 0.2109 against 0.2116, a loss of 0.0007, far smaller than any MMR step, with 5.36 categories on average: nearly free offline. `rerank.rerank caps` (filter plus 8 per category and 4 per shop) yields 0.2044 and 6.58 distinct categories. MMR at $\lambda = 0.9$ does slightly better (0.2081 with 6.78 categories), because it demotes items according to their similarity to the rest of the page, while the caps only react once a count passes a threshold.

There is a sharper difference in `recall@50`: 0.2956 for the caps against 0.3789 for the filter alone. `rerank` removes items without replacing them, so when a user's candidate pool is concentrated in a few categories the surviving list can be shorter than 50 (our likely explanation; we did not instrument it). Strict caps therefore need enough variety from the candidate generator upstream.

A caution: the offline labels come from users who saw whatever the *old* system showed. They cannot reward variety the user never saw, and the long-term boredom of a monotone page does not show up in a seven-day window. Ziegler and colleagues needed an online study to demonstrate the benefit, and you will too. Treat the offline curve as the *price list* and the A/B test (Chapter 14) as the place to decide how much to pay.

Beyond diversity: business rules in the re-ranker

Fairness to sellers

A marketplace is two-sided: buyers want relevance, sellers want to be seen. A ranker trained on engagement gives established shops more impressions, hence more reviews: a rich-get-richer loop that can starve a good new seller. The per-shop cap is a crude fairness rule. More principled approaches work with *exposure* (how much attention an item actually receives): since attention decays steeply with rank (Chapter 12), slot 15 is not slot 1. Singh and Joachims formalised constraints on the exposure items or groups receive relative to their merit and gave algorithms that maximise user utility (total benefit to shoppers) subject to them. In practice marketplaces implement simple versions, such as minimum slots for new sellers or a per-shop daily exposure budget.

Ads, boosts and sponsored slots

Almost every marketplace sells advertising placement. The design rule is **do not contaminate the ranker with ad money.** Ads are inserted *after* the ranker, into fixed positions (say slots 3, 8 and 13), chosen by a separate auction (sellers bid for the slot) that weighs the bid and a predicted click rate, and labelled "Sponsored". A campaign boost ("all 9.9 sale items get $+0.05$", meaning we add 0.05 to their score) should likewise be a visible term in the re-ranker with a start and end date, not a change to the training data; otherwise the next retraining treats boosted impressions as organic evidence (clicks that happened naturally, without a push from us) and bakes the campaign into the model.

Exploration in the re-ranker

In practice, run the pipeline as usual, then swap the item in a fraction ε of *slots* (not users) for an under-exposed candidate, preferably in the lower half of the page; for Thompson sampling, keep click

and impression counts per item in a feature store (a database of ready-to-use model inputs) and blend one Beta draw per candidate into the score. Either way, **log which slots were exploratory and with what probability**; otherwise the data stays as biased as before and cannot be corrected (inverse propensity weighting, Chapter 12: giving each logged example a weight that undoes how unlikely it was to be shown).

Pitfalls and production practice

Normalise relevance before MMR. The formula mixes a score with a similarity in $[0, 1]$, but LightGBM scores are unbounded and differ per user. Without rescaling (we use per-user min-max), λ means something different for every user.

Do not tune λ offline only. The curve above is the price list, not a prescription. Start mild (our 0.9 gives most of the first gains), A/B test on session-level metrics (orders per user, return rate), and move the dial from there.

Order of operations matters. Filters first, diversity next, ad insertion last; carve out exploration slots *before* caps, or a cap can silently remove the explored item.

Caps can shorten the page, and re-ranking cannot fix a bad pool. Decide whether to backfill, relax the caps or accept a short page (the repo accepts it). If everything in the pool is baby formula, MMR just picks the least-similar baby formula, so enforce diversity upstream too, for example with separate retrievers per interest cluster.

Summary

- Rankers score items independently, so the best-scored list can be repetitive; re-ranking optimises the whole list.
- Filters (already-bought durables, hidden, out of stock) cost almost nothing offline: 0.2109 versus 0.2116 ndcg@20.
- Caps give hard guarantees but are blind to similarity; here they cost more accuracy than mild MMR for similar variety.
- MMR picks greedily by $\lambda \cdot \text{rel} - (1 - \lambda) \cdot \max \text{sim}$; $\lambda = 0.9$ added about 1.45 categories for -0.0035 ndcg, while lower values lose accuracy quickly. Tune λ by A/B test.
- Keep fairness rules, ad slots and boosts in the re-ranker, outside the model, so they can be audited and switched off.
- Epsilon-greedy explores at a fixed rate; Thompson sampling explores in proportion to uncertainty. Log which slots were exploratory.

Quiz

Q11.1 In MMR with $\lambda = 1$, what does the algorithm return?

A. A list with maximal diversity, ignoring relevance.
B. The original relevance ordering, because the similarity penalty is multiplied by zero.

C. The reverse of the original ordering.
D. A random permutation.

Q11.2 Why does `rerank.rerank` exempt consumable categories from the already-bought filter?

A. Consumables are cheaper, so the business does not care.
B. Consumables cannot be hidden by users.
C. Buying consumables again (coffee, diapers) is a normal and desirable repeat purchase, unlike a durable such as a refrigerator.
D. Consumables do not have a `shop_id`.

Q11.3 What is the main advantage of Thompson sampling over epsilon-greedy?

A. It needs no logging of impressions.
B. It never shows the model's top item.
C. It explores in proportion to uncertainty, so well-measured poor items stop being shown and new items get chances without a fixed ε.
D. It is guaranteed to maximise offline NDCG.

Q11.4 Explain in two or three sentences why business rules such as sponsored slots and campaign boosts should be applied in the re-ranker and not as extra features or label changes in the ranker's training data.

Q11.5 In the results, `recall@50` for `rerank.rerank caps` is much lower than for the already-bought filter alone, while `recall@5` is almost identical. Give the most likely explanation.

Q11.6 Read the `rerank` function included in this chapter. A user has 30 scored candidates, all from the same category, coming from 10 different shops (3 items each), and none of them is a bought durable. With `MAX_PER_CATEGORY = 8`, `MAX_PER_SHOP = 4` and $k = 20$, how many items does the function return for this user, and which rule is responsible?

Q11.7 Use MMR with $\lambda = 0.5$ on three items with normalised relevance 0.9 (category X), 0.8 (category X) and 0.5 (category Y). Similarity is 1 for the same category and 0 otherwise. Give the final order, showing the MMR scores at each step.

Q11.8 An item starts with a uniform Beta(1, 1) prior and then receives 20 impressions and 5 clicks. What is its posterior distribution, its mean click rate and its standard deviation (to three decimals)? Compare the standard deviation with that of a new item (no impressions) and say what that means for Thompson sampling.

Q11.9 Which statement best matches the chapter's measured results?

A. MMR with $\lambda = 0.9$ cost about 0.0035 ndcg@20 and added about 1.45 distinct categories; $\lambda = 0.3$ cost about 0.0715 ndcg@20.
B. MMR at every λ raised coverage@20 above the baseline.
C. The already-bought filter alone cost about 0.05 ndcg@20.
D. The caps reduced the number of distinct categories below the baseline.

Q11.10 Hands-on: copy `book/code/ch11_rerank.py` to a scratch file and change `similarity` so that sharing a parent category gives 0.0 instead of 0.5 (only the exact same category counts as similar). Run it and compare ndcg@20 and `distinct_categories@20` at $\lambda = 0.5$ with the chapter's table. State what you expect to observe (hint: how does the penalty change for items in sibling categories?) and whether the result matches.

Chapter 12 — Cold start, context and bias

It is 00:01 on the night of a 12.12 sale. Within a single minute, tens of thousands of people who have never opened the marketplace before tap the app icon, because a friend sent them a flash-deal link. For the recommender these visitors are ghosts: no clicks, no searches, no history. At the same moment, three hundred sellers publish brand-new products that have never been shown to anybody, so no one has clicked them either. And in the background a data scientist is staring at a dashboard, puzzled: the item at the top of every home feed gets a click-through rate more than ten times higher than the item in the last slot, and the team cannot tell how much of that gap is "the top item is better" and how much is "the top item is at the top".

Three problems are hiding in that story, and this chapter takes them one at a time:

- **Cold start**: the model is asked about a user or an item that it has no behaviour data for.
- **Context**: the right answer depends on *when* and *where* the request happens, not just on *who* is asking.
- **Bias**: the clicks we train on were produced by the very system we are trying to improve, so they are a distorted picture of what people want.

What you'll learn:

- The three flavours of cold start (new user, new item, new system) and a practical ladder of fallbacks for each.
- How this repository degrades gracefully, from a personalised ranker, to trending items for anonymous users, to a plain SQL query when the Python service is down.
- What position bias is, how to see it in the impression log, and how inverse propensity weighting corrects for it.
- How a feedback loop turns a small popularity advantage into a large one, and the cheapest ways to resist it.
- Why request-time context (hour of day, device, surface) is *not* a feature of this book's weekly ranker, and what the real-time companion model looks like.
- How to measure all of this: ranker quality by user activity bucket, and how often new items reach the top 20.

Words used in this chapter

A few terms appear many times below. Here they are in everyday words, using shopper Sari and her phone-case search as the example.

Term	Plain meaning	Tiny example
Impression	One time an item was shown on screen.	The app shows Sari a phone case: 1 impression.
Click-through rate (CTR)	Clicks divided by impressions.	Shown 10 times, clicked 3 times: CTR $= 3/10 = 0.3$.
Candidate / retriever	The cheap first step picks a few hundred *possible* items (candidates); each rule that picks them is a retriever.	"Phone cases Sari viewed recently" is one retriever.
Ranker	The model that sorts those candidates, best first.	It puts the case Sari is most likely to want in slot 0.
LambdaMART	The kind of ranker this book uses: many small decision trees trained to sort lists well (chapter 10).	Only the name is new; treat it as "the ranker".
Position	The slot an item was shown in, counted from 0 at the top.	Position 0 is the first card; position 11 is the twelfth.
Surface	Which screen or area of the app showed the item.	Home feed, product page, search results.

Intuition: the new colleague, the restaurant menu and the escalator

Imagine you are the first person to greet a customer in a bookshop. A stranger walks in. You know nothing about them. What do you do? You do not stay silent. You point at the table of bestsellers, because bestsellers are the best guess for an unknown person. After the stranger picks up two cookbooks, you walk them to the cooking shelf. After ten minutes you are recommending a specific author. Cold start is exactly this ladder: *start with what is true for everyone, then narrow as evidence accumulates.* A recommender that skips the ladder either says nothing (an empty page) or pretends to know a person it has never met.

New items are the mirror image: a waiter who only recommends dishes others have ordered never promotes a brilliant new dish, so it is never ordered. Someone must spend a recommendation on something unproven (the exploration problem of chapter 11).

Position bias has a clean analogy: a supermarket shelf. Products at eye level sell more than products near the floor, even if you swap them. The shelf height changes how many people *see* the product, not how good it is. If you judged the products by sales alone, you would "learn" that eye-level products are better and then put them at eye level again. That self-confirming circle is a **feedback loop** (the system's own output becomes its next input, and the habit grows), and we will see it twice in this chapter, once for position and once for popularity.

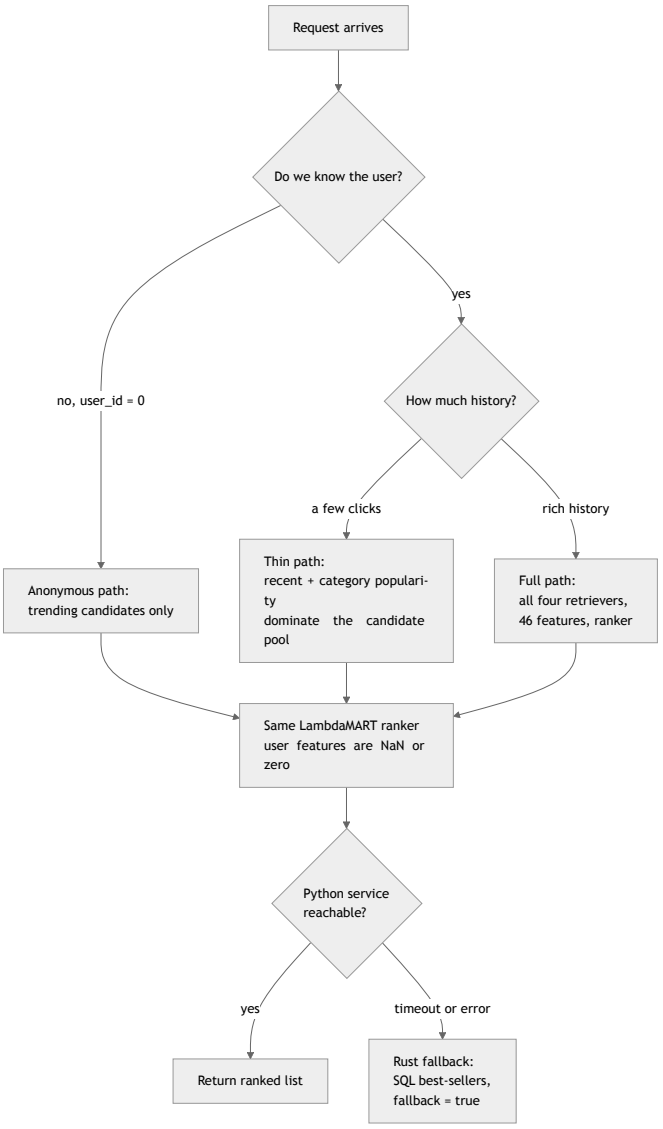


Figure 12.1: *the cold-start ladder. The same ranker serves every user; what changes is how much evidence the candidate generators and features can contribute¸ and an outer safety net catches service failures.*

The algorithm: ladders, propensities and loops

This section has four parts. First we look at where cold start bites in the pipeline we already built. Then we work through position bias and inverse propensity weighting with a small example. Then we model a feedback loop, and finally we decide where context belongs.

Where cold start bites

Cold start means *the model has no behaviour data about the thing it is being asked about*. It comes in three kinds:

Kind	Who or what is new	Example
New user	A person with no clicks yet.	Sari installs the app tonight, after a friend's flash-deal link.
New item	A product with no views, clicks or orders.	A phone case that a seller listed one hour ago.
New system	The whole service has no data.	Launch day, when nobody has clicked anything.

Without a plan, each of these produces either an empty page or a confident guess about someone the system has never met. The rest of this chapter is the plan.

Walk through the pipeline of chapters 8–10 and ask at each stage: *what happens if there is no data?*

1. **Candidate generation.** The four retrievers are `recent` (items the user viewed), `covisit` (items viewed together with those), `catpop` (popular items in categories the user likes) and `global_trending`. For a user with no history, the first three return nothing for that user. `global_trending` does not depend on the user, so *every* user, including a brand-new one, receives at least the globally trending items. That is the first rung of the ladder, and it is built into `candidates.generate`.
2. **Features.** A *feature* is one number the ranker reads about a user or an item, such as "clicks in the last 30 days". User-level features are built by adding up (aggregating) events. A user with no events has no row in that sum. The code first lists *every* user and then attaches the sums with a **left join** (keep every row on the left, even when nothing matches on the right), so the unmatched users get blanks, called missing values (`NaN`, "not a number"). The builder fills the blanks with explicit **sentinels** (placeholder values chosen on purpose): counters become 0, and "days since active" becomes a large constant (`NEVER_DAYS = 999`). LightGBM (the tree library behind the ranker) natively accepts missing values for the rest: at each split in a tree it learns which branch the blanks should go down.
3. **Ranking.** The ranker is one model for everyone. It has seen training examples with thin and empty histories, so it has learned what to do when `user_clicks_30d` is zero: lean on item-level features such as smoothed item CTR and conversion rate. **New items** are the harder case, because the usual strong signals are all behavioural. A product listed an hour ago has no clicks, no orders, no co-visits, and an item-level CTR that is the Bayesian-smoothed prior. The smoothing formula from chapter 3 is exactly the right tool: with prior rate p_0 and prior strength s,

$$\hat{p} = \frac{x + p_0\, s}{n + s}$$

where x is the number of clicks and n the number of impressions. With $x = 0$ and $n = 0$ the estimate is p_0, the catalogue average, not zero. A new item is therefore treated as "average until proven otherwise", rather than "terrible because nobody clicked it". Without smoothing, a brand-new item would have a CTR of $0/0$ (undefined) or 0 (hopelessly low). The ranker also receives `item_age_days`, so it can learn explicit freshness effects.

Here are the numbers for a tiny case. Say the catalogue-average CTR is $p_0 = 0.05$ and the prior strength is $s = 20$, which you can read as "pretend 20 imaginary impressions at the average rate". A

new phone case with $x = 0$, $n = 0$ gets $(0 + 0.05 \times 20)/(0 + 20) = 0.05$, the average. An older case with 2 clicks in 10 impressions gets $(2 + 1)/(10 + 20) = 0.10$: its real evidence pulls it above average, but not all the way to the raw $2/10 = 0.20$. The more impressions an item collects, the less the imaginary ones matter.

Position bias and inverse propensity weighting

Position bias means *an item gets more clicks simply because it was shown higher on the page, not because it is better*. Suppose phone case A sits in slot 0 and phone case B, an equally good case, sits in slot 5 (below the fold, where Sari must scroll). A collects more clicks. If we train a ranker on clicks without correcting this, it learns "A is better" and puts it in slot 0 again. The model is rewarding its own earlier placement. To correct for it we need a way to *separate* "was it seen?" from "was it liked?".

Now the standard model. A user clicks an item only if two things happen: they *examine* it (actually look at it) and they *like* it. The classic **examination hypothesis** writes this as a product, and a product is right because both must hold, like needing both a green light and an open road:

$$P(\text{click} = 1 \mid \text{item } i, \text{position } k) = \underbrace{\theta_k}_{\text{examination}} \cdot \underbrace{r_i}_{\text{relevance}}$$

Here $\theta_k \in [0, 1]$ is the probability that a user even looks at position k (the **propensity** of that position) and r_i is how much the user likes item i if they look. We want to learn r_i, but the logs only contain clicks, which mix θ_k and r_i. Training a ranker directly on clicks teaches it to rank *items that were shown high* on top, which is circular.

Try it with Sari: if she looks at slot 5 half the time ($\theta_5 = 0.5$) and likes a case with probability $r = 0.4$, she clicks it with probability $0.5 \times 0.4 = 0.2$. A case she likes just as much in slot 0 ($\theta_0 = 1.0$) gets $1.0 \times 0.4 = 0.4$. Same item quality, double the clicks, purely from position.

The **inverse propensity weighting** (IPW) idea in one sentence: *count a click at a rarely-seen position as more than one click, because that click was harder to get.* The fix is one line: weight each click by $1/\theta_k$, the inverse of the propensity. A click in a slot that is seen only 25% of the time counts as 4 clicks. "In expectation" means "averaged over very many repeats"; on average,

$$\mathbb{E}\left[\frac{c}{\theta_k}\right] = \frac{\theta_k \, r_i}{\theta_k} = r_i,$$

Read the formula left to right: c is 1 if the item was clicked and 0 if not, we divide by the chance θ_k that the item was looked at, and on average the result is r_i. The position cancels, which is the whole point. So the weighted click rate is an unbiased estimate of relevance regardless of where the item was displayed ("unbiased" means it is not pushed systematically too high or too low). Here is a worked example with toy numbers. Suppose the propensities are $\theta_0 = 1.0$, $\theta_1 = 0.5$, $\theta_2 = 0.25$. Item A was shown 100 times at position 0 and got 20 clicks. Item B was shown 100 times at position 2 and got 10 clicks.

1. Raw CTR: A is $20/100 = 0.20$, B is $10/100 = 0.10$. Raw clicks say A is twice as good.
2. Weight each click: every click on A counts $1/1.0 = 1$, every click on B counts $1/0.25 = 4$.
3. Weighted clicks: A is $20 \times 1 = 20$, B is $10 \times 4 = 40$.
4. Debiased relevance: A is $20/100 = 0.20$, B is $40/100 = 0.40$.

After correction B is twice as relevant as A. Raw clicks had it backwards, because B was penalised for being displayed in the third slot.

IPW has a price: **variance**, meaning how jumpy the estimate is from one batch of data to the next. A click at a position with $\theta_k = 0.02$ counts 50 times, so a single lucky click can dominate the training set. In practice we **clip** the weights, which means capping them at a maximum: $w = \min(1/\theta_k, M)$ with M around 10–50. Clipping accepts a little **bias** (a small systematic error) in return for a lot less variance.

The harder problem is estimating θ_k. The tempting shortcut is to read it off the CTR-by-position table: $\hat{\theta}_k \propto \mathrm{CTR}_k / \mathrm{CTR}_0$. In words: if position 1 gets half the CTR of position 0, guess that people look at position 1 half as often (for example CTRs of 0.20 and 0.10 give $\hat{\theta}_1 = 0.5$). That only works if the *same quality of items* appears at every position. In a logged system the ranker deliberately puts its best guesses at the top, so the table **confounds** examination with relevance, meaning the two effects are tangled and you cannot tell which one produced the gap. Honest propensity estimates need **randomisation**: for a small fraction of traffic, swap two adjacent items or shuffle a small slice of the page, so that quality no longer depends on position, and compare. The cheaper alternative is to treat position as a **model feature** during training (so the model can attribute part of each click to position) and then set it to one fixed value at serving time, which is what exercise 3 in `recommender/README.md` suggests.

Feedback loops

Position bias is the *local* feedback loop; the *global* one works at the catalogue level. Say the home feed shows item A, which has 10 more clicks than item B this week. The ranker, trained on clicks, gives A a higher score. A is shown more often, collects more clicks, and widens the gap. A toy model makes the dynamics concrete. Let each item's displayed share in week $t+1$ be proportional to its clicks in week t, and suppose clicks are proportional to displayed share times true relevance r. Then

$$s_i^{(t+1)} = \frac{s_i^{(t)} r_i}{\sum_j s_j^{(t)} r_j}.$$

Take $r_A = 0.12$, $r_B = 0.10$ and a 50/50 start. In words: s_i is item i's share of the screen time, and r_i is the chance a display gets a click. Week 1: A earns $0.5 \times 0.12 = 0.06$, B earns $0.5 \times 0.10 = 0.05$, total 0.11, so $s_A = 0.06/0.11 = 0.545$. Week 2 repeats with the new shares: A earns $0.545 \times 0.12 \approx 0.0655$, B earns $0.455 \times 0.10 = 0.0455$, total 0.111, so $s_A = 0.590$. The slightly better item keeps gaining share. Worse, an item C with a great $r_C = 0.30$ that starts with share $s_C = 0$ stays at 0 forever, because the formula multiplies by s_C and so is never discovered. The loop amplifies what the system already believes; it does not question it.

Chaney et al. (RecSys 2018, a research conference on recommenders) showed in simulation that training on data from already-recommended items makes everyone's behaviour more alike without adding value for users. The antidotes are:

- **Exploration**: deliberately show some items the model knows little about, to collect evidence. The classic framing is the **multi-armed bandit** (named after a row of slot machines, each "arm" being one item): you must balance *exploiting* the item that looks best so far with *exploring* arms you have barely tried. The simplest rule, epsilon-greedy, is "most of the time show the best-

looking item; with probability ϵ, say 1 slot in 20, show a random or new one". This is the exploration problem from chapter 11; this repository does not implement it, so treat it as a technique to add.

- **Diversity**: avoid filling the whole list with near-copies of one item.
- **Logging impressions with positions**, so that later models can correct for position bias as above.

Where does context belong?

Context means *the circumstances of one particular request*: the hour, the day of the week, the device, and the surface. Sari searching for a phone case at 23:00 on her phone in bed may want something different from Sari at lunchtime on a laptop; the same person, a different moment. Why does the ranker of chapters 9–10 ignore it? It predicts a 7-day outcome per user, so the request hour is identical for all of one user's candidates and there is no per-request label to learn from (`recommender/README.md`, "What's deliberately simplified").

In this chapter, request-time context means: hour of day, day of week, device, entry surface (the part of the app where the visit began). This repository logs all of them (the `sessions` table stores `surface` and `device_os`; `events` has `ts`, `position` and `surface`), and yet the LightGBM ranker of chapters 9–10 does **not** use them as features. That is deliberate, and `recommender/README.md` explains why in its "What's deliberately simplified" section.

The correct place for context is a **second, real-time model** trained on the impression log: each row is one displayed item, the label is whether it was clicked, and the features include hour, surface, device and the *position it was shown at*. That model re-scores the few dozen items that the weekly ranker already selected. Real-time **session features** belong to the same layer: "categories clicked in the last 30 minutes", "the last search query", "items added to the cart this session". The `recent` retriever and `Store.refresh()` already give this repository a taste of it, because a click posted two seconds ago changes the next response; but the ranker itself is retrained only periodically.

To picture the two layers, think of a shop assistant. The weekly ranker is the assistant who knows what you usually like and picks twenty items from the back room in the morning. The real-time model is the same assistant glancing at you at the counter ("it's late, you're holding a phone, you just looked at cases") and re-ordering those twenty on the spot. A *label* is simply the right answer a model learns from, here "was this displayed item clicked?".

> **Did you know?**
>
> **Users look at the top two results almost equally, but click the first far more often.** Joachims and colleagues tracked the eyes of students searching the web and found that the abstracts at ranks 1 and 2 were viewed with nearly equal frequency, yet users clicked the first link substantially more often than the second. In a follow-up experiment where the top two results were swapped without the users' knowledge, users still showed a strong bias to click whatever was in first position, even when the second result had been judged more relevant. The authors called this *trust bias* and concluded that clicks are "informative but biased". Source: Joachims et al., SIGIR, 2005

> **Did you know?**
>
> **Heuristics are surprisingly hard to beat when you know nothing.** Schein and colleagues benchmarked a probabilistic model for the cold-start problem, recommending items that nobody has rated yet, against trivial "heuristic recommenders". On the implicit-feedback task, simply ranking users by how many movies they had rated gave nearly the same global performance as their aspect model, which led the authors to argue that such heuristic baselines should always be included in this kind of evaluation. Source: Schein et al., SIGIR, 2002

Code walkthrough

We will follow the same ladder as the pipeline: how the retrievers cope with missing history, how the features fill in gaps, how the product page handles an item with no co-views, and finally what the Rust API does when the whole Python service disappears.

Candidates for everyone

Start with the one retriever that needs no user at all:

`recommender/recsys/candidates.py · global_trending`

```
def global_trending(hist: History, available: pd.DataFrame) -> pd.DataFrame:
    pop = item_popularity(hist, 3.0)
    pop = pop[pop.index.isin(available["item_id"])].nlargest(N_GLOBAL)
    return pd.DataFrame({"item_id": pop.index, "cand_global_pop_rank": np.arange(len(pop))})
```

`global_trending` takes the last three days of weighted interactions (click, cart and order events, each with its own weight), keeps items that are in stock, and returns the top `N_GLOBAL`. It takes no user argument. That one fact is the entire anonymous-user story: the function produces candidates that are the same for everyone.

Every user id in `generate` is **cross-joined** with the trending frame (every user is paired with every trending item), so no user ever has zero candidates and the ranker never receives an empty group. Missing source scores become 0.0 and the missing trending rank becomes 999. A cold user's candidates all have `cand_n_sources` (how many retrievers proposed this item) equal to 1, a pattern the ranker learns to recognise as "we know very little about this person".

Features that tolerate emptiness

In `features.user_features` every user has a row (it starts from the full `users` table), count merges are left joins, and `fillna` turns missing counts into zero. `_smoothed`, the formula $\hat{p} = (x + p_0 s)/(n + s)$, is called in `user_features` as `_smoothed(u["user_clicks_30d"], u["_impr_30d"], global_ctr, 20)`: a new user gets exactly `global_ctr`, and $s = 20$ acts like twenty imaginary impressions at the average rate.

After the join, `user_days_since_active` for users without events is filled with `NEVER_DAYS = 999`. This is a better sentinel than zero, because zero would mean "active just now", the opposite of the truth. Features for items follow the same philosophy in `item_features`: counts become zero, rates are smoothed towards the catalogue average, and `item_age_days` is computed directly from `listed_at`, which exists even for an item nobody has seen.

A product page with no co-views

The product page's "Similar items" strip faces a version of the problem for every freshly listed product:

recommender/recsys/pipeline.py · similar_items

```
def similar_items(tables: dict[str, pd.DataFrame], now: pd.Timestamp, item_id: int, k: int) ->
pd.DataFrame:
    """Product page "similar items": co-visited items first, then popular items of the same
category."""
    hist = data.History.at(tables, now)
    empty = pd.DataFrame({"item_id": pd.Series(dtype="int64"), "score":
pd.Series(dtype="float64"), "source": pd.Series(dtype="object")})
    if item_id not in set(hist.items["item_id"]):
        return empty
    in_stock = hist.items.loc[hist.items["in_stock"], ["item_id", "category_id"]]
    m = candidates.covisitation_matrix(hist)
    co = m[(m["item_id_a"] == item_id) & (m["item_id_b"] != item_id) &
m["item_id_b"].isin(in_stock["item_id"])]
    co = co.sort_values("w", ascending=False).head(k)
    out = pd.DataFrame({"item_id": co["item_id_b"].to_numpy(), "score": co["w"].to_numpy(),
"source": "covisit"})

    category = hist.items.loc[hist.items["item_id"] == item_id, "category_id"].iloc[0]
    pop = candidates.item_popularity(hist, 7)
    same = in_stock[(in_stock["category_id"] == category) & (in_stock["item_id"] != item_id) &
~in_stock["item_id"].isin(out["item_id"])]
    same = same.assign(score=same["item_id"].map(pop).fillna(0.0)).sort_values("score",
ascending=False, kind="stable")
    fill = pd.DataFrame({"item_id": same["item_id"].to_numpy(), "score": same["score"].to_numpy(),
"source": "same_category"})
    out = pd.concat([out, fill.head(k - len(out))], ignore_index=True) if len(out) < k else out
    return out.astype({"item_id": "int64", "score": "float64"}).reset_index(drop=True) if len(out)
else empty
```

This is a two-rung ladder in about twenty lines. First it asks the co-visitation matrix for items viewed together with `item_id`. A new item has no rows there, so `co` is empty. Then it fills the remaining slots with in-stock items from the *same category*, ordered by seven-day popularity, and labels them `same_category`. Notice `.fillna(0.0)`: items with no popularity at all are still allowed to appear, ranked last. The `source` column is what the UI uses to tell the user (and you) which rung answered.

The safety net: when Python is down

The last rung is not in Python at all. A **fallback** is a spare plan that takes over when the main one fails, like a spare tyre. The Rust API treats the recommender as an optional dependency, so a dead Python service does not mean a dead website. First the SQL fallback:

backend/src/recommender.rs · fallback

```rust
/// Used when the recommender is down: recent best-sellers, padded by rating count.
async fn fallback(
    pool: &PgPool,
    k: i64,
    category_id: Option<i64>,
    exclude_id: Option<i64>,
) -> Result<Vec<RecCard>, sqlx::Error> {
    let mut qb = QueryBuilder::new(format!("SELECT {CARD_COLS} {CARD_FROM} WHERE i.is_active AND
i.stock > 0"));
    if let Some(c) = category_id {
        qb.push(" AND i.category_id = ").push_bind(c);
    }
    if let Some(x) = exclude_id {
        qb.push(" AND i.id <> ").push_bind(x);
    }
    qb.push(
        " ORDER BY (SELECT count(*) FROM events e WHERE e.item_id = i.id AND e.event_type =
'order'
                    AND e.ts > now() - interval '7 days') DESC, i.rating_count DESC, i.id LIMIT
",
    )
    .push_bind(k);
    let cards = qb.build_query_as::<ProductCard>().fetch_all(pool).await?;
    Ok(cards
        .into_iter()
        .enumerate()
        .map(|(i, card)| RecCard {
            card,
            score: 0.0,
            source: "trending".into(),
            rank: i as i32 + 1,
            contributions: vec![],
        })
        .collect())
}
```

This function takes an optional `category_id` (used by "Similar items") and an optional `exclude_id`, and orders active, in-stock items by the number of `order` events in the last seven days, breaking ties by `rating_count`, then by id so that the output is deterministic. Every row gets `source: "trending"`, `score: 0.0` and an empty `contributions` vector, honestly reporting that nothing was personalised. Now the route that decides when to use it:

backend/src/recommender.rs · recommend_route

```
async fn recommendations(
    State(st): State<AppState>,
    MaybeUser(user): MaybeUser,
    Query(q): Query<RecsQuery>,
) -> AppResult<Json<Value>> {
    let k = q.k.unwrap_or(20).clamp(1, 50);
    let req = pb::RecommendRequest {
        user_id: user.unwrap_or(0),
        k: k as i32,
        surface: q.surface.unwrap_or_else(|| "home_feed".into()),
    };
    let mut rec = st.rec.clone();
    let res = tokio::time::timeout(Duration::from_secs(5), rec.recommend(req)).await;
    match res {
        Ok(Ok(resp)) => {
            let r = resp.into_inner();
            let items = hydrate(&st.pool, r.items).await?;
            Ok(Json(json!({
                "items": items,
                "model_version": r.model_version,
                "latency_ms": r.latency_ms,
                "candidates_considered": r.candidates_considered,
                "fallback": false,
            })))
        }
        other => {
            match other {
                Ok(Err(e)) => tracing::warn!("recommender error, using fallback: {e}"),
                _ => tracing::warn!("recommender timed out, using fallback"),
            }
            let items = fallback(&st.pool, k, None, None).await?;
            Ok(Json(json!({
                "items": items,
                "model_version": "fallback",
                "latency_ms": 0,
                "candidates_considered": 0,
                "fallback": true,
            })))
        }
    }
}
```

Three details deserve attention. A **timeout** is a deadline after which we stop waiting. The gRPC call (a fast way for one program to call a function in another) is wrapped in `tokio::time::timeout(Duration::from_secs(5), ...)`, so a *slow* service is treated the same as a *dead* one: a user is never left staring at a spinner longer than five seconds. The `match` has only one success arm; the catch-all `other` covers both the gRPC error and the elapsed timeout, logs a warning (distinguishing an error from a timeout) and falls through to `fallback(&st.pool, k, None, None)`. And the JSON it returns has exactly the same shape as the success path, plus `"fallback": true` and `"model_version": "fallback"`. The frontend does not need a second code path; it only needs to read the flag and show a banner.

The yellow banner on the home page (screenshot `fallback-banner.png`, shown in chapter 13's tour) is all the frontend needs to change.

The position table below exists only because the browser logs impressions *with positions* (`ProductGrid.vue`: `position` is the array index, one impression per item per grid). Without that log you can compute neither a CTR nor a propensity.

Run it

The companion script `book/code/ch12_coldstart_bias.py` computes three things on the same simulated data as every other chapter: CTR by display position over the impression log, ranker quality by user activity bucket, and the share of new items in each method's top 20. Run it from the `book/` directory:

```
uv run python code/ch12_coldstart_bias.py
```

Its stdout is stored in `results/ch12_coldstart_bias.txt`. We read it one section at a time. First, the position table:

TERMINAL OUTPUT

```
position
0               16770    3999  0.2385
1               16770    2908  0.1734
2               16770    2770  0.1652
3               16770    2490  0.1485
4               16770    2289  0.1365
5               16770    2093  0.1248
6               16770    1128  0.0673
7               16770     933  0.0556
8               16770     881  0.0525
9               16770     818  0.0488
10              16770     702  0.0419
11              16770     262  0.0156

position 0 vs position 11: 15.3x higher CTR
```

Then the activity buckets, which compare the popularity baseline `pop_7d` from chapter 3 with the LambdaMART ranker from chapter 10:

TERMINAL OUTPUT

```
=== Metrics by user activity (clicks before the cutoff) ===
                        users  recall@20  order_recall@20  ndcg@20
bucket      method
0-2 clicks  pop_7d         78     0.0795           0.0753   0.0241
            LambdaMART     78     0.1108           0.1774   0.0666
3-10 clicks pop_7d        258     0.0785           0.0938   0.0489
            LambdaMART    258     0.2143           0.3184   0.1395
>10 clicks  pop_7d        611     0.0785           0.1136   0.0499
            LambdaMART    611     0.3177           0.4764   0.2606
  0-2 clicks   LambdaMART / pop_7d ndcg@20 = 2.76x
  3-10 clicks  LambdaMART / pop_7d ndcg@20 = 2.86x
  >10 clicks   LambdaMART / pop_7d ndcg@20 = 5.22x
```

And finally the exposure of new items:

TERMINAL OUTPUT

```
=== Share of NEW items (listed < 7 days before the cutoff) in the top 20 ===
  catalogue (available items)   2.16%
  pop_7d                        0.00%
  LambdaMART                    0.25%
  future positives              2.68%
```

Results

The position table

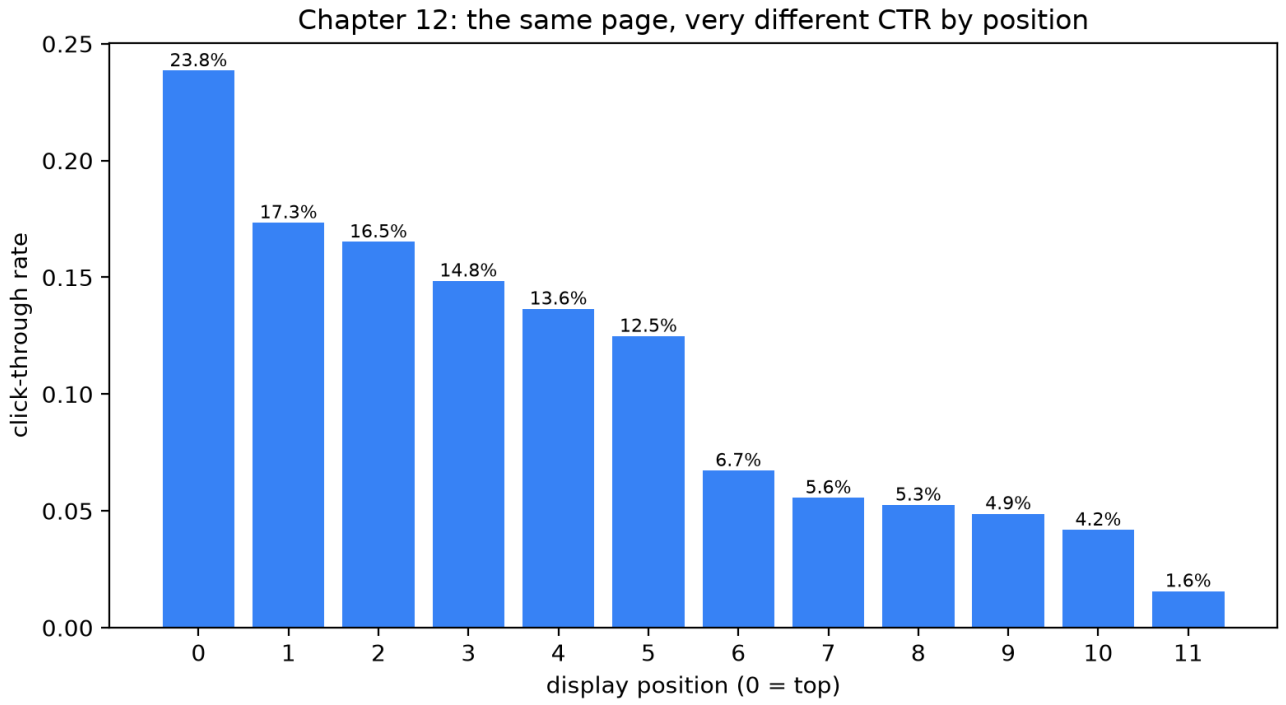


Figure 12.2: *CTR by display position over all impressions before the cutoff. Position 0 converts at 23.8% and position 11 at 1.6%.*

Figure: click-through rate by display position. Every position has exactly the same number of impressions, so the bars are directly comparable.

Each of the twelve positions has exactly 16,770 impressions, which means the denominators are equal and the bar heights are honest rates. Position 0 has a CTR of 0.2385 and position 11 has 0.0156, a ratio of 15.3 times. The decline is not smooth. From position 0 to position 1, CTR drops from 0.2385 to 0.1734; positions 1 to 5 slide gently from 0.1734 to 0.1248; then between position 5 (0.1248) and position 6 (0.0673) it nearly halves in a single step. That cliff is consistent with a page whose first six items are visible without scrolling. Joachims et al. observed a similar break at the fold in their eye-tracking data. Positions 6 to 10 then decline slowly, and position 11 drops sharply again.

Personalisation pays as history grows

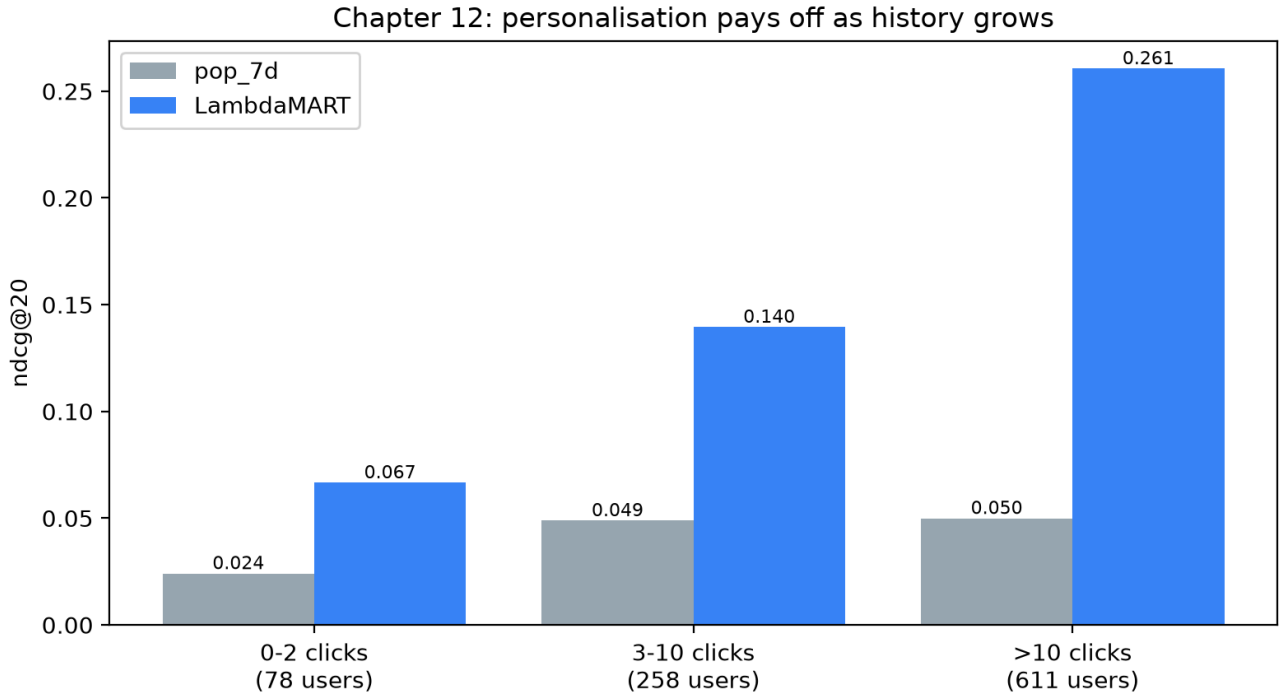


Figure 12.3: *ndcg@20 for pop_7d and LambdaMART across three user activity buckets.*

Figure: ndcg@20 by prior-click bucket. The popularity baseline barely moves with user history; the ranker climbs steeply.

The second table splits the 947 evaluation users (78 + 258 + 611) into *buckets* (groups) by how many clicks they had before the cutoff. A reminder of the two metrics: ndcg@20 scores how high the right items sit in the top 20 (higher is better, 1 is perfect), and recall@20 is the share of the right items that made it into the top 20 at all. Look at the `pop_7d` rows first (this is "recommend whatever was popular in the last 7 days"). Its ndcg@20 is 0.0241 for users with 0–2 clicks, 0.0489 for 3–10 clicks and 0.0499 for more than 10. Popularity ignores the user, so history does not help it; its recall@20 is the same 0.08 in all three buckets (0.0795, 0.0785, 0.0785).

The LambdaMART rows rise steeply: ndcg@20 of 0.0666, 0.1395 and 0.2606. For the heaviest users the ratio over `pop_7d` is 5.22x. For the coldest bucket it is 2.76x. Two points are worth drawing out.

First, **the ranker already beats popularity even for users with at most two clicks**. A plausible reason (we did not isolate it here) is that trending candidates plus item-level features such as smoothed CTR and conversion rate make a better popularity model than a raw seven-day count. The ratio for the 3–10 bucket (2.86x) is similar to that of the cold bucket (2.76x); the large jump happens only above ten clicks, where personalised retrievers (`recent`, `covisit`, `catpop`) and user-item features start to dominate the candidate pool and the ranking.

The new-item exposure table

The third table is the most uncomfortable one. Items listed less than seven days before the cutoff make up 2.16% of the available catalogue. Among the *future positives* (the items users actually went on to interact with during the label week) they make up 2.68%, slightly more than their catalogue share, so

users do want fresh items. What do the methods show them? `pop_7d` puts exactly 0.00% new items into its top 20s. LambdaMART puts 0.25%. Both are an order of magnitude below the 2.68% that users would have welcomed.

This is a feedback loop in action, caught in a single table. The ranker is not malicious: new items have weak behavioural features, so they are scored low, so they are not shown, so they stay weak. A model with a better offline ndcg than a popularity baseline can still be a worse citizen for sellers. It is a reminder that offline metrics reward predicting the past. They do not reward *giving the future a chance*.

Pitfalls and how it's done in production

Treating "new user" as one case. There are at least three: a visitor who has not logged in, a newly registered user with a profile but no behaviour, and a returning user who has been away for months. This repository handles the first with `user_id = 0`, which reaches the service through `Recommend` as an ordinary request and gets trending candidates. The second can use the `city`, `age_bucket` and `device_tier` columns that the registration form already collects. The third is best handled by features like `user_days_since_active` that expose the gap to the model.

Context in the wrong place. Adding "hour of day" to the weekly ranker gives you a feature that is constant within each group; at best it is ignored, at worst it lets the trees fit noise. Put context in the real-time layer, which has per-impression rows and labels. The repository README describes the production version of this design: a second, real-time ranker trained on the `impression` log that can use hour, surface and position, sitting after the long-term-taste stack.

Summary

- Cold start comes in three kinds (new user, new item, new system); the cure for each is a *ladder* of fallbacks that narrows from global to personal as evidence accumulates.
- Here `global_trending` guarantees every user, including `user_id = 0`, at least some candidates, and smoothing plus explicit sentinels (`NEVER_DAYS`) keep the features meaningful for users with no events.
- If the Python service is slow or down, the Rust API returns a SQL best-seller list with `fallback: true` after a five-second timeout, with the same JSON shape as a normal response.
- Position bias is visible in the impression log: CTR falls from 0.2385 at position 0 to 0.0156 at position 11, but raw CTR by position mixes examination with relevance and is not a propensity curve.
- Inverse propensity weighting divides each click by the probability that its position was examined; it is unbiased but high-variance, so weights are clipped and propensities are estimated with randomisation.
- The ranker beats the popularity baseline in all activity buckets, but the margin grows from 2.76x (0–2 clicks) to 5.22x (more than 10 clicks); cold cohorts are small, so their metrics are noisy.
- New items are 2.16% of the catalogue and 2.68% of future positives but only 0.25% of LambdaMART's top 20: an offline-better model can still starve the catalogue, which is a feedback loop.

- Request-time context belongs in a real-time impression-level model, not in a weekly "next 7 days" ranker.

Quiz

Q12.1 Which statement about `candidates.global_trending` is correct?

A. It returns items personalised from the user's most recent session.
B. It does not depend on the user, so it guarantees every user, including `user_id = 0`, some candidates.
C. It is only called when the Python service is down.
D. It ranks items by their price discount.

Q12.2 A brand-new item has 0 impressions and 0 clicks. The smoothed CTR in `features._smoothed` is computed as $(x + p_0 s)/(n + s)$. What does the model see for this item's CTR?

A. 0, because no one clicked it.
B. Undefined, which crashes the builder.
C. The prior p_0, the catalogue-average CTR.
D. 1, because new items are boosted.

Q12.3 Why is "hour of day" *not* a feature in this book's LightGBM ranker?

A. Hour of day has no predictive value in any e-commerce system.
B. LightGBM cannot handle numeric features that wrap around at 24.
C. The ranker predicts a 7-day outcome per user, so the request hour is the same for all of a user's candidates and has no per-request label to learn from.
D. The `events` table does not store timestamps.

Q12.4 In your own words, explain how a popularity feedback loop forms in a recommender trained on clicks, and name two techniques that reduce its effect.

Q12.5 The Rust route `recommendations` falls back to SQL after a 5-second timeout *and* when the gRPC call returns an error. Why is it sensible to treat a slow response the same way as a failed one? What two fields in the JSON tell the frontend that the fallback was used?

Q12.6 Read the included `fallback` function. Item P has 0 `order` events in the last seven days and `rating_count = 500`. Item Q has 1 `order` event in the last seven days and `rating_count = 5`. Both are active and in stock. Which ranks higher, and why does the query end with `i.id`?

Q12.7 Toy propensities are $\theta_0 = 0.8$ and $\theta_3 = 0.2$. Item A was shown 200 times at position 0 and received 40 clicks. Item B was shown 200 times at position 3 and received 20 clicks. Compute the raw CTR and the IPW-debiased relevance estimate for each item, and say which item is more relevant.

Q12.8 *Hands-on.* Open `book/code/ch12_coldstart_bias.py` and change `BUCKETS` so that it also contains a zero-click bucket, for example `("0 clicks", 0, 0)` placed first (and set the next bucket to start at 1). Re-run the script. What do you expect to observe about the number of users in the new

bucket, about the ratio `LambdaMART / pop_7d`, and about how much you trust that ratio? Report what you actually see.

Q12.9 Why do practitioners *clip* inverse propensity weights, for example at $w = \min(1/\theta_k, 20)$?

A. Because propensities can be negative.
B. To reduce variance: a click at a very low-propensity position would otherwise count for dozens of clicks and dominate training, at the cost of a small bias.
C. To make the weights sum to one.
D. Because LightGBM rejects weights above 20.

Chapter 13 — Building the system

It is a Tuesday evening, a few minutes after payday, and Dewi opens the marketplace app on her phone. She taps a cat-food listing, scrolls through the photos, decides the price is a little high, and presses back. She does not buy anything. She does not "like" anything. She did not even type a word. And yet, if she returns to the home page thirty seconds later, the feed should already look a little different: more pet supplies, fewer phone cases. The page she just left has to change what the system believes about her.

What you'll learn

- How the four processes (Vue, Rust/Axum, PostgreSQL, Python gRPC) divide responsibilities, and why the recommender only *reads* the database.
- The life of one click, from an impression to a changed recommendation, with the code at every hop.
- How impressions, clicks and dwell are captured, and why the order of two `pagehide` listeners decides whether a click is recorded.
- How sessions are derived, how the recommender refreshes an in-memory copy of the database, and how hydration, fallback and retraining work.
- How one `.proto` file generates both sides, and what the end-to-end latency is (we measure and profile it).

The intuition: a restaurant with a ledger

Think of a busy restaurant. Waiters take orders, run plates, and write every table's activity into a ledger: who sat down, what they looked at on the menu, what they ordered. The kitchen has a chef who is very good at suggesting the right dish, but the chef never talks to customers and never writes in the ledger. Whenever a waiter asks "what should I suggest to table twelve?", the chef glances at the ledger, thinks, and answers with a short list. If the chef is out sick, the waiter still has a printed card with the five best-selling dishes of the week and hands that to the table instead. Nobody leaves hungry.

That is exactly this architecture. The **ledger** is PostgreSQL. The **waiters** are the Rust API, which is the only writer: it registers users, takes cart and checkout actions, and appends the behavioural events. The **chef** is the Python recommender service, which reads the ledger through a read-only connection and returns ranked suggestions over gRPC. The **menu** that the customer sees is the Vue single-page application. The **printed card** is the fallback query inside the Rust API.

The division is deliberate, for three reasons.

1. **One writer means one set of rules.** Stock never goes negative, an order only moves `paid → shipped → completed`, and a client cannot forge an `order` event, all in one place.
2. **The recommender can be wrong or down without taking the shop down.** It sits behind a network boundary with a timeout and holds no write-capable connection: in `recsys/db.py`, `connect(read_only=True)` sets `default_transaction_read_only=on`, so an accidental `INSERT` fails loudly with `ReadOnlySqlTransaction`.

3. **Each side uses the right tool.** Rust for a concurrent, type-checked API with transactions; Python for pandas, LightGBM and the research code.

What "read-only connection" means. A database connection is a phone line from a program to the database. A *read-only* line can ask questions ("what did shopper Dewi click?") but cannot change anything. It is like letting the chef look at the ledger through a glass window: they can read every entry, but they cannot pick up a pen. Without it, one bug in the recommender, say a loop that updates the wrong table, could silently corrupt orders and stock. With it, the same bug just produces an error message.

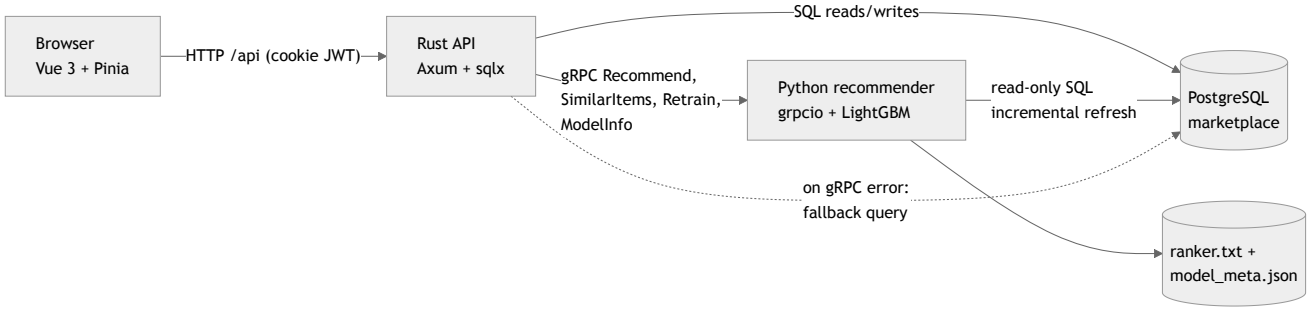


Figure 13.1: *The four processes. Rust is the only writer; Python reads Postgres through a read-only connection and is called over gRPC.*

Words you will meet in this chapter

Several technical terms appear in the next pages. Here is each one in plain words, tied to the restaurant. You can come back to this table whenever a term feels unfamiliar; each one is explained again, with an example, where it first matters.

Term	Plain meaning	In the restaurant
REST API	A set of web addresses (URLs) that a program can call to ask for something or to send something, for example `GET /api/recommendations` ("give me the feed") or `POST /api/events` ("here are some events"). The reply is usually JSON, a plain-text format for data.	The waiter's order window: the customer asks in a fixed way and gets an answer in a fixed way.
JWT cookie	After you log in, the server hands the browser a small signed note saying "this is Dewi" (a *JSON Web Token*, JWT). The browser stores it in a *cookie* and attaches it to every later request, so Dewi need not log in again for each click. `httpOnly` means page scripts cannot read the note, which protects it from theft.	A wristband given at the door: staff check it instead of asking for ID each time.
gRPC	A way for one program to call a function that lives in *another* program, possibly in another language, as if it were a local function.	The waiter calling through the kitchen hatch: "suggest something for table 12!"
Protobuf / `.proto`	*Protocol Buffers*: a short file that lists exactly which fields each request and reply contains. Both programs generate their code from this one file, so they cannot disagree.	A printed order form that waiter and chef both use.
Read-only connection	A database connection that can read but never write.	The glass window onto the ledger.
Beacon	A browser feature (`navigator.sendBeacon`) that hands a small message to the browser to deliver *after* the page has closed.	Dropping a letter into a postbox on your way out, instead of waiting at the counter for a reply.
Latency, p50, p99	Latency is how long one request takes. p50 is the typical (median) time; p99 is the time that 99 of every 100 requests beat.	How long a table waits for a suggestion; p99 is the worst waits, not the average ones.
Profiling	Timing each stage of a slow job to find where the time goes.	Timing each step of a dish to see which step makes the table wait.
Retrain	Building the model again from the newest data.	The chef revising the recipe notes after a busy week.
Fallback	A simpler plan B used when the main system fails.	The printed card of best-sellers.
Cutoff = now	In earlier chapters the *cutoff* was a past date we pretended was "today" to test fairly. When serving real shoppers, "today" is real, so the cutoff is simply the current time.	Nothing is hidden, because the future has not happened yet.

The algorithm: the life of one click

The "algorithm" of this chapter is a protocol: the sequence of steps that turns a user action into a changed ranking. Let us write it down as a sequence diagram first and then walk it step by step.

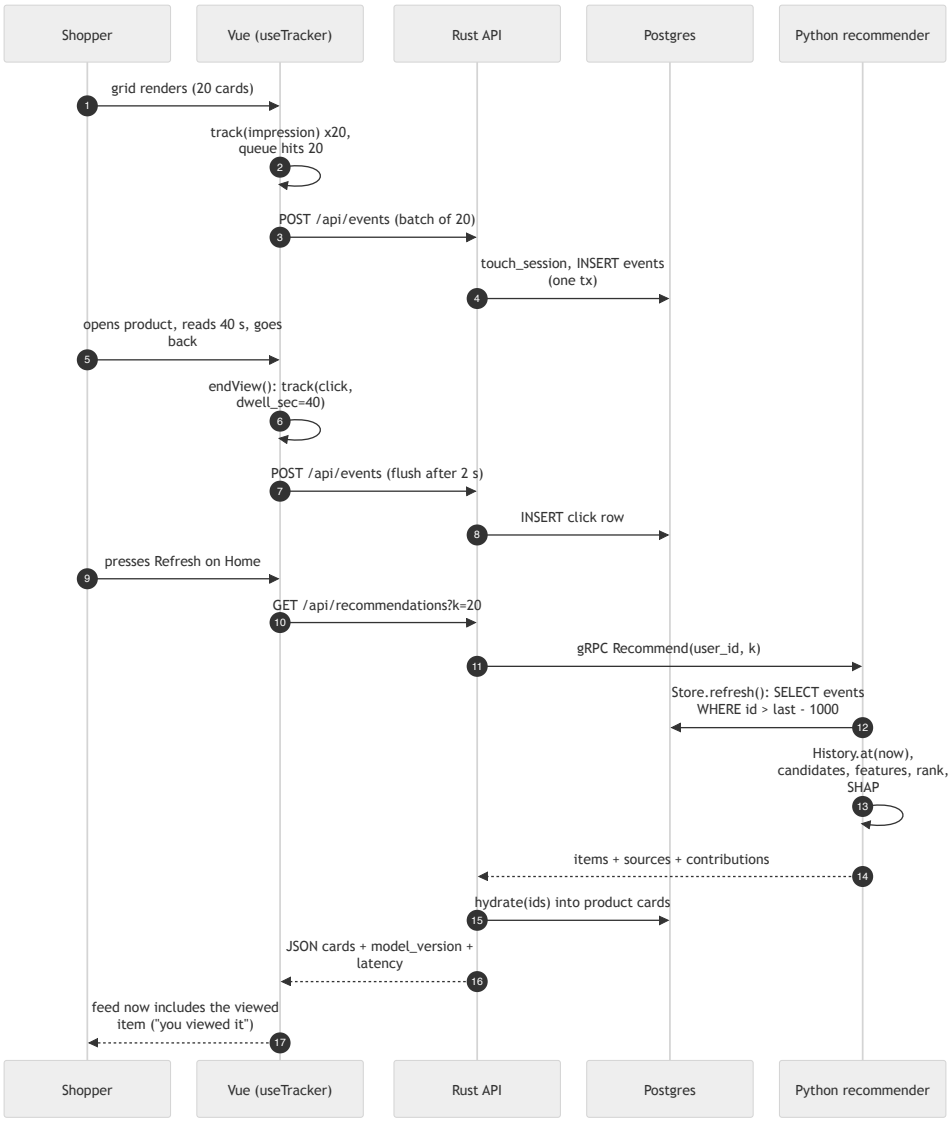


Figure 13.2: *One click, end to end. The first half records impressions and the click with its dwell time; the second half reads them back into a new feed.*

The numbered steps:

1. **Render and observe.** `ProductGrid.vue` calls `track` with an `impression` once per item per grid, with its index as `position` and the surface name as context.
2. **Queue and flush.** `useTracker.ts` flushes at 20 queued events or 2 seconds after the first, with one `POST /api/events`.
3. **Validate and persist.** `post_events` accepts only `impression`, `click` and `hide`, at most 100 per batch, and inserts them in one transaction with the user's current `session_id`.
4. **Dwell becomes a click.** When the shopper leaves a product page, `Product.vue` emits one `click` carrying `dwell_sec`, the strongest implicit signal: `ui_max_dwell_sec` is the top feature by gain on the model page.

5. **Ask.** The home page calls `GET /api/recommendations`; Rust calls the Python service with a 5-second timeout.
6. **Refresh.** `Recommend` first calls `Store.refresh()`, which reads only events newer than what it has seen.
7. **Run Chapters 8-11.** `History.at(now)`, candidates, `features.build`, LightGBM scoring, `rerank.rerank`, SHAP for the survivors.
8. **Hydrate and respond.** gRPC returns ids, scores, sources and contributions; Rust fetches the cards in one query.

A worked example with hand-sized numbers

How soon after a click does the ranking change? Let T_{flush} be the client flush delay, T_{refresh} the recommender's refresh throttle and L the time of the recommendation request itself. In the worst case the new row arrives just after the recommender refreshed, so

$$T_{\text{stale}} \le T_{\text{flush}} + T_{\text{refresh}} + L.$$

In this code base, `FLUSH_MS = 2000` in `useTracker.ts` and `MIN_REFRESH_SEC = 1.0` in `db.py`. Add the measured median request time later in this chapter, roughly 0.84 s, and the worst case is about $2 + 1 + 0.84 = 3.84$ seconds between "the shopper pressed back" and "the shopper sees a feed that knows about it". For a typical shopper who presses Refresh a few seconds after returning, the new item is already in the feed. That is the "recommendations are computed on request" decision from the project plan paying off: there is no nightly batch between behaviour and effect.

> **Did you know?**
>
> **Why this loop cannot be left to a notebook.** In "Hidden Technical Debt in Machine Learning Systems", Sculley et al. observe that only a small fraction of a real-world ML system is the ML code itself; the surrounding infrastructure for data collection, serving and monitoring is "vast and complex". In this repository the same proportion is visible: the ranking model lives in a handful of Python files, while the collection, storage and delivery of its inputs and outputs spans a frontend, an API, a schema and an RPC contract.
>
> Source: Sculley et al., NeurIPS, 2015

Code walkthrough, part 1: capturing behaviour in the browser

The tracker

`track()` first checks `if (!useAuth().user) return`: anonymous visitors generate no events (the server also answers `204` without storing), so new visitors only get trending items, the cold-start path of Chapter 12. Then comes the batching policy: send at `FLUSH_AT = 20`, otherwise start a 2-second timer. `flush()` swallows network errors on purpose: tracking is best-effort. The bottom of the region handles leaving the page, where the browser may cancel in-flight `fetch` calls; `navigator.sendBeacon` queues a small POST that the browser delivers after the page is gone. The `pageHideHooks` set carries a bug story, below.

What is a beacon, and why is it needed? When Dewi closes the tab, the browser tears the page down. A normal network request that is still on its way may be cancelled, like a customer leaving be-

fore the waiter has finished writing the order. `navigator.sendBeacon` fixes this by handing a small message to the browser itself, which promises to deliver it even though the page is gone. The cost is that you cannot wait for a reply or see an error, which is fine for "the shopper clicked this."

frontend/src/composables/useTracker.ts · tracker

```ts
export interface TrackEvent {
  item_id: number
  event_type: 'impression' | 'click' | 'hide'
  position?: number
  surface?: string
  dwell_sec?: number
}

const FLUSH_MS = 2000
const FLUSH_AT = 20
let queue: TrackEvent[] = []
let timer: number | null = null

function take(): TrackEvent[] {
  const batch = queue
  queue = []
  if (timer !== null) {
    clearTimeout(timer)
    timer = null
  }
  return batch
}

async function flush() {
  const events = take()
  if (!events.length) return
  try {
    await fetch('/api/events', {
      method: 'POST',
      credentials: 'same-origin',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ events }),
    })
  } catch {
    /* tracking is best-effort */
  }
}

export function track(e: TrackEvent) {
  if (!useAuth().user) return
  queue.push(e)
  if (queue.length >= FLUSH_AT) void flush()
  else if (timer === null) timer = window.setTimeout(() => void flush(), FLUSH_MS)
}

// Views that still need to record something when the tab closes (e.g. Product.vue's
// click + dwell) register here, so their events are queued before the beacon is sent.
const pageHideHooks = new Set<() => void>()

export function onPageHide(fn: () => void): () => void {
  pageHideHooks.add(fn)
  return () => pageHideHooks.delete(fn)
}

window.addEventListener('pagehide', () => {
  for (const fn of pageHideHooks) fn()
  const events = take()
  if (!events.length) return
  navigator.sendBeacon('/api/events', new Blob([JSON.stringify({ events })], { type:
'application/json' }))
})

export function useTracker() {
  return { track, flush }
}
```

Impressions

An impression is "this item was rendered for this user at this position on this surface". It is the denominator of every click-through rate in Chapter 9 (`item_impressions_7d`, `user_ctr_30d`) and the raw material for the position-bias analysis of Chapter 12. Without logged impressions you can only ever say what people clicked, never what they ignored.

frontend/src/components/ProductGrid.vue · impressions

```
const seen = new Set<number>() // one impression per item id per grid instance
const idsKey = computed(() => props.items.map((i) => i.id).join(','))
watch(
  idsKey,
  () => {
    props.items.forEach((item, position) => {
      if (seen.has(item.id)) return
      seen.add(item.id)
      track({ item_id: item.id, event_type: 'impression', position, surface: props.surface })
    })
  },
  { immediate: true },
)
```

The `seen` set enforces one impression per item id per grid instance; without it a re-render or an overlapping "Refresh" would inflate the denominator of every CTR. The watcher is keyed on the joined id list because Vue creates a new array on every fetch. And `position` is the render index, which Chapter 12 needs for position bias. Caveat: an item counts as seen when *rendered*, not when scrolled into view; production uses an `IntersectionObserver`.

Dwell time and the `pagehide` ordering bug

Every click event carries `dwell_sec`, and it is created in the product page. The idea is simple: record `performance.now()` when the product loads, and when the shopper leaves, emit one `click` with the elapsed seconds, the position they clicked from and the surface they came from. Those last two travel in the URL (`/products/:id?src=search&pos=3`), which is how a product page knows how it was reached.

frontend/src/views/Product.vue · dwell

```
let start = performance.now()
let viewItemId: number | null = null
let viewSurface = 'direct'
let viewPos = 0
let clickSent = true // guard: exactly one click per product view

function beginView() {
  start = performance.now()
  viewItemId = Number(route.params.id)
  viewSurface = String(route.query.src ?? 'direct')
  viewPos = Number(route.query.pos ?? 0)
  clickSent = false
}
function endView() {
  if (clickSent || viewItemId === null) return
  clickSent = true
  track({
    item_id: viewItemId,
    event_type: 'click',
    position: viewPos,
    surface: viewSurface,
    dwell_sec: Math.round(((performance.now() - start) / 1000) * 10) / 10,
  })
}
onBeforeRouteLeave(endView)
onBeforeRouteUpdate((to, from) => {
  if (to.params.id !== from.params.id) endView()
})
const stopPageHide = onPageHide(endView) // tab closed / full reload: queued before the beacon
flush
onUnmounted(() => {
  stopPageHide()
  endView()
})
```

`beginView()` and `endView()` bracket a "view". The guard `clickSent` guarantees exactly one click per product view, even though `endView` is wired to four different exits:

- `onBeforeRouteLeave`: the shopper navigates to another page of the app.
- `onBeforeRouteUpdate`: the shopper goes from one product to another, where Vue Router *reuses* the same component instance. Without this hook the first product's dwell would never be recorded.
- `onPageHide(endView)`: the tab is closed or the page is reloaded.
- `onUnmounted`: a final safety net.

The first version of the product page registered its own `pagehide` listener to record the click. The tracker's listener, which drains the queue into `sendBeacon`, was registered first and therefore ran first: it emptied the queue and sent the beacon, and only then did the product page queue its click. The click missed the last flush and was silently lost. The symptom was subtle: clicks were recorded when navigating inside the app but never when closing the tab.

The fix is the hook registry in the tracker region: a view calls `onPageHide(fn)`, and the tracker's single `pagehide` listener runs every hook (which queue their events) and *then* sends the beacon. The general lesson for any flush-on-exit design: give the flusher one exit listener and let producers register with it.

Did you know?

`pagehide` is not a guarantee. MDN's documentation for `navigator.sendBeacon()` explains that the legacy exit events (`unload`, `beforeunload` and `pagehide`) are unreliable, especially on mobile: if a user switches apps and later kills the browser, none of them fire. MDN recommends sending analytics on the `visibilitychange` event instead, and notes that the beacon payload is limited to about 64 KiB. Our tracker uses `pagehide`, which is simple and good enough to demonstrate the design; a production tracker would also flush when `document.visibilityState` becomes `hidden`.

Source: Mozilla Contributors, MDN Web Docs, 2026

Code walkthrough, part 2: the server writes it down

The events table

Everything the recommender knows about behaviour comes from one append-only table, `events` in `backend/migrations/0001_init.sql`: `ts`, `user_id`, `session_id`, `item_id`, `event_type`, `position`, `dwell_sec`, `price_paid`, `surface`.

backend/migrations/0001_init.sql · events_table

```
-- One row per thing a user did to an item. Append-only: the recommender's
-- whole view of user behaviour comes from this table.
CREATE TABLE events (
    id         BIGSERIAL PRIMARY KEY,
    ts         TIMESTAMPTZ NOT NULL DEFAULT now(),
    user_id    BIGINT NOT NULL REFERENCES users,
    session_id BIGINT NOT NULL REFERENCES sessions,
    item_id    BIGINT NOT NULL REFERENCES items,
    event_type TEXT NOT NULL CHECK (event_type IN
('impression','click','add_to_cart','order','hide','return')),
    position   INT NULL,
    dwell_sec  DOUBLE PRECISION NULL,
    price_paid DOUBLE PRECISION NULL,
    surface    TEXT NULL
);
CREATE INDEX events_user_ts_idx ON events (user_id, ts DESC);
CREATE INDEX events_item_id_idx ON events (item_id);
```

Its schema is the contract between "what the web app can observe" and "what the model can learn from".

Compared with the simulator schema of Chapter 1, real data adds `surface` and foreign keys that stop a client from writing events about nonexistent items. The `CHECK` on `event_type` is the last line of defence, and two indexes serve per-user history and per-item aggregates. There is no `UPDATE` anywhere: the log is append-only, and a return is a new `return` event.

From batch to rows: `post_events`

backend/src/events.rs · post_events

```rust
async fn post_events(
    State(st): State<AppState>,
    MaybeUser(user): MaybeUser,
    Json(b): Json<EventBatch>,
) -> AppResult<StatusCode> {
    let Some(uid) = user else { return Ok(StatusCode::NO_CONTENT) };
    if b.events.len() > 100 {
        return Err(AppError::BadRequest("too many events (max 100)".into()));
    }
    if b.events.iter().any(|e| !matches!(e.event_type.as_str(), "impression" | "click" | "hide"))
{
        return Err(AppError::BadRequest("event_type not allowed from client".into()));
    }
    let Some(first) = b.events.first() else { return Ok(StatusCode::NO_CONTENT) };
    let mut tx = st.pool.begin().await?;
    let sid = touch_session(&mut tx, uid, first.surface.as_deref().unwrap_or("direct")).await?;
    for e in &b.events {
        let r = insert_event(
            &mut tx,
            uid,
            sid,
            e.item_id,
            &e.event_type,
            e.position,
            e.dwell_sec,
            None,
            e.surface.as_deref(),
        )
        .await;
        match r {
            Ok(()) => {}
            Err(sqlx::Error::Database(d)) if d.is_foreign_key_violation() => {
                return Err(AppError::BadRequest("unknown item".into()));
            }
            Err(e) => return Err(e.into()),
        }
    }
    tx.commit().await?;
    Ok(StatusCode::NO_CONTENT)
}
```

`MaybeUser` never fails, so anonymous requests get `204`, which is what a beacon expects. The `matches!` check enforces what a browser may claim: impressions, clicks and hides are observations of the shopper's own screen. `add_to_cart` and `order` are consequences of authenticated server-side actions, so the API creates them itself; if a shopper could post `order` events, they could train the recommender to love any item.

One `touch_session` call resolves the session for the whole batch; a foreign-key violation becomes `400 unknown item`; and `tx.commit()` makes the batch visible atomically, so the recommender never sees half of one.

Sessions: `touch_session`

A session is a run of activity separated from the next by a quiet gap. The simulator's `sessions` table had that structure built in; the real app has to *derive* it.

backend/src/sessions.rs · touch_session

```rust
/// Returns the id of the user's current session, creating one if the latest
/// session has been idle for more than 30 minutes.
pub async fn touch_session(conn: &mut PgConnection, user_id: i64, surface: &str) -> Result<i64,
sqlx::Error> {
    let existing: Option<(i64,)> = sqlx::query_as(
        "UPDATE sessions SET last_seen_at = now() WHERE id = (
             SELECT id FROM sessions
             WHERE user_id = $1 AND last_seen_at > now() - interval '30 minutes'
             ORDER BY last_seen_at DESC LIMIT 1)
         RETURNING id",
    )
    .bind(user_id)
    .fetch_optional(&mut *conn)
    .await?;
    if let Some((id,)) = existing {
        return Ok(id);
    }
    let (id,): (i64,) = sqlx::query_as(
        "INSERT INTO sessions (user_id, surface, device_os) VALUES ($1, $2, 'web') RETURNING id",
    )
    .bind(user_id)
    .bind(surface)
    .fetch_one(&mut *conn)
    .await?;
    Ok(id)
}
```

One `UPDATE ... WHERE id = (SELECT ... LIMIT 1) RETURNING id` finds the latest session seen within 30 minutes, bumps `last_seen_at` and returns its id; otherwise a new session is inserted. Two concurrent requests could in theory both insert one; we accept that, since it only slightly dilutes co-visitation. Every server-side event (client batches, `add_to_cart`, `order`) goes through this function, because the co-visitation matrix depends on a valid `session_id`.

Checkout: the strongest signal

This is the best example of why a single writer pays off. In one transaction the handler loads the cart, locks the item rows with `SELECT ... FOR UPDATE ORDER BY id` (a deterministic order prevents deadlocks), rejects inactive or short-stock lines with `409`, groups lines by shop into one order per seller, stores `unit_price` at the final discounted price, decrements stock and stamps `out_of_stock_at` at zero, writes one `order` event per line through `touch_session`, and empties the cart. Two recommender-relevant consequences: the next `Recommend` stops proposing a sold-out product because `History.at` reads `out_of_stock_at`, and every `order` event corresponds to a real, paid order row.

backend/src/cart.rs · checkout

```
async fn checkout(
    State(st): State<AppState>,
    u: AuthUser,
    Json(b): Json<CheckoutBody>,
) -> AppResult<(StatusCode, Json<serde_json::Value>)> {
    let address = b.shipping_address.trim();
    if address.is_empty() {
        return Err(AppError::BadRequest("shipping address is required".into()));
    }
    let mut tx = st.pool.begin().await?;
    let cart: Vec<(i64, i32)> = sqlx::query_as("SELECT item_id, quantity FROM cart_items WHERE
user_id = $1")
        .bind(u.id)
        .fetch_all(&mut *tx)
        .await?;
    if cart.is_empty() {
        return Err(AppError::BadRequest("cart is empty".into()));
    }
    let ids: Vec<i64> = cart.iter().map(|(i, _)| *i).collect();
    // Row locks serialise concurrent checkouts of the same items.
    let locked: HashMap<i64, LockedItem> = sqlx::query_as::<_, LockedItem>(
        "SELECT id, shop_id, title, ROUND(price * (1 - discount_pct))::bigint AS final_price,
                free_shipping, stock, is_active
         FROM items WHERE id = ANY($1) ORDER BY id FOR UPDATE",
    )
    .bind(&ids)
    .fetch_all(&mut *tx)
    .await?
    .into_iter()
    .map(|i| (i.id, i))
    .collect();

    let mut by_shop: BTreeMap<i64, Vec<(&LockedItem, i32)>> = BTreeMap::new();
    for (item_id, qty) in &cart {
        let it = locked.get(item_id).ok_or(AppError::NotFound)?;
        if !it.is_active || *qty > it.stock {
            return Err(AppError::Conflict(format!("insufficient stock for {}", it.title)));
        }
        by_shop.entry(it.shop_id).or_default().push((it, *qty));
    }

    let sid = touch_session(&mut tx, u.id, "checkout").await?;
    let mut placed = Vec::new();
    for (shop_id, lines) in &by_shop {
        let subtotal: i64 = lines.iter().map(|(it, q)| it.final_price * *q as i64).sum();
        let shipping_fee = shipping_total(lines.iter().map(|(it, _)| (it.shop_id,
it.free_shipping)));
        let total = subtotal + shipping_fee;
        let (order_id,): (i64,) = sqlx::query_as(
            "INSERT INTO orders (buyer_id, shop_id, status, shipping_address, subtotal,
shipping_fee, total)
             VALUES ($1, $2, 'paid', $3, $4, $5, $6) RETURNING id",
        )
        .bind(u.id)
        .bind(shop_id)
        .bind(address)
        .bind(subtotal)
        .bind(shipping_fee)
        .bind(total)
        .fetch_one(&mut *tx)
        .await?;
        for (it, qty) in lines {
            sqlx::query("INSERT INTO order_items (order_id, item_id, quantity, unit_price) VALUES
($1, $2, $3, $4)")
                .bind(order_id)
                .bind(it.id)
                .bind(qty)
                .bind(it.final_price)
                .execute(&mut *tx)
```

```
                .await?;
            sqlx::query(
                "UPDATE items SET stock = stock - $2,
                        out_of_stock_at = CASE WHEN stock - $2 = 0 THEN now() ELSE out_of_stock_at
END
                 WHERE id = $1",
            )
            .bind(it.id)
            .bind(qty)
            .execute(&mut *tx)
            .await?;
            insert_event(
                &mut tx,
                u.id,
                sid,
                it.id,
                "order",
                None,
                None,
                Some((it.final_price * *qty as i64) as f64),
                Some("checkout"),
            )
            .await?;
        }
        placed.push(PlacedOrder { id: order_id, shop_id: *shop_id, total });
    }
    sqlx::query("DELETE FROM cart_items WHERE user_id = $1").bind(u.id).execute(&mut *tx).await?;
    tx.commit().await?;
    Ok((StatusCode::CREATED, Json(serde_json::json!({ "orders": placed }))))
}
```

Code walkthrough, part 3: the recommender reads it back

Keeping an in-memory copy fresh: `Store`

The recommender needs the entire event history as DataFrames because the pipeline from earlier chapters (`History.at`, `candidates.generate`, `features.build`) was written against pandas tables. Re-reading 277,000 rows from Postgres for every request would be slow. So `Store` keeps the tables in memory and refreshes them incrementally.

recommender/recsys/db.py · Store.refresh

```
def refresh(self) -> None:
    with self._lock:
        if self._tables and time.monotonic() - self._last_refresh < MIN_REFRESH_SEC:
            return
        try:
            self._refresh()
        except psycopg.OperationalError:
            self._conn = None  # reconnect once (e.g. Postgres restarted)
            self._refresh()
        self._last_refresh = time.monotonic()
```

`refresh()` does three things. It takes a lock, so concurrent gRPC threads do not issue overlapping refreshes. It applies a throttle: if the last refresh was less than `MIN_REFRESH_SEC` (one second) ago, it returns immediately, which caps database load at roughly one refresh per second no matter how many requests arrive. And it retries once on `OperationalError` after dropping the connection, which is what you want after a Postgres restart.

The four small tables (categories, shops, items, users) are reloaded in full because they change in place. Events and searches are fetched with `WHERE id > last_id - 1000`. The overlap matters because a `BIGSERIAL` id is assigned at *insert* but the row becomes visible at *commit*: if transaction Y commits event 501 before X commits event 500, a naive `id > 501` query would never fetch 500. Re-reading the last 1000 ids and de-duplicating in `_merge` (which also sorts by `ts`, as `History.at` assumes) tolerates out-of-order commits. All reads use the read-only connection.

The `Recommend` RPC

recommender/recsys/service.py · recommend_rpc

```
def Recommend(self, request, context):
    t = time.perf_counter()
    self.store.refresh()  # pick up events logged since the last request
    k = min(max(request.k, 1), 50)
    model, version = self.model, self.meta["version"]
    recs, n_candidates = pipeline.recommend(self.store.snapshot(), _now(), model, request.user_id,
k)
    sources = pipeline.source_of(recs) if len(recs) else []
    contribs = ranker.contributions(model, recs) if len(recs) else []
    items = [
        pb.RecommendedItem(
            item_id=int(item_id), score=float(score), source=source, rank=rank,
            contributions=[pb.Contribution(feature=f, value=v, contribution=c) for f, v, c in
contrib],
        )  # fmt: skip
        for rank, (item_id, score, source, contrib) in enumerate(
            zip(recs["item_id"], recs["score"], sources, contribs), start=1
        )
    ]
    latency_ms = (time.perf_counter() - t) * 1000
    log.info("recommend user=%d k=%d candidates=%d latency_ms=%.0f", request.user_id, k,
n_candidates, latency_ms)
    return pb.RecommendResponse(
        items=items, model_version=version, candidates_considered=n_candidates,
latency_ms=latency_ms
    )
```

The method refreshes the store, clamps `k` to 1..50, and takes references to the current model and version *before* any work, which makes hot-swapping safe. It then calls `pipeline.recommend`, the same function the offline CLI uses: offline evaluation and online serving share one code path, the best defence against training/serving skew. `source_of` labels each row with its strongest retriever (or `trending`), `ranker.contributions` adds the top five SHAP values, and one INFO log line records user, `k`, candidates and latency. Anonymous users (`user_id = 0`) need no special branch: no history means trending candidates and NaN user features, which LightGBM handles.

Retraining without downtime

Retraining means **building the ranking model again from the newest data**. A model is a snapshot of the patterns found in the data it was trained on. As new items are listed and tastes change, the snapshot goes stale, like a recipe book that never mentions this season's dishes. Without retraining, recommendations slowly get worse. The difficulty is doing it while customers keep ordering, which the next paragraphs solve.

The ranker's inputs drift as items appear and tastes shift, so the service retrains on two triggers: a background thread every `RECSYS_RETRAIN_INTERVAL_SEC` (six hours by default) and an admin-only gRPC `Retrain` call behind a button on the "How it works" page.

recommender/recsys/service.py · retrain

```
def retrain(store: Store, model_dir: Path = MODEL_DIR) -> tuple[lgb.Booster, dict, dict]:
    """Train + evaluate on the current DB snapshot, save ranker.txt + model_meta.json.

    Returns (model, meta, report) where meta is what's written to model_meta.json.
    """
    trained_at = datetime.now(timezone.utc).replace(microsecond=0)
    model, report = pipeline.train_and_evaluate(store.snapshot(), end=_now())
    metrics = {
        pipeline.metric_key(method, metric): float(v)
        for method, row in report["results"].iterrows()
        for metric, v in row.items()
    }
    metrics["candidate_recall"] = float(report["candidate_recall"])
    meta = {
        "version": trained_at.strftime("%Y%m%d-%H%M%S"),
        "trained_at": trained_at.isoformat().replace("+00:00", "Z"),
        "metrics": metrics,
        "importance": {k: float(v) for k, v in report["importance"].items()},
        "train_rows": int(report["train_rows"]),
    }
    model_dir.mkdir(parents=True, exist_ok=True)
    ranker.save(model, model_dir / "ranker.txt")
    (model_dir / "model_meta.json").write_text(json.dumps(meta, indent=2))
    return model, meta, report
```

The module-level `retrain` is shared by the service and the CLI. It trains on a store snapshot with `end=now`, so label windows slide with the clock, converts the evaluation table into flat keys such as `lightgbm_ranker/ndcg@20`, and writes `ranker.txt` and `model_meta.json`.

The method adds two safeguards. `retrain_lock.acquire(blocking=False)` makes a second concurrent request fail at once (`ABORTED`, which Rust maps to `409`). And the swap `self.model, self.meta = model, meta` is two reference assignments: in-flight `Recommend` calls keep the old booster and finish consistently, new calls see the new one, with no lock on the read path.

Why `end=now`: the cutoff at serving time. In the Preface and Chapter 9 we fixed a past *cutoff* (day 53), hid everything after it, and checked the model against what really happened next. Training still works that way: a model learns by seeing "as of this day, what did people do in the following week?" at many past days. But the retrain job simply uses the *current time* as the latest cutoff, so each retrain slides all of those windows forward. This is also why a retrain gives slightly different metrics even when no code changed. At serving time there is nothing to hide: the future has not happened, so the cutoff is `now`, and `History.at(now)` means "everything known up to this moment."

Code walkthrough, part 4: the Rust side of the bridge

The recommendations route

backend/src/recommender.rs · recommend_route

```rust
async fn recommendations(
    State(st): State<AppState>,
    MaybeUser(user): MaybeUser,
    Query(q): Query<RecsQuery>,
) -> AppResult<Json<Value>> {
    let k = q.k.unwrap_or(20).clamp(1, 50);
    let req = pb::RecommendRequest {
        user_id: user.unwrap_or(0),
        k: k as i32,
        surface: q.surface.unwrap_or_else(|| "home_feed".into()),
    };
    let mut rec = st.rec.clone();
    let res = tokio::time::timeout(Duration::from_secs(5), rec.recommend(req)).await;
    match res {
        Ok(Ok(resp)) => {
            let r = resp.into_inner();
            let items = hydrate(&st.pool, r.items).await?;
            Ok(Json(json!({
                "items": items,
                "model_version": r.model_version,
                "latency_ms": r.latency_ms,
                "candidates_considered": r.candidates_considered,
                "fallback": false,
            })))
        }
        other => {
            match other {
                Ok(Err(e)) => tracing::warn!("recommender error, using fallback: {e}"),
                _ => tracing::warn!("recommender timed out, using fallback"),
            }
            let items = fallback(&st.pool, k, None, None).await?;
            Ok(Json(json!({
                "items": items,
                "model_version": "fallback",
                "latency_ms": 0,
                "candidates_considered": 0,
                "fallback": true,
            })))
        }
    }
}
```

The handler reads the optional user, clamps `k` and builds the request. The `connect_lazy()` channel in `main.rs` lets the API start even when Python is down. The call is wrapped in a 5-second `tokio::time::timeout`; success is hydrated and returned with `"fallback": false`, while any gRPC error or timeout is logged and answered from the fallback query with `"model_version": "fallback"`, which the frontend turns into the yellow banner. Degraded is better than down.

Hydration

gRPC returns ids and scores; the browser needs titles and prices.

backend/src/recommender.rs · hydrate

```rust
/// Turn gRPC items (ids only) into full cards, in the recommender's order.
/// Ids that no longer exist or are inactive are skipped.
async fn hydrate(pool: &PgPool, items: Vec<pb::RecommendedItem>) -> Result<Vec<RecCard>,
sqlx::Error> {
    let ids: Vec<i64> = items.iter().map(|i| i.item_id).collect();
    let mut cards: HashMap<i64, ProductCard> = fetch_cards(pool, &ids).await?.into_iter().map(|c|
(c.id, c)).collect();
    Ok(items
        .into_iter()
        .filter_map(|it| {
            cards.remove(&it.item_id).map(|card| RecCard {
                card,
                score: it.score,
                source: it.source,
                rank: it.rank,
                contributions: it
                    .contributions
                    .into_iter()
                    .map(|c| ContributionJson { feature: c.feature, value: c.value, contribution:
c.contribution })
                    .collect(),
            })
        })
        .collect())
}
```

One SQL query with `id = ANY($1)` builds an id-to-card map, and the code walks the recommender's list *in the recommender's order*, skipping ids that vanished meanwhile. Fewer than `k` cards beats failing the whole feed.

The fallback

backend/src/recommender.rs · fallback

```rust
/// Used when the recommender is down: recent best-sellers, padded by rating count.
async fn fallback(
    pool: &PgPool,
    k: i64,
    category_id: Option<i64>,
    exclude_id: Option<i64>,
) -> Result<Vec<RecCard>, sqlx::Error> {
    let mut qb = QueryBuilder::new(format!("SELECT {CARD_COLS} {CARD_FROM} WHERE i.is_active AND
i.stock > 0"));
    if let Some(c) = category_id {
        qb.push(" AND i.category_id = ").push_bind(c);
    }
    if let Some(x) = exclude_id {
        qb.push(" AND i.id <> ").push_bind(x);
    }
    qb.push(
        " ORDER BY (SELECT count(*) FROM events e WHERE e.item_id = i.id AND e.event_type =
'order'
                    AND e.ts > now() - interval '7 days') DESC, i.rating_count DESC, i.id LIMIT
",
    )
    .push_bind(k);
    let cards = qb.build_query_as::<ProductCard>().fetch_all(pool).await?;
    Ok(cards
        .into_iter()
        .enumerate()
        .map(|(i, card)| RecCard {
            card,
            score: 0.0,
            source: "trending".into(),
            rank: i as i32 + 1,
            contributions: vec![],
        })
        .collect())
}
```

The printed card from our restaurant: active, in-stock items ordered by `order` events in the last seven days, then rating count. It labels every row `source: "trending"`, score 0, no contributions, so the frontend needs no separate path. It is deliberately non-personalised: a degraded experience that is the same for everybody cannot be wrong in a surprising, user-specific way.

The contract between languages: one `.proto`, two code generators

A **schema** is an agreed description of the data. Here, a request is "a user id and a count `k`", and a reply is "a list of items, each with an id, a score and some contributions". **gRPC** is the mechanism that carries these messages between two programs; **Protocol Buffers** (protobuf) is the compact format they are written in. The file with the agreed description, `recommender.proto`, is like a printed order form that both waiter and chef use: both are generated from it, so a request written by one side can always be read by the other. Without a shared form, each side would guess at the other's format, and silent mismatches would appear only in production.

The Rust API and the Python service share only a schema, one short file:

proto/recommender.proto

```
syntax = "proto3";
package recommender.v1;

service Recommender {
  // Personalised home feed. user_id = 0 means anonymous (gets trending-driven results).
  rpc Recommend(RecommendRequest) returns (RecommendResponse);
  // "Similar items" on a product page: co-visitation, then same-category popular.
  rpc SimilarItems(SimilarItemsRequest) returns (RecommendResponse);
  // Retrain the ranker on the current DB contents and hot-swap it.
  rpc Retrain(RetrainRequest) returns (RetrainResponse);
  // Model version, metrics, feature importance + feature descriptions (for the learning page).
  rpc ModelInfo(ModelInfoRequest) returns (ModelInfoResponse);
}

message RecommendRequest { int64 user_id = 1; int32 k = 2; string surface = 3; }
message SimilarItemsRequest { int64 item_id = 1; int32 k = 2; int64 user_id = 3; }

message Contribution {
  string feature = 1;       // schema feature name, e.g. "uc_click_share_30d"
  double value = 2;         // the feature's value for this (user, item)
  double contribution = 3;  // LightGBM SHAP contribution to the score (+ pushes up)
}
message RecommendedItem {
  int64 item_id = 1;
  double score = 2;
  string source = 3;  // "recent" | "covisit" | "category_popular" | "trending" | "same_category"
  int32 rank = 4;     // 1-based
  repeated Contribution contributions = 5;  // top 5 by |contribution|; empty for SimilarItems
}
message RecommendResponse {
  repeated RecommendedItem items = 1;
  string model_version = 2;
  int64 candidates_considered = 3;
  double latency_ms = 4;
}

message RetrainRequest {}
message RetrainResponse { string model_version = 1; map<string, double> metrics = 2; int64
train_rows = 3; }

message ModelInfoRequest {}
message FeatureInfo { string name = 1; string group = 2; string description = 3; double gain_share
= 4; }
message ModelInfoResponse {
  string model_version = 1;
  string trained_at = 2;               // ISO-8601 UTC
  map<string, double> metrics = 3;     // keys "<method>/<metric>", e.g.
"lightgbm_ranker/ndcg@20", plus "candidate_recall"
  repeated FeatureInfo features = 4;   // all schema.FEATURES, sorted by gain_share desc
  int64 events_loaded = 5;
  int64 train_rows = 6;
}
```

The pieces map directly to the product:

- `Recommend` and `SimilarItems` return `RecommendResponse` with `items`, `model_version`, `candidates_considered` and `latency_ms`, the numbers in the grey caption under "Recommended for you".
- `RecommendedItem.contributions` carries the top five SHAP contributions per item: the raw material of "Why this?".
- `Retrain` and `ModelInfo` feed the "How it works" page. IDs, not objects, cross the boundary, and the response carries the model version so any behaviour can be tied to the model that caused it.

> **Did you know?**
>
> **Field numbers are the wire format.** In Protocol Buffers, each field is identified on the wire by its number, not its name. The language guide states that the number cannot be changed once the message type is in use, that field numbers 1 through 15 take one byte to encode while 16 through 2047 take two, and that numbers of deleted fields should be reserved so they are never reused. In `recommender.proto`, the most frequently populated fields such as `item_id` and `score` therefore sit at the low numbers.
>
> Source: Google, Protocol Buffers Language Guide (proto 3), 2026

Generating code on both sides

Rust generates its client at build time:

backend/build.rs

```rust
fn main() -> Result<(), Box<dyn std::error::Error>> {
    println!("cargo:rerun-if-changed=../proto/recommender.proto");
    tonic_prost_build::configure()
        .build_server(false)
        .compile_protos(&["../proto/recommender.proto"], &["../proto"])?;
    Ok(())
}
```

`build_server(false)` generates only the client half, `rerun-if-changed` rebuilds on proto changes, and `tonic::include_proto!("recommender.v1")` pulls in typed items such as `pb::RecommendRequest`. A typo in a field name is a compile error, not a runtime surprise.

Python generates its stubs with one command, wrapped as a CLI subcommand so nobody has to remember the flags:

```
cd recommender && uv run recsys gen-proto
```

`grpc_tools.protoc` with `-I recsys/pb=../proto` makes the generated import read `from recsys.pb import recommender_pb2`. The stubs are committed, so running the service needs no codegen step; the price is remembering to regenerate after editing the proto, a fair trade for a teaching repository.

Run it

The script `ch13_latency.py` calls the *running* gRPC service directly (bypassing Rust and the browser) with `Recommend`, `k = 20`, for 200 random seeded users; the first call is a warm-up and is not counted.

```
# terminal A
cd recommender && uv run recsys serve
# terminal B (from the repo root)
cd book && uv run python code/ch13_latency.py
```

TERMINAL OUTPUT

```
=== Recommend(k=20) for 200 random users at 127.0.0.1:50051 ===
model_version=20261102-060535 (first call is a warm-up and not counted)
                            p50      p90      p99     mean      max
server latency_ms         839.9    892.5    985.0    840.1   1010.6
client wall ms            840.9    893.5    986.1    841.1   1011.8

candidates scored per request: mean 102, min 30, max 129
items returned per request:   mean 20.0
```

"server latency_ms" is the service's own measurement; "client wall ms" adds serialisation and the loop-back hop. They differ by about a millisecond, so essentially all the time is inside the recommender. Every request scored between 30 and 129 candidates and returned exactly 20 items.

What is profiling? When something is slow, guessing the cause usually fails. *Profiling* means timing each stage separately and reading the numbers, like timing every step of preparing a dish to learn whether the chopping, the cooking or the plating makes the table wait. The next script does exactly this for one `Recommend` call.

Where do 840 ms go? The second script, `ch13_profile.py`, runs offline on the simulated data through the very same `pipeline.recommend` path, timing each stage for three users with very different histories (median of 7 runs). It needs no running services:

```
cd book && uv run python tools/run_all.py --only ch13_profile
```

TERMINAL OUTPUT

```
=== Stage timings for one request (median of 7 runs, milliseconds) ===
catalogue: 2,500 items, 276,378 events, cutoff = END
user    inter.  cands  History.at  generate  features   score  rerank   shap   total
light        4    110           9       213       437       1       2      6     668
median      22    112           8       215       443       1       2      5     674
heavy      133     95           8       213       443       1       2      6     674

=== Inside features.build: every table is aggregated for ALL users/items, then joined ===
item_features              122 ms   item-level (same for every user)
user_features               57 ms   all users' events
user_item_features         152 ms   all users' events
user_category_features      57 ms   all users' events
user_shop_features          26 ms   all users' events
user_price_level            11 ms   all users' events
covisitation_matrix         23 ms   item-level (inside generate)

generate + features.build = 97% of the request on average.
Item-level work (item_features + covisitation_matrix): 144 ms;
user-side aggregations computed over every user, though one user asked: 303 ms.
Caching item-level frames saves the first number; filtering events to the requesting
user before the user-side aggregations is needed to remove the second.
```

The per-user table shows totals of 668 to 674 ms for users with 4, 22 and 133 interactions. `History.at` costs 8 to 9 ms, `generate` 213 to 215 ms and `features` 437 to 443 ms; `score`, `rerank` and `shap` together are a few milliseconds. The second section opens up `features.build`. Two kinds of work hide in there. Item-level work, the same for every user, is `item_features` (122 ms) plus `covisitation_matrix` (23 ms, inside `generate`): 144 ms in total. The larger part is user-side aggregation: `user_features` (57 ms), `user_item_features` (152 ms), `user_category_features` (57 ms), `user_shop_features` (26 ms) and `user_price_level` (11 ms) each aggregate *all users*' events, al-

though only one user asked, and only afterwards are the rows joined to that user's candidates. That adds up to 303 ms. The model is the cheap part.

Results

Latency

How to read p50, p90 and p99. Imagine ten requests that took (in seconds) 0.7, 0.7, 0.8, 0.8, 0.8, 0.8, 0.9, 0.9, 0.9 and 1.5. Sort them from fastest to slowest. The **p50** (the 50th percentile, or median) is the middle of the sorted list: half the requests were faster, about 0.8 s. The **p90** is the time that 90% of requests beat, about 0.9 s. The **p99** is the same idea for 99 of every 100 requests; with only ten requests it would sit near the slowest one, 1.5 s. We report percentiles rather than the average because a few slow requests barely move an average but are exactly what an unlucky shopper experiences. The p99 answers: "how bad is it for the unluckiest one shopper in a hundred?"

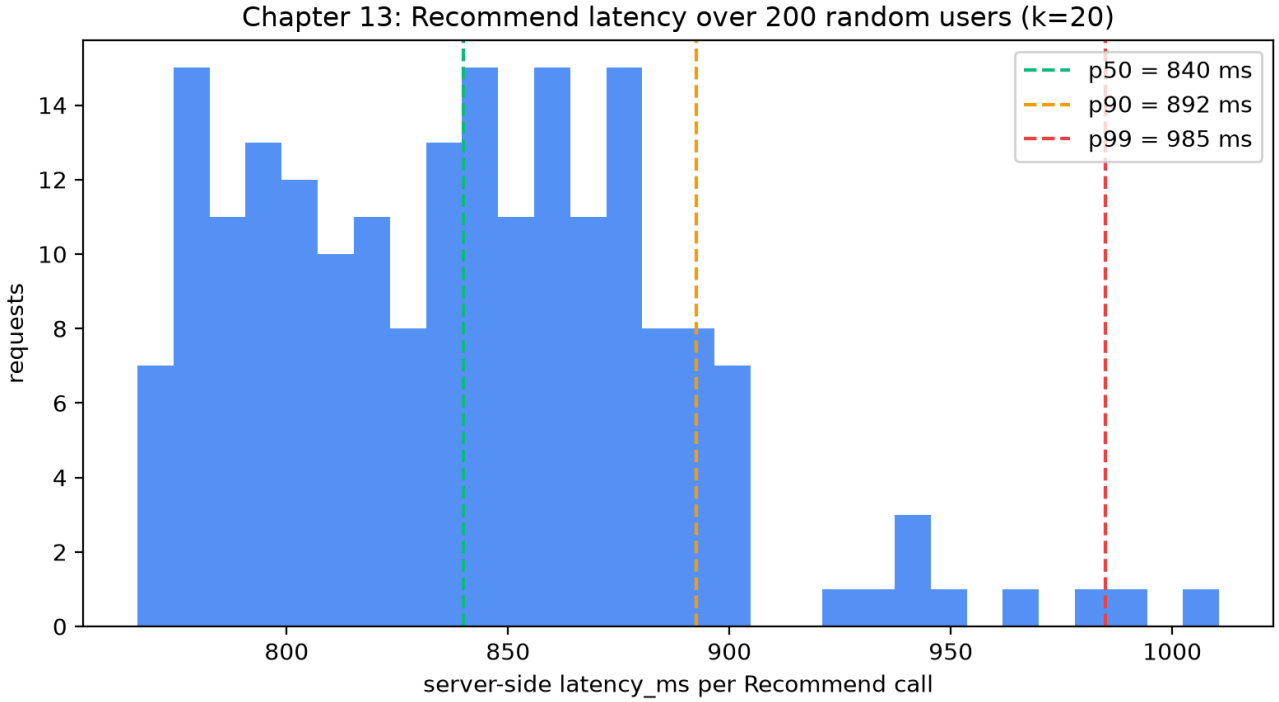


Figure 13.3: *Histogram of server-side Recommend latency over 200 random users, with p50, p90 and p99 marked*

The histogram shows a compact body between about 770 and 900 ms, and a short tail reaching about 1,010 ms. The median is 839.9 ms, the 90th percentile 892.5 ms and the 99th 985.0 ms. Two observations matter.

First, **latency barely depends on the user**. The profile explains it: `generate` plus `features.build` take 97% of the request on average, candidate counts are 95 to 112 whatever the history, and the work inside them is recomputed on every call: item-level frames and, more expensively, user-side aggregations over every user's events. Scoring, re-ranking and SHAP take about ten milliseconds together.

Second, **0.84 s is fine for a demo and too slow for a real home page**. Even the slowest observed call (1,010.6 ms) uses about a fifth of the 5-second budget, but a shopper will notice. The profile says the fix has two parts. Caching the item-level frames (co-visitation matrix and item features) for about 30

seconds would remove the 144 ms of item-level work; item statistics then lag by up to 30 s while the shopper's *own* clicks stay live. But that alone leaves the larger 303 ms: the user-side aggregations must also be restricted to the requesting user's events *before* aggregating, instead of aggregating every user and joining afterwards. A cache by itself does not fix the latency.

A screenshot tour

The screenshots come from the running application, captured with Playwright. The other pages follow the same pattern: login sets a JWT in an `httpOnly` cookie; search is plain SQL `ILIKE` that writes `searches` rows; the product page loads `/products/:id/similar` separately, so a failing recommender never blocks it; cart, checkout and orders exercise the writer side; and a freshly listed product in the Seller Center is the cold-start case of Chapter 12.

Two terms from that paragraph. A *JWT cookie* is the signed "this is user43" note described in the glossary above. The browser attaches it automatically, and the `httpOnly` flag stops page scripts from reading it, so even a buggy script cannot steal the login. And `ILIKE` is SQL's case-insensitive text match: searching "cat" finds "Cat food".

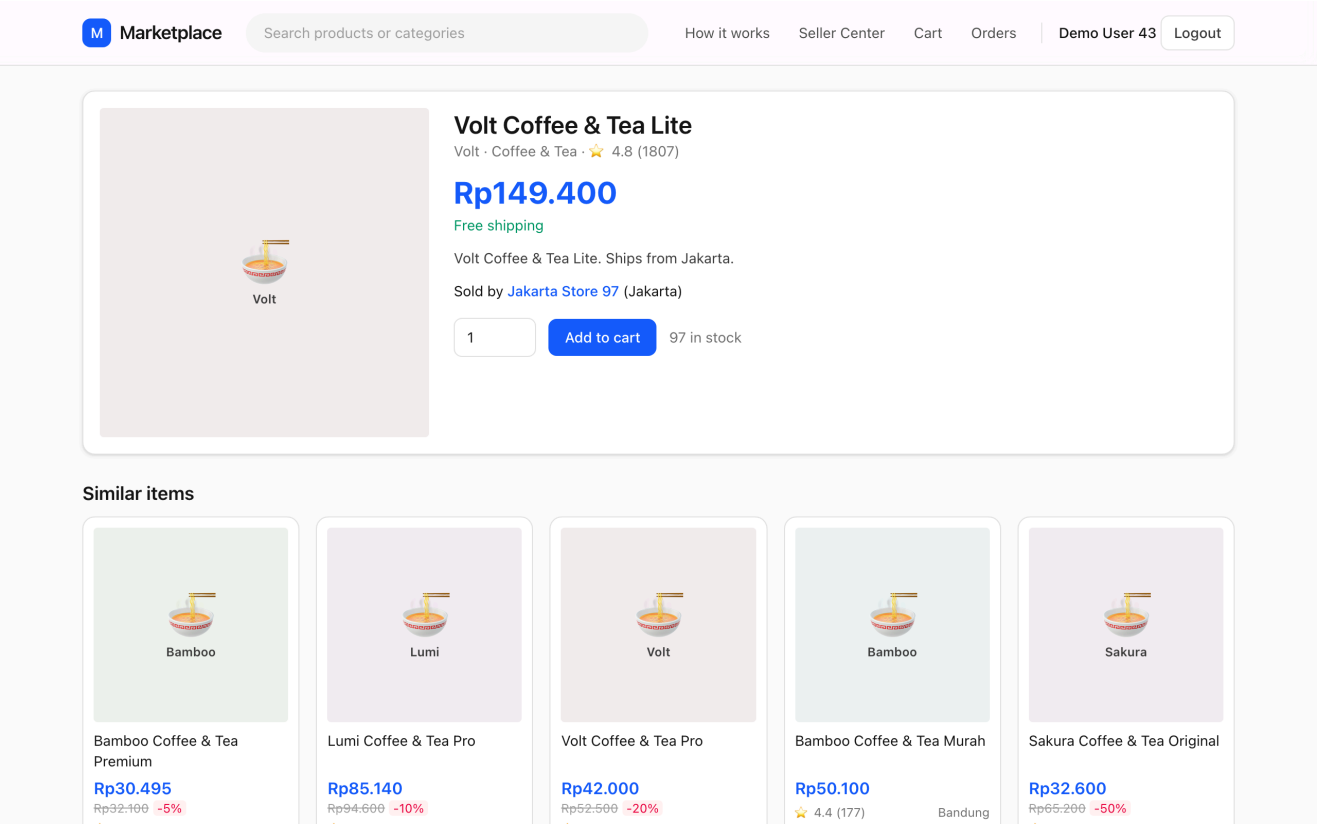


Figure 13.4: *A product page with an Add to cart button and a grid of similar items*

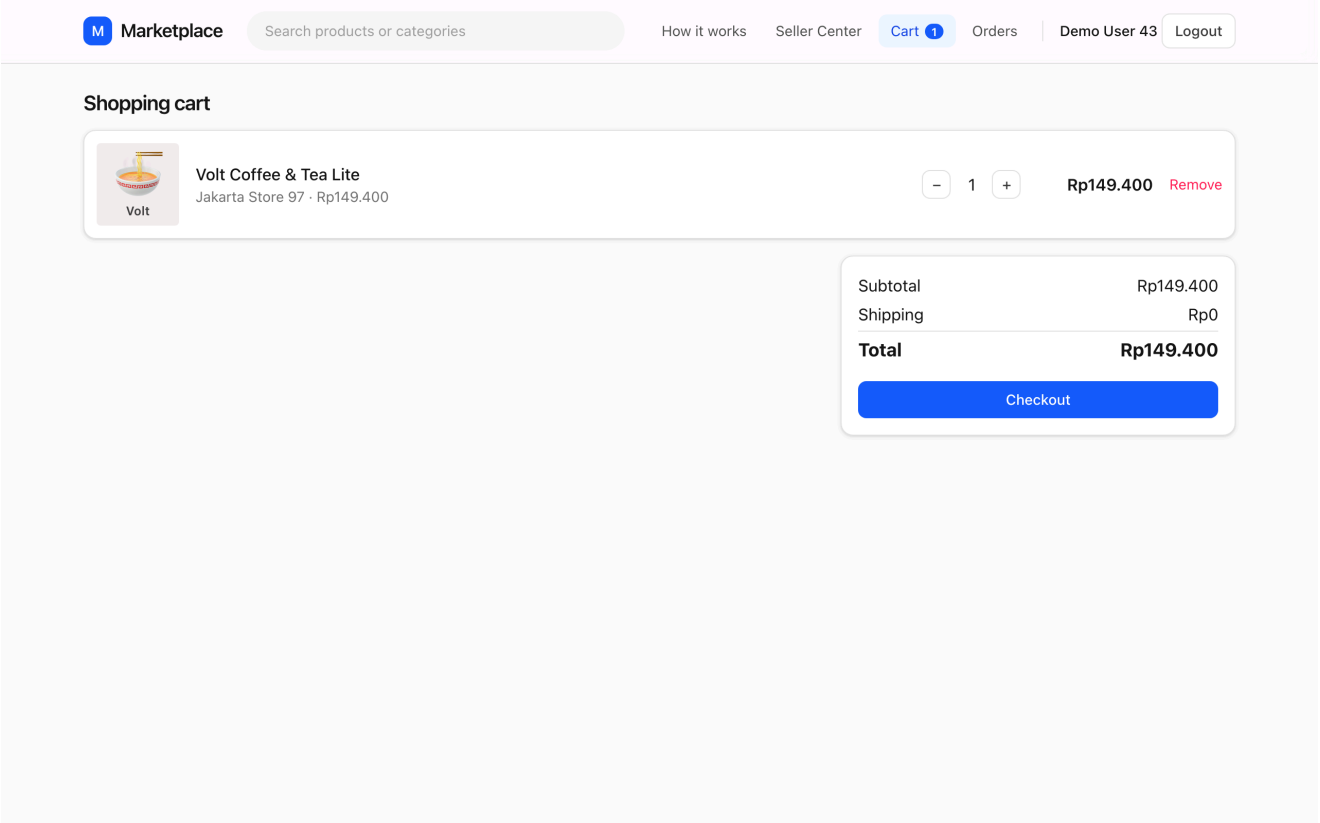


Figure 13.5: *The shopping cart with one line, subtotal, shipping and a Checkout button*

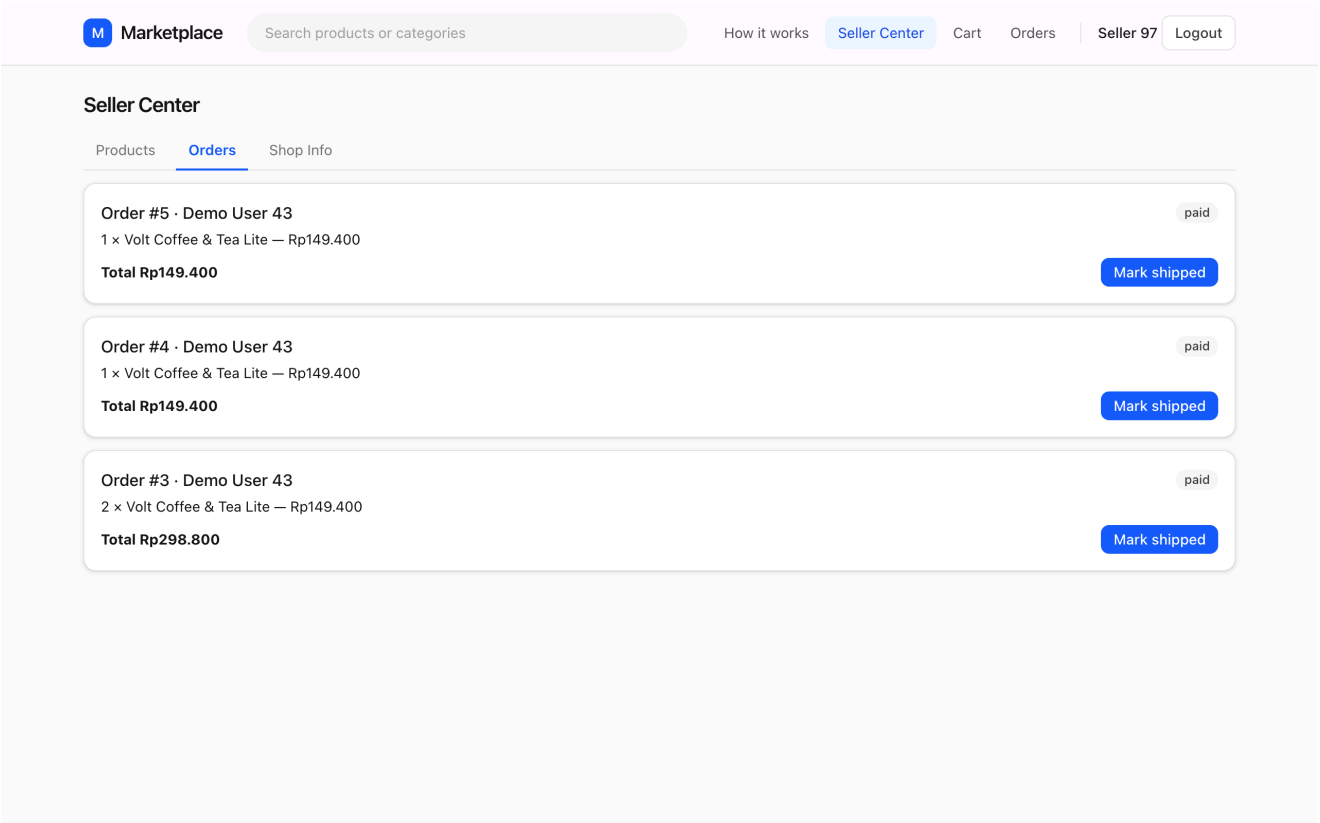


Figure 13.6: *Seller Center, Orders tab: paid orders with Mark shipped buttons*

The personalised home page. Logged in as `user43`, "Recommended for you" shows a caption with model version, candidate count and latency, and each card has "Why this?" and "Not interested" (which tracks a `hide` event). An anonymous visitor gets "Popular now" from `user_id = 0`, with no tracking.

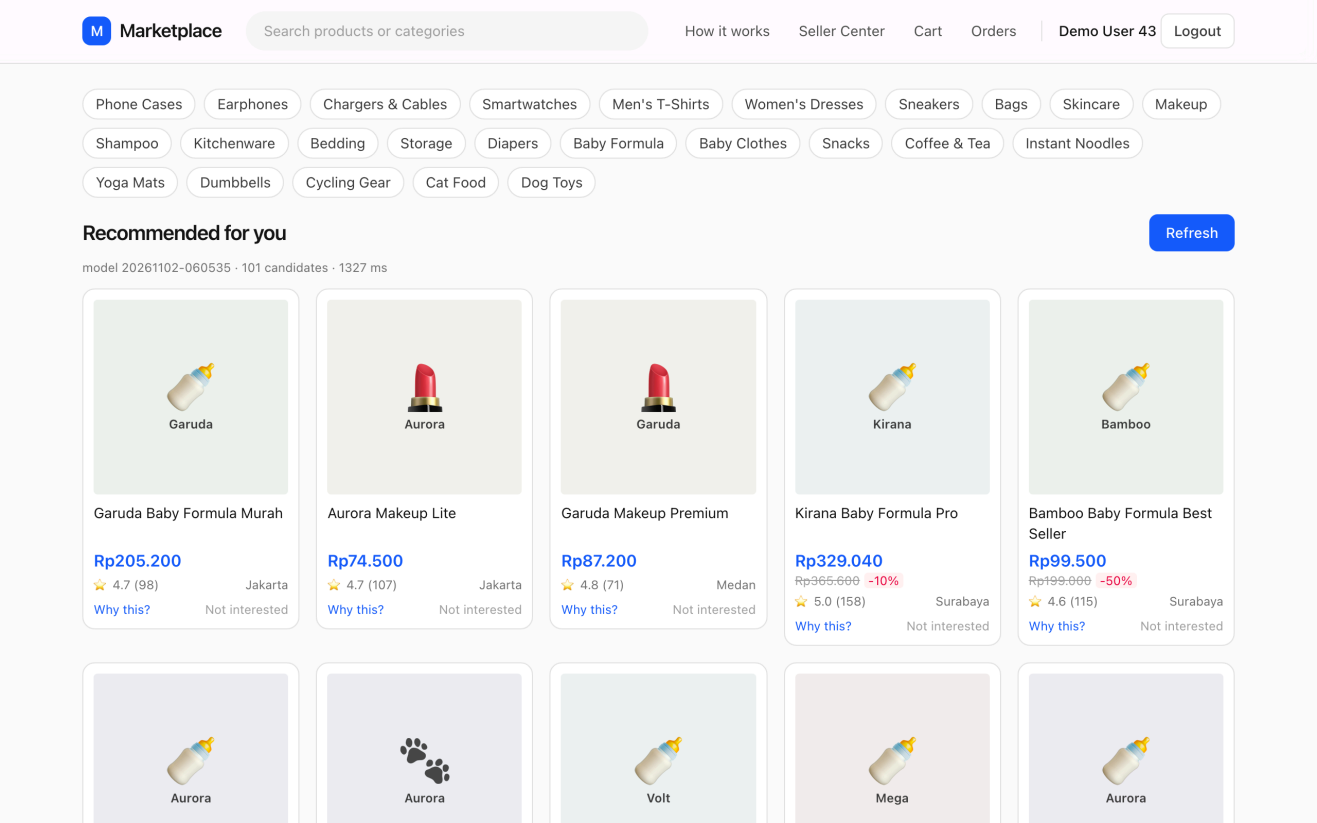


Figure 13.7: *Home page for user43: a personalised feed with Why this? and Not interested on each card*

Why this? The panel shows "Because: you viewed it", the score and rank, and five features with their signed contributions. The names come from `schema.FEATURES` via `ModelInfo`; the bars are the SHAP values computed inside `recommend_rpc`. If a card appears for a reason that makes no sense, the panel tells you which feature to investigate.

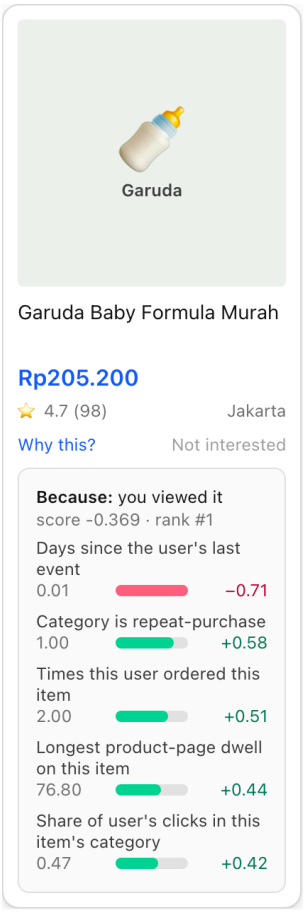


Figure 13.8: *The Why this? panel on the first recommendation*

Your behaviour, live

TIME	EVENT	ITEM	SURFACE	POS	DWELL
1:19:21 PM	order	Volt Coffee & Tea Lite	checkout		
1:19:19 PM	click	Volt Coffee & Tea Lite	search	0	1.8
1:19:19 PM	impression	Titan Coffee & Tea Premium	similar	11	
1:19:19 PM	impression	Lumi Coffee & Tea Best Seller	similar	10	
1:19:19 PM	impression	Volt Coffee & Tea Murah	similar	9	
1:19:19 PM	impression	Garuda Coffee & Tea Best Seller	similar	8	
1:19:19 PM	impression	Lumi Coffee & Tea Original	similar	7	
1:19:19 PM	impression	Volt Coffee & Tea Lokal	similar	6	
1:19:19 PM	impression	Titan Coffee & Tea Best Seller	similar	5	
1:19:19 PM	impression	Sakura Coffee & Tea Original	similar	4	
1:19:19 PM	impression	Bamboo Coffee & Tea Murah	similar	3	
1:19:19 PM	impression	Volt Coffee & Tea Pro	similar	2	
1:19:19 PM	impression	Lumi Coffee & Tea Pro	similar	1	
1:19:19 PM	impression	Bamboo Coffee & Tea Premium	similar	0	
1:19:19 PM	impression	Volt Coffee & Tea Lokal	search	23	
1:19:19 PM	impression	Lumi Coffee & Tea Pro	search	22	
1:19:19 PM	impression	Titan Coffee & Tea Premium	search	21	
1:19:19 PM	impression	Lumi Coffee & Tea Lokal	search	20	
1:19:18 PM	add_to_cart	Volt Coffee & Tea Lite	product		
1:19:17 PM	impression	Sakura Coffee & Tea Murah	search	19	
1:19:17 PM	impression	Nusa Coffee & Tea Best Seller	search	18	
1:19:17 PM	impression	Bamboo Coffee & Tea Murah	search	17	
1:19:17 PM	impression	Aurora Coffee & Tea Premium	search	16	
1:19:17 PM	impression	Lumi Coffee & Tea Lokal	search	15	
1:19:17 PM	impression	Lumi Coffee & Tea Lokal	search	14	
1:19:17 PM	impression	Mega Coffee & Tea Lite	search	13	
1:19:17 PM	impression	Garuda Coffee & Tea Murah	search	12	
1:19:17 PM	impression	Nusa Coffee & Tea Premium	search	11	
1:19:17 PM	impression	Lumi Coffee & Tea Import	search	10	
1:19:17 PM	impression	Mega Coffee & Tea Premium	search	9	

Recent searches
coffee
coffee
coffee
coffee
coffee
cat food
cat food
dog toys
dog toys
makeup

Figure 13.9: *The live events table on the How it works page: order, click with dwell, impressions by surface and position*

The model page. "How it works" shows the live event table (the same rows the recommender reads), then the model section: version, events loaded, the metric table for the four methods, candidate recall and the top features by gain, with `ui_max_dwell_sec` first. Admins also get a "Retrain now" button; after a retrain the version, metrics and training-row count differ slightly even though no code changed, because every cutoff is relative to `now` and the label windows slide with the clock.

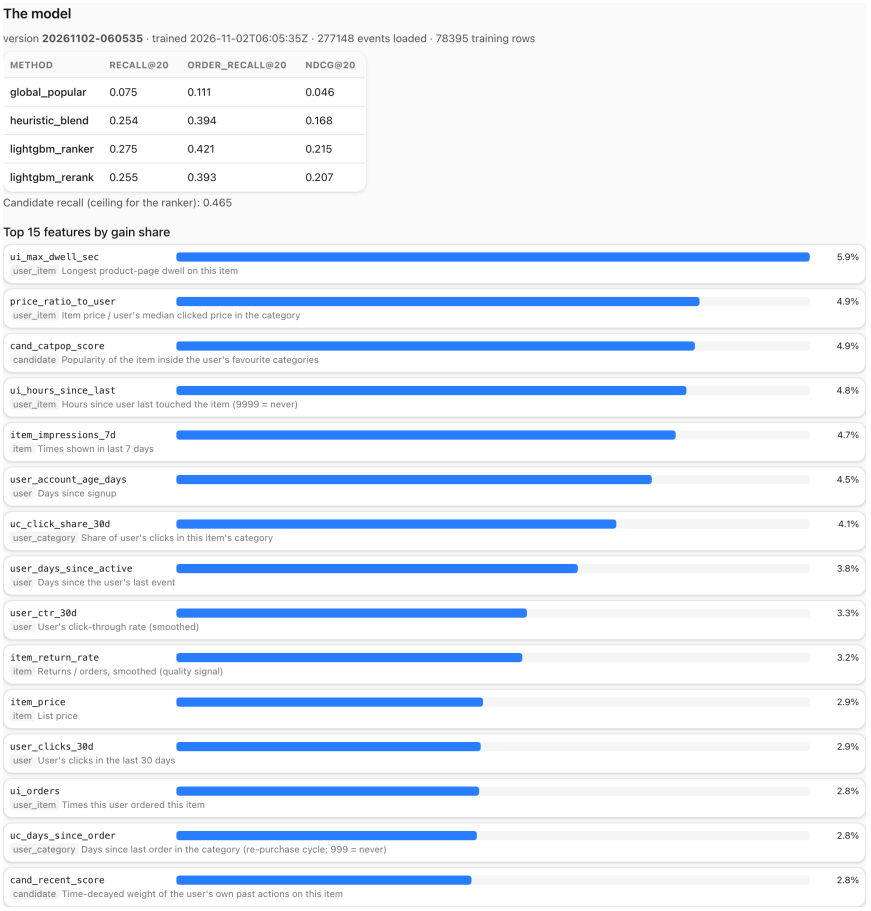


Figure 13.10: *The model section of the How it works page: version, metrics per method, candidate recall and top features*

Degraded mode. With the recommender stopped, the yellow banner replaces the model caption, cards come from the fallback query, and the "Why this?" panels are empty because trending items have no contributions. The user is slightly worse off; nobody is locked out.

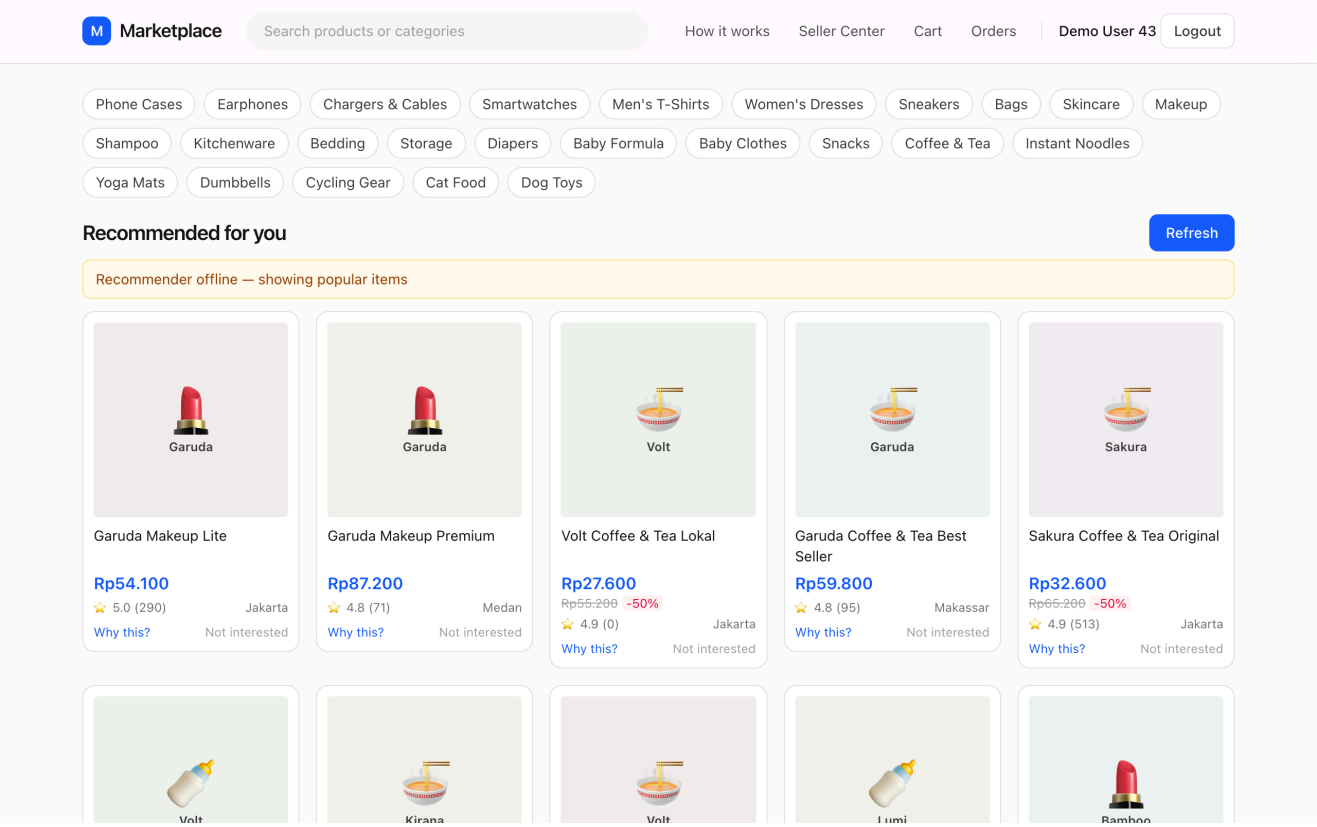


Figure 13.11: *Home page with the yellow "Recommender offline" banner*

Pitfalls and production practice

Event time versus server time. `events.ts` defaults to the database's `now()`: a client timestamp would let a skewed phone clock write events from the future. The cost is that a late-flushed batch is stamped with the flush time, irrelevant for a 2-second window but not for an offline-first app.

Training/serving skew. The service calls `pipeline.recommend` and the training job calls `pipeline.train_and_evaluate`; both use `History.at` and `features.build`. One implementation per feature, not two; a feature store (Chapter 14) industrialises this.

Latency budgets. At scale you do not have 840 ms; each stage gets a slice of a few hundred milliseconds. Measure percentiles, find the fixed cost, cache what can be cached, keep personalised work small.

Summary

- The marketplace has four processes: a Vue frontend, a Rust/Axum API that is the only writer, PostgreSQL as the shared ledger, and a Python gRPC recommender that reads Postgres through a read-only connection.
- A click becomes a changed ranking through impressions queued and flushed by `useTracker`, `post_events` and `touch_session` writing rows with a session id, `Store.refresh` reading them incrementally, and `Recommend` running the full pipeline on request. The worst-case staleness is the flush delay plus the refresh throttle plus the request time.

- Exit-time events need one ordered flusher: the `onPageHide` hook registry fixes the bug where a click was queued after the beacon had already been sent.
- The recommender stays fresh with an overlap-safe incremental refresh (`id > last − 1000`, then de-duplicate) because auto-increment ids are assigned at insert but become visible at commit.
- A single `.proto` file is the contract; Rust generates a client in `build.rs`, Python generates stubs with `recsys gen-proto`, and responses carry the model version, candidate count and latency for observability.
- Retraining is periodic and on demand, guarded by a non-blocking lock and hot-swapped by assigning two references, so serving never pauses.
- The Rust API hydrates ids into cards in the recommender's order and falls back to a non-personalised trending query on error or timeout, so a recommender outage degrades the page instead of breaking it.
- Measured on this machine, `Recommend` takes about 840 ms at the median and 985 ms at the 99th percentile, almost independent of the user. A stage profile attributes 97% of it to candidate generation and feature building: 144 ms of item-level work a short-TTL cache can absorb, and 303 ms of user-side aggregations over all users that only filtering events to the requesting user removes.

Quiz

Q13.1 Why does the Python recommender open its database connection with `default_transaction_read_only=on`?

A. It makes `SELECT` queries faster by skipping locks.
B. It guarantees that a bug in the recommender cannot modify application data; any write fails with an error.
C. It lets Postgres reuse the same connection for the Rust API.
D. It is required by the gRPC protocol.

Q13.2 Which set of event types does `POST /api/events` accept from the browser, and why is `order` not among them?

A. `impression`, `click`, `hide`; `order` is a consequence of a server-side action (checkout) and must not be forgeable from the client.
B. `impression`, `click`, `order`; `hide` is derived on the server.
C. All six types; the database `CHECK` constraint is the only protection.
D. Only `click`; impressions are computed by the server from page loads.

Q13.3 In the `pagehide` bug, what exactly went wrong in the first version of the product page?

A. The `fetch` call timed out after 2 seconds.
B. The product page's own `pagehide` listener ran after the tracker's listener had already drained the queue into the beacon, so the click was never sent.
C. `sendBeacon` cannot send JSON.
D. Vue Router unmounted the component before `pagehide` fired.

Q13.4 Explain in two or three sentences why `Store._refresh` re-reads the last 1000 event ids instead of querying `id > last_event_id`. What would go wrong without the overlap?

Q13.5 `Recommend` begins with `model, version = self.model, self.meta["version"]` and `retrain` finishes with `self.model, self.meta = model, meta`. Explain how these two lines let the service swap models during traffic without a lock on the request path.

Q13.6 (Code reading.) Read `touch_session` in `backend/src/sessions.rs`. A shopper triggers events at 10:00, 10:20 and 11:05 on the same day, with no other activity. How many sessions are created, and what is the `last_seen_at` of the first session at the end of the day? Justify using the 30-minute rule in the SQL.

Q13.7 (Numeric.) The Rust route times out the gRPC call after 5 seconds. Using the results in this chapter, what fraction of that budget does the slowest observed server-side call (1,010.6 ms) consume, and what is the worst-case staleness $T_{\text{stale}} = T_{\text{flush}} + T_{\text{refresh}} + L$ if $T_{\text{flush}} = 2$ s, $T_{\text{refresh}} = 1$ s and L is the observed p99 server latency of 985.0 ms? Give both answers.

Q13.8 What does the fallback return when the recommender is unavailable, and why does it label every item with `source: "trending"`, a score of 0 and empty contributions?

Q13.9 You rename a field in `recommender.proto` by changing its number from `3` to `7` and rebuild only the Rust API. The Python service still runs with the old generated stubs. What is the most likely outcome and why does the guide say field numbers should never be changed once in use?

A. Nothing; the wire format uses field names.
B. The two sides disagree about which bytes mean which field, so data is silently dropped or misread.
C. Rust will refuse to compile.
D. gRPC will automatically negotiate the new number.

Q13.10 (Hands-on.) Open `book/code/ch13_latency.py` and change `k=20` in both the warm-up and the measurement loop to `k=5`, then again to `k=50`, re-running the script against the live service each time. Record `p50` and the "candidates scored per request" line. What do you expect to observe about latency and candidate counts, and what does it tell you about where the time goes?

Chapter 14 — From prototype to Shopee scale

It is 12:00 on 9.9, and a flash sale has just opened. Within a few seconds the number of people on the marketplace jumps to many times the usual lunchtime peak. Every one of them lands on a home feed, and every feed has to be different. The ranker that you trained in Chapter 10 is a few hundred kilobytes of trees. It was built from a single Postgres table, and it takes about 840 milliseconds to answer one request on a laptop. Nothing about it is wrong. It is the right size for a book and for a weekend of learning, and it is almost the right shape for a production system. But it was never designed to survive that noon.

Something else matters just as much. The sale does not only stress the servers. It changes the data. Shoppers who normally browse sneakers are suddenly hunting for discounted rice cookers. A model trained last week has never seen a flash-sale crowd, and its features say that yesterday's popular items are still the popular ones. Scale is not only about throughput. It is also about freshness, about measuring whether a change really helped, and about keeping many teams, many objectives and many models honest at the same time.

This final chapter is a map. We will not build a Kafka cluster or train a transformer. Instead we walk through, one by one, the pieces that a company at the size of Shopee adds around the core you already understand, and we say precisely which problem each piece solves. For every piece, we point back to the part of this repository that it would replace or extend. By the end you should be able to read an architecture diagram from a large marketplace and recognise every box as a bigger, sturdier version of something you have already run.

What you'll learn

- How streaming logs (Kafka) and stream processing (Flink) turn the batch `events` table into real-time features, and why that changes what "fresh" means.
- What a feature store is for, and how it prevents training/serving skew.
- Why approximate nearest neighbour (ANN) search is unavoidable at billions of items, and how to estimate its memory cost.
- How Wide & Deep, DCN, DIN and multi-task MMoE rankers differ from the gradient-boosted ranker in this repo, and when they are worth their cost.
- How to combine several objectives (GMV, ads, seller fairness) into one score without hiding the trade-offs.
- How to run an A/B test, including a complete two-proportion z-test and a sample-size calculation worked by hand.
- What to monitor once the system is live, how to budget latency, and a concrete checklist of what to build next in this repository.

Intuition: the night market and the supermarket chain

Imagine a night-market stall that sells fried bananas. The owner knows every regular. She sees who walks up, remembers last week's order, adjusts the sugar when the weather turns, and decides on the spot what to recommend. That is our prototype. Everything fits in one head, which is the single Postgres database and the single Python process. The owner can try a new idea at lunch and know by dinner whether it worked.

Now imagine a national supermarket chain: hundreds of shops report sales through a stream of messages, a data team maintains one shared definition of "loyal customer", several teams run promotions, and finance insists that experiments are measured before a promotion goes national. The chain did not need a smarter owner. It needed *infrastructure for agreeing on facts, moving them quickly, and measuring change*.

That is the whole difference between our repository and a production marketplace. The ideas (candidates, features, ranking, re-ranking) survive intact. What multiplies is the plumbing around them. In the rest of the chapter we will keep asking the same three questions of every new component:

1. Which of the limits of the prototype does it remove: speed, freshness, scale, correctness, or the ability to experiment?
2. What new failure mode does it introduce?
3. Which file of this repository would it replace?

How big is "big"? Scale in everyday numbers

Three words come up all through this chapter, so we pin them down first.

- **Latency** is how long one request takes, from the moment you tap the screen to the moment the page appears. It is the wait at a food stall: from ordering to holding the food.
- **Throughput** is how many requests the system finishes per second. It is how many customers the same stall serves in an hour.
- **Offline vs online.** *Offline* means working on saved records: training a model, or scoring last week's logs. *Online* means serving real shoppers right now. The same model can look great offline and bad online, and much of this chapter is about why.

Now a little arithmetic to feel the size of the problem. In Chapter 13 our prototype needed about 840 milliseconds, or 0.84 seconds, to answer one request. One worker therefore finishes about $1/0.84 \approx 1.2$ requests per second. Imagine, purely as an illustration, 10,000 requests arriving every second at noon on 9.9. At 0.84 seconds each, that is $10{,}000 \times 0.84 = 8{,}400$ workers busy at the same time, every one of them holding its own copy of the data. No team wants to run that. So a big system has to do two things at once: make each request much *faster*, and *share* work between requests (compute something once, reuse it for thousands of shoppers). Every component in this chapter is one of these two moves, or a way to stay correct while making them.

The algorithm: the same pipeline, scaled out

Before the details, here is the whole picture. The multi-stage pipeline from Chapter 8 (retrieve, rank, re-rank) is unchanged. What changes is that every arrow in the diagram becomes a networked service with its own latency, capacity and owner.

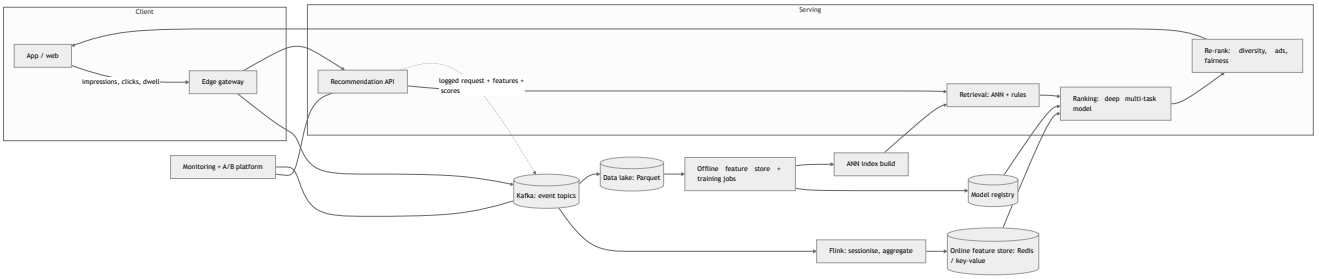


Figure 14.1: *A Shopee-style architecture. The dashed edge is the most important one: serving logs what it showed, with the features and scores it used.*

Read the diagram from left to right. Behaviour flows from the app into a message queue. From there it takes two paths. The fast path (Flink into an online feature store) keeps features fresh to the second. The slow path (the data lake into training jobs) produces models and ANN indexes every few hours or days. Serving reads from both: the latest model and index from the slow path, the freshest user features from the fast path. And, crucially, serving writes back the *exposure log*, i.e. what it displayed, so that the next training run knows what the user actually saw.

Each box in the diagram has an everyday meaning. Keep this table nearby; the sections below explain each row in detail.

Box	In plain words	Closest thing in this repository
Kafka (event log)	A shared logbook where events are only ever added at the bottom, and many readers each keep their own bookmark	The Postgres `events` table
Flink (stream processor)	A worker that reads the logbook non-stop and keeps running tallies	`db.Store.refresh`
Online feature store	A quick-lookup board holding each shopper's and item's *current* numbers	Nothing yet: features are recomputed on every request
Data lake	A huge, cheap archive of every old event, stored as files	The `events` table, read in bulk
Model registry	A shelf of numbered model versions, so you know which one is live	The model panel of `/how-it-works`
ANN index	A fast-lookup shelf of item vectors for finding similar items	The index of Chapter 7
Exposure log	A receipt of exactly what we showed each shopper, with the scores used	Nothing yet (checklist item 2)

In the prototype, the same loop exists but is shrunk to one machine. `useTracker.ts` is the client, the Rust `POST /api/events` route is the edge gateway, the Postgres `events` table is Kafka plus the data lake in one, `db.Store.refresh` is a stream consumer, `pipeline.train_and_evaluate` is the training

job, and the `Recommend` RPC is the serving tier. When a question starts with "how would we scale this?", map the question onto this list first.

Streaming logs and real-time features

There are two ways to keep a number such as "how many cat-food clicks has shopper Sari made in the last 30 days" up to date. **Batch** processing recomputes it from scratch on a schedule, like a shopkeeper who counts the till once, after closing. **Streaming** updates it the moment each event arrives, like a cashier's running total that changes with every sale. Batch is simpler. Streaming is fresher.

Suppose Sari clicks a cat-food bag at 12:00:05. With a nightly batch job, the ranker still believes she has never looked at cat food until tomorrow. With streaming, the number changes within seconds, so her next page refresh already reflects it. On 9.9, that gap is the difference between recommending rice cookers to someone who is hunting for them right now and showing yesterday's sneakers.

In the prototype, Rust inserts one row per event into Postgres, and the Python service polls (asks again and again) for rows newer than the last id it saw. This is simple and correct, and it works for a few hundred thousand rows. It breaks in three ways at scale:

- **Write throughput.** A single Postgres primary will not absorb millions of inserts per second, and event writes compete with orders and carts.
- **Fan-out.** The same event is wanted by many consumers: the feature pipeline, the ads system, fraud detection, the analytics warehouse. Polling a table makes every consumer a reader of the production database.
- **Replay.** When a bug is found in a feature definition, you need to re-process last month's events. A table that is also the system of record (the one true copy of the data) is a poor place for that.

The standard answer is a log-based message broker such as Apache Kafka. Picture a notebook where writers only ever add new lines at the bottom, nobody erases anything, and every reader keeps a bookmark of the line they have reached. Kafka's vocabulary maps onto that picture:

- A **topic** is one notebook for one kind of event, for example all clicks.
- A **partition** is one volume of that notebook. Splitting a topic into volumes lets many writers and readers work in parallel. We choose the volume by `user_id`, so all of Sari's events land in the same volume, in the order they happened. That is exactly what session logic needs.
- An **offset** is the bookmark: the line number a reader has reached.
- A **consumer group** is a team of readers sharing a topic. Each team has its own bookmark, so the ads team and the fraud team read the same notebook without disturbing each other.

Because the log is kept for days, a new reader can start at page one, and a fixed bug can be re-run over the old events (replay).

A stream processor such as Apache Flink reads the log continuously and computes features *incrementally*, meaning it updates a small running tally instead of recounting everything. Take the feature `user_clicks_30d` from Chapter 9. In SQL it is a `COUNT` over a window (here, the last 30 days). In a streaming system it becomes a stateful operator that keeps a small per-user tally, called the **state**:

Moment	What happens	Sari's `user_clicks_30d`
Before	Her state says 5 clicks in the window	5
New click arrives	Add one	6
A click from 30 days ago leaves the window	Subtract one	5

The general shape is *event in, state update, feature out*:

$$s_u^{(t+1)} = f\big(s_u^{(t)},\, e_{t+1}\big), \qquad x_u^{(t+1)} = g\big(s_u^{(t+1)}\big)$$

Here s_u is the state of user u (Sari's tally), e is a new event (her click), f is the rule that updates the state (add one), and g turns the state into the feature value x_u that the ranker will read (often just copying the tally). A decayed score such as the exponential decay of Chapter 3 is particularly pleasant in this form, because it can be updated in constant time: $s \leftarrow s \cdot 2^{-\Delta t/h} + w$, where h is the half-life (the time it takes the score to halve), Δt is the time since the last update and w is the event weight. Try it: with $h = 7$ days, a state of 4.0, and a new click of weight $w = 1$ arriving exactly 7 days later, the old score halves to $4.0 \times 0.5 = 2.0$ and then gains 1, giving 3.0. One multiplication and one addition; no need to re-read Sari's history.

Streaming adds subtleties. A click made in a tunnel may reach the server two minutes late, so the processor must group events by *event time* (when the click happened), not arrival time, and use a **watermark**, a marker that says "we believe we have now seen everything up to time T", to decide when a window may close. Restarts must not count an event twice. And computing a feature once in Flink and once in a batch job invites the skew discussed next.

Feature stores and training/serving skew

A **feature** is a number about a user, an item or a pair that the model reads, such as `user_clicks_30d` in Chapter 9. A **feature store** is a shared, versioned catalogue of feature definitions that feeds both training and live requests from the same recipe.

Recall the leakage demo in Chapter 9: a feature that peeked at the future made the offline ndcg@20 look like 0.5035 instead of 0.2116, and the model was useless live. Leakage is *cheating*. **Training/serving skew** is the quieter, honest cousin: the feature is legitimate in both places, but it is computed differently. Think of a restaurant chain whose test kitchen and branch kitchens each keep their own copy of a recipe. One copy says "drain the pasta", the other forgot. The dish tasted wonderful in the test kitchen and poor in the branches, and nobody did anything wrong in either place. The fix is one recipe card, not better cooks.

Here is a tiny example. CTR (click-through rate) is clicks divided by times shown. Suppose a phone case was shown 100 times and clicked 5 times, but 40 of those 100 showings went to automated bots that never click. The training job excludes bots, so it computes $5/60 \approx 0.083$. The serving code is a separate piece of application code that forgets to, so it computes $5/100 = 0.05$. The model learned "0.083 means a good item". Live, it sees 0.05 for the very same item and underrates it. Offline metrics stay excellent. Online metrics quietly sag, and nothing crashes, which is why skew is hard to spot.

A feature store prevents this by making each definition (for example `item_ctr_30d`) one versioned artefact with two materialisations, i.e. two places where its values are actually stored:

- an **offline store** (columnar files or a warehouse) that can answer "what was this feature's value for user u *as of* time t?" for millions of training rows at once; and
- an **online store** (a low-latency key-value store, like a giant lookup table) that answers "what is it now?" in about a millisecond.

The same definition feeds both, so the numbers agree by construction. The offline half is where *point-in-time correctness* lives. It performs an **as-of join**: for each training row with timestamp t, take the latest feature value with timestamp $\leq t$, never later. For example, on 1 March Sari's `user_clicks_30d` was 5, and by 5 March it was 9. A training row from 1 March must use 5. Using 9 would be leakage, because the row would learn from clicks that had not happened yet. In this repository `data.History.at` does exactly this, so the idea is the same as Chapter 9. The feature store simply industrialises it.

A second defence against skew is *logging the served features*. Instead of recomputing historical features for training, the serving layer writes the feature vector it actually used, together with the item's position and the model's score, into the **exposure log**: a receipt of what we showed Sari and why (the dashed edge in the diagram). Training then learns from precisely what was served. This costs storage, but it eliminates an entire class of bugs, and it gives you the propensities (the chance that each item was shown at that position) that you need for the position-bias corrections of Chapter 12.

ANN at billions of items

Nearest-neighbour search (Chapter 7) means: given Sari's vector, find the item vectors closest to it. **Exact** search compares Sari with every item. **Approximate** nearest neighbour (ANN) search accepts missing a few true neighbours in exchange for skipping most comparisons. Think of a library. Exact search means reading every spine in the building. Approximate search means walking to the right section and scanning only that shelf. You might miss a good book filed elsewhere, but you finish in seconds. (The IVF index of Chapter 7 works just like this: its clusters are the sections.)

At 2,311 in-stock items, brute force took 0.140 ms per query, and the index was slower to build than it was useful. At a hundred million or a billion items the picture reverses. Take a back-of-the-envelope memory estimate. One billion items, each a 64-dimensional vector of 4-byte floats. One item is $64 \times 4 = 256$ bytes. A billion of them:

$$10^9 \times 64 \times 4 \text{ bytes} = 2.56 \times 10^{11} \text{ bytes} = 256 \text{ GB}.$$

That is far more than one ordinary machine's memory, and scanning all of it per request is hopeless. Production systems therefore do three things at once.

1. Shard the index. A *shard* is one slice of the catalogue held on its own machine, like a library with several branches. A query goes to every shard, each shard returns its own best results, and one machine merges them. Tiny example: three shards each return their top 2 scores, (0.9, 0.7), (0.8, 0.6) and (0.5, 0.4). Merging gives the overall top 2: 0.9 and 0.8. This is correct because any item in the global top 2 must also be in the top 2 of its own shard.

2. Compress the vectors. *Product quantisation* (PQ) works like painting by numbers. Cut a vector into short pieces. For each piece, keep a palette of 256 typical pieces, and store only the number of the nearest palette entry (one byte). Tiny example: an 8-number vector (32 bytes) is cut into two pieces of 4 numbers, each replaced by a 1-byte palette id, so it shrinks to 2 bytes. You lose some precision, be-

cause each piece is only approximately restored, but memory drops sharply. For our 256-byte vectors, 16 pieces of 4 numbers with one byte each give 16 bytes, shrinking the same index to $10^9 \times 16 = 16$ GB.

3. Use a graph or clustered index so that each query touches only a tiny fraction of the vectors. IVF groups vectors into clusters and searches only the nearest few (Chapter 7). HNSW links every vector to its nearest "friends" and hops from friend to friend towards the query. IVF-PQ combines clusters with compression. ScaNN is Google's library of the same family. The trade-off is exactly the one in the Chapter 7 table, where probing 4 of 32 clusters kept recall@20 at 0.975 of brute force while scanning 12.5% of the items. At scale you tune the same dial (how much to look at against how much you might miss), only with a tighter latency target and a much larger catalogue.

Indexes are rebuilt, not edited, and the item tower (the half of the two-tower model that turns items into vectors) must be re-run for the whole catalogue whenever the model changes, so model and index versions deploy as a pair.

> **Did you know?**
>
> **Graph embeddings at web scale.** Pinterest's PinSage combined random walks with graph convolutions to learn embeddings of pins. It was trained on a graph with 3 billion nodes (pins and boards) and 18 billion edges, which its authors described as the largest application of deep graph embeddings to date at the time of publication.
>
> Source: Ying et al., KDD, 2018

Deep rankers

A **ranker** is the model that scores each candidate so that we can sort them. Gradient-boosted trees are a superb ranker for a few dozen to a few hundred dense features, and the leaderboard at the end of this chapter shows how far they get on our simulated data. At a large marketplace, the ranker's inputs look different: billions of *sparse ids* (a shop id, a user id, a search query: categories with huge numbers of values, each seen rarely), long behaviour sequences, and several outcomes to predict at once. Trees handle this poorly, because a tree cannot learn anything useful about an id it sees once a month. Neural rankers can, using **embeddings**: a short list of numbers, learned by the model, that acts as a fingerprint for an id, so that similar ids get similar numbers (Chapter 7). Two more words before we start. An **MLP** (multi-layer perceptron) is a small neural network: layers of simple weighted sums, each followed by a bend. A **logit** is a raw score, any number from minus to plus infinity, that the sigmoid function σ squashes into the range 0 to 1 so that it reads as a probability: $\sigma(0) = 0.5$, $\sigma(2) \approx 0.88$, $\sigma(-2) \approx 0.12$. Here are the four families you will hear about most.

Wide & Deep. The Google Play team observed two complementary needs. *Memorisation* is learning that a specific pair, such as "installed app A and was shown app B", is strongly predictive. *Generalisation* is handling pairs never seen together before, by way of embeddings. Think of two students preparing for an exam. One memorises last year's answers, which is perfect for repeated questions. The other learns the principles, which handles new questions. Wide & Deep hires both and adds their opinions. The "wide" part is a linear model over sparse cross-product features (a cross feature is an AND, such as "viewed A **and** B"). The "deep" part is an MLP over embeddings. Their outputs are summed into one logit:

$$P(y=1 \mid \mathbf{x}) = \sigma\big(\mathbf{w}_{\text{wide}}^{\top}[\mathbf{x}, \phi(\mathbf{x})] + \mathbf{w}_{\text{deep}}^{\top} a^{(L)} + b\big)$$

In words: $\mathbf{x}$ is the input features, $y=1$ means "the user clicks", $\phi(\mathbf{x})$ are the cross-product transformations, $\mathbf{w}_{\text{wide}}$ weighs the wide inputs, $a^{(L)}$ is the last hidden layer of the MLP, $\mathbf{w}_{\text{deep}}$ weighs it, and b is a constant offset. Toy numbers: suppose the wide part adds $+1.5$ (this pair is often clicked), the deep part adds $+0.5$, and $b=-1$. The logit is $1.5+0.5-1=1.0$ and $\sigma(1.0)\approx 0.73$, a 73% click probability. In our repository, the hand-built cross features of Chapter 9 (`uc_click_share_30d`, `price_ratio_to_user`) play the role of the wide part, built by an engineer and fed to a tree. In **DCN** (Deep & Cross Network), the crossing is learned: explicit cross layers multiply the input features with each other, layer after layer, so you stop hand-writing every interaction.

DIN. Alibaba's Deep Interest Network started from an observation about e-commerce. A user's history contains many interests, and only some matter for the item being scored. Compressing the whole history into one fixed vector wastes capacity. DIN adds a *local activation unit*: each history item gets a weight that says how relevant it is to the *candidate* item (the item we are scoring right now), so the same user gets a different interest vector depending on the candidate. Tiny example: Sari's history is a cat-food bag, a cat toy and a phone case. For the candidate "phone case", the weights might be roughly 0.1, 0.1 and 0.8, so the phone case dominates. For the candidate "cat litter", they might be 0.5, 0.4 and 0.1, so the cat items dominate. Same Sari, two different interest vectors. In pseudo-formula:

$$\mathbf{v}_u(a) = \sum_{j=1}^{H} w_j(a)\,\mathbf{e}_j, \qquad w_j(a) = \mathrm{MLP}\big(\mathbf{e}_j,\ \mathbf{e}_a,\ \mathbf{e}_j \odot \mathbf{e}_a\big)$$

Here $\mathbf{e}_j$ are the embeddings of the user's H past items (above, $H=3$) and $\mathbf{e}_a$ is the candidate's embedding. The symbol $\odot$ multiplies two embeddings number by number, which is large where they agree, so it gives the MLP a quick "how alike are these?" signal. The result $\mathbf{v}_u(a)$ is the weighted sum of the history embeddings. Compare this with the user tower of Chapter 7, which averaged the last 20 items with equal weights. DIN is that same idea, with the weights learned and conditioned on the candidate. Transformer-based sequence models, which are now common, generalise it further with self-attention.

Multi-task MMoE. A marketplace cares about at least three outcomes: click, add-to-cart and order. In this repository we squeeze them into one label (click is 1, cart 2, order 3) and train one LambdaMART model. That is cheap, but it hard-codes the trade-off and gives the model only a single, noisy target. A **multi-task** network predicts each outcome with its own **head** (the final small layer that outputs one prediction) and shares the lower layers. The naive version, one shared trunk, suffers when tasks conflict: what makes a user click (an eye-catching picture) differs from what makes them buy. The *Multi-gate Mixture-of-Experts* design is like a hospital with a panel of specialists. The shared "experts" are the specialists, and each task has its own coordinator, the **gate**, who decides how much to listen to each specialist:

$$y_k = h_k\Big(\sum_{i=1}^{n} g_k(\mathbf{x})_i\, f_i(\mathbf{x})\Big), \qquad g_k(\mathbf{x}) = \mathrm{softmax}(W_{g_k}\,\mathbf{x})$$

for task k, expert networks f_i, task head h_k, and a per-task gate g_k. The **softmax** turns any list of scores into weights that are positive and add up to 1. Example: scores $(2,1,0)$ become $e^2, e^1, e^0 =$

$7.39, 2.72, 1.00$, which sum to 11.11, so the weights are $(0.67, 0.24, 0.09)$ after rounding. Now say three experts output 0.4, 0.9 and -0.2 for one item. The click gate has weights $(0.67, 0.24, 0.09)$ and mixes them to $0.67 \times 0.4 + 0.24 \times 0.9 + 0.09 \times (-0.2) \approx 0.47$. The order gate has weights $(0.09, 0.24, 0.67)$ and mixes the same experts to ≈ 0.12. Same experts, different mix per task. If click and order are closely related, their gates will pick similar experts. If they conflict, the gates specialise. Because orders are rare compared with clicks (Chapter 1's funnel), the sparse `order` head also benefits from the representation that the dense `click` head learns. The production score then combines the heads, which brings us to the next topic.

> **Did you know?**
>
> **Attention over user history.** Alibaba's Deep Interest Network (DIN) replaced the fixed-length user vector of the standard Embedding-and-MLP recipe with a local activation unit that weights the user's past behaviours by their relevance to the candidate ad. It was trained on an Alibaba production dataset of over 2 billion samples and deployed in Alibaba's online display advertising system, serving the main traffic.
>
> Source: Zhou et al., KDD, 2018

> **Did you know?**
>
> **Wide & Deep in an app store.** Google's Wide & Deep paper jointly trained a linear model (memorisation) and a deep network (generalisation), and evaluated it on Google Play, a commercial app store with over one billion active users and over one million apps. Online experiments showed that it significantly increased app acquisitions compared with wide-only and deep-only models.
>
> Source: Cheng et al., DLRS Workshop, 2016

Multi-objective ranking: GMV, ads and sellers

Multi-objective ranking means the final order must serve several goals at once: shoppers finding things they like, the company earning money, and sellers getting a fair chance. Once the ranker emits $\hat{p}_{\text{click}}$, $\hat{p}_{\text{cart}}$ and $\hat{p}_{\text{order}}$ (the predicted probabilities of each action) separately, someone must decide how to blend them. The simplest form is a weighted sum, often with a business term for margin or for sponsored placements (an *ad*: a seller pays a **bid** to be shown, and the platform earns when it is clicked):

$$\text{score}(u, i) = \Big(w_1\, \hat{p}_{\text{click}} + w_2\, \hat{p}_{\text{cart}} + w_3\, \hat{p}_{\text{order}} \Big) \cdot \big(1 + \beta_{\text{boost}}(i)\big) + \lambda_{\text{ads}}\, \text{bid}(i)\, \hat{p}_{\text{click}}$$

The weights are not learned from clicks. They are policy: how much we value each action. Raising w_3 favours GMV (gross merchandise value, the total value of goods sold), and raising λ_{ads} favours advertising revenue. The boost $\beta_{\text{boost}}(i)$ encodes campaign rules, such as a shop-level subsidy for a mall brand during a sale (0.1 means a 10% uplift).

Walk through two items with imaginary probabilities. Item A, a phone case, has $(\hat{p}_{\text{click}}, \hat{p}_{\text{cart}}, \hat{p}_{\text{order}}) = (0.30, 0.10, 0.02)$. Item B, a rice cooker, has $(0.10, 0.08, 0.05)$. With weights $(1, 2, 3)$, A scores $0.30 + 0.20 + 0.06 = 0.56$ and B scores $0.10 + 0.16 + 0.15 = 0.41$, so A wins. Change the order weight to 10, meaning "we now care mostly about purchases", and A scores $0.30 + 0.20 + 0.20 = 0.70$ while B scores $0.10 + 0.16 + 0.50 = 0.76$. The ranking flips without retraining anything. That is why the weights are a business decision and not a technical detail.

This is the same structure as the re-ranking rules of Chapter 11, promoted from "caps in a Python function" to a formal part of the objective, and a good production system is explicit about it. Everyone can read the weights, argue about them, and see them in the experiment configuration. The danger is hidden coupling: a ranker trained to predict orders and then re-weighted by margin may chase expensive items that rarely convert.

Seller fairness enters in two ways. As a **constraint**, it is like the repository's cap of 4 items per shop in `rerank.rerank`. As an **objective**, you give new or small sellers a controlled amount of exposure, because a marketplace that only ever shows incumbent shops starves its own supply side. This is the exploration idea from Chapter 11 and Chapter 12, applied to sellers instead of items. Whatever you choose, *measure it separately*. Track seller-side metrics (share of exposure to shops under 30 days old, concentration across shops) next to buyer-side metrics in every experiment, and treat a drop in either as a cost.

> **Did you know?**
>
> **Multi-task learning for recommenders.** The MMoE paper adapted the Mixture-of-Experts structure to multi-task learning by sharing expert sub-models across all tasks while giving each task its own gating network. The authors showed that it performs better than baselines when tasks are less related, and demonstrated improvements on a large-scale content recommendation system at Google.
>
> Source: Ma et al., KDD, 2018

A/B testing, with the arithmetic

Every number in the leaderboard below is *offline*. It is computed from logs of what users did under a previous system. Offline metrics rank candidate models cheaply, but they cannot settle whether a model helps real users, for the reasons from Chapters 2 and 12: logged data reflects the old system's choices, position bias is baked in, and our metrics ignore diversity and revenue. The final arbiter is an **A/B test**: a fair taste test run on live traffic. Like a food stall that flips a coin for each customer to serve recipe A or recipe B, then compares how many come back, we randomly assign users to control (the current system) or treatment (the new one), wait, and compare a business metric. The coin flip is what makes the comparison fair: both groups are alike except for the recipe.

Running this at scale takes infrastructure, and each piece has a job:

- **An assignment service** that decides each user's variant (below).
- **Logging** that stamps every event with the user's variant, so no result can be attributed to the wrong group.
- **A metrics pipeline and dashboard** that computes the primary metric and the guardrails per variant.
- **Automatic checks**, such as the sample-ratio check below, that stop a broken experiment before anyone reads its numbers.

The mechanics follow a fixed recipe:

1. Choose a *primary metric* (say, order conversion per user) and *guardrail metrics* before launch. A guardrail is a metric that must **not** get worse even if the primary one improves, just like the rail

on a mountain road. Examples: latency, seller-side exposure, return rate. If the new ranker raises orders but doubles page latency, the guardrail stops the launch.
2. Compute the required sample size from the smallest effect you care about.
3. Randomise **by user**, with a stable hash, so that one user always sees one variant. A hash is a fixed scrambling function: the same input always gives the same output, but outputs look random. For example, take $\text{hash}(\text{user_id} + \text{experiment name}) \bmod 100$ to get a bucket from 0 to 99. If Sari's bucket is 37 and the rule is "0–49 is control", she is in control on every visit, and the other half of users land in treatment. Mixing the experiment name into the hash keeps different experiments independent.
4. Check the assignment itself. If you intended a 50/50 split and observe 50.6/49.4 on millions of users, something is broken (a *sample ratio mismatch*), and the results cannot be trusted. To see why, imagine 1,000,000 users. A fair coin gives a typical wobble of $\sqrt{1{,}000{,}000 \times 0.5 \times 0.5} = 500$ users, but 50.6% is 6,000 users above the middle, twelve times that wobble. Chance does not do this; a bug (say, bots filtered from one arm only) does.
5. Run for full weekly cycles (weekdays and weekends behave differently), then test the difference.

Here is a complete worked example. **Every number below is hypothetical**, invented to show the working. They are not outputs of this repository's scripts.

Suppose each variant receives $n = 100{,}000$ users. In control, 8,000 users place an order, giving $\hat{p}_C = 0.0800$. In treatment, 8,240 do, giving $\hat{p}_T = 0.0824$. The observed absolute lift is $0.0824 - 0.0800 = 0.0024$, a relative lift of 3%. Is it real or noise?

The question is like flipping two coins 100,000 times each and finding that one landed heads slightly more often. Was that luck, or a better coin? A **z-test** answers it in four steps. First, assume the **null hypothesis**: both variants truly have the same order rate, so any gap is luck. Second, measure how big a gap luck alone would typically produce (the standard error). Third, express the observed gap in units of that typical luck (the z statistic). Fourth, convert that into a **p-value**: if there really were no difference, how often would luck produce a gap at least this large? A small p-value means the gap is hard to blame on luck.

Under the null hypothesis, the best estimate of the shared rate is the pooled proportion (all orders divided by all users):

$$\hat{p} = \frac{x_C + x_T}{n_C + n_T} = \frac{8000 + 8240}{200000} = 0.0812$$

The standard error (SE) is the typical size of the luck-only gap between the two rates. Under the null, it is

$$\text{SE} = \sqrt{\hat{p}(1-\hat{p})\left(\frac{1}{n_C} + \frac{1}{n_T}\right)} = \sqrt{0.0812 \times 0.9188 \times \frac{2}{100000}} \approx 0.001222$$

Each piece has a meaning: $\hat{p}(1-\hat{p})$ is how "spread out" a yes/no outcome is, and the $\frac{1}{n}$ terms shrink the SE as we gather more users. The test statistic counts how many standard errors the observed difference is from zero:

$$z = \frac{\hat{p}_T - \hat{p}_C}{\text{SE}} = \frac{0.0024}{0.001222} \approx 1.965$$

So our gap is about two "typical luck-sized" steps from zero. For a two-sided test (a gap in either direction counts), the p-value is $2\,(1 - \Phi(1.965)) \approx 0.0494$, where Φ is the standard normal CDF (the table that converts a z value into a probability). At the conventional significance level $\alpha = 0.05$ (we accept a 5% chance of a false alarm) this just clears the threshold. The 95% confidence interval for the difference (a range of plausible true lifts; it uses the unpooled standard error, also about 0.001222) is roughly $[0.000006,\ 0.004794]$: it barely excludes zero, so the true lift could be anywhere from essentially nothing to 6% relative. That is the honest reading. "Significant" here means "we cannot comfortably rule out zero", not "the lift is 3%". If you had peeked at the dashboard daily and stopped the first time $p < 0.05$, your real false-positive rate would be far higher than 5%, just as checking a coin after every flip will eventually show a suspicious streak. The significance level assumes a single, pre-planned look.

How many users would we have needed? The answer depends on **power**: the chance that the test notices a real effect of the size we care about. A test with 80% power catches such an effect 8 times out of 10. For a target absolute effect δ at significance α and power $1 - \beta$, the per-variant sample size is approximately

$$n \approx \frac{2\,(z_{\alpha/2} + z_{\beta})^2\; p(1-p)}{\delta^2}$$

where $z_{\alpha/2}$ and z_{β} are standard cut-off values for the false-alarm rate and the power, and p is the baseline rate. With $p = 0.08$, $\delta = 0.0024$, $z_{\alpha/2} = 1.96$ and $z_{\beta} = 0.8416$ (80% power):

$$n \approx \frac{2 \times (2.8016)^2 \times 0.08 \times 0.92}{0.0024^2} = \frac{2 \times 7.849 \times 0.0736}{0.00000576} \approx 200{,}600$$

So about 200,600 users per variant, roughly twice what we ran. Our hypothetical test was *underpowered* for a 3% relative lift, which is why it landed on the boundary. Because the required n grows with $1/\delta^2$, halving the effect you want to detect quadruples the traffic you need. This is why large companies obsess over variance-reduction techniques (ways to make the metric less noisy so a smaller sample suffices), and why testing many small tweaks at once requires care.

Monitoring and drift

A deployed recommender can fail silently. The service keeps returning 200 OK (the "success" code) with twenty perfectly formatted items, while quality decays. **Drift** means the world or the data moves away from what the model was trained on. The usual causes are:

- **Data drift:** inputs change, for example a new app version stops sending `dwell_sec`.
- **Concept drift:** the feature-outcome relationship changes after a holiday: the same inputs now lead to different behaviour.
- **Feedback loops:** the model only learns about items it showed (Chapter 12).
- **Pipeline breakage:** a late stream, a stale feature, an index that did not rebuild.

Each has a monitor. For data drift, compare the distribution of each feature in the last hour against the training distribution. The *population stability index* does this by sorting values into buckets and com-

paring how many fall in each bucket now versus at training time; a bigger number means a bigger change. The Kolmogorov–Smirnov test is another way to measure how different two distributions are. Also alert when the share of missing values changes. For the model, watch the distribution of predicted scores, because a sudden shift is a cheap proxy for broken inputs. For outcomes, track online CTR and conversion by position and by user activity bucket, as Chapter 12 did offline, and compare against a small **holdout**: a few percent of users who always receive a simple baseline. The holdout is your honest ruler, like a shelf you leave unchanged to compare against the redesigned ones. It tells you whether the whole recommender is still beating popularity, which our leaderboard says is a 4x gap at least (pop_7d scores 0.048, the ranker 0.215). Finally, monitor *freshness* itself: the age of the newest event visible to the ranker, and the age of the index. In this repository, the equivalent of a freshness alert would be comparing `max(ts)` in the `events` table with the latest event seen by `Store.refresh`.

Latency budgets

A request is a chain of calls, and each link takes time. A page **budget** (say 300 ms at p99) is therefore sliced across stages, like splitting a trip's time across legs. The terms: **p50** is the median, the time that half of requests beat. **p99** is the time that 99 out of 100 requests beat; sort 100 request times from fastest to slowest and p99 is about the 99th. It describes the slow tail that real users still feel. An illustrative budget for a 300 ms page might read:

Stage	Budget (ms, illustrative)
Gateway and auth	20
User-feature fetch	20
Retrieval	60
Feature assembly	40
Ranking	80
Re-ranking	20
Hydration (fetching titles, images, prices) and network	60
Total	**300**

Three principles make budgets work.

First, **tails dominate**: a page that fans out to ten services is as slow as the slowest. If each service is slow just 1% of the time, the chance that *at least one* of ten is slow is $1 - 0.99^{10} \approx 9.6\%$, nearly one page in ten. So p99 matters more than the median. Our own result shows it at small scale: server p50 839.9 ms, p99 985.0 ms, maximum 1010.6 ms.

Second, **degrade rather than fail**: every stage needs a deadline and a cheaper fallback, like a waiter who brings bread when the kitchen is late, instead of leaving you with an empty table. That is why `GET /api/recommendations` returns a popularity list with `fallback: true` after a 5 second timeout.

Third, **cache what is shared**. A *cache* is a saved answer that is reused instead of recomputed, and its **TTL** (time to live) is how long it may be reused before being recomputed. Think of a shopkeeper who writes today's price list once in the morning instead of working out each price for each customer; the list is valid until tomorrow's. Item-level statistics change slowly and can be computed once for all

users. Suppose (hypothetically) 10,000 requests arrive per second and the item statistics are cached with a 60-second TTL: one computation then serves about 600,000 requests instead of 600,000 separate computations. The price is staleness of up to one TTL, so choose a TTL that is short compared with how fast the numbers truly change.

Code walkthrough: pseudo-code for the missing pieces

There is no included repository code here, because none of these components exist in the repository yet. The sketch below is pseudo-code: the business blend from the multi-objective section. Compare the serving flow with `RecommenderService.Recommend`: it has the same shape, plus a user-tower retriever, batch feature-store lookups, several task heads and an exposure log written at the end.

```
# Pseudo-code: blending task heads into one score.
def final_score(p, cands, w=(1.0, 2.0, 3.0), ads_weight=0.0):
    base  = w[0]*p["click"] + w[1]*p["cart"] + w[2]*p["order"]
    boost = np.array([c.campaign_boost for c in cands])   # e.g. 0.1 in a sale
    ads   = ads_weight * np.array([c.bid for c in cands]) * p["click"]
    return base * (1.0 + boost) + ads
```

The weights `(1.0, 2.0, 3.0)` echo the label scheme of this book (click 1, cart 2, order 3) but are now *explicit and tunable*.

Run it: the book's leaderboard

The chapter's figure aggregates every method evaluated in this book on the same simulated users, the same labels and the same 7-day evaluation window. Regenerate it, and the headline comparison it summarises, with:

```
cd book
uv run python tools/make_figures.py ch14
```

The figure reads `results/*.json` from every companion script. Its top three rows come from the comparison in Chapter 10, which looks like this:

TERMINAL OUTPUT

```
=== Three ways to order the same candidates ===
                   recall@5  recall@10  recall@20  recall@50  order_recall@20  ndcg@20
coverage@20
method
heuristic blend       0.1153     0.1769     0.2526     0.3753           0.3978   0.1674
0.5682
pointwise LightGBM    0.1577     0.2176     0.2836     0.3915           0.4250   0.2146
0.4777
LambdaMART            0.1503     0.2015     0.2725     0.3824           0.4210   0.2116
0.4245

candidate recall ceiling: 0.4645
```

The heuristic blend, the pointwise model and LambdaMART are all ordering the *same* candidates, whose recall ceiling is 0.4645. That ceiling is why even the best ranker reaches only a modest recall@20.

Results: reading the leaderboard

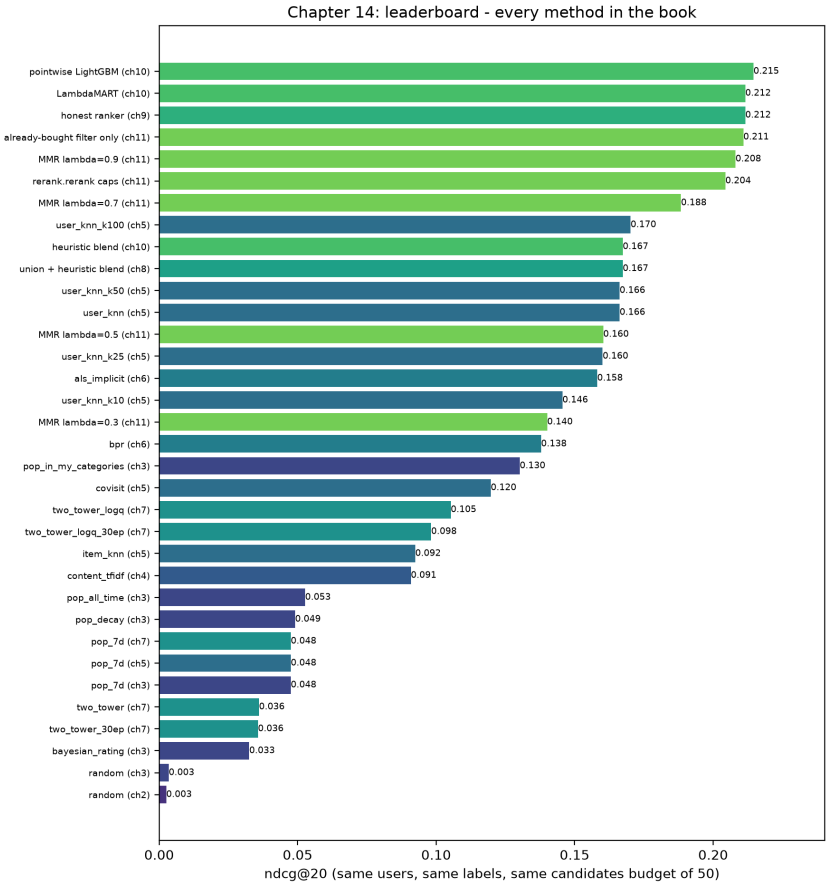


Figure 14.2: *Every method in the book on one axis: ndcg@20 on the same users, labels and candidate budget.*

The chart tells the story of the whole book in one picture, and it is worth reading from the bottom up.

Random and rating-only baselines (0.003 and 0.033) are the floor. A Bayesian-average rating alone, with no notion of what people actually buy, scores barely above chance. The popularity family lands around 0.048 to 0.053. Windowed 7-day popularity, all-time popularity and the decayed version are nearly indistinguishable, which is the honest outcome that Chapter 3 reported.

Content and collaborative models climb into the 0.09 to 0.17 range. TF-IDF content filtering reaches 0.091, item-kNN 0.092, ALS 0.158, and user-kNN with 100 neighbours 0.170. The family of personalised retrievers roughly triples the popularity baseline. Notice that the simple co-visitation retriever (0.120) is already stronger than the best two-tower model (0.105), and that the two-tower model only got there after the logQ correction of Chapter 7: the uncorrected run scored 0.036. A neural model is not automatically better. It needs the right loss, and on 2,495 items it has little room to shine.

The multi-stage system occupies the top of the chart. The heuristic blend over the union of candidate sources scores 0.167. Learned rankers over the same candidates reach 0.212 (LambdaMART) and 0.215 (pointwise LightGBM). The honest leakage-free ranker of Chapter 9 (0.212) sits on the same level, while its leaky twin from the same chapter scored 0.5035, which is why it is deliberately absent from the ranking: it is a bug, not a method. The ordering of the green bars is a reminder of two earlier lessons: the gap between the two ranker objectives is about one seed standard deviation (Chapter 10), and the rows for MMR with λ below 1 *lose* ndcg in exchange for diversity (Chapter 11).

Two cautions apply to every conclusion drawn from this chart. First, the data is simulated, and its hidden preference model is simpler than the real world. The *ordering* of methods transfers better than the absolute values. Second, these are offline numbers on a single weekly split. They justify trying a method in a real A/B test. They never replace the test.

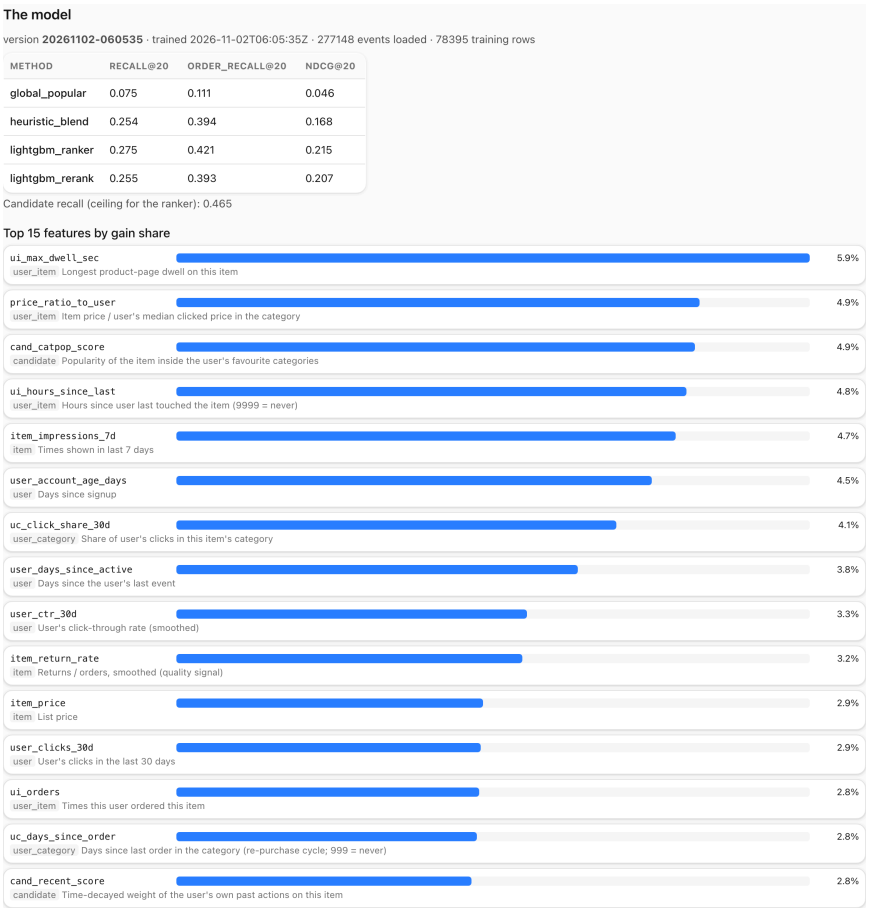


Figure 14.3: *The model page of the running app: the version, the evaluation metrics of the most recently trained ranker and the features it relies on most.*

The screenshot above is the "The model" panel of `/how-it-works`: the model version, events loaded and top features by gain share. At scale it becomes a model registry with history across versions.

Pitfalls and how it is done in production

Adding scale before you need it. Copying a big-company architecture for a small problem adds operational burden and new outages. If your events fit in one database and the model retrains in minutes, the prototype design is right; migrate on a concrete bottleneck.

Deep models without a strong baseline. Keep the boosted-tree ranker as benchmark and fallback; ship the deep model only if it wins online by enough to justify its cost.

What to build next in this repository

If you want to carry this project forward, here is a prioritised checklist. Each item is small enough for a weekend and teaches one idea from this chapter.

1. **Cache item-level frames and filter events per user.** The profile in Chapter 13 splits the request into two costs: item-level frames (`item_features`, `covisitation_matrix`) recomputed on every call, and user-side aggregations computed over every user's events although only one user asked, the larger share. Cache the item-level frames for a short TTL, keyed on the cache time, and filter events to the requesting user before the user-side aggregations. Measure after each part; neither alone removes the cost.
2. **Log the exposure.** Make `Recommend` write a row per displayed item (request id, model version, rank, score, features). Train on exposure instead of reconstructing it, and use it for propensity weighting (weighting each example by how likely it was to be shown).
3. **Add a holdout and a mini A/B framework.** Hash `user_id` into buckets; send 5% to the popularity fallback; log the bucket on every event; build a page that applies the z-test from this chapter to order conversion.
4. **Split the label into heads.** Train separate click, cart and order models (three LightGBM models are enough), expose the blend weights in configuration, and compare against the single graded label.
5. **Replace the poller with a queue.** Have Rust publish events to a broker, and have `Store` consume from it. Keep Postgres as the system of record. This is the smallest honest step towards the streaming architecture.
6. **Add a drift monitor.** Implement a drift check (for example the population stability index) for the top 15 features by importance and surface it on the `/how-it-works` page.
7. **Add the two-tower retriever as a fifth source.** Use the item index from Chapter 7 with the logQ correction, and check with the Chapter 8 script that it raises union recall before you pay for it in latency.
8. **Protect new sellers.** Add an exposure floor for shops listed in the last 7 days and measure its cost in ndcg, as Chapter 11 did for diversity.

> **Tip**
>
> Do the checklist in order. Items 1 to 3 give you the *measurement* machinery (speed, logs, experiments). Without them you cannot tell whether items 4 to 8 helped.

> **Did you know?**
>
> **Real-time training in production.** ByteDance's Monolith system argues that general-purpose frameworks separate batch training from serving and so cannot learn from customer feedback in real time. It introduces a collisionless embedding table (with expirable embeddings and frequency filtering to limit memory) and an online-training architecture, and it has been deployed in BytePlus Recommend.
>
> Source: Liu et al., arXiv, 2022

Summary

- Scaling a recommender keeps the pipeline (retrieve, rank, re-rank) and replaces each arrow with a service: events go through a log such as Kafka, and streaming jobs keep features fresh.
- A feature store defines each feature once, with an offline as-of materialisation for training and an online one for serving, which prevents training/serving skew. Logging the served features is

the second defence.
- At a billion items you shard, compress (product quantisation) and use ANN indexes; a 64-dimensional float32 index is 256 GB before compression, so memory arithmetic drives the design.
- Deep rankers add what trees cannot: embeddings for sparse ids (Wide & Deep, DCN), candidate-aware attention over history (DIN), and several heads for click, cart and order (MMoE).
- Blending heads into one score is a policy decision: make the weights, ads terms and seller-fairness rules explicit and measure them separately.
- An A/B test needs a pre-declared metric, a sample size computed in advance, a sample-ratio check and a single planned look. The worked example showed how a hypothetical $z \approx 1.965$ is only borderline significant, and why about 200,600 users per variant were needed.
- Monitor feature drift, score distributions, outcomes by position and activity, and freshness, against a holdout. Budget latency by the p99, give every stage a deadline and a fallback.
- The leaderboard says what this book found offline: personalised retrieval beats popularity, and a learned ranker over multi-source candidates beats everything else. A live experiment decides whether the gain is real.

Quiz

Q14.1 Training/serving skew most directly arises when:

A. The training set is larger than the serving set.
B. A feature is computed by different code or filters at training time and at serving time.
C. The model is retrained too often.
D. The ANN index returns approximate results.

Q14.2 What does the *local activation unit* of DIN do?

A. It compresses the whole user history into one fixed-length vector regardless of the candidate.
B. It weights the user's past behaviours by their relevance to the specific candidate item being scored.
C. It selects which retrieval source to use for each user.
D. It removes position bias from training labels.

Q14.3 In a multi-gate mixture-of-experts (MMoE) model, what is the purpose of a separate gate per task?

A. To let each task (click, cart, order) choose its own weighted combination of shared experts.
B. To train each task on a different dataset.
C. To guarantee that all tasks have equal loss.
D. To cache predictions between tasks.

Q14.4 In the latency result of Chapter 13, the server p50 is 839.9 ms and the p99 is 985.0 ms. Why do production teams budget for p99 rather than p50, and what does this repository do when a request is too slow?

Q14.5 Explain why a better position on the offline leaderboard does not by itself justify shipping a model. Use the leaky ranker (ndcg@20 0.5035 against 0.2116 for the honest one) as one of your ex-

amples, and name one other reason.

Q14.6 A hypothetical A/B test sends 50,000 users to each variant. Control has 5,000 orders and treatment has 5,150. Compute the pooled proportion, the standard error, the z statistic, and decide at $\alpha = 0.05$ (two-sided) whether the lift is significant. (The two-sided p-value for $z = 1.57$ is about 0.116.)

Q14.7 A catalogue has 2 billion items. Each item vector has 128 dimensions stored as 2-byte half-precision floats. How many GB does the raw index need? If product quantisation compresses each vector to 32 bytes, how many GB is the compressed index?

Q14.8 Read the `final_score` sketch in the chapter, with `w=(1.0, 2.0, 3.0)`, `ads_weight=0.0`. An item has `p_click = 0.2`, `p_cart = 0.05`, `p_order = 0.01` and `campaign_boost = 0.1`. What is its final score? What would the score be if the campaign boost were 0?

Q14.9 You plan a 50/50 A/B test and, after a day, observe 50.6% of users in control and 49.4% in treatment, out of several million users. What does this suggest, and should you read the conversion results?

A. It is expected noise; read the results.
B. It is a sample ratio mismatch, suggesting a bug in randomisation or logging; investigate before trusting any result.
C. It proves the treatment reduces traffic.
D. It means the sample size is too small.

Q14.10 *Hands-on.* Open `book/code/ch11_rerank.py` and add `0.8` to the `LAMBDAS` list (between 0.9 and 0.7). Run it from `book/` with `uv run python code/ch11_rerank.py`. State what you expect to observe for the new row's ndcg@20 and distinct categories relative to the 0.9 and 0.7 rows, then check it, and say whether this supports the claim in this chapter that diversity must be paid for in relevance.

Appendix A — Quiz answers

Answers to every end-of-chapter quiz, with a short explanation. Hands-on exercises describe what you should observe; your exact numbers depend on the change you make.

Chapter 1 — What a Recommender Does

Q1.1 — B. Retrieval cheaply narrows millions of items to a few hundred candidates and is judged on recall. Option A describes ranking, C describes re-ranking, and D is unrelated to serving. Whatever retrieval misses can never be recovered by the stages after it.

Q1.2 — C. Dwell time and add-to-cart are by-products of normal behaviour, not deliberate statements of opinion. Star ratings, written reviews and thumbs-up taps are all explicit feedback because the user chose to express a preference.

Q1.3 — B. A real platform only sees logs, so the simulator writes only what an app would log and keeps the latent variables private. Methods must therefore recover the signal from the same kind of data they would see in production. Because the truth exists, we also know the data is learnable, which helps when diagnosing a failing method.

Q1.4 — CTR is clicks divided by impressions, and the impression count is the denominator that says how often an item was actually shown. If only clicks are logged, a popular-looking item might simply have been shown very often, and you cannot tell "shown and ignored" from "never shown". You would also have no negatives for training and no way to study position bias.

Q1.5 — Sparsity is the fraction of user-item cells with no recorded interaction (here 99.388%). Each user has touched very few items, so an algorithm must infer taste from a handful of filled cells. Consequently methods share statistical strength across users and items: neighbourhood methods (chapter 5), low-rank factorisation (chapter 6), and content features. Any answer naming one such consequence is correct.

Q1.6 — A Saturday sale day has raw weight $1.2 \times 3.0 = 3.6$. A Sunday payday that is not a sale day has $1.2 \times 1.4 = 1.68$. The flags are returned because the simulation loop reuses them: on sale days the effective discount is raised by 0.2, and the order probability has payday and sale bonuses, so time changes behaviour as well as traffic volume.

Q1.7 — 251 orders. Multiply the three rates: $0.1059 \times 0.4295 \times 0.5517 \approx 0.02509$ (the 2.51% of impressions that end in an order), and $10{,}000 \times 0.02509 \approx 250.9$, which rounds to 251.

Q1.8 — Density = 50,000 / (1,000 × 10,000) = 50,000 / 10,000,000 = 0.5%, so sparsity = 99.5%. Even this "dense-looking" example leaves 199 of every 200 cells empty.

Q1.9 — After the change, the script prints the top 50% (1,250 items) receiving 92.8% of interactions, so the bottom half of the catalog shares only about 7% of them. The top 5% (125 items) gets 35.1%, which lies between the 12.5% of the top 1% and the 50.3% of the top 10%. This shows how steeply interactions concentrate in the head and how thin the tail is, which is why popularity baselines are strong.

Q1.10 — Two good reasons: (1) the user may never have noticed the item, because it sat low in the list or off-screen (exposure and position bias), so no click does not mean no interest; (2) the user may have been interested but chose a different item first or intended to come back later. Treating these as hard negatives teaches the model to repeat what was shown before and penalises items for being badly placed.

Chapter 2 — Measuring success

Q2.1 — B. With a random shuffle, some test events are older than training events, so the model learns from the future (popularity spikes, a user's later taste, other events from the same session). A is not the problem, C is nonsense, and D is unrelated to how data is split. A time-based split removes the leak.

Q2.2 — C. Both lists contain the same set of items, so recall@10, precision@10 and coverage are identical. NDCG discounts by position, so the list that puts the ordered item (gain 7) first scores higher than the one that puts it last.

Q2.3 — B. Candidate recall is the share of future positives that appear in the candidate set, so no ranker that only reorders those candidates can exceed it. It is not "always lower" than final recall (A is wrong: it is an upper bound), it is not a breadth measure (C is coverage), and improving it needs better retrievers, not a better ranker (D).

Q2.4 — Because AP should also punish positives that never appear. Missing positives add nothing to the numerator but still count in the denominator R. Dividing by the number of hits would reward a list that finds one item at rank 1 and ignores four others as much as one that finds all five.

Q2.5 — With an inner join (or a merge that starts from `recs`), future positives that were never recommended would disappear from the table. The number of rows per user would then be the number of hits instead of R, so recall would be hits/hits = 1 for everyone who has any hit, and users with no hits would vanish rather than score 0. The left merge from the labels keeps every positive in the denominator.

Q2.6 — `dropna(subset=["pos"])` removes the future positives that were not recommended in the top k (their `pos` is NaN after the left merge), so only hits contribute to DCG. The constant is `+ 2` because `pos` is zero-based while the formula uses the one-based rank i: $\log_2(i+1) = \log_2(\text{pos}+2)$. Using `+ 1` would divide by $\log_2 1 = 0$ at the first slot.

Q2.7 — DCG@3 = 4.131, IDCG@3 = 7.631, NDCG@3 = 0.541. DCG = $0/\log_2 2 + 1/\log_2 3 + 7/\log_2 4 = 0.631 + 3.5 = 4.131$. The ideal order puts the order first: $7/\log_2 2 + 1/\log_2 3 = 7 + 0.631 = 7.631$. The ratio is $4.131/7.631 \approx 0.541$.

Q2.8 — precision@5 = 0.4 and recall@5 = 0.5. Two hits in five slots gives $2/5 = 0.4$; two of the four future positives were found, giving $2/4 = 0.5$.

Q2.9 — NDCG is an average over users of ranking quality and does not see breadth or the pipeline ceiling. Coverage reveals a model that shows the same few bestsellers to everyone (good NDCG on a popularity-dominated week, tiny share of the catalogue ever shown). Candidate recall reveals that the first stage never retrieves many of the items users later buy, so the ranker has nothing to choose from and NDCG improvements from retuning it will be capped; NDCG alone does not tell you whether the bottleneck is retrieval or ranking.

Q2.10 — Coverage@20 should drop to about $20/2311 \approx 0.0087$, because only 20 distinct items are ever shown across all users (the number of available items is 2311 in the printed output). Accuracy numbers will differ from the random run, but they will not be personalised. The exercise shows that coverage and accuracy are separate axes: random had coverage 1.0 with near-zero accuracy, and a constant list has near-zero coverage.

Chapter 3 — Popularity baselines

Q3.1 — B. In the simulator each item has a fixed hidden quality, so popularity does not drift. When the underlying popularity is stable, the 53-day count is a less noisy estimate of it than the 7-day count. On real marketplaces with flash sales and seasonality the window often wins, so tune it on held-out data.

Q3.2 — C. With $v = 0$ the weight on the item's own rating, $v/(v+m)$, is zero, so the score equals the global mean C (4.324 in the printout). The stated 4.4 or 4.5 is just a placeholder written to new items. Smoothing treats it as "no information".

Q3.3 — B. Coverage@20 is the number of distinct recommended items divided by the available catalogue. A global ranking gives the same 20 items to everybody, so only 20 items are ever shown. This is the root of popularity bias and the feedback loop.

Q3.4 — Pass `num` = total stars (R · v), `den` = number of ratings v, `prior` = the global mean C and `strength` = m. The function then returns (R·v + C·m)/(v + m). Splitting the fraction gives v/(v+m)·R + m/(v+m)·C, which is the Bayesian average. The same call works for CTR (clicks, impressions, global CTR, pseudo-impressions).

Q3.5 — Items shown at the top get more impressions and clicks, which raises their popularity score, which keeps them at the top, while items never shown cannot collect evidence. Any of these weakens it: personalisation, exploration (ε-greedy, Thompson sampling), exposure-normalised rates such as smoothed CTR, position-bias correction or diversity re-ranking.

Q3.6 — No. The filter is `age_days < window_days`, strict, so an event with age exactly 7.0 is excluded. Every surviving interaction keeps its full weight `w` (1, 3 or 5) with no decay, because this is a hard window. Changing `<` to `<=` would include an event aged exactly 7 days; to include the whole 8th day you would call it with `window_days=8` (or use `<= 8`).

Q3.7 — About 4.98. Order: 5 · 2^(−7/7) = 5 · 0.5 = 2.5. Click: 1 · 2^(−14/7) = 0.25. Add-to-cart: 3 · 2^(−3/7) ≈ 3 · 0.743 = 2.229. Sum = 2.5 + 0.25 + 2.229 ≈ 4.98.

Q3.8 — With v = 20: 20/120·4.8 + 100/120·4.0 = 0.8 + 3.333 = 4.133. With v = 400: 400/500·4.8 + 100/500·4.0 = 3.84 + 0.80 = 4.64. More ratings move the estimate from the prior (4.0) towards the item's own mean (4.8).

Q3.9 — Half-life = 10 days / 2 = 5 days, because 25% = 2^(−2), i.e. two half-lives have elapsed at age 10. A smaller half-life concentrates the weight on the most recent events, so the score reacts quickly to new behaviour but rests on fewer effective events, which makes it noisier.

Q3.10 — With a very short half-life (2 days) pop_decay behaves like a short window: its behaviour should move towards the short-memory pop_7d row, since it rests on few effective events. With a very long half-life (30 days) the weights are close to 1 for the whole 53-day history, so pop_decay approaches pop_all_time. This matches the explanation that, with stable simulated popularity, more effective history means less noise. Exact values depend on your run; the ordering is what to check.

Chapter 4 — Content-based filtering

Q4.1 — B. IDF is large for rare terms and small for terms present in many documents, so a word like "sneakers" counts for more than "home". Length preference (A) is not a goal; normalisation actually removes it, and TF-IDF does not make vectors orthogonal or remove stop-words by itself.

Q4.2 — B. For unit vectors the denominator of cosine is 1, so cosine equals the dot product, which is why one sparse product `X @ X.T` yields all pairwise similarities. It is not Euclidean distance (although the two are monotonically related for unit vectors), it is non-negative for TF-IDF vectors, and normalisation removes the dependence on text length.

Q4.3 — C. The content-based scorer needs only the item's own attributes, which exist from the moment it is listed. Popularity, co-visitation and order-based methods all need interactions that a brand-new item does not have.

Q4.4 — The two items have identical text after tokenisation (same title, category and brand words), so their TF-IDF vectors are identical and the angle between them is zero. A content-only recommender cannot tell them apart and may fill the top slots with copies of the same product; breaking the tie needs behavioural signals such as price, rating, popularity or seller quality, or a diversity step.

Q4.5 — Over-specialisation means the system only recommends items that resemble the user's history, so it never surprises them or covers a new need. Remedies: blend with collaborative signals such as co-visitation, re-rank for diversity (MMR or category caps), or reserve slots for exploration. Any two of these are acceptable.

Q4.6 — Without the guard, a user with no positive events has `wsum == 0` and `1 / wsum` becomes infinity, so multiplying the all-zero row by it yields NaN. With the guard the divisor is 1, the row of `W @ X` stays all zeros, and `normalize` leaves a zero vector. The profile is therefore all zeros and every item scores 0, which is why the script adds a tiny popularity tie-break for these users.

Q4.7 — 0.571. Item A has weight $1 \times 2\hat{}0 = 1$. Item B has weight $3 \times 2\hat{}(-14/7) = 3 \times 0.25 = 0.75$. The share of A is $1 / (1 + 0.75) = 0.5714$, i.e. 0.571. The older cart event, despite its higher strength, contributes less than the fresh click.

Q4.8 — 0.48. The vectors are unit length, so cosine is the dot product: $0.6 \times 0.8 + 0.8 \times 0 + 0 \times 0.6 = 0.48$. They share only the first dimension, which is why the similarity is moderate rather than high.

Q4.9 — Expected observation: very short half-lives such as 1 day throw away most of the history, so profiles rest on a few events and quality typically drops; very long ones such as 30 days approach an unweighted average and may react slowly, so a moderate half-life near the baseline of 7 days is typically best. Your actual numbers may differ, because the simulator's interests are fairly stable. Whatever you observe, compare ndcg@20 and recall@20 to 0.0909 and 0.1378 and treat differences within a few thousandths with caution.

Chapter 5 — Collaborative filtering

Q5.1 — B. User-based CF assumes that people who agreed in the past will probably agree again, so the items liked by users with overlapping histories are good recommendations. It does not look at titles (A), it is not about popularity alone (C), and it does not assume static taste (D) — which is why time decay is used elsewhere in the book.

Q5.2 — C. Item-to-item CF computes the expensive item–item similar-items table offline. At request time it only looks up neighbours of the items in the user's history, so cost is independent of catalogue size and customer count. User-kNN must compare the target to all users per request.

Q5.3 — B. Jaccard only asks "present or not", which fits binary signals such as same-session membership and stops heavy users from dominating through counts. When weights carry information (order = 5 clicks), cosine on the weighted matrix keeps that information. Both measures are symmetric.

Q5.4 — Each row is normalised so that the outgoing weights of every source item sum to 1. Without it, pairs involving very popular items would have huge raw counts and would dominate every candidate list regardless of how strongly the items are actually related. Normalising makes weights comparable across source items and ranks neighbours of each item by relative strength; it also makes the matrix asymmetric.

Q5.5 — Both items have few interactions and a couple of shared users, so cosine is inflated by tiny support (the table's maximum similarity is 1.000, typically a single common user). Fixes: require a minimum number of common users (support threshold), shrink similarity by $n/(n+\lambda)$, or cap/sample the data. Any one of these is acceptable.

Q5.6 — It collapses all events of the same item inside one session into the single strongest weight, so refreshing a page ten times counts once. With `.sum()` an item clicked, carted and ordered in one session would count 1+3+5 = 9 instead of 5, and repeat-clicking loops or bots would inflate pair weights, distorting the intended strength scale.

Q5.7 — Cosine 0.632, Jaccard 0.333. The dot product is $2{\cdot}1 + 0{\cdot}1 + 1{\cdot}0 = 2$; the norms are $\sqrt{5} = 2.236$ and $\sqrt{2} = 1.414$, so cosine $= 2/3.162 = 0.632$. The user sets are X = {u1, u3} and Y = {u1, u2}: intersection 1, union 3, so Jaccard $= 1/3 = 0.333$.

Q5.8 — 0.55. Score(C) = Σ recency(seed) · w(seed→C) $= 0.5{\cdot}0.3 + 1.0{\cdot}0.4 = 0.15 + 0.40 = 0.55$. This is the sum in `covisit (cand_recent_score * w`, grouped and summed per user and candidate). Two seeds pointing to the same candidate reinforce each other.

Q5.9 — B. item_knn has coverage@20 0.8399 (highest), and ndcg@20 0.0925, below user_knn (0.1662) and covisit (0.1197). user_knn does not have the highest coverage (A false); covisit's recall@5 (0.0795) is below

user_knn's (0.1230) (C false); pop_7d's ndcg (0.0475) is below item_knn's (D false).

Q5.10 — In a scratch run, ndcg@20 went 0.1662 (k=50), 0.1702 (k=100), 0.1706 (k=200), 0.1700 (k=400) and recall@20 0.2273, 0.2419, 0.2409, 0.2405: gains stop after about k=100 and the curve is flat or slightly down. Beyond a point extra neighbours are only weakly similar users whose votes add noise rather than signal, and the similarity weights make them contribute little anyway.

Chapter 6 — Matrix factorisation

Q6.1 — B. A latent factor is one learned dimension of the user and item vectors. Nobody labels it in advance; the optimiser discovers it because giving every user and item a value on that dimension helps reconstruct the interactions. It is not a raw matrix column, a hyperparameter or a seller-entered attribute.

Q6.2 — B. p_{ui} is the binary target "did this user show interest" ($\mathbb{1}[R_{ui} > 0]$), and $c_{ui} = 1 + \alpha R_{ui}$ weights how much the squared error on that cell matters. Unobserved cells keep $c = 1$, so they are weak zeros, while orders and carts are strong positives.

Q6.3 — C. $Y^\top Y$ does not depend on the user, so it is computed once per sweep. The correction $Y^\top (C^u - I) Y$ has non-zero rows only for the n_u items the user touched, so the per-user cost scales with their history, not the catalogue. The loss is still not jointly convex, and the trick says nothing about regularisation or explicit ratings.

Q6.4 — Empty cells mostly mean "not seen", not "disliked". Fitting them as true zeros would claim that almost all of the catalogue is hated by every user and would push the model to predict near zero everywhere. Implicit ALS keeps them as zeros but gives them only the floor confidence $c = 1$, while observed cells get $c = 1 + \alpha R$, so the loss mainly cares about explaining the interactions we are sure about.

Q6.5 — With near-zero vectors every gap $x_u \cdot (y_i - y_j)$ is about 0. Then $\sigma(0) = 0.5$ and the loss $-\ln 0.5 = \ln 2 \approx 0.693$ for every triple. Seeing this value at epoch 1 is a sanity check that the model starts as an uninformed coin flip and the loss is wired correctly.

Q6.6 — $x_u = 23/34 \approx 0.676$. Confidences are $c = 1 + 4R = (5, 13)$. Numerator: $\sum c\,p\,y = 5 \cdot 1 \cdot 2 + 13 \cdot 1 \cdot 1 = 23$. Denominator: $\sum c\,y^2 + \lambda = 5 \cdot 4 + 13 \cdot 1 + 1 = 34$. Dividing gives 0.676.

Q6.7 — $\sigma(-\ln 3) = 1/(1+3) = 0.25$; loss $= -\ln 0.25 = \ln 4 \approx 1.386$; $s = \sigma(\ln 3) = 0.75$. The gap is negative, so the pair is ordered wrongly (the negative outscores the positive). The multiplier $s = 0.75$ is large, so the update is large, which is exactly the behaviour BPR wants for badly ordered pairs.

Q6.8 — `sig` equals $\sigma(-\text{gap}) = 1 - \sigma(\text{gap})$, which is $-\partial\,\text{loss}/\partial\,\text{gap}$. By the chain rule every gradient is `sig` times the derivative of the gap with respect to that parameter: $y_i - y_j$ for the user, x_u for the positive item, and $-x_u$ for the negative item. `gj` is negative because raising the gap requires lowering the negative item's score, so y_j moves away from the user vector. Pairs already ordered correctly have `sig` near 0 and barely change anything.

Q6.9 — Recall@20 only asks whether the user's future items appear anywhere in the top 20; NDCG also rewards placing them near the top. ALS finds more of the right items, but orders them less sharply than user-kNN. That profile suits a candidate generator (judged by recall) feeding a later ranking stage (judged by NDCG), such as the LightGBM ranker in this repo.

Q6.10 — Exact numbers depend on your run (the script is seeded and deterministic), so compare against the 0.1581 ndcg@20 for `als_implicit` in `results/ch06_mf.txt`. Expect `alpha=1.0` to change ndcg@20 noticeably, because with a small alpha observed and unobserved cells carry almost the same confidence and the model behaves differently; the direction is not guaranteed, so check it. Expect `factors=8` to drop well below the 32-factor run: 8 dimensions cannot separate the tastes spread over many categories and thousands of items. The final ALS loss is not comparable across `alpha` values because the loss itself changes with the weights. The lesson

is that sweeping hyperparameters against the evaluation labels tunes on the test set; use an earlier validation window and report the test week once.

Chapter 7 — Embeddings and two-tower retrieval

Q7.1 — B. Dividing cosines by a small τ stretches the gaps between scores, so the softmax becomes sharper and the loss concentrates on the top of the ranking, punishing hard negatives. A flatter softmax (A) is what a high temperature gives. The temperature does not change the embedding dimension or the batch contents.

Q7.2 — C. A batch is a random sample of positive pairs, so each item appears in proportion to its interaction count. Popular items therefore show up as negatives in many rows and receive more downward push, although serving scores all items uniformly. L2 normalisation fixes norms to 1, so it is not the cause.

Q7.3 — A. Item vectors depend only on item attributes, so they can be computed once and stored in an index. The score is an inner product, which is what ANN structures are built for. A tower that took the user's history as an item input (D) would break this.

Q7.4 — It subtracts $\log q_j$, the log of the probability that item j is drawn into the batch, from the logit of column j. Popular items (large q_j) get a small boost and rare ones a large boost, which cancels the extra push-down popular items get from being over-sampled as negatives. The correction is only used in training; at serving the plain dot product is used.

Q7.5 — It measures how many of the exact brute-force top-20 items the approximate search still returns. An ANN index only needs to approximate exact search, so fidelity to brute force isolates the index's error from the model's quality. NDCG would mix both and could not tell you which one lost accuracy.

Q7.6 — It replaces occurrences of the target item in the user's own history with the padding index, so the user tower cannot see the answer. If removed, the model could copy the target from the history, so training loss would fall faster and look better, but evaluation on the held-out week, where no such shortcut exists, would be worse. This is a small instance of target leakage.

Q7.7 — About 43,159 dot products, or about 0.43% of brute force. Centroid scoring costs 4,096; each cluster has $10{,}000{,}000 / 4{,}096 \approx 2{,}441$ items, so 16 probes scan $16 \times 2{,}441 = 39{,}062.5$. The total is $4{,}096 + 39{,}062.5 = 43{,}158.5$, and $43{,}158.5 / 10{,}000{,}000 \approx 0.43\%$, roughly 230 times less work.

Q7.8 — By $\ln(0.04 / 0.0004) = \ln 100 \approx 4.605$ logit units. The correction adds $-\ln q$: $-\ln 0.04 = 3.219$ for the popular item and $-\ln 0.0004 = 7.824$ for the rare one, and the difference is 4.605. In probability terms, the rarer item's exponentiated score is multiplied by 100 relative to the popular one.

Q7.9 — Expected observations (verify on your run): the spec `two_tower` stays well below `two_tower_logq` in ndcg@20 at every temperature, because the popularity bias comes from the sampling scheme, not the temperature. Temperature changes how sharp the loss is, so ndcg and especially coverage@20 will shift between runs; a very low value such as 0.02 tends to be harder to optimise. The point of the exercise is that the logQ gap persists and temperature is a tuning knob, not the cure. Restore results with `git checkout book/results`.

Chapter 8 — Multi-source candidate generation

Q8.1 — B. An item missing from the candidate set can never reach the ranker, so the first stage is judged on how many future positives it contains. False positives are cheap because the second stage, with richer features, removes them. The ranker does not need a clean input (D is false), and precision can be computed on implicit feedback (A is false).

Q8.2 — C, `catpop`. Its unique pair recall is 0.0968, against 0.0524 for `covisit`, 0.0298 for `recent` and 0.0253 for `global_trending`. It also has the highest standalone recall (0.3274), because it covers users through both clicks and searches.

Q8.3 — B. The positive rate rises from 0.0090 to 0.1817 as more sources agree, so the number of sources and each source's score are informative features. Dropping single-source candidates (A, C) would throw away most of the recall, since 76,612 candidates have a single source. Counting sources cannot replace a ranker (D), which also uses user, item and cross features.

Q8.4 — C. Only `global_trending`. `recent` is empty without history, `covisit` is seeded from `recent`, so it is empty too, and `catpop` needs category affinity from clicks or searches. The outer join only fills missing score columns; it does not invent candidates (A, D).

Q8.5 — The union contains 103.94 candidates per user on average, so cutting the ranked list at K=100 drops the candidates at the end of the longer lists, and some of those are positives. The ceiling of 0.4645 is the recall of the whole candidate set with nothing cut. Recall@K can only reach it when K is at least the size of the pool for every user.

Q8.6 — Hiding is explicit negative feedback, and a learned feature only changes probabilities: the model could still show the hidden item occasionally. A hard filter on the pool guarantees it never reappears, is cheaper (fewer rows to score) and is easy to audit. Users tolerate an irrelevant recommendation but not one they have explicitly rejected.

Q8.7 — It would be 3 instead of 1. After the swap the three score columns are already filled with 0.0, so `notna().sum(axis=1)` counts all three of them for every row, and the trending term adds nothing because the item is not in that list. The feature would no longer separate single-source from multi-source candidates and would lose the signal that raised the positive rate from 0.0090 to 0.1817.

Q8.8 — The out-of-stock items in the top 30 would be removed after `nlargest`, leaving fewer than 30 trending items, and the list would shrink whenever a popular item sells out. Filtering first and then taking the top 30 always returns 30 in-stock items (if enough exist).

Q8.9 — 12.1880 + 35.8321 + 54.9102 + 30.0000 = 132.9303, minus the union size of 103.9366 gives 28.9937, i.e. about 29.0 duplicate (user, item) proposals per user, which is 28.9937 / 132.9303 = 21.8% of the sum. That is the overlap between sources, which is what `cand_n_sources` counts.

Q8.10 — Expect `catpop cands_per_user` to rise (more categories joined, up to 5 × 20 items per user instead of 3 × 20) and its recall to rise slightly, since a user's future positives sometimes lie in the 4th and 5th categories. The union's candidates per user and recall rise a little, with diminishing returns because the extra categories have a low affinity share. The ranker then has to score more rows per request, which increases latency, and most of the added candidates are negatives, so the ranker's job of separating positives from negatives gets harder. The results are overwritten by the run, hence the `git checkout` to restore them.

Chapter 9 — Feature engineering for ranking

Q9.1 — B. Point-in-time correctness means every feature for a training example is computed only from information that existed before that example's cutoff time. Normalisation, equal event counts and refresh frequency are unrelated engineering choices. Violating the rule is called leakage and inflates offline metrics.

Q9.2 — B. Events stamped exactly at the cutoff belong to the label window `[cutoff, cutoff + days)`, which `data.labels` selects with `ts >= cutoff`. The strict `<` in `History.at` therefore puts each event on exactly one side of the boundary, so no event is both a feature input and a label.

Q9.3 — B. With smoothing, the single impression and click are blended with `s` imaginary impressions at the global rate, so the estimate lands close to `p0`. It moves away only as real impressions accumulate and outweigh the prior. The rate still depends on the item's own clicks; it just trusts them in proportion to the evidence.

Q9.4 — Zero would mean "touched just now", which is the strongest possible recency signal, so it would confuse "never" with "this very minute". A large sentinel such as 9999 lies far outside the real range, so a tree can separate "never touched" from every real value with a single threshold. (Linear and neural models need an explicit missing flag instead.)

Q9.5 — An item feature describes the item for every user (e.g. `item_ctr_30d`). A user feature describes the user regardless of the item (e.g. `user_clicks_30d`). A cross feature is an aggregate over a pair of keys (e.g. `ui_clicks`, `uc_click_share_30d`, `us_orders`). Cross features carry most of the personalisation, because only they differ between two users for the same item. Item features mostly carry quality signal.

Q9.6 — Yes, the test would fail. The test rewrites every post-cutoff event into an order of item 0 in the tampered tables. A feature reading the full unfiltered table would therefore change its value (item 0 would gain many orders, and other items would lose theirs), so the rebuilt frame would differ from the original. `pd.testing.assert_frame_equal(before, _build(tampered, cutoff))` compares the two frames cell by cell and raises as soon as one differs.

Q9.7 — Smoothed CTR = (3 + 20·0.10) / (10 + 20) = 5/30 ≈ 0.167. The raw CTR is 3/10 = 0.30. Smoothing pulls the estimate more than half-way back to the global 0.10 because the item has only 10 impressions against a prior strength of 20.

Q9.8 — item_trend = (6/3 + 0.1) / (14/14 + 0.1) = 2.1 / 1.1 ≈ 1.91. The value is above 1, so the item is heating up: the last three days average 2 clicks a day against 1 click a day over the last two weeks.

Q9.9 — Expected: with a 1-day leak window the leaked feature covers only a seventh of the label window, so most positives get a zero value and the leak weakens; NDCG@20 of the leaky ranker should fall below the 0.5035 of the 7-day leak, and the gain share of `leaky_future_clicks` should fall too. It will most likely still stay clearly above the honest 0.2116, and the feature will still rank near the top. Even a small slice of the future is enough to make the offline metric untrustworthy, which is why the only safe rule is a hard cutoff with no exceptions.

Chapter 10 — Learning to rank

Q10.1 — B. Pointwise losses score each row independently, pairwise losses compare two items from the same user, and listwise losses look at the whole ranked list. Model family (trees, networks) is orthogonal to this distinction, and all three can use graded or binary labels.

Q10.2 — C. Each new tree is fit to the residuals, which for squared error equal the negative gradient of the loss at the current predictions. The prediction is then updated by η times the tree's leaf values.

Q10.3 — D. The NDCG change from swapping a pair is large near the top and for big gain gaps, so those pairs get large lambdas. The multiplication does not make NDCG differentiable; it replaces the gradient by a hand-defined one.

Q10.4 — LightGBM's `group=` argument is a list of consecutive group sizes, not user ids. If a user's rows are not adjacent, the boundaries cut across users, so "queries" mix different users' rows and the pairs and NDCG are computed over meaningless lists. There is usually no error, only a worse model, which makes the bug hard to notice.

Q10.5 — Few positives per list (about a hundred candidates, few actions) give list-aware losses little extra signal over a simple binary label; graded labels (orders) are rare and noisy; and the candidate recall ceiling squeezes differences between good rankers. The gap is about one seed standard deviation (0.2147 vs 0.2125 mean ndcg@20, std 0.0014 and 0.0021). I would switch only if a larger-data offline test with many seeds, or an online A/B test, showed a consistent advantage.

Q10.6 — `pred_contrib=True` returns one column per feature plus a final bias column (the baseline score). Slicing `[:, :-1]` removes the bias so only features remain. Without it the bias column, a large constant like −2.9, would often be the largest absolute value and be reported as a "feature" at an index beyond the feature names, causing an index error or a bogus top explanation.

Q10.7 — 0.393. Gains are 1, 0, 7; IDCG = 7·1 + 1·0.6309 + 0 = 7.631. Swapping positions 1 and 3: gain gap 7 − 1 = 6, discount gap 1 − 0.5 = 0.5, so the DCG change is 3.0 and 3.0/7.631 = 0.393.

Q10.8 — About +1.47. Contributions plus bias equal the score, so the total contribution is 1.277 − (−2.917) = 4.194. Subtracting the eight shown (2.726) leaves 1.468, which rounds to +1.47 (±0.003 from rounding).

Q10.9 — C. The drop is about 1.6 noise standard deviations, which is suggestive, not definitive. A fixed "exact effect" (A) ignores seed noise, the 0.01 threshold (B) is arbitrary, and the negative sign (D) is exactly what a useful group would produce when removed.

Q10.10 — Expected (hypothesis, to be checked by running): making orders worth the same as clicks removes the extra reward for the strongest actions, so the model may rank carted and ordered items less sharply and `order_recall@20` could drop. Whether ndcg@20 changes by more than the seed standard deviation of 0.0021 is what you must judge from your run; a smaller change is noise. The exercise teaches that `label_gain` encodes business value and that a change in it must be judged against seed noise.

Chapter 11 — Re-ranking, diversity and business rules

Q11.1 — B. With $\lambda = 1$ the term $(1 - \lambda) \max \mathrm{sim}$ is zero, so every step picks the highest remaining relevance. The result is the original ordering; this is why the `MMR lambda=1.0` row in the results equals the unmodified LambdaMART baseline.

Q11.2 — C. Re-buying coffee or diapers is the normal behaviour that the recommender should encourage, while a user who already owns a refrigerator does not need that exact one again. The rule therefore drops an item only when `ui_orders > 0` and `item_is_consumable == 0`.

Q11.3 — C. Thompson sampling draws from each item's posterior, so items with little data have wide distributions and sometimes win a slot, whereas well-measured poor items almost never do. Exploration therefore shrinks automatically as evidence accumulates, with no fixed ε.

Q11.4 — Rules should be explicit, time-boxed and auditable, which you cannot guarantee inside a trained model. If boosted or sponsored impressions enter the training data as ordinary evidence, the next retraining bakes the campaign into the model permanently and distorts relevance estimates. In the re-ranker a boost is a visible term with a start and end date that can be switched off and tested without retraining.

Q11.5 — The caps remove items and do not backfill, so for users whose candidate pool is concentrated in a few categories or shops the surviving list can be shorter than 50, and deep recall falls. `recall@5` is nearly unchanged because the top few items rarely hit a cap. (This is the most likely explanation; the script does not log list lengths.)

Q11.6 — 8 items, and the per-category cap is responsible. All 30 candidates share one category, so only those whose position within the category is below `MAX_PER_CATEGORY = 8` survive; the shop cap (at most 3 items per shop here, under 4) removes nothing, and `head(20)` has fewer than 20 rows to keep.

Q11.7 — Order: item 1, item 3, item 2. Step 1 scores are $0.5\times$ rel $= 0.45, 0.40, 0.25$, so item 1 (0.9) is chosen. Step 2: item 2 scores $0.5 \cdot 0.8 - 0.5 \cdot 1 = -0.10$ (same category as item 1), item 3 scores $0.5 \cdot 0.5 - 0 = 0.25$, so item 3 is chosen and item 2 comes last.

Q11.8 — Beta(6, 16), mean $6/22 \approx 0.273$, standard deviation ≈ 0.093. The variance is $ab/((a+b)^2(a+b+1)) = 96/(484 \cdot 23) \approx 0.00862$. A new item has Beta(1, 1) with standard deviation ≈ 0.289, about three times larger, so its samples vary far more and it will often draw high enough to win a slot until data narrows it.

Q11.9 — A. The results show -0.0035 ndcg and $+1.45$ categories at $\lambda = 0.9$, and -0.0715 ndcg at $\lambda = 0.3$. B is false (coverage@20 fell from 0.4245 to 0.3129), C is false (the filter alone cost 0.0007), and D is false (the caps raised categories from 5.33 to 6.58).

Q11.10 — Expected (not run here): with sibling categories no longer penalised, items from the same parent group stop being pushed down, so ndcg@20 at $\lambda = 0.5$ should be higher than the 0.1603 in the table, while the mean distinct categories can stay high, since distinct categories are still counted per category. The page may however concentrate in fewer parent groups (for example only Pets). Compare your measured numbers with this prediction and with the baseline row.

Chapter 12 — Cold start, context and bias

Q12.1 — B. `global_trending` takes no user argument and `generate` cross-joins its result onto every user id, so even a user with no history, or the anonymous `user_id = 0`, receives the globally trending items. It is not session-based, it runs on every request rather than only on failure, and it ranks by weighted recent interactions, not discounts.

Q12.2 — C. With $x = 0$ and $n = 0$ the formula reduces to $p_0 s / s = p_0$, the catalogue-average CTR. This treats a new item as average until evidence says otherwise, instead of as zero or undefined. Smoothing therefore removes the penalty for having no data.

Q12.3 — C. The ranker predicts what a user will do in the next 7 days, with labels aggregated over a week. The request hour is identical for every candidate in a user's group and has no per-request label to learn from. Context belongs in a second, real-time model trained on the impression log.

Q12.4 — The ranker is trained on clicks, so an item with a few more clicks gets a higher score and is shown more. More exposure produces more clicks, which widens the gap, while items never shown never get a chance to prove themselves. Techniques that reduce the loop include exploration (epsilon-greedy or Thompson sampling, reserving slots for unproven items), diversity re-ranking, and logging impressions with positions so propensity weighting can correct for exposure. Any two of these are acceptable.

Q12.5 — A slow recommender harms users just as a dead one does, so a timeout bounds the worst-case wait to five seconds and then serves something useful. The JSON carries `"fallback": true` and `"model_version": "fallback"`. The frontend reads the flag and shows the "Recommender offline" banner.

Q12.6 — Q ranks higher. The query orders first by the count of `order` events in the last seven days descending (Q has 1, P has 0) and only uses `rating_count` to break ties. The final `i.id` makes the order deterministic when both earlier keys tie, so repeated requests return the same list.

Q12.7 — Raw CTR: A is $40/200 = 0.20$ and B is $20/200 = 0.10$. Debiased relevance: A is $0.20/0.8 = 0.25$ and B is $0.10/0.2 = 0.50$ (equivalently, each click on A is weighted 1.25 and each click on B is weighted 5). B is twice as relevant as A even though its raw CTR is half. The raw numbers were misleading because B was shown in a position that few users examine.

Q12.8 — Hands-on. Expect the new "0 clicks" bucket to contain far fewer users than the other buckets, so its metrics are noisy and the `LambdaMART / pop_7d` ratio there should be trusted less than the ratio for the large bucket. With no history, the ranker can only use trending candidates and item-level features, so look for whether its advantage over `pop_7d` shrinks relative to the 5.22x of the heavy-user bucket. If the bucket is empty, the script's evaluation users all have at least one prior click and the row will not appear. The point is that cold cohorts are small and noisy.

Q12.9 — B. A click at a position with a tiny propensity gets weight $1/\theta_k$, which can be dozens or hundreds, so one lucky click can dominate training. Clipping bounds the weight, trading a small bias for a large variance reduction. Propensities are probabilities and cannot be negative, weights need not sum to one, and LightGBM accepts large weights.

Chapter 13 — Building the system

Q13.1 — B. A read-only default transaction means any `INSERT`, `UPDATE` or `DELETE` from the recommender fails with `ReadOnlySqlTransaction`. This protects the system of record from bugs in feature or model code and keeps the Rust API as the single writer. It does not speed up reads and has nothing to do with gRPC.

Q13.2 — A. The browser may report only `impression`, `click` and `hide`, which are observations of the shopper's own screen. `order` (and `add_to_cart`) are consequences of authenticated server-side actions, so the API writes them itself; accepting them from the client would let anyone forge strong positive signals for any item.

Q13.3 — B. The tracker's `pagehide` listener was registered first, so it emptied the queue and sent the beacon before the product page's own listener queued the click. The click was added after the last flush opportunity and was lost. The fix is the `onPageHide` hook registry: hooks run first, then the queue is drained and beaconed.

Q13.4 — Auto-increment ids are assigned at insert time but rows become visible at commit time. If a transaction holding id 500 commits after one holding id 501, a refresh that already saw 501 would never fetch 500 with `id > last`. Re-reading the last 1000 ids and de-duplicating by `event_id` catches such late commits at the cost of a few redundant rows.

Q13.5 — Each `Recommend` call copies the references to the current booster and version into local variables at the start, so it keeps a consistent model/version pair for its whole duration. `retrain` builds the new booster separately and then swaps both attributes with plain reference assignments. In-flight requests finish with the old model, new requests use the new one, and no lock is needed on the read path.

Q13.6 — Two sessions. The 10:00 event creates session A; at 10:20 the latest session was seen 20 minutes ago (inside the 30-minute window), so A is reused and its `last_seen_at` becomes 10:20. At 11:05 the gap since 10:20 is 45 minutes, which exceeds 30, so a new session B is inserted. The first session ends the day with `last_seen_at` = 10:20.

Q13.7 — About 20.2% of the budget and 3.985 s. The fraction is 1010.6 / 5000 = 0.2021. The worst-case staleness is 2 + 1 + 0.985 = 3.985 seconds (flush delay + refresh throttle + request latency).

Q13.8 — It returns active, in-stock items ordered by the number of `order` events in the last 7 days, then by rating count. Items are labelled `source: "trending"` with score 0 and no contributions because they come from a non-personalised popularity query and no model scored them; this lets the frontend reuse the same card rendering. The response also carries `fallback: true` and `model_version: "fallback"` so the UI can show the offline banner.

Q13.9 — B. Protobuf identifies fields on the wire by number, not by name, so two sides with different numbering for the same field disagree about which bytes mean what and data is dropped or misread without a compile error. That is why a field number must never change or be reused once the message is in use; delete fields by reserving their numbers instead.

Q13.10 — Expect `p50` latency to change very little (all runs stay in roughly the same band as the k=20 run), and the candidates-per-request line to be identical in all three runs, because `candidates.generate` and `features.build` do not depend on `k`; only the final re-rank cut and the number of SHAP rows change. This indicates that the time is dominated by fixed per-request work (history and feature building), not by scoring or by response size.

Chapter 14 — From prototype to Shopee scale

Q14.1 — B. Skew means the feature is legitimate in both places but computed differently (different code, different filters such as bot traffic), so the model learns on one distribution and is queried with another. Dataset size, retrain frequency and ANN approximation are separate issues. A feature store with one definition and logging the served features are the standard defences.

Q14.2 — B. DIN's local activation unit weights each past behaviour by its relevance to the candidate being scored, so the user's interest vector changes with the candidate. Option A describes the Embedding-and-MLP recipe that DIN was designed to improve on. The other options describe unrelated components.

Q14.3 — A. Each task has its own softmax gate over a shared pool of experts, so related tasks can pick similar experts and conflicting tasks can specialise. Gates do not change the datasets, equalise the losses or cache anything.

Q14.4 — Users experience the slowest part of a request, and a page that fans out to many services is as slow as the slowest one, so tail latency (p99) decides the experience; the median hides the slow requests (here 985.0 ms

at p99 versus 839.9 ms at p50). The Rust API gives the gRPC call a 5 second timeout; on error or timeout it returns a popularity-based fallback list with `fallback: true` instead of failing the page.

Q14.5 — Offline numbers come from logs of a previous system, so they can be inflated by leakage (the leaky ranker scored 0.5035 versus 0.2116 for the honest one, and would be useless live). They also inherit position bias and ignore revenue, diversity and seller effects. Another valid reason: selection bias and feedback loops (the model is only scored on items the old system showed). Only a randomised A/B test measures the real effect.

Q14.6 — Pooled proportion $\hat{p} = (5000 + 5150)/100000 = 0.1015$. SE $= \sqrt{0.1015 \times 0.8985 \times 2/50000} \approx 0.001910$. The difference is $0.1030 - 0.1000 = 0.0030$, so $z = 0.0030/0.001910 \approx 1.57$. The two-sided p-value is about 0.116, which is above 0.05, so the lift is not statistically significant: the data cannot rule out zero effect, and the test was probably underpowered.

Q14.7 — Raw: $2 \times 10^9 \times 128 \times 2 = 5.12 \times 10^{11}$ bytes $= 512$ GB. Compressed: $2 \times 10^9 \times 32 = 6.4 \times 10^{10}$ bytes $= 64$ GB. Product quantisation therefore gives an 8x reduction here, at the price of some recall, which is tuned with the number of probes and re-scoring.

Q14.8 — Base $= 1.0 \times 0.2 + 2.0 \times 0.05 + 3.0 \times 0.01 = 0.2 + 0.10 + 0.03 = 0.33$. With the boost, $0.33 \times (1 + 0.1) = 0.363$ (no ads term). With boost 0 the score is 0.33. The boost multiplies the whole blend, so a campaign lifts items proportionally to how good the model already thinks they are.

Q14.9 — B. With millions of users, a 0.6-point deviation from 50/50 is far outside chance and indicates a sample ratio mismatch: a bug in randomisation, logging, bot filtering or a redirect that loses one arm. Results from such a test are not trustworthy until the cause is found.

Q14.10 — Expect the $\lambda = 0.8$ row to lie between the 0.9 and 0.7 rows on both measures: ndcg@20 between about 0.2081 and 0.1884, and more distinct categories than at 0.9 but fewer than at 0.7 (between about 6.78 and 11.03). The monotone trade-off supports the claim: every step towards more diversity costs relevance, so the diversity weight is a product decision to be tested online.

Appendix B — Glossary

Terms are listed alphabetically. Each entry gives a plain-language meaning, sometimes with a tiny example. The chapter where a term is introduced and explained in full is in brackets.

Term	Meaning
A/B test	An experiment where a random half of shoppers see the old system (control) and the other half see the new one (treatment), and a statistical test tells us whether the difference in a metric is real or just luck. [14]
Ablation	Retraining a model with one group of features removed to see how much worse it gets; the drop shows how much that group was worth. [10]
Add-to-cart	A logged event for a shopper putting an item in the cart; it shows stronger interest than a click. [1]
ALS (Alternating Least Squares)	A way to learn taste cards (see *Matrix factorisation*): fix the item cards and solve for the best user cards, then fix the user cards and solve for the items, and repeat. [6]
ANN (Approximate Nearest Neighbour)	A search that finds items that are *almost* the closest to a query, much faster than comparing against every single item. [7]
AP (Average Precision)	A score for one list: at every rank where a relevant item appears, note the share of relevant items so far, then average those shares. [2]
Bayesian average	A rate pulled toward the overall average until an item has enough data, so 1 click out of 1 view is not treated as a perfect 100%. Also called *smoothing*. [3]
Beacon	A browser feature that hands a small message to the browser to deliver after the page has closed, like dropping a letter in a postbox on your way out. [13]
Beta distribution	A curve that shows how plausible each click rate is for an item, given its clicks and non-clicks so far; it is narrow when we have lots of data and wide when we have little. [11]
BPR (Bayesian Personalised Ranking)	A training goal that says: for each shopper, score an item they interacted with higher than one they did not. [6]
Candidate	An item that a first-stage retriever proposes for a shopper; only candidates are scored by the ranker. [8]
Candidate recall	The share of the items a shopper will actually engage with that made it into the candidate list. It is the recall ceiling: the ranker cannot recommend what was never proposed. [8]
Co-visitation	Counting how often two items are used in the same session; it powers "people who viewed this also viewed that". [5]
Cold start	A new shopper or item with little or no history, so there is nothing yet for a personal method to learn from. [12]
Collaborative filtering	Recommending from the behaviour of many shoppers (similar shoppers, or items used together), without reading what the items are. [5]
Content-based filtering	Recommending items whose descriptions (text, category, brand) look like items the shopper already liked. [4]
Cosine similarity	A score from 0 to 1 for how alike two vectors are, based on the angle between them: 1 means the same direction, 0 means nothing in common. [4]
Coverage	The share of the catalogue that appears in anyone's top K; low coverage means many items never get shown. [2]
CSR (Compressed Sparse Row)	A way to store a mostly empty table by keeping only the non-empty cells and where they sit, row by row. [5]
CTR (Click-through rate)	Clicks divided by impressions: out of 100 views, 5 clicks is a CTR of 5%. [1]
Cross feature	A feature about a pair, such as *this shopper* × *this item*, rather than about the shopper or the item alone. [9]

Term	Meaning
Cutoff	The moment we freeze time at: features may use only events before it, and what happens after it is the answer key. [2]
DCG / NDCG	DCG gives credit for each relevant item, with less credit the further down it sits ($1/\log_2(\text{rank}+1)$); NDCG divides by the best score possible, so it runs from 0 to 1. [2]
Decay (half-life)	Counting older events for less. With a 7-day half-life, an action a week old counts half as much as one today. [3]
Dense	Stored in full, every cell written down even when it is zero; the opposite of *sparse*. [6]
Dot product	Multiply two lists of numbers position by position, then add the results: $(1,2)\cdot(3,4)=1\times3+2\times4=11$. A bigger result means the two lists point the same way. [4]
Dwell time	How many seconds a shopper stays on a product page; longer usually means more interest. [1]
Embedding	A short list of numbers learned for a shopper or an item, arranged so that similar things get similar lists. [7]
Epoch	One full pass through all the training data. [6]
Event	One logged action by a shopper on an item: impression, click, add-to-cart, order, hide or return. [1]
Explicit feedback	Ratings or reviews that shoppers give on purpose. [1]
Exploration	Deliberately showing items we are unsure about so we can learn about them, at a small short-term cost (ε-greedy and Thompson sampling are two ways). [11]
Exposure	Whether, and how often, an item was actually shown to a shopper; an item never shown cannot get clicks. [1]
Fallback	What the system shows when the recommender is unavailable or the shopper is unknown; here, recent best sellers. [12]
Feature	A number that describes a shopper–item pair at one moment, such as `user_clicks_30d`, and that the model reads as input. [9]
Feature store	A shared place that defines each feature once and serves the same numbers to training and to live requests. [14]
Feedback loop	The recommender decides what is shown, what is shown decides the clicks, and the clicks become its next training data, so it can end up reinforcing its own habits. [12]
Gradient	For each number the model can change, a signal that says which direction (up or down) makes the loss smaller, and by how much. [6]
Gradient boosting	Building a model from many small decision trees, each one trained to fix the mistakes left by the trees before it. [10]
Guardrail metric	A number that must not get worse (such as coverage or page speed) even while we try to improve the main metric. [2]
Hard filter	A rule that removes candidates outright, such as out of stock or hidden by the shopper. [8]
Hit rate@K	The share of shoppers for whom at least one of the top K items was a hit; if 4 of 10 shoppers found something, it is 0.4. [2]

Term	Meaning
HNSW (Hierarchical Navigable Small World)	An ANN method that links each vector to its neighbours in a layered graph and finds matches by walking along the links. [7]
Hyperparameter	A setting you choose before training (such as the embedding size or the window length in days), as opposed to a parameter that training learns. [3]
Impression	A moment when an item was shown on screen. [1]
Implicit feedback	Behaviour that hints at interest without being a rating: views, clicks, purchases. [1]
In-batch negatives	When training on a batch of examples, treating the other shoppers' positive items in that batch as the "wrong answers" for each shopper. [7]
IPW (Inverse propensity weighting)	Giving each logged click more weight when it was unlikely to be seen in the first place (weight = 1 ÷ chance of being shown), to correct for position and exposure bias. [12]
Item-to-item	Recommending items that are similar to the ones a shopper used, using a table of how alike item pairs are. [5]
IVF (Inverted file index)	An ANN method that sorts vectors into groups and searches only the few groups nearest to the query. [7]
k-means	A method that sorts points into k groups: pick k centres, assign each point to its nearest centre, move each centre to the average of its points, and repeat. [7]
Label	What actually happened after the cutoff (0 nothing, 1 click, 2 cart, 3 order); the answer the model learns to predict. [9]
Label window	The stretch of time right after the cutoff (here, one week) whose events become the labels; nothing from it may be used for training or features. [2]
LambdaMART	A gradient-boosted ranker whose training signal is larger for pairs of items where swapping them would change NDCG a lot. [10]
Latency, p50, p99	Latency is how long one request takes. p50 is the typical (median) time; p99 is the time that 99 of every 100 requests beat, so it shows the slow ones. [13]
Latency budget	The time each stage of a request is allowed to take so that the whole response arrives before its deadline. [14]
Latent factor	A hidden taste dimension (such as "strong and black" versus "sweet and milky") that nobody labelled but the model discovers from the data. [6]
Leakage	Using information from after the cutoff when building features; it is cheating, and it inflates offline scores. [9]
Learning rate	How big a step the model takes each time it adjusts its numbers; too big overshoots, too small is slow. [6]
Learning to rank	Training a model to put items in the right order for a shopper, rather than to predict each item's value on its own. [10]
Logit	A raw score before it is turned into a probability. [7]
logQ correction	Subtracting the log of how often an item is picked as a negative from its score, so popular items are not unfairly punished when training with in-batch negatives. [7]
Long tail	The many items that each get little attention but together make up a large share of the catalogue. [1]
Loss	A single number for "how wrong is the model right now"; training tries to make it small. If it predicts 3.5 and the truth is 5, the error behind the loss is 1.5. [6]

Term	Meaning
Matrix factorisation	Summarising the big shopper × item table as two thin tables of short "taste cards", one per shopper and one per item, whose dot product guesses the missing cells. [6]
MMR (Maximal Marginal Relevance)	A re-ranking rule that picks the next item by balancing how relevant it is against how similar it is to items already chosen. [11]
MRR (Mean Reciprocal Rank)	For each shopper take 1 ÷ the rank of their first hit (rank 1 gives 1, rank 2 gives 0.5, rank 4 gives 0.25), then average over shoppers. [2]
Multi-task learning	One model that predicts several outcomes (click, cart, order) at once, sharing its inner layers. [14]
Overfitting	A model memorising its training data, like a student who memorises last year's exam answers and then fails this year's exam. [6]
PCA (Principal Component Analysis)	A way to squash many-number vectors down to two numbers by keeping the directions along which the points spread out most, so they can be drawn on a page. [7]
Point-in-time correctness	Computing every feature using only the data that existed at its cutoff, as if we were standing in that moment. [9]
Popularity bias	Recommenders favouring items that are already popular, which then get more exposure and become even more popular. [3]
Position bias	Items shown higher on the page get more clicks even if they are no better. [12]
PQ (Product quantisation)	Compressing a vector by cutting it into short pieces and replacing each piece with the number of its closest entry in a small palette, like painting by numbers. [14]
Precision@K / Recall@K	Precision@K is the share of the top K that is relevant; recall@K is the share of all the relevant items that landed in the top K. [2]
Profiling	Measuring where a program spends its time, stage by stage, so that you speed up the part that is actually slow. [13]
Propensity	The chance that an item was shown to a shopper in the first place. [12]
Query group	All the rows that belong to one shopper's list; ranking is compared only inside a group. [10]
Recall ceiling	See *Candidate recall*. [8]
Recency	How recently something happened; newer events usually matter more. [4]
Regularisation	Adding a penalty for large numbers to the loss so the model stays simple and does not overfit. [6]
Re-ranking	A final step that applies page-level rules (limits per category, removing already-bought items) to the model's scores. [11]
Retrieval	The first stage: cheaply picking a few hundred candidates out of the whole catalogue. [8]
Sentinel value	A deliberately extreme number (such as 9999) that means "this never happened". [9]
Session	One burst of shopper activity; here a gap of 30 minutes starts a new session. [13]
Shard	One slice of the catalogue kept on its own machine, like one branch of a library; a query asks every shard and merges the answers. [14]
SHAP	A method that splits one prediction into a contribution from each feature, and those contributions add up to the model's output. [10]

Term	Meaning
Smoothing	See *Bayesian average*. [3]
Softmax	A recipe that turns any list of scores into probabilities that add up to 1, giving bigger scores bigger shares. [7]
Sparse matrix	A big table in which almost every cell is empty, so we store only the non-empty cells. [4]
Sparsity	The share of the shopper × item table that is empty; usually above 99%. [1]
SRM (Sample ratio mismatch)	The two groups of an A/B test came out a different size than planned (for example 50.6/49.4 instead of 50/50 on millions of users), which signals a bug, so the results cannot be trusted. [14]
Temperature	A dial in softmax: a low temperature makes the model sharper and more confident, a high one makes it flatter and more cautious. [7]
TF-IDF	A word weight: high for a word that is common in one description but rare across all descriptions, so it describes that item. [4]
Thompson sampling	An exploration method: draw one random plausible click rate for each item from its Beta distribution and show the items with the highest draws. [11]
Training/serving skew	A mismatch between how features are computed for training and how they are computed live, which quietly hurts the model. [14]
Two-tower model	A retrieval model with one small neural network for shoppers and one for items, whose output vectors are compared with a dot product. [7]
TTL (Time to live)	How long a saved (cached) answer may be reused before it must be recomputed. [14]
Vector	A list of numbers in a fixed order; we turn each product or shopper into one so a computer can compare them. [4]
Weak learner	A very simple model, such as a tree with one question and two answers, that is only slightly better than guessing and is useful only in large numbers. [10]

Appendix C — Reproducing every result

Every number, chart and screenshot in this book is produced by code in this repository. This appendix lists the exact commands, in order. All paths are relative to the repository root.

1. Tools

Tool	Used for
PostgreSQL at `localhost:5432` (user `postgres`, password `postgres`)	the marketplace database (chapter 13, screenshots, latency)
Rust toolchain (`cargo`) and `protoc`	the API server in `backend/`
Node.js and `npm`	the web app in `frontend/`, and mermaid/KaTeX for the PDF build
`uv`	every Python environment: `recommender/` and `book/`

2. The offline companion scripts (chapters 1–12)

These need nothing but Python. They all use the same simulated marketplace (seed 7), cached in `book/build/sim.pkl` on first use.

```
cd book
uv sync                                  # installs recsys from ../recommender in editable mode
uv run python tools/run_all.py --skip-live
```

`run_all.py` runs `code/ch01_explore.py` … `code/ch12_coldstart_bias.py` in order. Each script prints the text quoted in its chapter, which is saved to `results/<script>.txt`, and writes its numbers to `results/<script>.json`. The run stops at the first script that fails. To re-run a single chapter's script:

```
uv run python tools/run_all.py --only ch06_mf
```

The scripts are deterministic: the same machine gives identical files. Across machines, timings (milliseconds per query in chapter 7) differ, and LightGBM and PyTorch may differ in the last decimal places.

3. The live system (chapter 13)

One-time setup:

```
PGPASSWORD=postgres createdb -h localhost -U postgres marketplace
cp backend/.env.example backend/.env
(cd backend && cargo run -- migrate)
(cd recommender && uv sync && uv run recsys seed-db --yes)
(cd frontend && npm i)
```

Then start the three services, each in its own terminal:

```
cd recommender && uv run recsys serve
cd backend && cargo run
cd frontend && npm run dev
```

With the recommender running, the latency script calls it directly over gRPC:

```
cd book && uv run python tools/run_all.py --only ch13_latency
```

Because `seed-db` shifts the simulated history so that it ends at the moment you run it, the live model's metrics and latencies will differ slightly from the ones printed in this book.

4. Charts

```
cd book && uv run python tools/make_figures.py
```

Reads `results/*.json` and writes every PNG in `assets/charts/`. Each chapter's charts are drawn by `tools/figures/chNN.py`.

5. Screenshots

With all three services running and the app at `http://localhost:5173`:

```
cd book && uv run python tools/screenshots.py
```

This logs in through the real UI as `user43@demo.test`, places an order, then logs in as that shop's seller and as `admin@demo.test`, and writes the PNGs in `assets/screenshots/`. The fallback banner needs the recommender to be **stopped**:

```
# stop `recsys serve`, then:
cd book && uv run python tools/screenshots.py --only fallback
```

6. The PDFs

```
cd book
uv run playwright install chromium
npm i                                    # mermaid + KaTeX, rendered inside the page
uv run python tools/check.py --lang en
uv run python tools/check.py --lang id
uv run python tools/build.py --lang all
```

`check.py` verifies every include, output file and image exists, and that each chapter has its fact boxes, diagram, figures, quiz and minimum length. `build.py` renders Markdown to HTML, opens it in headless Chromium, waits for the diagrams and formulas to render, and prints `dist/recsys-book-en.pdf` and `dist/recsys-book-id.pdf` with their page counts.

Bibliography

Works cited in the text and in the "Did you know?" boxes, alphabetical by first author.

- Abdollahpouri, H., Mansoury, M., Burke, R., & Mobasher, B. (2019). The Unfairness of Popularity Bias in Recommendation. Workshop on Recommendation in Multi-stakeholder Environments (RMSE), RecSys 2019.
- Amatriain, X., & Basilico, J. (2012). Netflix Recommendations: Beyond the 5 stars (Part 1). Netflix Technology Blog.
- Anderson, C. (2004). The Long Tail. Wired.
- Burges, C. J. C. (2010). From RankNet to LambdaRank to LambdaMART: An Overview. Microsoft Research Technical Report MSR-TR-2010-82.
- Burges, C. J. C., Svore, K. M., Bennett, P. N., Pastusiak, A., & Wu, Q. (2011). Learning to Rank Using an Ensemble of Lambda-Gradient Models. Proceedings of the Learning to Rank Challenge (PMLR 14), 25–35.
- Carbonell, J. and Goldstein, J. (1998). The use of MMR, diversity-based reranking for reordering documents and producing summaries. Proceedings of the 21st Annual International ACM SIGIR Conference (SIGIR).
- Chaney, A. J. B., Stewart, B. M., & Engelhardt, B. E. (2018). How algorithmic confounding in recommendation systems increases homogeneity and decreases utility. Proceedings of RecSys 2018.
- Chapelle, O. and Li, L. (2011). An empirical evaluation of Thompson sampling. Advances in Neural Information Processing Systems (NeurIPS/NIPS).
- Cheng, H.-T., Koc, L., Harmsen, J., Shaked, T., Chandra, T., Aradhye, H., et al. (2016). Wide & Deep Learning for Recommender Systems. Proceedings of the 1st Workshop on Deep Learning for Recommender Systems (DLRS).
- Covington, P., Adams, J., & Sargin, E. (2016). Deep Neural Networks for YouTube Recommendations. Proceedings of the 10th ACM Conference on Recommender Systems (RecSys '16).
- Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. Annals of Statistics, 29(5), 1189–1232.
- Funk, S. (2006). Netflix Update: Try This at Home. sifter.org blog, 11 December 2006.
- Gomez-Uribe, C. A., & Hunt, N. (2015). The Netflix recommender system: Algorithms, business value, and innovation. ACM Transactions on Management Information Systems.
- Google. (2026). Language Guide (proto 3). Protocol Buffers Documentation.
- Guo, R., Sun, P., Lindgren, E., Geng, Q., Simcha, D., Chern, F., & Kumar, S. (2020). Accelerating large-scale inference with anisotropic vector quantization. ICML (PMLR 119).
- He, X., Pan, J., Jin, O., Xu, T., Liu, B., Xu, T., Shi, Y., Atallah, A., Herbrich, R., Bowers, S., & Candela, J. Q. (2014). Practical Lessons from Predicting Clicks on Ads at Facebook. Proceedings of the Eighth International Workshop on Data Mining for Online Advertising (ADKDD, KDD workshop).
- Hu, Y., Koren, Y., & Volinsky, C. (2008). Collaborative Filtering for Implicit Feedback Datasets. Proceedings of the IEEE International Conference on Data Mining (ICDM).
- IMDb. (2026). Ratings FAQ: How do you calculate the rank of movies and TV shows on the Top 250 charts? IMDb Help Center.
- Joachims, T., Granka, L., Pan, B., Hembrooke, H., & Gay, G. (2005). Accurately interpreting clickthrough data as implicit feedback. Proceedings of SIGIR 2005.
- Joachims, T., Swaminathan, A., & Schnabel, T. (2017). Unbiased learning-to-rank with biased feedback. Proceedings of WSDM 2017.
- Johnson, J., Douze, M., & Jégou, H. (2021). Billion-scale similarity search with GPUs. IEEE Transactions on Big Data.
- Järvelin, K., & Kekäläinen, J. (2002). Cumulated gain-based evaluation of IR techniques. ACM Transactions on Information Systems, 20(4).

- Kaufman, S., Rosset, S., Perlich, C., & Stitelman, O. (2012). Leakage in Data Mining: Formulation, Detection, and Avoidance. ACM Transactions on Knowledge Discovery from Data, 6(4).
- Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., & Liu, T.-Y. (2017). LightGBM: A Highly Efficient Gradient Boosting Decision Tree. Advances in Neural Information Processing Systems 30 (NeurIPS).
- Krichene, W., & Rendle, S. (2020). On Sampled Metrics for Item Recommendation. Proceedings of the 26th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD).
- Linden, G., Smith, B., & York, J. (2003). Amazon.com Recommendations: Item-to-Item Collaborative Filtering. IEEE Internet Computing, 7(1), 76–80.
- Liu, Z., Zou, L., Zou, X., Wang, C., Zhang, B., Tang, D., Zhu, B., Zhu, Y., Wu, P., Wang, K., & Cheng, Y. (2022). Monolith: Real Time Recommendation System With Collisionless Embedding Table. 5th Workshop on Online Recommender Systems and User Modeling (ORSUM), RecSys 2022.
- Lundberg, S. M., & Lee, S.-I. (2017). A Unified Approach to Interpreting Model Predictions. Advances in Neural Information Processing Systems 30 (NeurIPS).
- Lundberg, S. M., Erion, G., Chen, H., DeGrave, A., Prutkin, J. M., Nair, B., Katz, R., Himmelfarb, J., Bansal, N., & Lee, S.-I. (2020). From local explanations to global understanding with explainable AI for trees. Nature Machine Intelligence, 2(1), 56–67.
- Ma, J., Zhao, Z., Yi, X., Chen, J., Hong, L., & Chi, E. H. (2018). Modeling Task Relationships in Multi-task Learning with Multi-gate Mixture-of-Experts. KDD 2018.
- Malkov, Y. A., & Yashunin, D. A. (2020). Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. IEEE TPAMI.
- Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). Efficient estimation of word representations in vector space. arXiv:1301.3781.
- Mozilla Contributors. (2026). Navigator: sendBeacon() method. MDN Web Docs.
- Rendle, S., Freudenthaler, C., Gantner, Z., & Schmidt-Thieme, L. (2009). BPR: Bayesian Personalized Ranking from Implicit Feedback. Proceedings of the 25th Conference on Uncertainty in Artificial Intelligence (UAI).
- Resnick, P., Iacovou, N., Suchak, M., Bergstrom, P., & Riedl, J. (1994). GroupLens: An open architecture for collaborative filtering of netnews. Proceedings of the ACM Conference on Computer Supported Cooperative Work (CSCW).
- Sarno, D. (2009). Netflix awards $1-million prize to recommendation wizards, announces 2nd contest. Los Angeles Times.
- Sarwar, B., Karypis, G., Konstan, J., & Riedl, J. (2001). Item-Based Collaborative Filtering Recommendation Algorithms. Proceedings of the 10th International World Wide Web Conference (WWW).
- Schein, A. I., Popescul, A., Ungar, L. H., & Pennock, D. M. (2002). Methods and metrics for cold-start recommendations. Proceedings of SIGIR 2002.
- Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.-F., & Dennison, D. (2015). Hidden Technical Debt in Machine Learning Systems. Advances in Neural Information Processing Systems (NeurIPS) 28.
- Singh, A. and Joachims, T. (2018). Fairness of exposure in rankings. Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD).
- Spärck Jones, K. (1972). A statistical interpretation of term specificity and its application in retrieval. Journal of Documentation, 28(1), 11–21.
- Thompson, W. R. (1933). On the likelihood that one unknown probability exceeds another in view of the evidence of two samples. Biometrika, 25(3/4).
- Westergren, T. (2011). Tim Westergren, Founder of Pandora (radio interview). NPR.
- Yi, X., Yang, J., Hong, L., Cheng, D. Z., Heldt, L., Kumthekar, A., Zhao, Z., Wei, L., & Chi, E. (2019). Sampling-bias-corrected neural modeling for large corpus item recommendations. RecSys.
- Ying, R., He, R., Chen, K., Eksombatchai, P., Hamilton, W. L., & Leskovec, J. (2018). Graph Convolutional Neural Networks for Web-Scale Recommender Systems. KDD 2018.
- Zhou, G., Song, C., Zhu, X., Fan, Y., Zhu, H., Ma, X., Yan, Y., Jin, J., Li, H., & Gai, K. (2018). Deep Interest Network for Click-Through Rate Prediction. KDD 2018.

- Ziegler, C.-N., McNee, S. M., Konstan, J. A. and Lausen, G. (2005). Improving recommendation lists through topic diversification. Proceedings of the 14th International Conference on World Wide Web (WWW).